

Metheus

FAIR ACCOUNTABLE TRANSPARENT

Machine Intelligence Audit

LING XIAO

5th April 2026

Draft: 5th April 2026:Metheus: Ling Xiao

Abstract

There are two mega trends ripping through the world today:

1. Artificial intelligence (AI) permeating every aspect of our lives.
2. Currency and supply-chain multipolarity advancing across the globe on the heels of war and sanctions.

Metheus builds a set of market protocols that anneals these two separate trends into a united front. On the AI front we stand with the public, and push open-governance of artificial intelligence as a fundamental right. This is a call for AI reform around the principles of digital sovereignty. Metheus builds tools that allow anyone to audit AI models, thereby converting passive consumers into active decision makers. Specifically:

1. Just as material commodities have physical supply chains, AI models are products of digital supply chains. It spans data collection, data annotation, architecture exploration, model development, and finally refinement towards end-user integration.
2. Just as people are judged by IQ and EQ metrics, there are similar measures for AI models. IQ corresponds to the quality of models on impartial performance metrics, while EQ measures models along socially normative standards of fairness and equality.

Metheus constructs protocols so that the public may audit AI agents according to openly defined IQ and EQ standards, over the entire model development process.

1. In this autonomously financed supply chain, auditing pays for model development. So that as the supply chain is audited, it is also financed.
2. Concurrently, capable auditors are rewarded for discovering performant models and data. Thus the relationship between public auditors and the model developers is not an adversarial one. Instead they collude to produce socially moderated AI agents.
3. Critically, open auditing creates a public repository of communally maintained data and models. Thereby promoting new digital commons and standards in the presently fragmenting digital Westphalia.

Since Metheus decentralized auditing is equipped with financial incentives, we tap the flow of this auditing circuit to construct an *Entropy Digital Currency*. It is an analogue to the Petrodollar in the AI supply chain, and denominates the fundamental quanta of value along this circuit: information. Thus Metheus positions itself in the second trend. However, whereas others see multi-currency mercantilism as nations trading in sovereign currencies, Metheus rezones the playing field as industries trading in programmable domain-specific currencies, exchanged along commonly defined technical rails and standards.

In conclusion, Metheus develops protocols that allow people to realize their fundamental right of governing artificial intelligence. We then leverage this AI auditing-circuit to bootstrap an industry-specific digital currency. Together they define a unified techno-social horizon upon which all may reside and prosper.

Contents

Abstract	i
Contents	ii
1 Introduction	1
1.1 Overview	1
1.2 Machine Intelligence	2
1.3 Decentralized Finance	3
1.4 The Path Forward	4
1.5 Decentralized Machine Intelligence Audit	5
1.6 Metheus System Overview	6
1.7 How to Read this Monograph	15
I Core Protocol	17
2 Machine Intelligence Quality Audit	18
2.1 Introduction	18
2.2 QAudit Incentive Structure	23
2.3 QAudit Core	28
2.4 QAudit Machine Learning	46
2.5 QAudit Elementary Analysis	49
2.6 Fundamental Theorems of QAudit	59
2.7 Perpetual QAudit	70
2.8 Conclusion	72
3 Machine Intelligence Fairness Audit	73
3.1 Introduction	73
3.2 FeaAudit Elementary Analysis	80
3.3 Tokenizing FeaAudit	83
3.4 Conclusion	87
II Stablecoin Protocol	89
4 FAT and Stable	90
4.1 Introduction	90
4.2 FAT Issuance and Redemption	90
4.3 FAT Exchange Rate	95
4.4 FAT Valuation	96
4.5 Conclusion	105

4.6	Postscript	105
Appendices		107
A	Monograph Notation	108
B	Definitions	110
B.1	Randomized Algorithms	110
B.2	Game Theory	111
C	Proofs	113
C.1	Agents-Agents Sub-Game	113
C.2	Agents-Auditors Sub-Game	115
C.3	Auditors-Auditors Sub-Game within every Stage	117
C.4	Fundamental Theorem across Stages	123
C.5	FeaAudit	128
D	Sequential Parimutuel Prediction Markets	131
D.1	Introduction	131
D.2	Exponential Family Sequential Prediction Market	136
E	Machine Learning	143
E.1	Introduction	143
E.2	Hilbert Space	143
E.3	Learning in Hilbert Space	145
E.4	Neural Tangent Kernel	152
F	Exponential Family Distributions	171
F.1	Derivation	171
F.2	Canonical Representation	172
F.3	Mean Representation	173
F.4	Kullback-Leibler Divergence	173
Bibliography		176

CHAPTER 1

Introduction

1.1 Overview

Recent decades have witnessed deep ruptures in how the modern economy operates. In the old model, intelligent people design mindless machines, and together they produce the goods and services. All of which are ultimately denominated in: the Petrodollar. The new age is just dawning, but it is becoming increasingly clear that in the future, machine intelligence will provide many of the services, and design many of the "goods." Presently, the fate of the Petrodollar is under fierce litigation. But the outcomes of the early innings suggest that its last act will be a violent one. This is a sea change that will torch industries, set entire communities adrift, and unmoor nations.

There are decades where nothing happens; and there are weeks where decades happen.

This is disruption par excellence, a climate change of social relations. The way of Metheus is to *place as many people on the right side of this change as possible*. We address the problem wholistically by fielding protocols that mediate:

1. how machine intelligence is developed and its quality audited;
2. how data providers are motivated, audited, and compensated;
3. how algorithms are moderated for its social implications;
4. how the general public may partake and directly profit from this auditing process;
5. and most importantly: how this burgeoning economy is monetized.

Here "monetization" does not mean "making money." Rather, it quite literally means measuring the value created in this underlying economy by minting new currencies. Said numéraire are recorded on a single-purpose decentralized ledger, native to the Metheus ledger is the token-pair:

1. $\mathfrak{F}\mathfrak{a}\mathfrak{t}$: the Fair Accountable Transparent stablecoin;
 2. $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$: the Machine Intelligence Audit Token.
-

$\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is the world's first **entropy digital currency**.

Together they measure the productive potential of the **Metheus** economy, one that is built on quantitative and normative introspection of machine learning algorithms. Thus **ƒat** is the world's first **entropy digital currency**:

1. its underlying commodity of choice is not petroleum but raw information;
2. its refineries are graphic processing units (GPUs);
3. its refined oil is trained models.

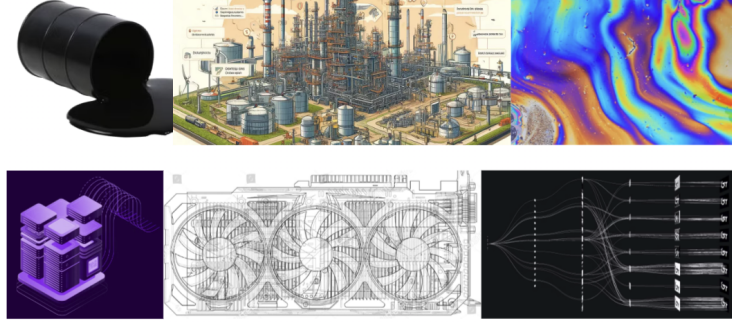


Figure 1.1: Metheus mints the fair, accountable, and transparent stablecoin (**ƒat**). **ƒat** is the world's first entropy digital currency. Instead of feeding the perpetual violence of the Petrodollar economy, **ƒat** denominates the burgeoning machine intelligence world by auditing it.

This shift in underlying commodity of value is profound. Whereas the Petrodollar tracks population growth, the entropy digital currency harnesses growth in artificial intelligence. Whereas the Petrodollar is built upon perpetual conflicts, **Metheus** makes wagers, not war.¹ The moment to spring is now. This is clear in light of recent history.

Metheus makes wagers, not war.

1.2 Machine Intelligence

Much ink has been spilled over the transformative potential artificial intelligence (AI) ever since the first conversational agent ELIZA was conceived at MIT in 1964. Since then the field has seen its ups and downs. Highlights include:

1. from the disillusionment of neural networks in the 1980s due to insufficient data and compute power, to its 2014 redemption by internet-scale data lakes and GPUs.
2. From the utter infeasibility of solving GO with classical tree search, to its rousing resolution with deep reinforcement learning.
3. From the inconceivably futuristic sci-fi nature of human-like dialogue systems, to its now mundane applications due to massive generative models.

¹Normally entropy means disorder. In this case entropy is a measure of information, see [Sha48].

Each trough of disillusionment is followed by the subsequent highs of a seemingly inevitable breakthrough. Once again genuine progress has been made in this cycle. Moreover, many of the AI models that enable this step-change are in the public domain, ready to be recapitalized on by intrepid opportunists (see remark 1.6.1). Presently, industrial scale artificial neural networks (ANNs) are generating images, videos, and holding conversations at a high enough fidelity to replace artists, human assistants, and even teachers. Consumer applications notwithstanding, the long-term trend of consequence is the massive investments at the infrastructure level. Since 2021, billions of GPU related hardware have been built by major corporations. However this is now followed by a growing sense of unease, as the current generations of generative models prove too error-prone for use-cases demanding high precision.² It is becoming increasingly clear that an entire crop of startups may not see revenue commensurate with the volume of investor money spent on training. As this consensus diffuses through the system, deal flow taper as investors conclude it is *they* who have been harvested by the likes of Microsoft. Meanwhile, Deepseek released the R1 model that is on parity with OpenAI's o1, at a mere fraction of the training and inference cost. This dramatic step-change fundamentally negates the thesis that massive compute capex is necessary precondition for winning the "AI race." SV companies scramble as they justify their massive fund raise and eye-popping valuation. The fear has spilled into main-stream media. The next AI winter must be coming.

However there are historical reasons to remain optimistic. This moment is similar to the burst of the dot-com bubble, whereby a torrent of broad-band infrastructure was built in anticipation of demand, which eventually came with the advent of web 2.0. In summary, the present juncture is as follows: there will be a deluge of compute capacity for model training and inference, and comparatively fewer customers using them. The pressing question for those who have sunk billions into GPU farms is how to survive this winter. They seek:

1. new use cases for existing models to recoup their astronomical training costs;
2. better ways of prospecting novel model architectures in anticipation of the next spring;
3. new data/model moderation tools to keep the regulators at bay.

Metheus offers one such avenue by bridging machine intelligence onto a decentralized environment. Thus tapping a broader capital base, new use cases, and new ways of socializing the technology.

Metheus bridges machine intelligence with DeFi capital, pegging the entropy-stablecoin to the open-source AI economy.

1.3 Decentralized Finance

Similar to artificial intelligence, decentralized finance (DeFi) as an industry has seen even greater boom and deeper busts.

1. In the first cycle, purist, libertarians, and anarchists showed that it is possible to build a permissionless payment system around Bitcoin.

²Overparameterized statistical models trained on past data tend to have high recall and low precision. This is fine in entertainment-related applications such as generating digital influencers, but may prove deadly in sensitive domains.

2. This is followed by a decentralized computational environment in Ethereum, featuring a native on chain currency **Eth**.
3. This new model of networked computation is complemented by a distributed storage environment in IPFS, featuring native currency **filecoin**.

However wild fluctuation in prices of Bitcoin and **Eth** render their value as a numéraire dubious. This has inspired a family of stablecoins whose value is pegged to 1 unit of any currency. Broadly speaking, there are many ways to peg a stablecoin:

1. (*Fully collateralized fiat backed stablecoin*): the users are essentially lending money to the platform by depositing 1 fiat of choice, and receiving one minted stablecoin in return. Tether is the most well-known example with a market cap of USD 138.3B. Although Tether can be redeemed in theory, in practice it is not straightforward. Moreover, since customer deposits are reinvested in difficult-to-audit ways, de-pegging is a very real risk [Har+20],[AS22].
2. (*Central Bank Digital Currencies or CBDC*): one subset of projects adjacent to the category above are CBDCs [D25]. As of 2023, over 100 countries representing more than 95% of global gross domestic product, are actively engaging in research, development, pilots or have fully-launched CBDC initiatives [SM+23]. The leader here is project mBridge, a permissioned blockchain project for whole-sale CBDC. Its primary purpose is intra-central-bank settlements, cross-border atomic settlement of tokenised primary issuance, and supply chain financing [[hkm22],[UBS21]].
3. (*Over-collateralized stablecoin backed by a "diversified" basket of cryptocurrencies*): similar to an exchange traded fund, the "vault controller" assembles a basket of cryptocurrencies and issues shares of the underlying tracking any fiat of choice. Stability here requires active market making. A more fundamental flaw is that DeFi coins are highly correlated, so this "diversified" basket may not even exist [AM+23].
4. (*Algorithmic stablecoins*): or Ponzi scheme. Luna is the most notorious example. It imploded after a catastrophic deleveraging spiral, and its founder is now in jail [KP+23].

Despite its arduous history, the utility of stablecoins stands true. Technically speaking, any fiat currency bridging onto a DeFi environment requires an on-chain mirror anyways. This unit of account is by construction a stablecoin. Furthermore, any usable decentralized economy must stabilize the price of its native currency. So Metheus pushes the field forward by pegging the **fat** stablecoin to a true economy with sustainable future value.

1.4 The Path Forward

Charting a path forward requires a keen understanding of the intrinsic rhythm of how both industries have progressed. In AI, there is an exploration-exploitation trade off in prospecting models. In the exploration phase, many architectures across data verticals are tried, so one must justify scaling the model to an industrial scale.³ Once a viable model architecture is (re)discovered, the industry enters the exploitation phase. Here models are packaged and sold as enterprise software, and sometimes consumer products. While AI summer is exploitation, the subsequent winter is a time for exploration. Naturally, winter lasts longer than summer. So given its longer duration and requisite capital outlays, exploration is done in universities and well-funded industry labs.

Metheus reforms this process by supplementing open-source data and models with distributed governance and *open-source capital*. The Metheus ecosystem transforms industrial-scale data annotation and model exploration into a profitable decentralized protocol.

³For example, training dialogue systems using recurrent neural networks is not a new idea, but training them at the scale of OpenAI was.

Concurrently, we will address the structural issues around open machine intelligence governance that is only possible in a truly decentralized environment. So that the fruits of everyone's labor shall exist outside of major companies, in a way that the products of OpenAI and DeepMind did not.

AI as an industry is built on open-source models, free data, and transparent benchmarks. Metheus pushes it further with distributed governance, and open-source capital.

In DeFi the cycles are difficult to predict, so no attempt is made here. However, history suggests activities in this space are downstream to global events and local disruptions. It is quite clear that the next five years will bring a cluster of unexpected events across the globe that may even exceed the previous five years, this bodes while for the industry as a whole. Meanwhile, Bitcoin and Eth have gained institutional acceptance, so au contraire the detractors they are here to stay. Although *fat* is designed to be fiat-collateralized, the existence of a decentralized currency network and a persistent permissionless compute environment gives us additional legs to stand on. The groundwork is laid for a decentralized capital and compute environment for auditing machine intelligence.

1.5 Decentralized Machine Intelligence Audit

The blazing speed with which AI has developed in the last few years may rouse the technically inclined, but its "life-like" outputs has spooked main street and regulators alike. Rather than "move fast and down with the luddites," a wiser approach may entail bringing a broader swath of people into the research and development process. This is in accordance with the logic of history, where technology tend to be socialized in two waves:

1. first an aggressive first mover pushes the technology into mainstream consciousness as a viable mass-consumer product.
2. However, this first wave is often deemed too aggressive or even illegal. So what follows is the slower next wave that benefits more parties, thus finding long-term success.

The path from Napster to Spotify is the canonical example. Metheus follows this wave of generative AI by socializing passive users into active decision makers. They negotiate the nature of this technology by way of *auditing*. Now the word "audit" has a precise definition in the field, however we pursue a more expansive interpretation, this allows Metheus to protocol a larger segment of the AI development lifecycle.

Definition 1.5.1. (*Machine Intelligence Audit*): is the quantitative and qualitative introspection of machine learning models, the data they are trained on, and the entire development process. This process proceeds along all conceivable dimensions, such as:

- 1.5.1.1. *full spectrum quality audit*: of data to be used to train models.
- 1.5.1.2. *performance introspection*: along well-defined metrics of models trained on data.
- 1.5.1.3. *annotator assessment*: of people who provide labels to data.

1.5.1.4. *fairness, equality, and equity inspection*: according to fluid and socially defined norms of what is "acceptable" behavior for AI agents.

The vectors defined here are far from exhaustive. Moreover, since the community is constantly developing new and better metrics to characterize each point, the variety of auditing protocols is in effect unbounded. In more basic terms, "auditing" is fundamentally a *protection mechanism*. Metheus steers the relationship the general public has with machine intelligence, and protects the public from the prospect of intelligent machines. All the while giving the people a chance to profit from this arrangement.

Decentralized auditing is a public safety mechanism. **Metheus** mediates people's relationship with machine intelligence, and protects the public from the prospect of intelligent machines.

In allegorical terms, "auditing" is AI governance. Whereby the machine learning algorithms are the governed, while the people form the government. From a positioning standpoint, auditing as a basis for open protocol is well situated amongst industry stakeholders:

1. *Model foundries*: we complement well-capitalized foundational model developers by *social proofing AI* in a public certification process. People quite literally buy in.
 2. *Compute vendors*: auditing complements their hardware and improve their revenue outlook. The Metheus auditing process creates communally maintained pools of data and models that are continuously updated. At the transactional level, auditing is a prolonged inference session, so it smooths out the demand curve for existing AI models.
 3. *The public*: auditing gives the average person a stake in the algorithm development process, it is a new way of relating to AI that may become mainstay of culture. Think nutrition labels on the back of food packages that have become legislated must-haves.
 4. *Regulators*: their attempts to regulated AI as new abstraction over the internet is understandable, and their contempt towards DeFi is a perennial given. Metheus gives regulators a way to steer one tech cycle with tools from another, in effect "using poison to fight poison."
-

Fair and accountable AI is the new ESG

1.6 Metheus System Overview

System Components

This section outlines the silhouette of the Metheus economy, with a particular emphasis on its main components.

Data Centers and GPU Farms

Infrastructure-level support will enter the Metheus ecosystem through third-party vendor partnerships. They are used to host Metheus-associated models and data. They *will not* be used to mine cryptocurrency.

Metheus Consortium (Side) Chain

On the technical front, the consortium sidechain is a refactored version of **Ethereum**, whereby the native **Eth** is removed to prevent speculation. It is the settlement layer for all market making activities in Metheus. Critically, it also hosts the primary-market **ƒat** stablecoin. Price action on the Metheus sidechain keeps the **ƒat** price pegged to 1 fiat of choice. On the partnership front, the Metheus consortium chain is the technical fabric that binds a new form of Blockchain Industry Association: one that is built around the notion of AI governance. Vendors along the entire AI supply chain can participate following this template:

"consortium chains would typically involve those active in, and responsible for, the production, distribution, storage and transport of commodities in question, as well as other stakeholders such as regulators and financial services actors."
[Pow23, p. 132]

In the context of AI industry, the commodity in question are data and models, as well as resellers and distributors of any hardware components. The consortium members determine how the chain should operate, observing the tradition whereby:

"membership of associations can readily map to a technical topology, whereby different members with varying authorities can perform different kinds of functions in the overall information system." [Pow23, p. 136]

There is an analogue of this very dynamic in the Riskstream collaborative, which proxies the entire insurance industry. It defines the compliance needs of industry players, trains the insurance agents, certifies them, and fields a consortium chain for settling claims. The difference is that where as Riskstream is a cost center to which insurance companies must pay their dues, Metheus is a revenue generator that finances the development of AI models as they are audited.

Mainnet

Primary-market **ƒat** tokens on the Metheus sidechain factors into two secondary-market tokens on a production mainnet environment. Any EVM compatible chain will suffice. This process is depicted in figure 1.2, and is a critical component of the incentive package geared towards **Auditors**. **ƒat- α** in particular captures the upside of the underlying AI audit economy mediated by the Metheus market protocols. See chapter 4 for more information on how **ƒat- α** and **ƒat- β** are valued.

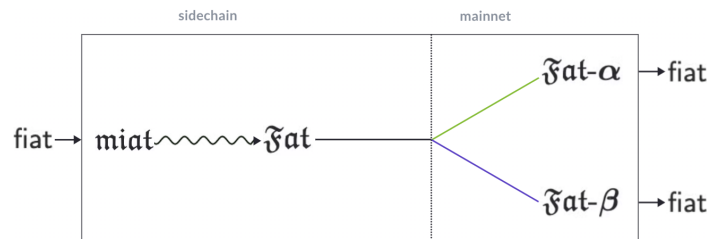


Figure 1.2: The sidechain stablecoin **ƒat** captures the economic upside of the underlying Metheus economy due to fluctuations in **miat** supply. The upside of the underlying is passed onto **ƒat- α** , while **ƒat- β** remains the fiat-pegged fully collateralized stablecoin.

Metheus Market Protocols

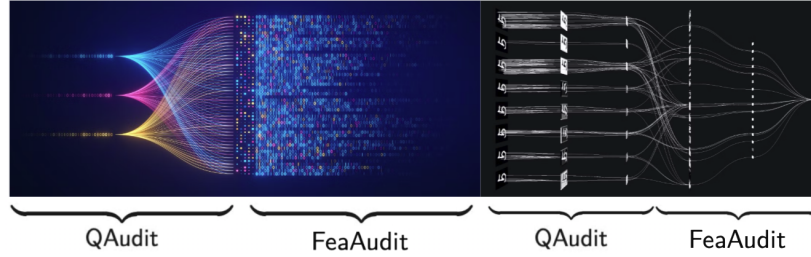


Figure 1.3: In traditional machine development pipelines:

1. data is collected and sanitized by private firms as "back-office" work;
2. annotation is sometimes outsourced completely to third parties;
3. since data carries biases that percolate into the models trained on them, the data itself is also audited along social-compliant measures;
4. foundational model development is done by private firms, and are published only after they have achieved some level of performance.

Metheus opens up this lifecycle completely, so that the "kitchen is in the dining room," so to speak. Metheus market participants now:

1. audit the data w.r.t to fairness/quality measures, and profit in doing so.
2. Model development occurs in the open in competition-like environments. Model "checkpoints," are tokenized into mints, and marketed to Auditors to fund model development.
3. Auditors trade mint tokens to judge their relative performance against other models in the same class.
4. Model output are also audited for fairness and equity infractions. The users' judgments are tokenized and marketed for compensation.

Metheus fields two major classes of auditing protocols:

1. **QAudit**: or machine intelligence quality audit. This protocol satisfies the "full spectrum quality audit" and "performance introspection" of definition 1.5.1. In particular, **QAudit** mediates both:
 - a) *data markets*: where participants can sell their data to third parties, so that the quality of said data is evaluated as they are listed.
 - b) *model markets*: where a distributed team of machine learning engineers coalesce to develop AI models. This setting is similar to Kaggle, however Metheus incentives are significantly better for all stakeholders involved.

From a user-interface standpoint, **QAudit** as a product is a new kind of computational environment where AI engineers do not pay to train models. Rather they are *paid* to train models when they push intermediate states of the unfinished model, or "*checkpoints*." The more checkpoints pushed, the more they are paid. ⁴

⁴The Metheus **QAudit** protocol is based on work done at Harvard, funded by the Ethereum Foundation [[HW19], [WFA15]]. Amazon AI Labs is also fielding a variation of ensemble crowd-sourced machine learning competition [GH+24].

2. **FeaAudit**: satisfies the "annotator assessment" and "fairness, equality, and equity inspection" of definition 1.5.1. It socializes the broader populace into AI development by giving them a consumer-facing annotation protocol. However **FeaAudit** is far from some trivial pursuit, it is the lever to shape the "social intelligence" of AI software. The nature of **FeaAudit** depends on the perspective of the participants:
 - a) from the perspective of annotators, **FeaAudit** is a game whereby they answer questions to survive an elimination game, and are paid if they "hit a run" of acceptable answers.
 - b) From the perspective of the market owner, **FeaAudit** is an annotation service whereby they are not paying for data labels, but rather are *paid* while procuring data labels.

In simpler terms, **QAudit** is model and data development, while **FeaAudit** is data annotation and annotator quality assessment. Together, they protocol a significant portion of the data, model, and product development process (see fig 1.3). Any **Metheus** participants can launch *instances* of these protocols in a permissionless manner, so as to capitalize their own AI development pipeline. Hence the name "decentralized audit." Finally, auditing is a persistent process that never terminates. "Cooking" the product in the open demystifies machine learning, and the public will see that AI agents are fundamentally huge regression models with limited operational domain.⁵

While others profit from the public's ignorance,
Metheus benefits from the public's vigilance.

Metheus Token Ensemble

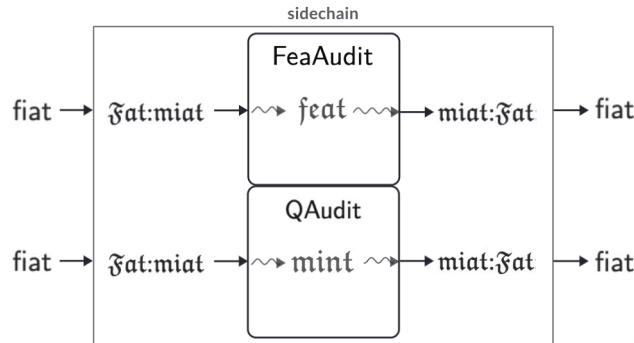


Figure 1.4: In the Metheus economy, fiat is converted to ⚡at on 1:1 basis, and then converted to ERC20 token *miat* at a floating exchange rate with partial bonding curve (see fig 4.7). This *miat* buys *mint* in **QAudit** markets, and *feat* in **FeaAudit** markets. Both are intermediate currencies issued and consumed within one audit session, where their *miat*-denominated upside is realized in *that* session. This *miat* exits the sidechain at spot *miat:⚡at* exchange rate, and convert back to fiat at 1:1. See fig 1.5 for full conversion lifecycle.

⁵One aspect of AI development we do not touch is the productization thereof, especially when it involves with company's private data.

The underlying commodities in QAudit and FeaAudit markets are denominated with:

1. (*mint*): *the machine intelligence nonfungible token*, akin to an ERC-1155. This is effectively a share of model or data. It denotes the quality of the underlying commodity. An overview of its tokenization process is provided in remark 1.6.1. *mint* tokens are denominated only in *miat*.
2. (*feat*): *the fair and equal annotation token*, akin to an ERC-721. This denotes the quality of an annotator or Witness. It is also denominated only in *miat*.

Referencing figure 1.4, we see that:

1. if the underlying commodity performs, then *mint* and *feat* can be redeemed for fiat at well-defined market prices. Thus they have *exchange value*.
2. Because *feat* and *mint* can only be purchased with *miat*, they also endow *miat* with *exchange value*.

In sum, *miat*, *mint*, and *feat* are *currencies*. Meanwhile, the underlying data, labels, and models possess clear use value. Additionally, unlike nonfungible tokens of the last cycle, *mint* and *feat* do not have infinite lifespans. Instead they are:

1. issued at specific moments within QAudit/FeaAudit market lifecycles;
2. have *bounded* upside that is rigorously defined per protocol rules;
3. and most importantly: have fixed expiration dates.

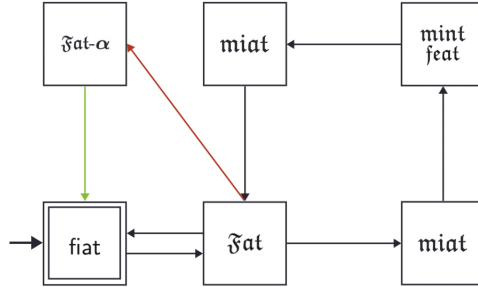


Figure 1.5: The conversion cycle begins when Auditors enter the market with fiat, and proceeds along the path: fiat → fat → miat → (mint, feat) → miat → fat → fiat. Observe the red arrow from fat to fat-α, this is known as *miat-inflation-dividend*. The upside of which accrues to holders of fat-α tokens. See chapters 2 and 4 for full discourse.

Figure 1.5 depicts the token ensemble conversion lifecycle in the Metheus economy, observe:

1. the supply of fat and miat are unbounded, however their rate of growth is not.
2. In particular, the growth in miat supply is a function model, data, and annotation volume in the Metheus ecosystems. It is a measure of the system's capacity to process information. Its rate of growth can never exceed this capacity.
3. Concurrently growth in fat supply is proportionate to fluctuation in miat supply, and its induced miat-inflation dividend. Just as miat supply measures "temperature" of the underlying AI economy, fat is the "entropy digital currency" of this economic system.
4. Moreover, relative fat and miat supply fluctuates within a fixed exchange-rate envelope (see fig 4.7). This envelope maintains the positive miat-inflation invariant.

5. Critically, the ceiling and floor of this exchange rate envelope are determined by $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ and/or $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ holders. So $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ is not just a fiat-collateralized stablecoin, it is also a *governance token* that determines the allowed inflation dividend $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ holders receive.

In sum $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ and $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ are *loosely coupled*, their fluctuating exchange rate contribute to the $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ -inflation dividend of fig 1.5. This is the source of $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin's upside. This monograph will show that Metheus not only improves DeFi tokenomics of past projects, it also betters the incentives of AI-development as well. See chapter 2, 3, and 4 for in-depth discussion.

Market Participants

The core Metheus protocol addresses a spectrum of stakeholders, with different profiles from retail inclined to professionally oriented. The professional entrants are:

1. (PA principle auditor): they launch instances of **QAudit** and **FeaAudit** with the intention of procuring/inspecting data sets or models.
2. (Agents): these can be teams of machine learning engineers. They develop models in context of **QAudit**-machine learning markets, and receive fiat-denominated subsidies on training, in exchange for pushing their model checkpoints into the **QAudit** competition.

The retail side include:

1. (Auditors): these are retail DeFi players who seek new tokens to trade. The principle contribution of Metheus protocol is massaging their trading activity into socially-positive auditing activity.
2. (Witnesses): they are **FeaAudit** annotators who judge AI agents for fairness infractions. Their roles can be further generalized to annotation tasks of any kind. The **FeaAudit**-Witness paradigm is a direct substitution for Amazon mechanical Turk farm. However unlike other annotation services, in **FeaAudit** the annotators are not paid, instead they "pay to label." Chapter 3 shows how this is possible.
3. (Agents): in addition to being teams, they can also be consumers who have data to sell or annotate. In which case they can deposit their goods into a **QAudit**-data-market instance for Auditor-judgment, and receive fiat rewards.

Finally, there is the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -creditor. This monograph covers $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -offerings suitable for professional and retail counterparties. Ideally most $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ holders are retail, thereby socializing the upside of the AI economy broadly. This is not an euphemism: $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -creditors should never spend more than 1 fiat of choice per $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin, and its value is always *at least* 1 fiat. Chapter 4 provide a rigorous valuation framework for $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoins.

Remark 1.6.1. (QAudit bespoke tokenization process). Metheus is a protocol developer that enable principle auditors (PA) to launch their own market instances.⁶ However these markets can be complex. **QAudit** for example is a multi-sided market, cold-starting them is challenging. The most difficult set of participants is the **Agents**, they must agree to submit models for wager on the public market. However **Agents** are professional algorithm developers with well-defined workflows, they may not even be allowed to engage in "DeFi activities."

This is why **QAudit** can run in "single-set-of-player" mode, where the three sided market of PA, **Agents**, and **Auditors** is reduced to PA and **Auditors**. The PA now just has to convince

⁶In crude terms, the PA owns the market instance; i.e. Jeff Bezos own Amazon, so he would be a PA. The Amazon sellers would be the **Agents**, while the buyers are the **Auditors**. **QAudit** is set up so that Amazon does not need sellers, just Bezos and buyers.

Auditors (read: DeFi players) to wager on existing open-source models. Figure 1.6.1 below describes how QAudit can be configured for this task.

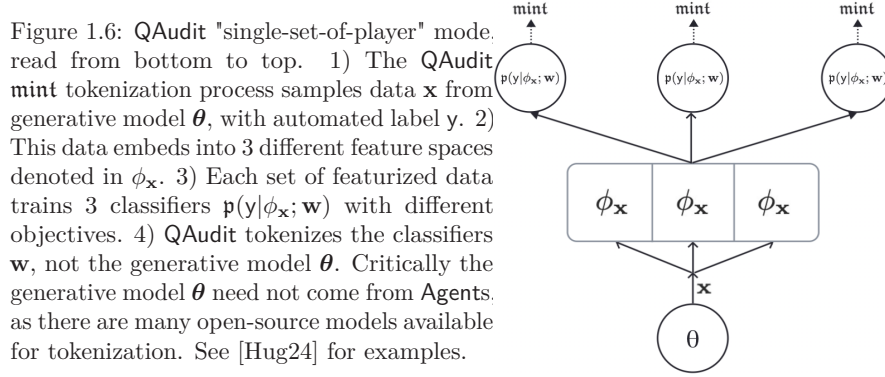


Figure 1.6: QAudit "single-set-of-player" mode. read from bottom to top. 1) The QAudit mint tokenization process samples data $\mathbf{x}$ from generative model θ , with automated label y . 2) This data embeds into 3 different feature spaces denoted in $\phi_{\mathbf{x}}$. 3) Each set of featurized data trains 3 classifiers $p(y|\phi_{\mathbf{x}}; \mathbf{w})$ with different objectives. 4) QAudit tokenizes the classifiers $\mathbf{w}$, not the generative model θ . Critically the generative model θ need not come from Agents, as there are many open-source models available for tokenization. See [Hug24] for examples.

In the example of figure 1.6.1, given one class of open source generative model, the PA launches $3 \times 3 = 9$ different *derivative* QAudit markets wagering on their quality. The protocol trains the classifier $\mathbf{w}$'s, so no Agents participation is needed. This is not just business sense, there are strong technical reasons for this tokenization process. See chapter 2, Appendix D, E, and F for full technical discussion.

Operating Model

This section ties the various factors together into the Metheus operating model. Referencing fig 1.7, reading from top to bottom. The model proceeds as follows:

1. (**miat** mine): in the Metheus machine intelligence auditing economy, **miat** is mined from a global pool at a rate commensurate with market activity platform wide.
2. (Audit markets): when Auditors wager in QAudit and FeaAudit market instances, they purchase **miat** with fiat of choice at the current **fiat**:**fiat**:**miat** conversion rate. Then they use **miat** to buy model shares **mint**, and Witness shares **feat**.
3. (Inflation pool): as shares are traded and **miat** mined, the markets realize both fees and **miat-inflation-dividend**. The fees are taken by the Metheus operating company, while the **miat-inflation-dividend** are placed into the inflation pool. Critically, since the Auditors buy into the QAudit/FeaAudit market with fiat, this inflation dividend is denominated in fiat, not **miat**. The inflation pool is an off-chain traditional finance (TradFi) facility.
4. (Collateral pool): Metheus then issues **fiat** stablecoin, pegged to fiat of choice. This **fiat** is partially collateralized by the Metheus operating company, with additional contributions from **fiat** creditors. Note this means the **fiat** creditor is spending less than 1 fiat to purchase 1 **fiat** stablecoin of that denomination. However given Metheus's contribution, **fiat** is still fully collateralized. This bears repetition: Metheus will not debase the **fiat** stablecoin. The fiat collateral are placed in off-chain TradFi credit facility, where KYC/AML is part of the **fiat** issuance process.
5. (TradFi bridge): the collateral pool places deposits in safe assets, and earn yields. The yields fund Metheus, while **miat-inflation-dividends** are given to **fiat** or **fiat- α** holders as seen in step 3.

In summary, $\mathfrak{F}at$ holders trade yield in their fiat currency of choice for $miat$ -inflation-dividends. They are swapping one low-yield but stable source of interest, for a high-yield high-risk instrument in $\mathfrak{F}at-\alpha$. However the value of their $\mathfrak{F}at-\beta$ never depreciates modulo TVM.

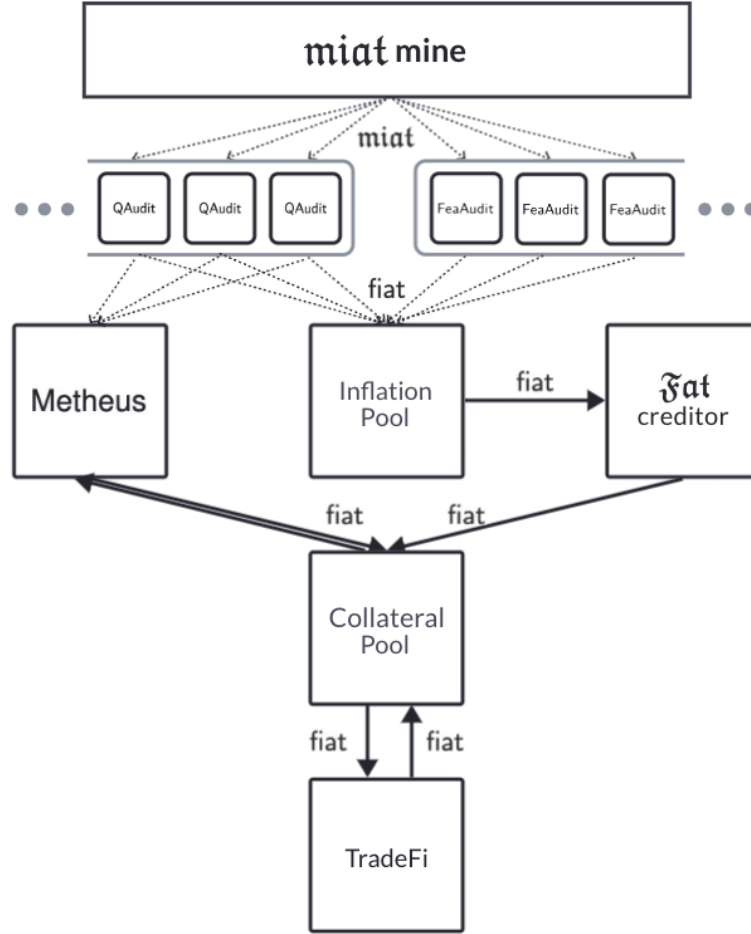


Figure 1.7: The Metheus operating model.

System Calendar

The operating diagram of fig 1.7 can be further enriched by a model of the Metheus's operating calendar. In this view, Metheus is an orchestrator that syncs the rhythm of the DeFi sector with the development process of AI models. This occurs most in QAudit, which proceeds along well-defined *stages*, whereby in each stage models are submitted by Agent, wagered on by Auditors, and the preliminary winners announced. $\mathfrak{F}at-\alpha$ distribution tracks this submission/wager cycle, so that $\mathfrak{F}at-\alpha$ -holders receive regular payouts.

Moreover, QAudit mint auction rules are such that mint token prices oscillate along predictable trajectories over this calendar, in an overall sweep that resembles Bitcoin. The Metheus market protocol allows a *controlled* amount of speculation, but resets the market on a regular schedule so that prices do not exceed the use-value of the underlying. This is depicted in the top curve in fig 1.8. Predictable price oscillation is part of the broader Metheus productization process, whereby commodities such as data, models, and annotations

from one sector of the economy are presented in an attractive manner, so as to appeal to market participants from the DeFi world.

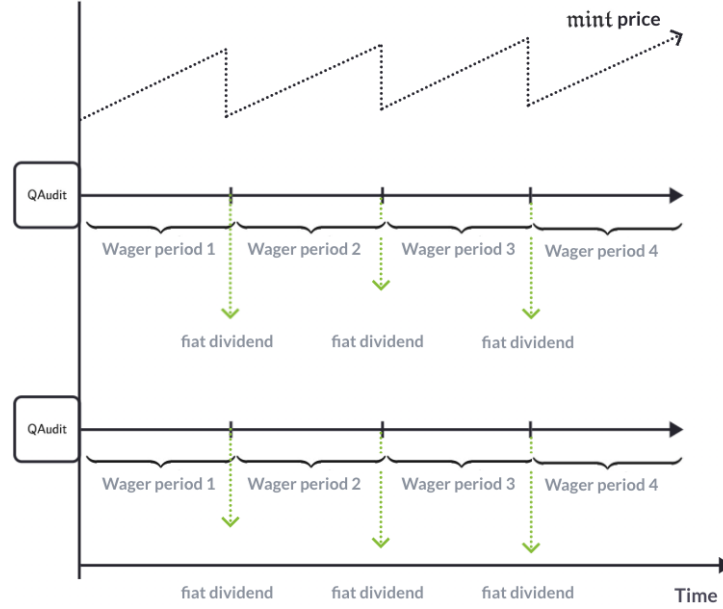


Figure 1.8: The top level user interface of Metheus is the system calendar, whereby multiple QAudit competitions and FeaAudit oracle sessions are running (only QAudit instances shown). Each one features wagering session that may last 1 week, where Agents submits model checkpoints or raw data, and Auditors wager on the quality of models/data submitted. Each weekend, their positions are liquidated. The winning proceeds are distributed, and mint-inflation dividend paid out to $\mathfrak{F}\mathfrak{a}\mathfrak{t}/\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ holders. This AI-auditing calendar syncs the rhythm of DeFi players with AI model development cycles, and ensure the activity of one sector (DeFi) subsidizes the expenditure of another (AI). In Metheus, *trading subsidizes training*.

Philosophically, one may conceptualize QAudit as standardized exams for AI models, while FeaAudit instances are standardized quizzes for annotators. Then the "Metheus Calendar" is in fact a regular examination schedule. This is a very old institutional design pattern that collates talent from afar and near: be it performant AI models, machine learning engineers, or quality annotators. In the very long run, the Metheus decentralized institution may contend with deep-pocketed Industry Labs for the future of AI development. This includes new talent, new technical standards, and new culture of machine learning research and production.

System Incentives

We have characterized two ways of viewing Metheus: 1) as a set of markets protocoled by QAudit and FeaAudit, and 2) a calendar. The third way is to view Metheus as a system of incentives - this is the product in its purest form. In QAudit you will find these principles:

1. Trading subsidizes training.
2. Agents: eat what you bartered with.
3. Auditors: eat what you paid for. But if you defect, you shall regret.
4. Auditors and Agents hunt together, but eat separately.

In FeaAudit we have desiderata:

1. The principle **Auditor** is hidden.
2. The vice **Auditor** is exposed.
3. The **Witness** is hidden.
4. The **Cohort** is exposed.

These are no mere normative statements, rather they are defined precisely along mathematical lines. In the main body and Appendix C, we show these principles prompt *incentive compatible* behavior that yields a socially positive **decentralized market consensus mechanism**, and a **privacy-preserving data-release mechanism**. And finally in $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ issuance we simply incite an arrangement so that **auditors are creditors**. This last tenet is critical, it ties the system end-to-end so that the most challenging stakeholders are incentivized to hold $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ for the long run, thus improving the quality of activity system wide.

1.7 How to Read this Monograph

This ends the overview to the Metheus ecosystem. The rest of the monograph will see a marked tonal shift. You will find a rigorous layer-by-layer deconstruction of Metheus protocols in **QAudit**, **FeaAudit**, and $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -issuance mechanism. The audience who trolls arxiv.org and/or are used to reading technical documents will find it a breeze. Those who are at home in commercial whitepapers will find this book dense. Nevertheless, we hope this introduction piqued your interest sufficiently, so that both sets of audience give it an honest attempt. If you are reading this monograph for the first time, you may find this order amenable:

1. Chapter 2 on **QAudit**: read section 2.1 to understand the basic rules of the game. Then pay particular attention to the tables in section 2.2 for how price and rewards are set in the **QAudit** staggered sequential parimutuel prediction market. You may skip the text and code block in sections 2.3 and 2.4, however skimming the diagrams and their caption will enrich your understanding of protocol details. Pay close attention to examples 2.3.1 and 2.3.2 to understand **Agent** and **Auditor** payouts under $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}:\mathfrak{F}\mathfrak{a}\mathfrak{t}$ conversion. The proof sections can be skipped entirely. However you should read definition 2.6.8 on *regime change*, which is an important market mechanism. Finally read section 2.7 as it is short, and it describes how auditing could be stretched into a perpetual process that 1) does not terminate, and 2) precipitously elevate **mint** token prices per market participation.
2. Chapter 3 on **FeaAudit**: read section 3.1 to understand why **FeaAudit** is a necessary process in the context of Metheus ecosystem. Read the diagram captions to understand the **FeaAudit** annotation game. Section 3.3 describes how the **FeaAudit** oracle can be tokenized. You may skip the math equations if you are not used to reading dense notation, however the text blocks are approachable and should be read.
3. Chapter 4 on $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ issuance: if you are reading this monograph to understand the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin, then this chapter may be read in full. In particular, pay close attention to section 4.3 to understand how the conversion bonding curve affects $\mathfrak{f}\mathfrak{i}\mathfrak{a}\mathfrak{t}:\mathfrak{F}\mathfrak{a}\mathfrak{t}:\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ exchange rates. This monograph also offers a valuation framework for $\mathfrak{F}\mathfrak{a}\mathfrak{t}-\alpha$ in section 4.4. This valuation method is in fact the *Metheus system dynamics*, so it reinforces the high-level ecosystem view we presented here, albeit in more precise terms.

If you are technically inclined and wish to examine this monograph closely, then there is an alternative reading order for you. In the inverted order, this monograph specs out an academic discipline or curriculum. This discipline ingests a machine-learning based economy, and outputs a stablecoin pegged to any fiat currency of choice. The order is as follows:

1. Appendix F: derives the exponential family distributions. The convex property of this family's partition function **A** is the basis for many proofs in this monograph. This chapter should be supplemented by books [WJ08] and [WJ04].

2. Appendix E: is an introduction to machine learning on a *narrow* pedagogical track. In this path, we dispense with the normal curriculum and focus on kernel-based methods only. In particular, we review recent work on *neural tangent kernels* in E.4. The knowledge here is not necessary to understand **QAudit** per-se, but it does help you understand the basic motivation for its price, reward, and tokenization process. This chapter is supplemented by [GB16], [JM24], [SC04], [SS16], and [RW06].
3. Appendix D gives a minimal introduction to the theory of sequential prediction markets, especially as it applies to machine learning algorithms. It motivates our tokenization process whereby the discriminative proxy is judged and tokenized in **mint**. This chapter should be supplemented by [CV10], [AK+14], [Kut15], and [ACV12].
4. Appendix B gives some of the basic definitions used in the proofs. They are drawn from [MR95], [Tad13], and [Rou14]. Observe many of the definitions on games are lifted from computer science, not microeconomics.
5. Now read chapter 2, 3, and 4 in context of the proofs in Appendix C, for a deeper understanding of this discipline. Although we did not conceptualize **FeaAudit** along orthodox lines as it is a consumer product, it should be read in the context of the canonical treatments found in [MP+24] and [DR14].

Finally in Appendix A you will find the major notations used in this monograph.

Metheus:

Open Source **AI**

Open Source Capital

Open **AI** Governance

PART I

Core Protocol

Draft: 5th April 2026:Metheus: Ling Xiao

CHAPTER 2

Machine Intelligence Quality Audit

2.1 Introduction

Metheus does not market one single protocol, but rather deploys a *family* of protocols, each introspecting machine learning (ML) systems through a different lens. The foundational protocol in Metheus evaluates the performance of ML algorithms under certain pre-defined performance metrics. This protocol runs a set of machine learning competition markets akin to Kaggle, albeit under significantly better incentive structure. Unlike Kaggle which can be winner take all, every Metheus market participant who submits model or data "in good faith" receives awards commensurate with their contribution. The competition proceeds as follows:

1. Any *principle auditor* or PA may initiate an auditing market by launching an instance of the *machine intelligence quality audit protocol*, or QAudit. The PA launches the auditing process with the intention of procuring models or recruiting top machine learning talent, using the model performance as judge. The QAudit protocol is agnostic to the underlying submission so long as it emits data in the right format. Thus the PA may also launch a QAudit instance sourcing raw data, in which case this particular QAudit session is a *data market*.
2. The PA deposits:
 - a) a fiat-denominated bounty to the protocol;
 - b) defines how many *stages* T the QAudit session lasts;
 - c) the time duration of each stage t .
 - d) how many winners R there are at *every stage* t of the competition.

For example, if an audit session may span 1) $T = 60$ stages, 2) each stage is 24 hours, 3) it admits $R = 3$ top teams out of a total of 100 teams, and 4) the bounty is v_{pa} , then we denote this audit session QAudit(60, 24, 100, 3, v_{pa}). Submissions are judged in a preliminary manner as they are submitted in every stage t . But rewards are given after the competition ends, when all the models are evaluated against a PA-provided test set.

3. The **Agents** or *teams* then submit their model into the competition. Then in every stage t , each **Agent**'s model is judged against all other **Agents**' model submitted within *the same stage*. In the QAudit(60, 24, 100, 3, v_{pa}) instance, QAudit admits daily winners announced *ex post facto* at market close after 60 days. Note that $R = 3$ implies 3 **Agents** win every stage for 60 stages, so that there are $3 \times 60 = 180$ winning **Agents** per QAudit(60, 24, 100, 3, v_{pa}) session, along with the **Auditors** who wagered on the winning **Agents**' models. Moreover, neither the losing **Auditors** nor the losing **Agents** in each stage are eliminated. So an **Agent** that lost one day can always win the next day. **Agents** can win as often as they perform well, so that i.e. it is possible for an **Agent** to win 60 times. See figure 2.10 for a visual representation of how models are compared.

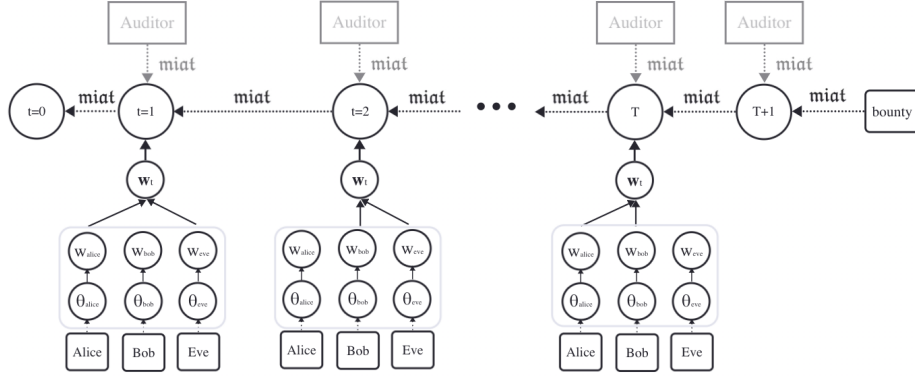


Figure 2.1: QAudit is a staggered sequential parimutuel market where in every stage t , Agents such as Alice, Bob, and Eve submit generative models θ , while Auditors wager on these models with *miat*. The data sampled from each generative model is used to train a corresponding discriminative model w as part of the tokenization process. The best performing discriminative models in every stage is summarized by an ensemble discriminative model $w^\dagger$. The winning Agents in each stage t are paid by those Auditors who wagered on the losing Agents on the same stage t . The winning Auditors in each stage t are paid by losing Auditors in stage $t+1$. In the last stage $T+1$, the winning Auditors are paid by the PA-deposited bounty. Notably, Agents are not eliminated across stages. They can submit as often as they like, and win as many times as they perform. The diagram continues in fig 2.2. See Appendix D for simplified version of sequential parimutuel model markets.

The Agents' winnings do not come from the PA's bounty, but rather from the losing Auditors' wagers. The Auditors' winnings also stem from other Auditors, except for the Auditors who wagered on stage $T = 60$, in which case their winning come from the bounty v_{pa} .

4. Auditors approach the market sequentially in every stage t , and wager on the *previous* stage's submissions (that is $t-1$). They may wager according to QAudit-protocol's *prompted rankings* over Agents' models, or the Auditors may report their own ranking. QAudit provides nonfungible-tokens for every Agent $_k$'s model at every time stage, each nonfungible token is denoted mint_k^t . Each mint_k^t token represents the quality of Agent $_k$'s model in a particular stage. For example in the QAudit(60, 24, 100, 3, v_{pa}) instance, there are 100 Agents participating, each Agent submits one model per day. If the average number of tokens sold per model is 10 mints. Then a total of $10 \times 100 \times 60 = 60,000$ mint tokens will be sold over the course of 60 days. Specifically, if Auditors have purchased 20 mint_4^{12} and 10 mint_1^{12} tokens on stage $t = 12$, then QAudit interprets this buy-signal as stating that Agent $_4$'s submission is twice as likely to be the best model as that of Agent $_1$'s submission on the 12th day.
5. Critically, every QAudit instance runs an automated market maker (Amm) so that each time the Auditor wagers by buying mint tokens, the price of model shares change in accordance to the Auditor's wager. When the market closes after 24 hours, the 3 winners of the day are chosen as the top 3 Agents most likely to win per market consensus. However, this *does not* imply the top winning Agents or the associated Auditors will be paid immediately. It simply means the top R Agents will be judged against the previous stage's top R teams at test time after 60 days. The Amm resets the price on every stage t , so Agent models' closing price on one stage does not affect the opening price in the next stage.

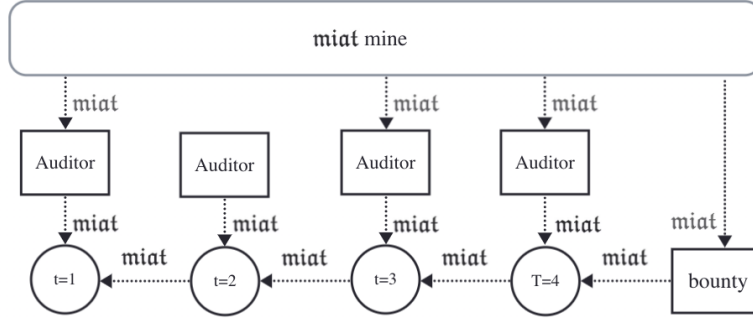


Figure 2.2: In QAudit sequential parimutuel market, Auditors wager with on-chain currency **miat** that is sold at the current **miat:fiat** exchange rate. Upon each purchase event, new **miat** is minted for Auditors. The second Auditor in the diagram has enough **miat** to wager, so no additional token is mined. Bounty **miat** always come from mines. See section 2.2 to understand how QAudit achieves incentive compatibility and budget balance.

6. At competition close after $T + 1$ stages, the test set is revealed and all the submissions are judged against it. In QAudit(60, 24, 100, 3, v_{pa}), the top 3 submissions at any given day is judged against the top 3 submissions in the previous day *collectively*. Specifically, QAudit builds an collective model from the 3 individual submissions and judge this ensemble, so that:
 - a) if the ensemble model of the day beats the previous day's ensemble on the test set, then the Agents who submitted the top models are rewarded *individually* from the losing wagers of that day, in proportion the marginal information value add they provided on the collective model. This is the principle of **judging collectively and rewarding individually**.
 - b) The Auditors who wagered on the top model are paid from the top wagers of the *next day*, in proportion to how much they have wagered. This is the principle of **eat what you pay for**. A betting regime whereby each person is paid off by the next person in line is called a *sequential parimutuel market*.¹
 - c) Auditors who wagered on the last day are paid from the PA-provided bounty v_{pa} .

Most importantly, QAudit does not engage in staking but rather wagering. So Auditors who bet on the losing models are not paid. Agents who submit losing models lose only time and effort. Observe QAudit is parimutuel for Agents, and sequential parimutuel for Auditors. In order to win:

1. Agents need Auditors to wager generously on their models, so that they may be selected by the protocol to compete against the previous day's top model ensemble.
2. Auditors need Agents to submit good models, so that their wager is not lost in vain.
3. Auditors and Agents are paid from different pools: Agents rake today's losing wagers, while Auditors take tomorrow's winning wagers.

In sum, Auditors and Agents **hunt together but eat separately**. This arrangement of incentives is critical as it breaks the conflict of interest that tend to exist between the Auditors and the audited, that is the Agent. Instead of evading each other, the QAudit sequential game play induces a *Nash Equilibrium*, whereby the Agents and Auditors collude to submit the most performant model possible, and wager to move it to the top of the line.

¹See [Han02],[Han03],[CV10],[Agr+09],[CA10],[ACV12],[CP07],[GR07],[FPW23],[AK+14],[Pen04],[HW19], and [For+07].

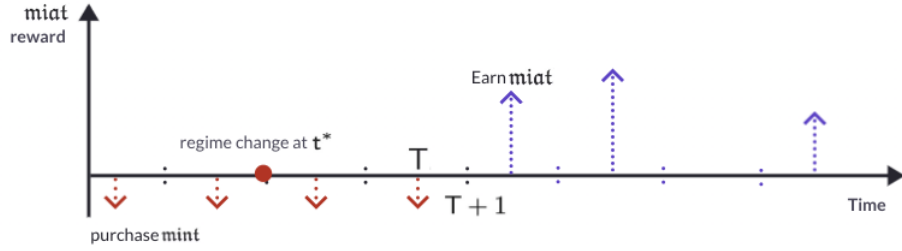


Figure 2.3: QAudit miat-flow diagram. The model/wager elicitation stages lasts from $t = 1 \dots T + 1$, where **Agents** submit models, and **Auditors** buy *machine intelligence nonfungible tokens* (mint) issued from each model. They buy mints with the *machine intelligence audit token*, or miat. The models are scored against a validation data set in a preliminary basis in each stage t . After $T + 1$ stages, QAudit replays the entire market history, and judges the **Agent**-submitted models on the test set. It allocates miat-denominated winnings to the **Agents** and **Auditors** who submitted/wagered on the best models in each stage t . Market participants can win as many times as they submit models or wager, and more than one participant can win each stage. Note the *regime change* at stage t^* , this is an important topic covered in section 2.6.

The introduction above leaves out one last stakeholder: the protocol developer **Metheus**. The tried-and-true way to profit is with trading fees. We do this whilst putting the settlement layer on chain, so that wagering is passed through three sets of tokens with floating exchange rates: 1) the $\mathfrak{F}$ at stable coin, 2) the **miat** audit token, and 3) the **mint** machine intelligence nonfungible token. Their floating exchange rate is **Metheus**'s margin opportunity. This is the essence of the system, but the devil is in the subtleties. The rest of the chapter deconstructs the protocol in layers, and proceeds as follows:

1. section 2.2 below tables the protocol incentives at a high level. In particular it introduces the notion of *staggered sequential parimutuel prediction market*. And qualitatively argue that it aligns the incentives of the **Auditors** and **Agents**, so that they collude to report and wager the best models.
2. Section 2.3 specify the protocol in detail, and introduces all the terminology and data types that will be used to prove the protocol's properties.
3. Section 2.4 highlight's QAudit's machine learning subroutines. A primer on machine learning may be found in Appendix E.
4. Section 2.5 proves these elementary properties of QAudit:
 - a) lemma 2.5.1 shows that the QAudit reward structure is *incentive compatible* from the perspective of the **Agents**, so that they will report their best model subject to resource constraints. Lemma 2.5.2 then shows that reporting the truth is a *pure strategy equilibrium* for every **Agent**.
 - b) Lemma 2.5.3 states that collusion is a *mixed-strategy equilibrium* between **Agent** and **Auditor**. Whereby "collusion" implies the **Agents** report their best model in every stage t , and the **Auditors** wager on the best reported model in this stage.
5. Section 2.6 follows the "spirit" of [CV10], and view QAudit as 1) a market protocol, 2) a game, and 3) a randomized algorithm. In this spirit, we state the *wager-complexities* of QAudit - loosely defined as the number of wagers needed for the market to find the

correct ranking $\mathfrak{s}_r^*$ over **Agents**' submitted models. In particular, we have for every stage $t = 1 \dots T + 1$:

- a) lemma 2.5.8 states that in the event where every **Auditor** _{r} accepts the prompted rankings $\mathfrak{s}_r^*$ provided by **QAudit** instance, then the **Auditors** will collectively find the correct permutation in $\text{poly}(K!)$ number of wagers. Where K is the number of **Agents** participating in some instance of **QAudit** market.
- b) Lemma 2.5.10 states the "stability" of equilibrium state when the **Auditors** behave selfishly according to unobserved interest. We find these sensitivity bounds:
 - i. If some $1 - \alpha_r$ ($0 < \alpha_r < 1$) portion of **Auditors** wager according to unprompted rankings, then the market settles on some distribution $\mathbf{p}$ of over the rankings. It deviate from the delta distribution over the prompted ranking $\tilde{\delta}_r$ by:

$$\|\mathbf{p} - \tilde{\delta}_r\|_1 \leq \frac{2}{1 - \alpha_r} |\Delta \alpha_r|.$$

- ii. The deviation in the **QAudit** equilibrium as a function of the **Auditors**' private preference over rankings ($\mathbf{e} + \Delta \mathbf{e}$) is:

$$\|\tilde{\delta}_r - \mathbf{p}\|_1 \leq (1 - \alpha_r) \|\Delta \mathbf{e}\|_1.$$

- iii. The sensitivity of distribution on rankings as a function of the **Auditor** _{r} 's' wager strategy $\mathbf{M}_r$ is:

$$\|\tilde{\delta}_r - \mathbf{p}\|_1 \leq \frac{\alpha_r}{1 - \alpha_r} \|\Delta \mathbf{M}_r\|_\infty.$$

- c) Lemma 2.5.11 states that in the event where the **Auditors** are partitioned into two blocks, so that some of the **Auditors** accept the prompted permutation, denoted $\mathfrak{s}_r^*$, and some of which hold a different opinion. If the proportion of **Auditors** who hold private belief $\mathfrak{s}_r^*$ is greater than 50%, then under some very mild conditions on the rate of wagering, the likelihood of finding one of the two accepted solution states may be significantly greater than $\frac{1}{2}$.
6. Section 2.6 then builds on the previous lemmas, and states *three fundamental theorems of QAudit*. The first theorem captures market behavior when **Auditors** possess "singular conviction." The second theorem explains the rationality of "herd behavior" in **QAudit** markets. Notably, both conviction and herd behavior lead to correct rankings in expectation. The third theorem states that despite the intrinsic oscillation of the price bonding curve, **QAudit** is profitable for winning **Auditors**.
- a) Theorem 2.6.4, or the *QAudit Protocol Consensus Theorem*, stating that for every stage t , there exists a *Correlated Equilibrium*, s.t. the market consensus at market close reflects the true permutation $\mathfrak{s}_r^t$ in expectation, provided the permutation is prompted by **QAudit** _{t} at market open in every t .
 - b) Theorem 2.6.6, or the *QAudit Social Consensus Theorem* shows that even if **Auditors** differ in their belief over the optimal ranking over **Agents**, rational herd behavior will push the market towards the correct ranking. Specifically, we state that in the event where the protocol suggests many possible rankings over models, then in every protocol stage t , there exists a point in time τ^* bounded by:

$$\tau_f - N - K \log(K) \leq \tau^* < \tau_f - 1 - K \log(K),$$

s.t. it is rational for every **Auditor** to switch to wagering on one unique ranking.

- c) Theorem 2.6.10: QAudit is not a Loser's Game. That is to say the QAudit sequential parimutuel reward arrangement has positive expected value for winning Auditors. In particular, we find that Auditors profit if the per-stage- t earnings Δv is above this threshold:

$$\Delta v \geq \frac{8v_{t^*}}{\sum_{t=2}^{t^*} (1+r)^{t^*-t}}.$$

Where:

- i. r is *regime change risk* as seen in definition 2.6.9.
 - ii. v_{t^*} is the Auditor $_i$'s wager at the moment of regime change t^* .
 - iii. the interval $[t, t^*]$ defines the stages between consecutive regime changes.
7. Section 2.7 introduces *perpetual QAudit*, where the basic QAudit market is extended so that the game has infinite time horizons. Then it shows how this game can recreate an asset curve over *mint* tokens that is comparable to Bitcoin. Featuring periodic price oscillations similar to Bitcoin halving events, and an over-all upward drift in price ceiling over time.

2.2 QAudit Incentive Structure

This section offers a high-level explanation of the incentive structure of the QAudit automated market maker (Amm). It begins by reviewing a basic sequential prediction market, and use its deficiency to motivate the QAudit price-bonding curve. At a very basic level, QAudit is an information market. They are similar to stock markets, but emphasize 1) incentive compatible pricing rules, or proper scoring rules; and 2) budget balance. A *proper scoring rule*:

1. elicits the truthful report of private information from agents,
2. collate all the reports a probability distribution that reflects market consensus,
3. and compensates the agents in accordance to the quality of their report.

Reports in information markets are always judged against some objective ground truth at the end of the competition. For example, if some information market is designed to elicit the likelihood of raining next Tuesday, then the players are judged by whether it does rain on Tuesday. Thus unlike stock markets where price can be conflated for value and traders are rewarded for pumping the stock, in information markets the users are rewarded for telling the truth. Moreover the QAudit Amm is *budget balanced*:

1. the current stage's winners are paid by the next stages' wagers.
2. The last tranche of players to participate in the QAudit market is paid by the bounty paid by the PA.

Thus neither the PAs nor Metheus will go bankrupt operating QAudit protocols. See Appendix D for more on sequential parimutuel information markets. Presently, using prediction market to run collaborative machine learning projects has been proposed by the Ethereum Foundation in [HW19], and fielded at Amazon with [GH+24]. QAudit is similar to these proposed sequential parimutuel markets in appearance, but differs in some important ways:

1. typical prediction markets do not differentiate between principles who make the bets, and Agent who report the information without paying, in fact they may be paid for reporting good models. QAudit separates Agent who submit the model checkpoints, and Auditor who wager on the models.² This segregates the two classes of users who have different relationship with technology. Most importantly, Agent-Auditor dichotomy

²A checkpoint is the intermediate state of a model during training. They are useful for diagnostics.

ensures the machine learning scientists do not have to "pay-to-play," which is unnatural from their perspective to put it charitably. However, separation of reporting and profiting introduces principle-agent conflict of interests, as **Agents** are no longer directly rewarded or punished for their submissions. In the main body, we argue QAudit induces an equilibrium so that they *do* collude to report and wager on the best models.

Agent reports model θ or $\mathbb{D}$, Auditor wagers on reports

Player	Report	Amm update
Agent _k	θ_k^t	$\theta_k^t \rightarrow \mathbf{w}_k^t \rightarrow \log p_k^t$
Agent _k	$\mathbb{D}_k^t$	$\mathbb{D}_k^t \rightarrow \mathbf{w}_k^t \rightarrow \log p_k^t$
Auditor _i	$\mathbf{b}_i^t = (b_{1,i}^t, \dots, b_{K,i}^t)$	$\mathbf{s}^t = \mathbf{s}^t + \mathbf{b}_i^t$

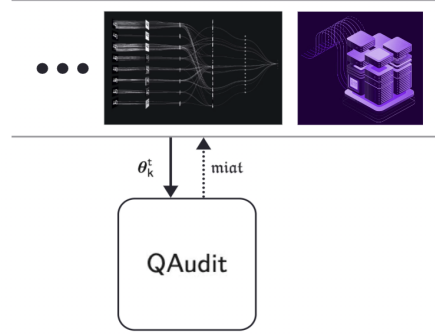
Table 2.1: In QAudit, **Agents** report either their generative model θ or raw data set $\mathbb{D}$. **Auditors** report a buy vector $\mathbf{b}$, where each scalar $b_{k,i}^t$ denotes how many shares of model k Auditor_i will buy. The QAudit_{Amm} then trains a logistic regression classifier $p(y|\mathbf{x}; \mathbf{w}_k^t) = \sigma(\langle \mathbf{w}_k^t, \mathbf{x} \rangle)$ by sampling from either the reported data set $\mathbb{D}$, or the generative model $p(\mathbf{x}_\tau | \mathbf{x}_{\tau-1}; \theta)$. This model outputs a likelihood p_k^t that scores the **Agent**'s reported model θ . Similarly, the QAudit protocol updates the market inventory with the **Auditor**'s orders when he reports a buy vector. Here $\mathbf{s}^t + \mathbf{b}_i^t$ is element-wise vector addition.

2. Classical formulation of prediction markets for machine learning models assume the agents are all working on the same underlying model. They update the weights in private before submitting the candidate updates to a shared global model state [[HW19], [GH+24]]. This assumption is very limiting and defeats the purpose of holding a competition, which is to explore different solutions. In normal machine learning competition, teams work on separate repositories, and often use different model architectures. QAudit extends the sequential prediction market design pattern to accommodate a more natural use case whereby the teams share the same objective, but select their own model architectures. In fact, QAudit can be used even if teams do not submit models at all.
3. Proper market scoring rules must be convex functions, so the naive formulation of machine learning sequential prediction markets requires the model to be drawn from the exponential family [[GR07], [HW19], [AK+14]]. For example, logistic regression satisfies this constraint (see Appendix E.3.3, and Appendix F for more). However this is a highly contrived setting, there is no need to crowd source regression. Instead QAudit aims for production level assets. In particular, the protocol elicits two types of products from **Agents**:
 - a) (*generative models*): of form $p(\mathbf{x}_\tau | \mathbf{x}_{\tau-1}; \theta)$, where $\mathbf{x}_{\tau-1}$ is the query and $\mathbf{x}_\tau$ is the response. These deep artificial neural networks (ANNs) do not have convex partition functions. Moreover, most commercially interesting ANNs such as large language models (LLMs) and image generation models do not output a probability at all, instead they output real values. Thus QAudit trains a surrogate classifier $p(y|\mathbf{x}; \mathbf{w})$ on responses sampled from generative models θ , and some noise control η or natural data set $\mathbb{D}$. The classifier learns to discriminate noise from generated sample, or generated sample vs real data set. This fraud-detection game is somewhat similar to generative adversarial nets of [GP+14]. However in our case the objective functions are not tied, and the QAudit protocol is interested in the discriminative model, not the generator.
 - b) (*raw data*): similar to the previous point, raw data is not equipped with a convex log partition function. But it can be used to train a convex surrogate for scoring. A QAudit market instance that is eliciting data from agents is in fact a *data market*,

the PA and Auditors are buying information from users. This is an important aspect of the Metheus economy. Data markets significantly expand the surface area of who can participate in the games. Most people cannot train machine learning models, but most people do have access to some data that can be sold.

In both cases, if the Agent-submission is of high quality, then it would refine the decision boundary between the source and the control learned by the classifier $\mathbf{w}$. If the submission is low quality, then it degrades this boundary. This is reflected in changing classification error w.r.t. the test set provided by the principle auditor PA. QAudit measures the improvement or degradation in quality of the classifier, and uses it to reward the Agents and Auditors.

Figure 2.4: QAudit accepts two forms of submissions from Agents: 1) machine learning models θ (such as LLM), which accepts a query and outputs response with $p(\phi_{\mathbf{x}_\tau} | \phi_{\mathbf{x}_{\tau-1}}; \theta)$; or 2) raw data sets $\mathbb{D}$ that can be used to train models downstream. In the first case, QAudit in a model market, in the second case, it is a data market.



4. However scoring a convex surrogate introduces a problem: QAudit must train the surrogate classifier by sampling from the reported models first, and then publicize the price. Hence the QAudit market refactors auditing into two steps: *model elicitation step* and *wager elicitation step*. After all the generative models or datasets have been reported, QAudit runs discriminative model on the reported assets and then publish the relative performance of these discriminative models proxies.
5. Additionally, splitting the market into two steps bottle necks the trading or auditing activity, so that the Auditors have a full stage (i.e. 24 hours) to trade in and out of all K models. If we allow the Agents to submit whenever they "feel like it" as is the case with [HW19], then some agents may game the system by submitting early and often. This skews the incentive and turns the auditing process into a spam-like popularity contest. Note the two steps are interleaved, so that i.e. if one time stage is 24 hours, then the Auditors are always trading yesterday's models.
6. In the classical formulation, the price charged is computed in expectation over data samples, whilst the reward is computed on a particular data set. Should there be any biases in the sample, then there is a mismatch between wagers collected and rewards given. The market maker must then make up the difference - a prohibitive ask.

In sum, QAudit differs from standard prediction markets in that we 1) consider a wider set of stakeholders, 2) think deeply about the consequence of protocol design on user experience, and 3) the protocol accept both models with non-convex partition functions, and datasets with no partition functions at all. Finally, note that the auditors in QAudit are not staking but rather *wagering*. Staking implies the principle is returned, while wagering make no such guarantees. Lost wagers will be used to subsidize cost of training for Agents. This is an important point: **trading pays for training**.

QAudit Staggered Sequential Parimutuel AMM

QAudit builds upon previous work in sequential parimutuel prediction markets. However we must solve two problems:

1. the Auditors are not staking but wagering. And since Auditors are paid from the next stages' wagers, an amount of which he cannot control, it is possible for a Auditor to lose money even if he selects the correct model, but overpays for the current one.
2. Since Agents are not staking on their own submission but rely on Auditors to pick them, QAudit must 1) incentivize the Agents to report the best model they can, and 2) prevent the Auditors from selecting a lesser model over a better one.

The next three points builds towards the full QAudit incentive structure that remedy the principle-agent problems.

1. (QAudit *simple*) In this simplified version, Agents submit their models θ at every stage t , while Auditors wager on the submitted models from each stage $t - 1$. Then during the reward phase, QAudit allocate the rewards depending on how well the top Agents selected by the Auditors have performed. The problem is that Auditors may wager as much as they want per stage, with no *price floor* on the *mint* model shares. So that Auditors may lose money even if they win the stage. For example referencing table 2.2, at $t = 1$ Auditor₁ wins the stage by wagering $v_{\text{alice},1}^1$. But he does not control how much the winning Auditors at stage $t = 2$ wagers. So that if $v_{\text{alice},1}^1 \geq v_{\text{alice},2}^2 + v_{\text{alice},3}^2$, then Auditor₁ may still lose money.

QAudit: Agent reports θ , Auditor wagers with no price floor

Stage t	Auditor wagers $v_{k,i}^t$	Agent reports θ	reward v_{Agent}^t	reward v_{Auditor}^t
0	0	$\theta_{\text{Amm}}^0 \rightarrow \log 0.5$	0	0
1	v_{alice}^1 $v_{\text{bob},2}^1 + v_{\text{bob},3}^1$	$\theta_{\text{alice}}^1 \rightarrow \log 0.6$ $\theta_{\text{bob}}^1 \rightarrow \log 0.3$	$v_{\text{bob},2}^1 + v_{\text{bob},3}^1$ 0	$v_{\text{alice},2}^2 + v_{\text{alice},3}^2$ 0
2	$v_{\text{alice},2}^2 + v_{\text{alice},3}^2$ $v_{\text{eve},3}^2$	$\theta_{\text{alice}}^2 \rightarrow \log 0.7$ $\theta_{\text{eve}}^2 \rightarrow \log 0.5$	$v_{\text{eve},3}^2$ 0	$v_{\text{bob},1}^3 + v_{\text{bob},2}^3$ 0
$T = 3$	$v_{\text{bob},1}^3 + v_{\text{bob},2}^3$ $v_{\text{eve},3}^3$	$\theta_{\text{bob}}^3 \rightarrow \log 0.8$ $\theta_{\text{eve}}^3 \rightarrow \log 0.9$	$v_{\text{eve},3}^3$ 0	bounty v_{pa} 0

Table 2.2: A simple audit example where $T = 3$. The Agents report model θ used to train classifier w (not shown here). QAudit then judges the quality of model on test data. In this case a higher probability value in $\log p$ is a better estimate. Note $T = 3$, so the last Auditor who wagers correctly is rewarded with the bounty value. This is called budget balance since the QAudit Amm is not paying for any of the reports.

2. (QAudit *with Dutch auction*) The solution is to vary the price floor in every stage with the function v_{ρ}^t . So that we can maintain some invariant of form:

$$v_{\rho}^1 \cdot v_{\text{alice},1}^1 \leq v_{\rho}^2 (v_{\text{alice},2}^2 + v_{\text{alice},3}^2),$$

if the shares "sell well" over the stages. And the inverse inequality:

$$v_{\rho}^1 \cdot v_{\text{alice},1}^1 > v_{\rho}^2 (v_{\text{alice},2}^2 + v_{\text{alice},3}^2),$$

in the event where the price is too high so the shares fail to sell. This reversion in price is known as *regime change* (see section 2.6). The Auditor is effectively evaluating an asset θ_k^t , whose performance or fundamental value is increasing every stage with upward drift in price. He must decide when and how often to buy *mint* _{k} ^{t} shares of θ_k^t . Naively the Auditor should buy in one of the earlier stages when bonding factor v_{ρ}^t is a lower value.

QAudit: Agent reports θ , Auditor wagers with variable price floor

Stage t	Auditor wagers $\mathbf{v}_{k,i}^t$	Agent reports θ	reward $\mathbf{v}_{\text{Agent}}^t$	reward $\mathbf{v}_{\text{Auditor}}^t$
0	0	$\theta_{\text{Amm}}^0 \rightarrow \log 0.5$	0	0
1	$\mathbf{v}_{\rho}^1 \cdot \mathbf{v}_{\text{alice},1}^1$ $\mathbf{v}_{\rho}^1(\mathbf{v}_{\text{bob},2}^1, \mathbf{v}_{\text{bob},3}^1)$	$\theta_{\text{alice}}^1 \rightarrow \log 0.6$ $\theta_{\text{bob}}^1 \rightarrow \log 0.3$	$\mathbf{v}_{\rho}^1(\mathbf{v}_{\text{bob},2}^1 + \mathbf{v}_{\text{bob},3}^1)$ 0	$\mathbf{v}_{\rho}^2(\mathbf{v}_{\text{alice},2}^2 + \mathbf{v}_{\text{alice},3}^2)$ 0
2	$\mathbf{v}_{\rho}^2(\mathbf{v}_{\text{alice},2}^2, \mathbf{v}_{\text{alice},3}^2)$ $\mathbf{v}_{\rho}^2 \cdot \mathbf{v}_{\text{eve},3}^2$	$\theta_{\text{alice}}^2 \rightarrow \log 0.7$ $\theta_{\text{eve}}^2 \rightarrow \log 0.5$	$\mathbf{v}_{\rho}^2 \cdot \mathbf{v}_{\text{eve},3}^2$ 0	$\mathbf{v}_{\rho}^3(\mathbf{v}_{\text{bob},1}^3 + \mathbf{v}_{\text{bob},2}^3)$ 0

Table 2.3: In QAudit with ascending price factor, the Amm scales the price computed by the logarithmic market scoring rule by factor $\mathbf{v}_{\rho}^t$. However the QAudit Amm does not increase the price monotonically across stages $t = 1..T$, this would eventually price all buyers out of the market. Instead, QAudit features periodic *regime changes*, whereby the price is reset to some initial point. See 2.6.8 for more. Last row omitted in table above.

QAudit: Agent reports θ , Auditor wagers with Dutch-auction-set price

Stage t	Auditor wagers $\mathbf{v}_{k,i}^t$	Agent reports θ	reward $\mathbf{v}_{\text{Agent}}^t$	reward $\mathbf{v}_{\text{Auditor}}^t$
1	$\mathbf{v}_{\rho}^1$ $(\mathbf{v}_{\rho}^1, \mathbf{v}_{\rho}^1)$	$\theta_{\text{alice}}^1 \rightarrow \log 0.6$ $\theta_{\text{bob}}^1 \rightarrow \log 0.3$	$2\mathbf{v}_{\rho}^1$ 0	$2\mathbf{v}_{\rho}^2$ 0

Table 2.4: Simplified $t = 1$ betting history. The Auditors' wagers are set to $\mathbf{v}_{k,i}^t = 1$, so they bid the price fixed by bonding function $\mathbf{v}_{\rho}^t$. The other stages are omitted for clarity.

3. (QAudit *full*) There is still a problem with table 2.2 in time stage $t = 3$: note that even though θ_{eve}^3 outperformed θ_{bob}^3 , Auditor₁ and Auditor₂ have wagered on Agent_{bob} instead, thus Agent_{eve} lost unfairly. The solution is to penalize the Auditors with regret factor ρ_k^t , and allocate this regret to eve. *Regret* expresses how well the ensemble model could have done had the Auditors selected the best models. It prevents the Auditors from defecting from the best model on each stage, and prompts the Agents to submit their best model knowing they will be rewarded by either the Auditors, or the QAudit regret compensation. The exact value of this regret depends on how well θ_{eve}^3 does, see table below and subsection 2.3 for more information.

QAudit: Agent reports θ , Auditor wagers, variable price floor, regret penalty

Stage t	Auditor wagers $\mathbf{v}_{k,i}^t$	Agent reports θ	reward $\mathbf{v}_{\text{Agent}}^t$	reward $\mathbf{v}_{\text{Auditor}}^t$
0	0	$\theta_{\text{Amm}}^0 \rightarrow \log 0.5$	0	0
1	$\mathbf{v}_{\rho}^1 \cdot \mathbf{v}_{\text{alice},1}^1$ $\mathbf{v}_{\rho}^1(\mathbf{v}_{\text{bob},2}^1, \mathbf{v}_{\text{bob},3}^1)$	$\theta_{\text{alice}}^1 \rightarrow \log 0.6$ $\theta_{\text{bob}}^1 \rightarrow \log 0.3$	$\mathbf{v}_{\rho}^1(\mathbf{v}_{\text{bob},2}^1 + \mathbf{v}_{\text{bob},3}^1)$ 0	$\mathbf{v}_{\rho}^2(\mathbf{v}_{\text{alice},2}^2 + \mathbf{v}_{\text{alice},3}^2)$ 0
2	$\mathbf{v}_{\rho}^2(\mathbf{v}_{\text{alice},2}^2, \mathbf{v}_{\text{alice},3}^2)$ $\mathbf{v}_{\rho}^2 \cdot \mathbf{v}_{\text{eve},3}^2$	$\theta_{\text{alice}}^2 \rightarrow \log 0.7$ $\theta_{\text{eve}}^2 \rightarrow \log 0.5$	$\mathbf{v}_{\rho}^2 \cdot \mathbf{v}_{\text{eve},3}^2$ 0	$\mathbf{v}_{\rho}^3(\mathbf{v}_{\text{bob},1}^3 + \mathbf{v}_{\text{bob},2}^3)$ 0
$T = 3$	$\mathbf{v}_{\rho}^3(\mathbf{v}_{\text{bob},1}^3, \mathbf{v}_{\text{bob},2}^3)$ $\mathbf{v}_{\rho}^3 \cdot \mathbf{v}_{\text{eve},3}^3$	$\theta_{\text{bob}}^3 \rightarrow \log p_{\text{bob}}^3$ $\theta_{\text{eve}}^3 \rightarrow \log p_{\text{eve}}^3$	$\mathbf{v}_{\rho}^3 \cdot \mathbf{v}_{\text{eve},3}^3$ $\mathcal{L}_{\text{eve}}^3$	$\mathbf{v}_{\text{pa}} - \mathcal{L}_{\text{eve}}^3$ 0

Table 2.5: Full incentive structure of QAudit, featuring parimutuel payout per stage for Agents, and sequential parimutuel payouts for Auditors, featuring Dutch ascending auction factor $\mathbf{v}_{\rho}$ and regret ϱ penalty.

2.3 QAudit Core

This section presents QAudit, highlighting how Agents, Auditors and $\mathfrak{F}\text{at}$ holders can profit. The details of the protocol are factored into multiple subroutines. In the future, certain subroutines may compose with open source variations. QAudit is subtle, it introduces:

1. Auditor-Agent dichotomy.
2. It separates model elicitation step from wager elicitation step.
3. And converts amongst three tokens ($\mathfrak{F}\text{at}$, miat , mint) with variable exchange rates.

QAudit differs dramatically from the "straight-forward," but naive solutions of vanilla sequential-parimutuel machine learning markets of Appendix D.2. Aside from fiat, the following set of commodities are "in play" for every QAudit instance:

1. ($\mathfrak{F}\text{at}$): the fully-backed stable coin pegged to 1 fiat of choice. When Auditors buy into the market, they must purchase miat token using fiat. This purchase is then exchanged to miat at the current floating miat to $\mathfrak{F}\text{at}$ exchange ratio. The miat may come from a pre-mined miat pool, or if the pool is empty then miat will be minted for conversion. This conversion occurs in the backstage and do not require user decisions.
2. (miat): the machine intelligence audit token. Auditors use miat to purchase model shares over stages $t = 1 \dots T + 1$, and are rewarded in miat if their models win. This miat is then convertible back to $\mathfrak{F}\text{at}$ at a floating exchange rate, and at user discretion from $\mathfrak{F}\text{at}$ to fiat at 1:1 basis.
3. (mint_k^t): the machine intelligence nonfungible token for model k indexed by stage t , or simply "model shares." Model shares express the upside the Auditor is entitled to in the event where model k in stage t is performant relative to previous stage $t - 1$. Thus mint_k^t is a claim on the next stage's wager. Once purchased, the Auditor may resell mint at their own discretion. Note however, since QAudit allocates the losing Auditor's wagers to the winning Agents at every stage, mint_k^t only has value *if* model θ_k^t wins its stage.³ The mint_k^t is effectively an out of the money call option purchased at time t for model k . It matures at the end of the audit stage $T + 1$, and represents a contingent claim on the wagers of stage $t + 1$. The fact that this option is liquid induces a secondary market on mint 's that further enriches the QAudit ecosystem.
4. (*data source* θ or $\mathbb{D}$): θ denotes some generative model, while $\mathbb{D}$ is a data set drawn from nature. This $\mathbb{D}$ may emit data such as real images, videos, and text.
5. (*surrogate classifier* $\mathbf{w}$): each QAudit market keeps a set of latent states, whereby at any stage t , the protocol trains a surrogate proxy of every model θ_k^t or data set $\mathbb{D}_k^t$ to construct: $\{\{\mathbf{w}_1^1, \dots, \mathbf{w}_K^1\}, \dots, \{\mathbf{w}_1^T, \dots, \mathbf{w}_K^T\}\}$ over the course of the full audit session.

Require Statements

Before competition commences, the principle auditor (PA) defines:

1. A fiat-denominated bounty v_{pa} is charged to the principle auditor using the function $\text{QAudit}_{v_{\text{pa}}}$. This bounty is allocated to the final set of Agents who submit high quality models as mining reward.
2. *Competition duration* T over stages $t \in \{1, \dots, T + 1\}$. For example, T may be 60 days where each stage t is 24 hours. On day or stage 0, QAudit initiates the submissions with a randomly initialized model θ^o and surrogate classifier $\mathbf{w}^o$. See Appendix E for more on deep learning and classical learning methods. Then on day 1, the market opens and accept model submissions in private for stage $t = 1$.

³A nonfungible token is the bar-code for some digital merchandize.

Protocol 1 Machine Intelligence Quality Audit Protocol (QAudit)**Require:** Bounty v_{pa} , training stage T , qualifiers per stage R **Require:** hidden data set $\mathbb{D}$, public query set $\mathbb{Q}$, kernel embedding function ϕ ,**Require:** noise generator η , dutch auction function v_ρ^t , permutation distance $a \parallel b$ **Require:** agent accounts $\{\text{Agent}_k\}_{k=1..K}$, and auditor accounts $\{\text{Auditor}_i\}_{i=1..N}$

```

1: commitment phase:
2:  $(\mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}}) \leftarrow \text{QAudit}_{\mathbb{D}}(\mathbb{D}, \eta, \phi)$ 
3: QAudit reveal validation set  $\mathbb{D}_{\text{val}}$ 

4: audit phase:  $\triangleright$  accept submissions/wagers on models
5:  $(\theta^o, \mathbf{w}^o) \leftarrow (\epsilon \mathbf{I}, \epsilon \mathbf{I})$ 
6:  $(\Theta, \mathbf{W}, \text{logprob}) \leftarrow ([\{\theta_k^o\}_K], [\{\mathbf{w}_k^o\}_K], [])$ 
7:  $(\mathbf{B}, \mathbf{P}) \leftarrow (\{\}, \{\})$ 
8: for  $t \in 1 \dots T + 1$  do
9:   if  $t \leq T$  then  $\triangleright$  model elicitation phase
10:      $(\Theta^t, \mathbf{W}^t, \text{logprob}^t) \leftarrow \text{QAudit}_{\Theta}(\mathbb{D}_{\text{val}}, \mathbb{Q}, \eta, \{\mathbf{w}_k^{t-1}\}_K, \{\text{Agent}_k\}_K)$ 
11:      $(\Theta, \mathbf{W}, \text{logprob}) \leftarrow ([\dots, \Theta^t], [\dots, \mathbf{W}^t], [\dots, \text{logprob}^t])$ 
12:   end if
13:   if  $t \geq 2$  then  $\triangleright$  wager elicitation phase with Amm
14:      $(\mathbf{B}^t, \rho^t) \leftarrow \text{QAudit}_v(v_\rho, v_\rho^o, \mathbb{D}_{\text{val}}, \mathbf{W}^{t-1}, \mathbf{W}^t, \text{logprob}, \{\text{Auditor}_i\}_N)$ 
15:      $(\mathbf{B}, \mathbf{P}) \leftarrow ([\dots, \mathbf{B}^t], [\dots, \rho^t])$ 
16:   end if
17: end for

18: reward phase:  $\triangleright$  replay game history and allocate reward
19:  $\mathbf{V} \leftarrow \{\}$ 
20:  $(\Delta, \Delta_{\text{Agent}}, \Delta_{\text{Auditor}}) \leftarrow ([], \{\}, \{\})$ 
21: for  $t \in 1 \dots T$  do
22:    $(\varrho^t, \dots) \leftarrow \text{QAudit}_{\varrho}(a \parallel b, R, \mathbf{W}^t, \rho^t, \mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}})$ 
23:    $(\text{logprob}_{\mathbb{D}_{\text{test}}}^t, \dots) \leftarrow \text{QAudit}_{\text{infer}}(R, \mathbb{D}_{\text{test}}, \rho^t, \mathbf{W}^t)$ 
24:    $(\text{logprob}_{\mathbb{D}_{\text{test}}}^{t-1}, \dots) \leftarrow \text{QAudit}_{\text{infer}}(R, \mathbb{D}_{\text{test}}, \rho^{t-1}, \mathbf{W}^{t-1})$ 
25:   if  $\exp(\text{logprob}_{\mathbb{D}_{\text{test}}}^t) > \exp(\text{logprob}_{\mathbb{D}_{\text{test}}}^{t-1})$  then  $\triangleright$  distribute reward
26:     if  $t < T$  then
27:        $\mathbf{V}^t \leftarrow \text{QAudit}_v(R, 0, \rho^t, \rho^{t+1}, \mathbf{B}^t, \mathbf{B}^{t+1})$ 
28:        $\mathbf{V} \leftarrow [\dots, \mathbf{V}^t]$ 
29:     else
30:        $\mathbf{V}^T \leftarrow \text{QAudit}_v(R, v_{pa}, \rho^t, (), \mathbf{B}^t, \{\})$ 
31:        $\mathbf{V} \leftarrow [\dots, \mathbf{V}^T]$ 
32:     end if
33:     if  $\varrho^t > 0$  then
34:        $(\varrho^t, \Delta_{\text{Agent}}^t, \Delta_{\text{Auditor}}^t) \leftarrow \text{QAudit}_{\varrho v}(a \parallel b, R, \mathbf{W}^t, \rho^t, \mathbf{B}^t, \mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}})$ 
35:        $(\Delta, \Delta_{\text{Agent}}, \Delta_{\text{Auditor}}) \leftarrow ([\dots, \varrho^t], \{\dots, \Delta_{\text{Agent}}^t\}, \{\dots, \Delta_{\text{Auditor}}^t\})$ 
36:     end if
37:   else  $\triangleright$  allocate regret penalty
38:     if  $\varrho^t > 0$  then
39:        $(\varrho^t, \Delta_{\text{Agent}}^t, \Delta_{\text{Auditor}}^t) \leftarrow \text{QAudit}_{\varrho v}(a \parallel b, R, \mathbf{W}^t, \rho^t, \mathbf{B}^t, \mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}})$ 
40:        $(\Delta, \Delta_{\text{Agent}}, \Delta_{\text{Auditor}}) \leftarrow ([\dots, \varrho^t], \{\dots, \Delta_{\text{Agent}}^t\}, \{\dots, \Delta_{\text{Auditor}}^t\})$ 
41:     end if
42:   end if
43: end for
44: return  $(\Theta, \mathbf{W}, \mathbf{V}, \Delta, \Delta_{\text{Agent}}, \Delta_{\text{Auditor}})$ 

```

Between stages 1 and 2, QAudit samples from the generative models θ and updates the discriminative model w . During stage 2, the market reveals stage 1's submissions and accept wagers for top model, *while* accepting submissions in private for stage 2. This continues until stage/day 61, the market opens one last time to elicit wagers on models submitted during stage 60.

3. *Winners per stage* $R \leq \frac{K}{2}$. This constant determines the top R teams from each time stage that qualify for wagering rewards. Models submitted by the qualifying R Agents will be used to determine test error during the QAudit reward phase. QAudit competition is effectively memory-less:
 - a) R is constant throughout the stages, so the Agents and their models are *not* eliminated across stages. It determines the top R experts (Agent-submitted models) from each stage, whose performance will be *collectively* evaluated on the test $\mathbb{D}_{\text{test}}$ during the reward phase.
 - b) Agents who were selected in the previous stage in the winning pool by Auditors are no more likely to be selected in the next stage. That is assuming the Auditors also wager in a manner that is conditionally independent across stages.
4. *Data set* $\mathbb{D}$ used to judge all the submitted models, and optional query set $\mathbb{Q}$. The two canonical QAudit use cases are:
 - a) (*determine the quality of generative model*) of form $p(\phi_{x_\tau} | \phi_{x_{\tau-1}}; \theta)$. That is to say the model accepts query $x_{\tau-1} \in \mathbb{Q}$, and outputs a response x_τ . If the model is an "inverted-auto-captioning" system, then this query $x_{\tau-1}$ may be: "generate an image of a horse riding a motorcycle on the beach," and the model will output real valued image x_τ of just such an image ⁴. The data set is then of form: $\mathbb{D} = \{(x_{\tau-1}, x'_\tau)\}_{j \geq 1}$, where $x_{\tau-1}$ is human generated query prompt, and x'_τ a human response. The model θ is judged against this human response.
 - b) (*buy data sets* $\mathbb{D}$). In this case the query set $\mathbb{Q}$ is empty, and the PA provides an example data set $\mathbb{D}$. This $\mathbb{D}$ is too small to be useful, so Agents are expected to expand its size and scope.
5. *Noise generator* η used to generate negative examples for consumption by QAudit_{train} and QAudit_{infer}. They train a discriminative surrogate to determine the fidelity or quality of data sampled from Agent-submitted data or generative models. This setup superficially resembles generative adversarial networks (GANs) of [GP+14], but is in fact closer to online kernel regression as seen in [ZJ+12]. In this case data sampled from generative model θ_k^t are used to train a discriminative model $p(y | \phi_x; w_k^t)$. This discriminative surrogate w_k^t is then used as a proxy to judge quality of generative model θ_k^t or data submitted by Agent_k at time t . Note this noise generator η may simply generate random signal (i.e. "snow" in the case of images), or it may be an *earlier* version of some generative model, whose output are of "lower quality."
6. *Kernel embedding function* ϕ . Raw data x cannot train a surrogate discriminative classifier in a meaningful manner. Instead this x must be kernelized into ϕ_x . A data set $\mathbb{D}$ so kernelized will span a common space, and all surrogate model comparisons will occur in this space constructed by the data set ⁵ Note there is a subtlety here in that the Agent-submitted models θ_k are neural networks (ANN) that typically construct ϕ end-to-end in some opaque manner. In fact, many intermediate layers of said θ_k can be used as kernel mappings, see Appendix E.4 [[JG+18], [Yan20], [YL20], [AW+21], [HB+20], [AD+19], [Dom20]]. However we do not use the intermediate layers of the Agent-submitted models as feature mappings for a few reason:

⁴Note $x_{\tau-1}$ is a real valued sentence vector, while x_τ is a real valued $m \times n$ pixel image.

⁵See Appendix E for definition of Hilbert space, and Representer theorem.

- a) doing so would entail a literal "change of frame-of-reference" when judging model quality. Instead the machine learning community has many such embedding models ϕ for every conceivable class of data. For example, if the models θ_k are language generative models, then "Word2Vec" may be one possible embedding model [MC+13].⁶ In practice, Metheus will have pre-set QAudit classes with community-approved embedding functions, so the principle auditor do not have to consider this problem.
- b) The ANN may not be publicly viewable to QAudit protocol. Alternatively if the ANN is public, selecting the "right" layer may be labor intensive.
- c) Agent may be submitting raw data instead of ANN. In which case some common feature map ϕ is needed to send all submitted data to a common Kernel space.

Commitment Phase

This is the pre-competition phase where data assets are prepared and the principle auditor is charged the bounty. The PA provided dataset $\mathbb{D}$ is partitioned into private test and public validation sets using subroutine $\text{QAudit}_{\mathbb{D}}$. The validation set is revealed and used to judge the improvement in surrogate discriminative model from $\mathbf{w}_k^{t-1}$ to $\mathbf{w}_k^t$ on a preliminary basis. The test set $\mathbb{D}_{\text{test}}$ remains hidden, and is revealed in the reward phase as the parimutuel rewards are calculated. See $\text{QAudit}_{\mathbb{D}}$ for dataset construction process.

Protocol 2 $\text{QAudit}_{\mathbb{D}}$ data commitment subprotocol

Require: data set $\mathbb{D}$, noise generator η , embedding ϕ

```

1:  $(\mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}}) \leftarrow ([], [])$ 
2:  $k \leftarrow 0$ 
3: for  $\mathbf{x}_i \in \mathbb{D}$  do
4:    $\mathbf{z}_i \leftarrow \eta()$  ▷ out of class sample
5:   let  $v = \{(\phi_{\mathbf{x}_i}, 1), (\phi_{\mathbf{z}_i}, 0)\}$ 
6:   if  $k \leq \frac{|\mathbb{D}|}{4}$  then
7:      $(\mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}}) \leftarrow ([\dots, v], \mathbb{D}_{\text{val}})$  ▷ fill tests set
8:   else
9:      $(\mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}}) \leftarrow (\mathbb{D}_{\text{test}}, [\dots, v])$  ▷ fill validation set
10:  end if
11:   $k += 1$ 
12: end for
13: return  $(\mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}})$ 
```

Public Audit Phase

This phase lasts $T+1$ time stages, and it alternatively fills the queue Θ with model checkpoints submitted by Agents from the current stage t , while the Auditors audit the checkpoints from the last time stage $t-1$ by wagering on their quality. The data updated at each stage include:

1. (Θ *model parameter history*): history of model submission over all T stages. Each t 'th element of Θ is a set Θ^t of length K , featuring model submission from all K Agents. Agents can submit as often as they like while the private submission phase is open, but QAudit only stores the latest submission.

$$\Theta = [\Theta^1, \dots, \Theta^T] \quad \text{where } \Theta^t = \{\theta_1^t, \dots, \theta_K^t\}.$$

⁶Note Word2Vec is over 10 years old. In some sense QAudit is akin to giving SATs to machine learning models in *Latin*. It is not the definitive judge of quality, but it is a *standardized one*.

2. (**W classifier parameter history**): history of surrogate models over all T stages. Each t 'th element of $\mathbf{W}$ is a set $\mathbf{W}^t$ of length K , featuring model surrogates trained on submissions from all K Agents.

$$\mathbf{W} = [\mathbf{W}^1, \dots, \mathbf{W}^T] \quad \text{where } \mathbf{W}^t = \{\mathbf{w}_1^t, \dots, \mathbf{w}_K^t\}.$$

$\mathbf{w}_k^t$ is trained from samples drawn from θ_k^t using $\text{QAudit}_{\text{train}}$. The interpretation of $\mathbf{w}$ is as follows:

- in the case where QAudit is eliciting generative model from Agent_k , then vector $\mathbf{w}_k^t$ parameterize a logistic regression model of form $p(y|\mathbf{x}; \mathbf{w}_k^t)$, with $y \in \{0, 1\}$ and some real valued $\mathbf{x} \in \Omega$. The model classifies the likelihood that $\mathbf{x}$ was generated by the model θ_k^t given parameter $\mathbf{w}_k^t$ (see example E.3.3).
 - In the case where QAudit elicits raw data from Agent_k from some PA-defined class, then the $p(y|\mathbf{x}; \mathbf{w}_k^t)$ describes the likelihood that the data belongs to this class. For example, if the PA wish to buy images of "socks on the floor" from Agents to train a robot to pick up socks, then $p(y|\mathbf{x}; \mathbf{w}_k^t)$ is a classifier for socks, trained on images of socks submitted by Agent_k .
3. (**logprob log likelihood history**): storing how well every proxy classifier $\mathbf{w}_k^t$ performed on the *validation set* $\mathbb{D}_{\text{val}}$, measured by the log-likelihood of sampling this data from nature as defined by $p(y|\phi_{\mathbf{x}}; \mathbf{w}_k^t)$. This loss is public and should be used by Auditors to wager on the underlying model θ_k^t .

$$\text{logprob} = \{\text{logprob}^1, \dots, \text{logprob}^T\} \quad \text{where } \text{logprob}^t = \{\text{logprob}_1^t, \dots, \text{logprob}_K^t\}$$

4. (**B share history**): Stores the bid history throughout stage t for every Auditor_i . During stage t , each Auditor_i submits bid vectors $\mathbf{b}_i^t = \{b_{1,i}^t, \dots, b_{K,i}^t\}$, where $b_{k,i}^t \geq 0$ is a scalar value denoting the number of shares Auditor_i purchased for model k at time t :

$$\mathbf{B} = [\dots, [\mathbf{b}_1^t, \dots, \mathbf{b}_i^t, \dots, \mathbf{b}_N^t]_t, \dots]_{t=1..T+1} \quad \text{where } \mathbf{b}_i^t = (b_{1,i}^t, \dots, b_{K,i}^t).$$

Some additional notes:

- within each stage, the auditors may bid as often as they like, thus each $\mathbf{b}_i^t$ may appear more than once.
 - The share $b_{k,i}^t$ has associated purchase price ρ_i^t denominated in miat , thus the Auditor's purchase value for this vector is $\langle \mathbf{b}_i^t, \rho^t \rangle$.
5. (**P price history**) stores the price of buying a share of each model θ_k^t at market close for stage t :

$$\mathbf{P} = [\rho^1, \dots, \rho^{T+1}] \quad \text{where } \rho^t = (\rho_1^t, \dots, \rho_K^t).$$

The vector ρ^t is initialized by subroutine QAudit_{ρ^0} , it computes the Bregman divergence from $\mathbf{w}_k^{t-1}$ to $\mathbf{w}_k^t$. This is the initial price or prior likelihood that the model $\mathbf{w}_k^t$ improves the previous model $\mathbf{w}_k^{t-1}$. As the Agents submit buy orders, the current market price is updated by the logarithmic market scoring rule in QAudit_b to reflect the current market consensus.

For every $(t-1, t)$ time stages, the competition alternate between two phases:

- (*model elicitation step*). At time $t-1$, Agents submit their checkpoints into Θ^{t-1} . Agents may submit as often as like, and the order of submission does not matter, as QAudit will build the final Θ^{t-1} from the latest submission from each agent k at time t . Before Θ^{t-1} is published, QAudit_{Θ} computes the proxy classifiers $\mathbf{W}^{t-1}$, and publishes it along with associated log-data-likelihood history logprob^{t-1} . The losses

will be revealed to Auditors at market open in t , and maybe used as one indicator of quality, alongside other metrics, i.e. Frechet inception distance score of the submitted generative models. Note the surrogate model weights $\mathbf{W}^{t-1}$ are not published as this would skew incentives, in that Agents may train models to "beat" the surrogate classifiers.

Protocol 3 QAudit_o Agent model elicitation subprotocol

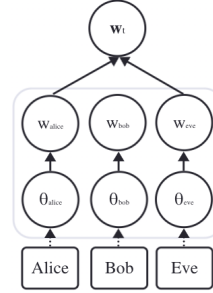
Require: validation set $\mathbb{D}_{\text{val}}$, noise generator η

Require: surrogate model weights from previous stage $\{\mathbf{w}_k^{t-1}\}_{k=1..K}$

Require: agent accounts $\{\text{Agent}_k\}_{k=1..K}$

- 1: $(\Theta^t, \mathbf{W}^t, \text{logprob}^t) \leftarrow (\{\}, \{\}, \{\})$
 - 2: **while** submission open **do**
 - 3: $\theta_k^t \leftarrow \text{Agent}_k$
 - 4: $(\mathbf{w}_k^t, \text{logprob}_k^t) \leftarrow \text{QAudit}_{\text{train}}(\theta_k^t, \eta, \mathbf{w}_k^{t-1}, \mathbb{D}_{\text{val}}, \phi, 10)$
 - 5: $(\Theta^t, \mathbf{W}^t, \text{logprob}^t) \leftarrow (\{\dots, \theta_k^t\}, \{\dots, \mathbf{w}_k^t\}, \{\dots, \text{logprob}_k^t\})$
 - 6: **end while**
 - 7: **return** $(\Theta^t, \mathbf{W}^t, \text{logprob}^t)$
-

Figure 2.5: In every stage QAudit elicits generative models θ from Agents such as alice, bob, and eve. Data is sampled from said generators to train three respective discriminative models $p(y|x; \mathbf{w}_k)$. In this diagram, the best $R = 2$ performing discriminative models from alice and eve are used to build an ensemble model $\mathbf{w}^t$ representing the winners of stage t . This ensemble is judged against the Auditor-selected ensemble in the previous stage $\mathbf{w}^{t-1}$.



2. (*wager elicitation step*). At stage t , the previous submission stage's queue Θ^{t-1} is published, along with losses logprob^{t-1} . QAudit_o then initialize the price vector $\{\rho_k^{t-1}\}_{k=1..K}$ using the private parameter set $\mathbf{W}^{t-1}$ and $\mathbf{W}^{t-2}$. Note: QAudit is a memory-less protocol, so that the price vector is *re-initialized* on every stage. This means models who performed well with Auditors last stage are not at a price (dis)advantage this stage. The only factor (aside from the price ascending constant) that determine the initial price quote in every stage is the Bregman divergence between the model $\mathbf{w}_k^{t-1}$ in the current trading stage, and the corresponding model $\mathbf{w}_k^{t-2}$ from the previous trading stage (see Appendix (F.4)). Auditors approach the protocol and observes the model θ_k^{t-1} , in particular they see:
 - a) current price ρ_k^{t-1} ,
 - b) log likelihood of data logprob_k^t computed on the validation set $\mathbb{D}_{\text{val}}$,
 - c) and any published metrics associated with model θ_k^{t-1} .

Based on these metrics, Auditors may buy a share (denoted mint_k^{t-1}) of model θ_k^{t-1} if they believe it is under-priced relative to its expected performance under auditor's belief. Each time the Auditor purchases shares of some model, its share price is updated by line 6-7 of QAudit_o using the logarithmic market scoring rule (LMSR). Each model in every submission stage are competing for R slots, and are eligible for parimutuel or mining rewards. Note the two steps are interleaved, so that i.e. if one time stage is 24 hours, then the Auditors are always trading yesterday's models.

Protocol 4 QAudit_v Auditor wager elicitation subprotocol with LMSR-based Amm

Require: dutch auction function v_ρ , price init constant v_ρ^o
Require: consecutive surrogate model weights $\mathbf{W}^{t-1}$, and $\mathbf{W}^t$
Require: log-data-likelihood logprob over models $\{\mathbf{w}_k^t\}_{k=1..K}$
Require: validation set $\mathbb{D}_{\text{val}}$, auditor accounts $\{\text{Auditor}_i\}_{i=1..N}$

- 1: $\mathbf{B}^t \leftarrow []$
- 2: $(\mathbf{s}^t, \rho_{\tau_o}^t) \leftarrow \text{QAudit}_{\rho^o}(\mathbb{D}_{\text{val}}, v_\rho^o, \mathbf{W}^{t-1}, \mathbf{W}^t)$
- 3: **while** market open **do**
- 4: $\mathbf{b}_i^t \leftarrow \text{Auditor}_i$ where $\mathbf{b}_i^t = (b_{1,i}^t, \dots, b_{K,i}^t)$
- 5: let $\mathbf{s} = (\dots, s_k^t + b_{k,i}^t, \dots)_{k=1..K}$
- 6: let $\mathbf{d} = \exp(s_1^t) + \dots + \exp(s_K^t)$ where $s_k^t \in \mathbf{s}$
- 7: let $\rho = v_\rho^t \cdot \left(\frac{\exp(s_1^t)}{\mathbf{d}}, \dots, \frac{\exp(s_K^t)}{\mathbf{d}} \right)$ where $s_k^t \in \mathbf{s}$ $\triangleright$ update price by LMSR
- 8: $(\mathbf{s}^t, \rho^t) \leftarrow (\mathbf{s}, \rho)$ $\triangleright$ update current price for stage t
- 9: $\mathbf{B}^t \leftarrow [\dots, \mathbf{b}_i^t]$
- 10: **end while**
- 11: **return** $(\mathbf{B}^t, \rho^t)$

Commentary on the QAudit_v automated market maker (Amm):

- a) QAudit_v references the data log-likelihood logprob computed by QAudit_o over the set of models $\{\mathbf{w}_k^t\}_{k=1..K}$, and suggests a set of possible rankings $\{\mathbf{s}_r^*\}_{r \geq 1}$ over the $K!$ possible permutation over Agents. This should aid the Auditors in deciding who to wager on.
- b) The price vector $\rho^t = (\rho_1^t, \dots, \rho_K^t)$ is a probability distribution over the events:

$$\Omega = \{(\text{Agent}_1, \text{wins stage } t), \dots, (\text{Agent}_K, \text{wins stage } t)\},$$

where every $\rho_k^t \in \rho^t$ denotes the likelihood that the k th Agent will submit the best model at stage t . Thus Auditors engaging in "subset betting," where the betting language is $\{(1, 1)^t, \dots, (K, 1)^t\}$, each tuple k denotes Agent _{k} will finish first at stage t . From the Bayesian perspective, the initial value of ρ^t is the prior distribution, while every Auditor's buy signal is "data observation." Every buy vector $\mathbf{b}_i^t = (b_{1,i}^t, \dots, b_{K,i}^t)$ is filled with values $b_{k,i}^t \geq 0$ denoting how many shares of Agent _{k} 's model the Auditor _{i} would like to own. Recall the word "share" is synonymous with model token mint_k^t .⁷

- c) Once the order for some model k is placed, the Agent receives $b_{k,i}^t$ shares of the Machine Intelligence Nonfungible They point to a specific model $\mathbf{w}_k^t$ at stage t , and is a claim on the upside if model k is one of R top-ranked models in t . The QAudit_v Amm then incorporates the change in share inventory, and updates the market-estimated likelihood that any Agent _{k} will win stage t . This "posterior update after data fusion" is written:

$$p(\mathbf{s}^t | \mathbf{b}_i^t; \rho^t) = \left(\frac{\exp(s_1^t + b_{1,i}^t)}{\sum_{k=1}^K \exp(s_{k,i}^t + b_{k,i}^t)}, \dots, \frac{\exp(s_K^t + b_K^t)}{\sum_{k=1}^K \exp(s_k^t + b_{k,i}^t)} \right).$$

Line 6-7 in QAudit_v computes this value up to some value v_ρ^t that is a function of the current stage t . See example D.1.2.

⁷In some markets, auditors can sell shares back to the Amm, so that the Amm covers all buy/sell scenarios. QAudit_v does not offer this.

- d) Now note the presence of PA-defined constant v_ρ^t in the price update in line 7. In the canonical setting defined in example D.1.2, every value in the price vector is a true probability value. That is: $0 \leq \rho_k^t \leq 1$ for every $\rho_k^t \in \rho^t$, and the payout if event k occurs is \$1.00. However in our case the price is denominated in *miat*, and can be scaled by scalar value v_ρ^t due to Dutch auction constraint. Similarly the reward is some variable amount of *miat*. Note v_ρ^t is a function whose value differs as a function of stage t . Share prices increase monotonically in the increasing v_ρ^t regime, so the cost of buying into each stage increases. Thus:
- i. incentivizing the auditors to participate in the earlier stages, as the cost of acquiring winning wagers is lower.
 - ii. Allow auditors in every stage to profit as the overall betting pool increases each stage - assuming the betting volume is constant throughout the stages.
- e) QAudit_b uses this descending likelihood estimation of every Agent's odd of winning the stage as a *heuristic* to order and select the top R teams. Notice this winning track wager is a far simpler betting language than permutation betting or subset betting [For+07]. For example if QAudit_b offered permutation betting instead, then it must offer R! options to express the likelihood of every ordering of the top R candidates. Depending on the user-defined R value, the PA may have to market an insurmountable array of Arrow-Debreu options.
- f) Finally, observe the model θ_k^t and w_k^t do not change within each wagering stage t , but its price does. QAudit_b Amm increases the price of the k th model each time Auditors buy shares of model k . The rational Auditor should buy early *within* each stage t at a lower price, if he believes model k is more performant than its price suggests. Recall QAudit is sequential parimutuel in payout for auditors, that is auditors are paid with a pro-rata percentage of the wagers on stage $t + 1$ in proportion to how many shares of the winning models he purchased on stage t . Therefore the auditor is rewarded for buying the right model early *within each stage* t . If he buys the top model w_k^t after other auditors have already moved, then the price of shares mint_k^t would have increased, and his cost of buying the next stage's wager is higher. Conversely, when Auditors buy shares of model k , every model not indexed k decrease in price as ρ^t is a scaled probability distribution. So a Auditor seeking model k may also wait until other Auditors have purchased other models.

Protocol 5 QAudit _{ρ^o} price initialization subprotocol using Bregman Divergence

Require: validation set $\mathbb{D}_{\text{val}}$, price init constant v_ρ^o

Require: consecutive surrogate model weights $\{w_k^t\}_{k=1..K}$, and $\{w_k^{t-1}\}_{k=1..K}$

- 1: $(s, \rho_{\tau_o}^t) \leftarrow ((), ())$
- 2: **for** $k \in 1..K$ **do**
- 3: let $\rho_k^t = v_\rho^o D(w_k^{t-1} || w_k^t)$ where
- 4:

$$D(w_k^{t-1} || w_k^t) = \sum_{(\phi_x, y) \in \mathbb{D}_{\text{val}}} p(y | \phi_x; w_k^{t-1}) \cdot \log \frac{p(y | \phi_x; w_k^{t-1})}{p(y | \phi_x; w_k^t)}$$

- 5: $(s, \rho_{\tau_o}^t) \leftarrow \left((\dots \log(\rho_k^t)), (\dots \rho_k^t) \right)$
 - 6: **end for**
 - 7: **return** $(s, \rho_{\tau_o}^t)$
-

Commentary on QAudit _{ρ^o} :

- a) Normally, a LMSR is initialized with zero inventory across all options offered. However in this case QAudit_{ρ^o} initializes market inventory state $\mathbf{s}^t$ with the KL divergence between model $\mathbf{w}_k^{t-1}$ and $\mathbf{w}_k^t$, for every $k = 1 \dots K$ (again scaled by a factor v_{ρ}^o). So that for example if QAudit_{ρ^o} places an order $b_{k, \text{Amm}}^o$, then $b_{k, \text{Amm}}^o$ shares of mint_k^t model shares are minted and allocated to Metheus. Thus Metheus is a shareholder of every model k , in every stage t , across every instance of QAudit.
- b) The initialized price vector is the prior likelihood of Agent' odds of winning. The implication is that every model update is an improvement, and the more an Agent updates a model, the more likely this model will win the stage. Naive Auditors may examine the price only and buy the cheapest model at the beginning of each stage. If the Agents expect this behavior, they may be incentivized to update their model θ_k^{t-1} as little as possible. This incentive must then be balanced by the fact that the Agents also wish to make the top R slots per stage, so as to qualify for parimutuel rewards. The balance of the two opposing incentives is tilted by the regularization parameter R , defined by the principle auditor.
- c) Bregman divergence or "KL-divergence in the primal form" computes the distance between two distributions. In this case, line 4 of protocol QAudit_{ρ^o} estimates the distance between two distributions by logistic regression models $p(y|\phi_{\mathbf{x}}; \mathbf{w}_k^{t-1})$ and $p(y|\phi_{\mathbf{x}}; \mathbf{w}_k^t)$, in the space spanned by the data set $\mathbb{D}_{\text{val}}$ kernalized by feature map ϕ . The regression models are trained in $\text{QAudit}_{\text{train}}$ by sampling from the Agent-submitted models θ_k^t and θ_k^{t-1} , and the KL-divergence is estimated w.r.t $\mathbb{D}_{\text{val}}$. See Appendix (F.4) and/or [WJ08] for derivation of KL divergence.

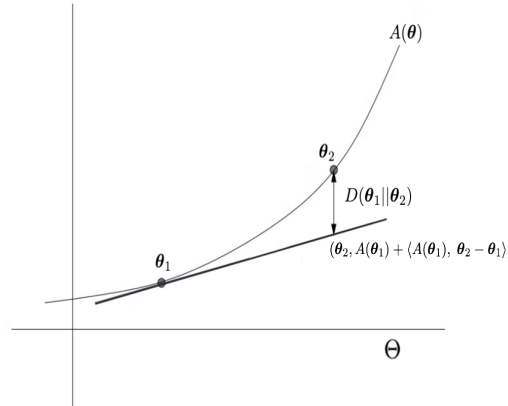


Figure 2.6: The Bregman divergence $D(\theta_1 || \theta_2)$ of two distributions parameterized by θ_1 and θ_2 , is the gap between the actual value of log-likelihood curve $A(\theta_2)$, and the first order polynomial approximation of value at $A(\theta_2)$, using the slope $\nabla A(\theta_1)$ at θ_1 [WJ04].

- d) QAudit_{ρ} then initializes the market state vector:

$$\mathbf{s} = \left(s_1^t, \dots, s_K^t \right) \quad \text{where} \quad s_k^t = \log \left(v_{\rho}^o D(\mathbf{w}_k^{t-1} || \mathbf{w}_k^t) \right)$$

Note the scaling factor v_{ρ}^o determines Metheus' margin per trade, so that the larger the v_{ρ}^o , the greater the margin. This is because Auditors will be paid out of the wagering pool by the percentage of model shares they own, and QAudit is in effect "moving first" by purchasing $\log(v_{\rho}^o \cdot D(\mathbf{w}_k^{t-1} || \mathbf{w}_k^t))$ shares of model $\mathbf{w}_k^t$ at no cost.

- e) In the naive implementation of exponential family prediction market of Appendix D, there are no Auditors. Instead every Agent submits his model $\mathbf{w}^t$ with some stake that is computed with $D(\mathbf{w}_k^{t-1} || \mathbf{w}_k^t)$, this is a signal that the Agent has reported his true model. Translated into this context, when an Auditor buys the first share of model $\mathbf{w}_k^t$ at price $\log(v_{\rho}^o D(\mathbf{w}_k^{t-1} || \mathbf{w}_k^t))$, it is a signal that the Auditor believes the Agent_k has revealed his best model at stage t .

Reward Allocation Phase

Protocol 6 QAudit_v Auditor/Agent reward allocation subprotocol

Require: regularization factor R , fiat denominated bounty ρ_{PA}^T

Require: final price vector at end of stage $(t, t+1)$: (ρ^t, ρ^{t+1})

Require: agent accounts $\{\text{Agent}_k\}_{k=1..K}$, auditor accounts $\{\text{Auditor}_i\}_{i=1..N}$

Require: bid history at stage t and $t+1$:

$$\mathbf{B}^t = \{\mathbf{b}_i^t\}_{i=1..N} = \{\{\mathbf{b}_{1,1}^t, \dots, \mathbf{b}_{K,1}^t\}, \dots, \{\mathbf{b}_{1,N}^t, \dots, \mathbf{b}_{K,N}^t\}\}$$

$$\mathbf{B}^{t+1} = \{\mathbf{b}_i^{t+1}\}_{i=1..N} = \{\{\mathbf{b}_{1,1}^{t+1}, \dots, \mathbf{b}_{K,1}^{t+1}\}, \dots, \{\mathbf{b}_{1,N}^{t+1}, \dots, \mathbf{b}_{K,N}^{t+1}\}\}$$

- 1: $\{\text{Agent}_r^t\}_{r=1..R} \leftarrow \text{sort}(\{\text{Agent}_k\}_{k=1..K}, \rho^t)[1 : R]$
- 2: $\{\text{Agent}_s^{t+1}\}_{s=1..R} \leftarrow \text{sort}(\{\text{Agent}_k\}_{k=1..K}, \rho^{t+1})[1 : R]$
- 3: $\text{agentPairs} \leftarrow \text{zip}(\{\text{Agent}_r^t\}_{r=1..R}, \{\text{Agent}_s^{t+1}\}_{s=1..R})$
- 4: $\mathbf{V}^t \leftarrow [(0x1, 0)_1, \dots, (0x1, 0)_{R \times (R+N)}]$ $\triangleright$ payouts with 0x1 placeholder addresses
- 5: $\triangleright$ select losing wagers on all $(K - R + 1)$ models at stage t from Auditors for agent payout
- 6: $\{d_{R+1,1}^t, \dots, d_{K,N}^t\} \leftarrow \text{select}(\mathbf{B}^t, [R + 1 : K])$

$\triangleright$ loop over R winners in pairwise descending order: ie $(r, s) = (1, 1)$ $(r, s) = (R, R)$

- 7: **for** $(\text{Agent}_r^t, \text{Agent}_s^{t+1}) \in \text{agentPairs}$ **do**
- 8:
- 9: $\{\mathbf{b}_{r,1}^t, \dots, \mathbf{b}_{r,M_t}^t\} \leftarrow \text{select}(\mathbf{B}^t, [r])$ $\triangleright$ get all M_t wagers on model R at stage t
- 10: $(\{\log\text{prob}_r^t\}_{r=1..R}, \dots) \leftarrow \text{QAudit}_*(R, \mathbb{D}_{\text{test}}, \mathbf{W}^t)$

$$\text{let } v_r^t = \underbrace{\frac{e^{\log\text{prob}_r^t}}{e^{\log\text{prob}_1^t} + \dots + e^{\log\text{prob}_R^t}} \cdot \rho_r^t \cdot (d_{R+1,1}^t + \dots + d_{K,N}^t)}_{\text{"eat what you bartered with"}} \cdot \underbrace{\frac{\text{fiat}}{\rho_r^t} \cdot \frac{\mathfrak{F}\text{at}}{\text{fiat}} \cdot \frac{\text{miat}}{\mathfrak{F}\text{at}}}_{\text{conversion}}$$

- 11: $\text{Agent}_r \leftarrow \text{Agent}_r + v_r^t$ $\triangleright$ payout agent r
- 12: $\mathbf{V}^t \leftarrow [\dots, (\text{Agent}_r, v_r^t)]$
- 13:

- 14: **for** $i \in 1..N$ **do** $\triangleright$ reward Auditors
- 15: $\text{let } v_{r,i}^t = 0$
- 16: $\text{let } b_{r,i}^t \in (\mathbf{b}_{1,i}^t, \dots, \mathbf{b}_{K,i}^t)_i$ $\triangleright$ select auditor i 's wager on model r
- 17: $\text{let } q_{r,i}^t = b_{r,i}^t / s_r^t$ where $s_r^t \in \mathbf{s}^t$ $\triangleright$ auditor i 's wager as % of all wagers on r
- 18: **if** $(\rho_{PA}^T = 0)$ **then** $\triangleright$ parimutuel reward

$$v_{r,i}^t = \underbrace{q_{r,i}^t \cdot s_s^{t+1} \cdot \rho_s^{t+1}}_{\text{"eat what you paid for"}} \cdot \underbrace{\frac{\text{fiat}}{\rho_s^{t+1}} \cdot \frac{\mathfrak{F}\text{at}}{\text{fiat}} \cdot \frac{\text{miat}}{\mathfrak{F}\text{at}}}_{\text{conversion}} \text{ where } s_s^{t+1} \in \mathbf{s}^{t+1}$$

- 19: **else** $\triangleright$ mining reward

$$v_{r,i}^t = \underbrace{q_{r,i}^t \cdot v_{pa}}_{\text{\% of bounty pool}} \cdot \underbrace{\frac{\mathfrak{F}\text{at}}{\text{fiat}} \cdot \frac{\text{miat}}{\mathfrak{F}\text{at}}}_{\text{conversion}}$$

- 20: **end if**
 - 21: $\text{Auditor}_i \leftarrow \text{Auditor}_i + v_{r,i}^t$
 - 22: $\mathbf{V}^t \leftarrow [\dots, (\text{Auditor}_i, v_{r,i}^t)]$
 - 23: **end for**
 - 24: **end for**
 - 25: **return** $\mathbf{V}^t$
-

As the $T + 1$ elicitation stages end, QAudit replays the market history from $t = [1, \dots, T + 1]$, and distributes the rewards to all Auditors and Agents based on the quality of their wagers and submissions, respectively.

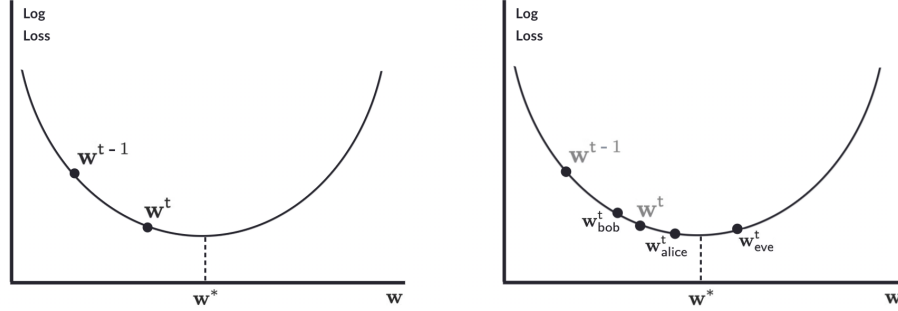


Figure 2.7: QAudit_{train} trains logistic regression models of form $p(y|\phi_{\mathbf{x}}; \mathbf{w}_k^t) = \sigma(\langle \phi_{\mathbf{x}}, \mathbf{w}_k^t \rangle)$. This class of models have convex log-loss function due to the $\sigma(\cdot)$ function. Over the $T + 1$ stages of QAudit, the protocol trains K different logistic regression models $\{\mathbf{w}_1, \dots, \mathbf{w}_K\}$. Every model has a history of $(T + 1)$ iterations: $\{\mathbf{w}_k^1, \dots, \mathbf{w}_k^{T+1}\}$. These models can be compared on the same log-loss curve, where lower values are better. In the left diagram, the ensemble model $\mathbf{w}^t$ outperforms that of the previous stage $\mathbf{w}^{t-1}$. So Agents whose models are used to build the ensemble $\mathbf{w}^t$ are eligible for parimutuel rewards, as are the Auditors who wagered on them. In the right graph PA defined $R = 3$, so that ensemble $\mathbf{w}^t$ is constructed from models submitted by alice, eve, and bob. Since alice's model has the lowest log-loss value, she takes the most reward from the pot at stage t , eve take the 2nd most, and bob takes the least reward. Similarly, Auditors who wager on alice are eligible for the most sequential parimutuel reward drawn from stage $t + 1$, and so on. Note this comparison across all $(T + 1) \times K$ models cannot be made with the underlying Agent-submitted generative models, as they do not have convex loss functions. See Appendix E.3.3 for more on kernel logistic regression, see D.2.3 for more on logistic regression and LMSR.

Protocol QAudit_o replays the wager history three stages at a time: $[t - 1, t, t + 1]$, and compares the current surrogate composite model $\mathbf{w}^t$ against the previous composite model $\mathbf{w}^{t-1}$. This composite/ensemble model $\mathbf{w}^t$ is constructed from the top R models $\{\mathbf{w}_1^t, \dots, \mathbf{w}_R^t\}$. The prediction of the composite model on every point in the dataset $(\phi_{\mathbf{x}}, y) \in \mathbb{D}_{\text{test}}$ is the weighted average of their individual predictions, weighed by the Auditors' wager on each model $\mathbf{w}_r^t$:

$$p(y|\phi_{\mathbf{x}}; \mathbf{w}^t) := p(y|\phi_{\mathbf{x}}; \{\mathbf{w}_r^t\}_{r=1..R}) = \sum_{r=1}^R \frac{\rho_r^t}{\rho_1^t + \dots + \rho_R^t} \cdot p(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t).$$

In other words, the more an Auditor wagers on the model θ_r^t , the more the prediction of surrogate classifier $\mathbf{w}_r^t$ affects the ensemble prediction.

Next, QAudit_o compares the data likelihood of test set $\mathbb{D}_{\text{test}}$ for $\mathbf{w}^{t-1}$ against $\mathbf{w}^t$, and determine whether to reward the relevant parties. Under the principle of "**judge collectively but reward individually**," if the current composite model $\mathbf{w}^t$ is performant then:

1. every winning Agent_r is paid from the losing wagers from the current pool at stage t , pro-rata to their respective model $\mathbf{w}_r^t$'s marginal information value add to the prediction of $\mathbf{w}^t$. So payout is parimutuel within every stage t for the winning Agents, and all payouts are denominated in mia_t . Note the Agent_r's payout and his model prediction share is different:

$$\text{split to Auditor} = \frac{\rho_r^t}{\rho_1^t + \dots + \rho_r^t + \dots + \rho_R^t},$$

$$\text{split to Agent} = \frac{e^{\log \text{prob}_i^t}}{e^{\log \text{prob}_1^t} + \dots + e^{\log \text{prob}_i^t} + \dots + e^{\log \text{prob}_R^t}}.$$

Moreover they are taken from different pools, the winning pool in the next stage for the Auditors and the losing pool from this stage for the Agents. Naturally, this discrepancy creates a misalignment of incentives between the Auditor and Agents. Thus regret penalty q_i^t is taken from the Auditor and given onto Agents to force an alignment.

Example 2.3.1. (*Agent submission scenario*) Agent_r submits model θ_r^t at stage t , which is used to train the surrogate classifier $\mathbf{w}_r^t$. Now suppose $\mathbf{w}_r^t$ finished 2nd in the winning pool. Moreover, suppose the average miat to $\mathfrak{F}\text{at}$ conversion rate at end of last trading stage $T + 1$ is 110 : 1. Then Agent_r 's payout is:

- a) at market close for stage t , the total wager pool for model $\mathbf{w}_r^t$ has 250 shares, with an average price per share of 4.5 miat /share, for a total of

$$250 \text{ mint}_r^t \cdot \frac{4.5 \text{miat}}{\text{mint}_r^t} = 1125 \text{miat}$$

tokens in the pool.

- b) Now suppose the principle auditor defined $R = 3$, and the total amount of miat wagered over all three winning models at the current stage t is 3000 miat , including model-R. Next, suppose the remaining losing Agents' models have collectively elicited 5000 miat in wagers at time t . Then Agent_r 's pro-rata share of the losing pool is:

$$\frac{1125}{3000} \cdot 5000 \text{miat} = 1875 \text{miat}.$$

This converts to fiat at:

$$1875 \text{miat} \cdot \frac{1 \mathfrak{F}\text{at}}{110 \text{miat}} \cdot \frac{1 \text{fiat}}{1 \mathfrak{F}\text{at}} = \text{fiat} 17.046.$$

Note although Agent_r 's pro-rata share of the losing pool is determined by how much the Auditors' wagered on his model, the agent himself have not wagered any amount of money, be it in fiat or miat . He only contributed time and effort, and yet have pocketed 17.05 in fiat in this scenario. In real life, this amount can be credited to Agent_r 's developer account, so that his future training sessions are subsidized. This is an important point: **trading subsidize training**.

2. The winning Auditors who wagered on model $\mathbf{w}_r^t$ are paid from the *corresponding* betting pool of model $\mathbf{w}_s^{t+1}$, where r and s are the matching positional index of the model rankings. So that QAudit is sequential parimutuel for the winning Auditors, and payouts are denominated in miat .

Example 2.3.2. (*Auditor wager scenario*) Suppose Auditor $_i$ owns shares in model $\mathbf{w}_r^t$. As before, assume $\mathbf{w}_r^t$ finished 2nd in the winning pool:

- a) Auditor $_i$ owns 100 shares of $\mathbf{w}_r^t$, which he purchased at 3 miat per share. So his total spend is:

$$100 \text{ mint}_r^t \cdot \frac{3 \text{miat}}{1 \text{mint}_r^t} = 300 \text{miat}.$$

- b) Auditor $_i$ purchased the miat at 100:1 miat to $\mathfrak{F}\text{at}$ conversion rate, and $\mathfrak{F}\text{at} : \text{miat}$ conversion rate is pegged at 1:1 as $\mathfrak{F}\text{at}$ is a stable coin. The auditor's fiat spend was:

$$300 \text{miat} \cdot \frac{1 \mathfrak{F}\text{at}}{100 \text{miat}} \cdot \frac{1 \text{fiat}}{1 \mathfrak{F}\text{at}} = \text{fiat} 3.00$$

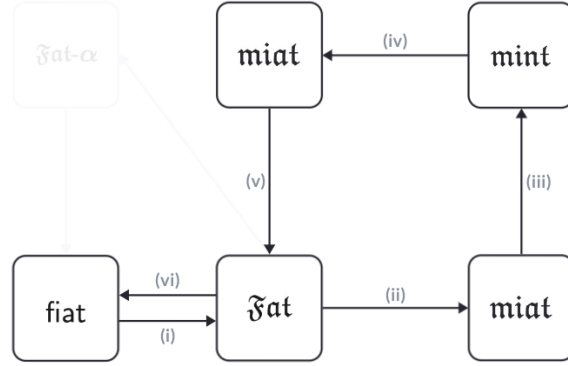


Figure 2.8: The QAudit conversion cycle:

- i. QAudit wager conversion process begins when Auditors buy into QAudit with fiat, which converts to "phantom ⚡at" token.
- ii. Conceptually, this phantom ⚡at is minted and exchanged for miat at the current floating exchange rate. This ⚡at is then immediately burnt so that total ⚡at supply stays constant.
- iii. Auditors use miat to wager on models by buying machine intelligence nonfungible tokens, or mints from Agents. If the respective models perform, then Auditors win more miat.
- iv. Auditors receive their reward by depositing their mint and receive miat owed to them. New miat is minted and issued to winners. This is called "miat-mining." Observe as models improve, fresh miat is mined.
- v. Auditors may convert their miat back to ⚡at at the prevailing conversion rate. The miat is recycled back to the global pool awaiting resale.
- vi. Finally, this ⚡at is converted back to pegged-fiat at 1:1 exchange rate.

Auditors whose winnings have been diluted away by miat inflation may recover this loss by purchasing ⚡at-α tokens and receiving inflation dividends. This is depicted in the faded bubble. Finally, there is an anti-deflation clause whereby if the miat:⚡at exchange rate falls within the conversion cycle duration, then Auditors/Agents must liquidate their position at an exchange rate no lesser than what they bought into the market. See eqn 4.2 and fig 4.6 in chapter 4 for more information.

- c) At market close of stage t , the price $\rho_s^t \in \rho^t$ of $\mathbf{w}_r^t$ is 5 miat per share. This implies Auditor _{i} purchased his shares at a lower price. The total wager pool for model $\mathbf{w}_r^t$ has 250 shares at market close for time t , with an average price per share of 4.5 miat/share, for a total of

$$250 \text{ miat}_r^t \cdot \frac{4.5 \text{ miat}}{1 \text{ miat}_r^t} = 1125 \text{ miat}$$

tokens in the pool. Moreover, suppose the average miat to ⚡at conversion rate for the pool is 105 : 1, this could occur if miat is mined over the wagering stage. Thus the fiat value of the pool is:

$$1125 \text{ miat} \cdot \frac{1 \text{ ⚡at}}{105 \text{ miat}} \cdot \frac{1 \text{ fiat}}{1 \text{ ⚡at}} = \text{fiat}10.71.$$

- d) Next, the corresponding 2nd place finish model $\mathbf{w}_s^{t+1}$ at time stage $t + 1$ features a total wagering pool of 300 shares of mint_s^{t+1} , with average price/share of 6 miat , so that its miat pool value is:

$$300 \text{ mint}_r^t \cdot \frac{6\text{miat}}{1\text{mint}_s^{t+1}} = 1800 \text{ miat}.$$

The miat to $\mathfrak{F}\text{at}$ average conversion at stage $t + 1$ is $110 : 1$, so the fiat value of the pool is:

$$1800 \text{ miat} \cdot \frac{1\mathfrak{F}\text{at}}{110\text{miat}} \cdot \frac{1\text{fiat}}{1\mathfrak{F}\text{at}} = \text{fiat}16.36.$$

- e) Now suppose the collective model $\mathbf{w}^t$, of which $\mathbf{w}_r^t$ is a component outperformed the respective model $\mathbf{w}^{t-1}$. Then at reward time Auditor_i's payout for model $\mathbf{w}_r^t$ is:

$$\mathbf{v}_{s,i}^t = \frac{300\text{miat}}{1125\text{miat}} \cdot 1800\text{miat} = 480\text{miat}.$$

Observe Auditor_i is only raking a percentage of the corresponding sth model at $t + 1$, namely $\mathbf{w}_s^{t+1}$. Auditor_i is *not* entitled to what was bet on the 1st, 3rd,...,Rth winning model at stage $t + 1$. This is "judge collectively but reward individually." Now his reward may be converted back to fiat, as more miat mining reward has been minted, the miat to $\mathfrak{F}\text{at}$ conversion is now $112 : 1$. So Auditor_i's intake is:

$$480\text{miat} \cdot \frac{1\mathfrak{F}\text{at}}{112\text{miat}} \cdot \frac{1\text{fiat}}{1\mathfrak{F}\text{at}} = \text{fiat}4.285.$$

So his profit is $4.285 - 3.00 = \text{fiat}1.285$. Since the Auditors hold mint_k^t shares, the Auditors must deposit mint_k^t tokens to receive miat , which then converts to $\mathfrak{F}\text{at}$ and fiat. Once deposited, QAudit burns the mint_k^t shares. If however model k does not achieve one of the top R spots at time t , then the mint_k^t shares all have value zero. The mint are options on the upside.

3. In the terminal stage T , the winning Agents are paid out of the current stage as before, but the Auditors are paid out of the bounty $\mathbf{v}_{pa}$ paid by the principle auditor. As before, this bounty is converted to miat and paid to Auditors, that is miat is mined. This increases the total miat supply. See $\mathbf{v}_{r,i}^t$ of line 23.
4. Agents who submit losing models are not rewarded, however they are not eliminated either. Meanwhile Auditors who bet on losing models lose their wagers.

In QAudit, Auditors profit when:

1. the model they wagered on is one of the top R winning models at stage t .
2. The corresponding winning model at the same position at stage $t + 1$ has a bigger pool in proportion to what Auditors contributed on this current pool.
3. They buy shares at a lower price than its final price at market close for time t .

Conversely, adverse events that affect auditor's profit (aside from losing) are:

1. (*dilution*). miat is mined or minted throughout the competition. And there may be many competitions occurring concurrently, each one of which is mining. Thus growth in miat supply relative to $\mathfrak{F}\text{at}$ supply may dilute the auditor's winning.

2. (*contracting wagering pool size across stages*). QAudit is sequential parimutuel in payout, thus the auditor's earnings depend on the next stage's wagers. If the crowd lose interest and wagers less and less, then Auditors may lose money even if they select the proper winning model.
3. (*compression of share price across stages*). If price per share of models reduces across stages, then Auditors may lose money even if the overall pool size measured in shares do not contract. This may be remedied by price floor function v_ρ^t , which could force the share price to increase across stages t . In the event where purchasing volume is not decreasing, v_ρ^t turns QAudit into an ascending price auction, whereby on every stage the price of the underlying model θ_k^t increases. Increase v_ρ^t values forces the Auditors to buy shares of the model early across the stages.

Factors influencing Auditor/Agent's profits are also *margin opportunities* for Metheus:

1. (*miat dilution*). As total **miat** supply expands relative to **fiat** supply, Auditors who buy in any given conversion rate are diluted. So i.e. if he who buys in at 100 : 1 **miat** to **fiat**, and then cashing out at 120 : 1 **miat** to **fiat** conversion, then this delta contributes to Metheus' margin.
2. (*price initialization constant v_ρ^o*). This constant is invoked on every t th stage for a total of T stages. In effect, Metheus is buying shares of every model at each t th stage with the buy vector $\mathbf{b}^t = (b_{1,1}^t, \dots, b_{1,K}^t)$ for free. So that at the end of the competition, Metheus is holding a set of shares: $\{\mathbf{b}^1, \dots, \mathbf{b}^T\}$ over every model in every stage. These shares dilute every Auditor and Agent's claim on the overall wagering pool over the T stages.
3. (*2.5% trading fee*). This fee is charged when Auditors buy in with fiat.

Remark 2.3.3. (Redemption of model nonfungible token mint) One aspect is missing in the documentation thus far: how Auditors redeem their token **mint** for **miat** currency. Recall that mint_r^t is a *conditional* claim on **miat** in the event its associated model θ_r^t wins in stage t . Suppose the model θ_r^t does win, and there are in total 100 shares of mint_r^t claiming 2000 **miat** tokens. Then if Auditor _{i} holds 10 shares, he is entitled to $10/100 \cdot 2000$ **miat** tokens. Meanwhile Auditor holding losing shares mint_s^t ($s \neq r$) are entitled to nothing. Their fiat wagers denominated in **miat** are instead given to winning Agents in stage t .

Regret Adjustment Phase

After every reward payment stage t , there is a regret adjustment phase, whereby **Agents** who should have been selected as one of the top winning R models are paid with regret penalty taken from the winning **Auditors**. The operative word here is *should have*:

1. **Agents** who were already chosen by **Auditors** are not compensated with regret.
2. If the individual models in the worse ensemble are ranked according to one of the suggested rankings, then the **Auditors** are not penalized.

Regret penalty is designed specifically to dis-incentivize malicious **Auditors** from pumping the market by purchasing cheap and low quality model shares, thus manipulating the rankings and causing the **Agents** who submit good models to exodus the market.

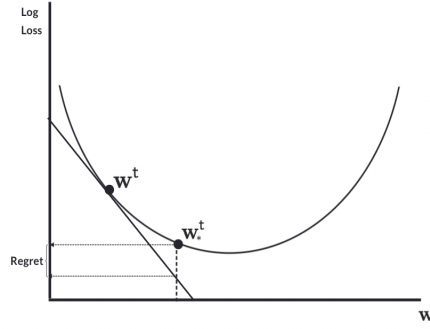


Figure 2.9: This diagram shows the difference in performance between the two models in the log-loss curve. However in QAudit_ℓ we compute the KL divergence $D(\mathbf{w}^t || \mathbf{w}_*^t)$. Remark E.3.4 states that they are comparable.

Protocol 7 QAudit_* maximum performing ensemble subroutine

Require: regularization parameter $R \leq K/2$

Require: model parameters $\{\mathbf{w}_k^t\}_{k=1..K}$, test set $\mathbb{D}_{\text{test}}$

- 1: $\text{logprobs} \leftarrow []$ $\triangleright$ log-likelihood on test set for every model
- 2: **for** $\mathbf{w}_k^t \in \{\mathbf{w}_k^t\}_{k=1..K}$ **do**
- 3: let $\text{logprob}_k^t = \sum_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}_{\text{test}}} \log p(y | \phi_{\mathbf{x}}; \mathbf{w}_k^t)$ where

$$\log p(y | \phi_{\mathbf{x}}; \mathbf{w}_k^t) = y \log \sigma(\langle \mathbf{w}_k^t, \phi_{\mathbf{x}} \rangle) + (1 - y) \log \sigma(1 - \langle \mathbf{w}_k^t, \phi_{\mathbf{x}} \rangle)$$

- 4: $\text{logprobs} \leftarrow [..., \text{logprob}_k^t]$

5: **end for**

- 6: $\{\mathbf{w}_s^t\}_{s=1..R} \leftarrow \text{sort}(\{\mathbf{w}_k^t\}_{k=1..K}, \text{logprobs})[1 : R]$ $\triangleright$ select best R models in hindsight

- 7: let $\text{logprob}^{t*} = \sum_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}_{\text{test}}} \log p(y | \phi_{\mathbf{x}}; \mathbf{w}^t)$ where

$$\begin{aligned} \log p(y | \phi_{\mathbf{x}}; \mathbf{w}^t) &= y \log p(y = 1 | \phi_{\mathbf{x}}; \mathbf{w}^t) + (1 - y) \log p(y = 0 | \phi_{\mathbf{x}}; \mathbf{w}^t) \\ &= y \log \left(\sum_s^R \beta_s \sigma(\langle \mathbf{w}_s^t, \phi_{\mathbf{x}} \rangle) \right) + (1 - y) \log \left(\sum_s^R \beta_s \sigma(1 - \langle \mathbf{w}_s^t, \phi_{\mathbf{x}} \rangle) \right) \\ \text{and } \beta_s &= \frac{e^{\text{logprob}_s^t}}{e^{\text{logprob}_1^t} + \dots + e^{\text{logprob}_R^t}} \end{aligned}$$

- 8: **return** $(\text{logprobs}, \text{logprob}^{t*}, \{\mathbf{w}_s^t\}_{s=1..R})$
-

Commentary on QAudit_* : it selects best R classifier models based on their individual performance on the test set logprob_s^t . QAudit_* then uses the weighted ensemble of $\mathbf{w}_s^t$'s to predict the label y given some input $\phi_{\mathbf{x}}$ drawn from the test set $\mathbb{D}_{\text{test}}$. Notice the weight β_s is the individual model performance of each $\mathbf{w}_s^t$.

Protocol 8 QAudit_Q compute Bregman's regret subprotocol**Require:** test set $\mathbb{D}_{\text{test}}$, validation set $\mathbb{D}_{\text{val}}$, regularization parameter $R \leq K/2$,**Require:** acceptable permutation difference $\mathbf{a} \parallel \mathbf{b}$ **Require:** model params $\mathbf{W}^t$, price vector for stage t at market close: ρ^t 1: $(\text{logprob}_{\text{val}}^t, \{\mathbf{w}_u^t\}_{u=1..R}) \leftarrow \text{QAudit}_{\text{infer}}(R, \mathbb{D}_{\text{val}}, \rho^t, \mathbf{W}^t)$ 2: $(\text{logprob}_{\text{test}}^t, \{\mathbf{w}_r^t\}_{r=1..R}) \leftarrow \text{QAudit}_{\text{infer}}(R, \mathbb{D}_{\text{test}}, \rho^t, \mathbf{W}^t)$ 3: $(\text{logprob}_s, \text{logprob}_*^t, \{\mathbf{w}_s^t\}_{s=1..R}) \leftarrow \text{QAudit}_*(R, \mathbb{D}_{\text{test}}, \mathbf{W}^t)$ 4: $\varrho^t \leftarrow 0$ $\triangleright$ total regret denominated in *miat*5: $\{\beta_s\}_{s=1..R} \leftarrow [0]_{s=1..R}$ 6: let $\mathbf{s}_{\text{val}} = \text{induceOrdering}(\{\mathbf{w}_u^t\}_{u=1..R})$ 7: let $\mathbf{s}_{\text{test}} = \text{induceOrdering}(\{\mathbf{w}_r^t\}_{r=1..R})$ 8: **if** $\exp(\text{logprob}_*^t) > \exp(\text{logprob}_{\text{test}}^t)$ & $(\mathbf{s}_{\text{val}} \parallel \mathbf{s}_{\text{test}}) > (\mathbf{a} \parallel \mathbf{b})$ **then**9: $\varrho^t \leftarrow v_{\rho}^{t+1} \cdot D(\mathbf{w}^t \parallel \mathbf{w}_*^t)$ where $\triangleright$ regret is *kl-div* between models

$$D(\mathbf{w}^t \parallel \mathbf{w}_*^t) = \sum_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}_{\text{test}}} \mathbf{p}(y | \phi_{\mathbf{x}}; \mathbf{w}^t) \log \frac{\mathbf{p}(y | \phi_{\mathbf{x}}; \mathbf{w}^t)}{\mathbf{p}(y | \phi_{\mathbf{x}}; \mathbf{w}_*^t)} \quad \text{where}$$

$$\mathbf{p}(y | \phi_{\mathbf{x}}; \mathbf{w}^t) = y \log \left(\sum_r \alpha_r \sigma(\langle \mathbf{w}_r^t, \phi_{\mathbf{x}} \rangle) \right) + (1-y) \log \left(\sum_r \alpha_r \sigma(1 - \langle \mathbf{w}_r^t, \phi_{\mathbf{x}} \rangle) \right)$$

$$\mathbf{p}(y | \phi_{\mathbf{x}}; \mathbf{w}_*^t) = y \log \left(\sum_s \beta_s \sigma(\langle \mathbf{w}_s^t, \phi_{\mathbf{x}} \rangle) \right) + (1-y) \log \left(\sum_s \beta_s \sigma(1 - \langle \mathbf{w}_s^t, \phi_{\mathbf{x}} \rangle) \right)$$

$$\text{where } \alpha_r = \frac{\rho_r^t}{\rho_1^t + \dots + \rho_R^t} \quad \beta_s = \frac{e^{\text{logprob}_s^t}}{e^{\text{logprob}_1^t} + \dots + e^{\text{logprob}_R^t}} \quad (r, s \in 1..R)$$

10: **end if**11: **return** $(\varrho^t, \{\beta_s\}_{s=1..R})$

Commentary on QAudit_Q: it computes the amount of regret value ϱ^t that is owed to all relevant Agents at time stage t .

1. In line 1 and 2, QAudit_Q computes the data-likelihood of each model w.r.t the test set and validation set. Recall the Auditors have access to each model $\mathbf{w}_k^t$'s individual performance on the validation set at every wagering stage t . Moreover, recall by QAudit_D, the test data and training data are drawn from the same distribution, so that the expected ranking is known. In line 7, QAudit_Q check if the individual classifier ranking under $\mathbb{D}_{\text{test}}$ is the same as $\mathbb{D}_{\text{val}}$. If they are different then it is due to the natural variation between the test and validation sets, so the Auditors are not penalized.
2. Line 3 computes the best expert ranking, using QAudit_{*} it select the top R models by their individual data-likelihood, and computes the ensemble score. This logprob_*^t is the best the models $\mathbf{W}^t$ could have done at around t , any deviation from this best prediction is *regret*.
3. In line 9, QAudit_Q checks that the best expert ensemble in hindsight does do better than the Auditor-selected ensemble, *and* that the Auditors could not have selected this ensemble using the log-likelihood information given.
4. The block of expressions between lines 9-10 computes the regret value denominated in *miat*, which is the Bregman divergence between the best ensemble the Auditors have created $\mathbf{w}_{\text{Auditor}}^t$, and the best expert ensemble according to data likelihood ranking $\mathbf{w}_*^t$. The intuition is that if the Auditors choose the wrong ensemble, then the protocol

QAudit_g forces the Auditors to play one additional "phantom" stage between t and $t + 1$. In this phantom stage, all the winning Auditors buy shares of the best expert model $\mathbf{w}_*$. Similar to QAudit_{po}, this share price is initialized as the Bregman divergence from the Auditor model to the expert model.

Protocol 9 QAudit_g Bregman's regret payment subprotocol

Require: regularization parameter $R \leq K/2$, permutation distance function $a \parallel b$

Require: model parameters $\{\mathbf{w}_k^t\}_{k=1..K}$

Require: test set $\mathbb{D}_{\text{test}}$, validation set $\mathbb{D}_{\text{val}}$

Require: price vector for stage t at market close: ρ^t , bid history at stage $\mathbf{B}^t$

Require: agent accounts $\{\text{Agent}_k\}_K$, auditor accounts $\{\text{Auditor}_i\}_N$

1: $(\log\text{prob}_{\text{auditors}}^t, \{\mathbf{w}_r^t\}_{r=1..R}) \leftarrow \text{QAudit}_{\text{infer}}(R, \mathbb{D}_{\text{test}}, \rho^t, \mathbf{W}^t)$

2: $(\log\text{probs}, \log\text{prob}_*^t, \{\mathbf{w}_s^t\}_{s=1..R}) \leftarrow \text{QAudit}_*(R, \mathbb{D}_{\text{test}}, \mathbf{W}^t)$

3: $\triangleright$ select unpaid best Z agents in hindsight, & Auditors who wagered on winning models

4: $\{\text{Auditor}_j\}_{j=1..M} \leftarrow \text{select}(\{\text{Auditor}_i\}_{i=1..N}, \rho^t, \mathbf{B}^t)$

5: $\{\text{Agent}_s\}_{s=1..R} \leftarrow \text{sort}(\{\text{Agent}_k\}_{k=1..K}, \log\text{probs})[1 : R]$

6: $\{\text{Agent}_z\}_{z=1..Z} \leftarrow \text{subsetDiff}(\{\text{Agent}_s\}_{s=1..R}, \rho^t)$

7: $\Delta_{\text{Agent}}^t \leftarrow []$

$\triangleright$ regret comp/penalty for Agents

8: $\Delta_{\text{Auditor}}^t \leftarrow []$

$\triangleright$ regret comp/penalty for Auditors

9: $(\varrho^t, \dots) \leftarrow \text{QAudit}_g(a \parallel b, R, \mathbf{W}^t, \rho^t, \mathbb{D}_{\text{test}}, \mathbb{D}_{\text{val}})$

10: **if** $\varrho^t > 0$ **then**

11: **for** $\text{Agent}_z \in \{\text{Agent}_z\}_{z=1..Z}$ **do**

$\triangleright$ compensate unpaid agents for regret

12: **let**

$$\beta_z = \frac{e^{\log\text{prob}_z^t}}{e^{\log\text{prob}_1^t} + \dots + e^{\log\text{prob}_Z^t}}$$

13: **let** $\varrho_z^t = \beta_z \cdot \varrho^t$

14: $\text{Agent}_z \leftarrow \text{Agent}_z + \varrho_z^t$

15: $\Delta_{\text{Agent}_z}^t \leftarrow [\dots, (\text{Agent}_z, \varrho_z^t)]$

16: **end for**

17: **for** $\text{Auditor}_j \in \{\text{Auditor}_j\}_{j=1..M}$ **do**

$\triangleright$ penalize Auditors for regret

18: $(\mathbf{b}_{1,j}^t, \dots, \mathbf{b}_{R,j}^t) \in \mathbf{B}^t$

$\triangleright$ select auditor j 's wager for winning models

19: $(\mathbf{b}_{1,1}^t, \dots, \mathbf{b}_{R,M}^t) \in \mathbf{B}^t$

$\triangleright$ select all winning Auditors' wager for winning models

20: **let** $\mathbf{q}_j^t = \frac{\mathbf{b}_{1,j}^t + \dots + \mathbf{b}_{R,j}^t}{\mathbf{b}_{1,1}^t + \dots + \mathbf{b}_{R,M}^t}$

21: **let** $\varrho_j^t = \mathbf{q}_j^t \cdot \varrho^t$

22: $\text{Auditor}_j \leftarrow \text{Auditor}_j - \varrho_j^t$

23: $\Delta_{\text{Auditor}_j}^t \leftarrow [\dots, (\text{Auditor}_j, \varrho_j^t)]$

24: **end for**

25: **end if**

26: **return** $(\varrho^t, \Delta_{\text{Agent}_z}^t, \Delta_{\text{Auditor}}^t)$

Commentary on QAudit_g:

1. This protocol computes the regret and allocates it to the aggrieved Agents, and penalize the bad Auditors. Line 4-6 pick out the Auditors who selected the winning models, and the Agents who were had "good" models but were not selected by said Auditors. Then in line 12-17, the aggrieved Agents are paid pro-rata to how well their model predicted the likelihood of the test data. Line 19-26 then subtract the pro-rated regret penalty from the Auditors' accounts.

2. Referencing the top level QAudit protocol, QAudit_{ev} is run in two branches of the code, once when the current ensemble improves that of the previous stage, and once if it does not. The notion is that in both cases, there may be aggrieved Agents.

2.4 QAudit Machine Learning

This section introduces the subroutines used to determine the quality of a generative model θ_k^t . This is part of the "tokenization" process whereby a generative model (or any data source) is turned into a machine intelligence nonfungible token (mint). It proceeds as follows:

1. at every stage t for every model $\theta_1 \dots \theta_K$, a corresponding classifier w_k^t is trained on this model. Each w_k^t measures the quality of the current generative model snapshot w.r.t some PA-defined ground truth. This training is done via *online logistic regression* [ZJ+12], which produces classifier versions: $\{w_k^1, \dots, w_k^{T+1}\}$. In online regression, the model is not trained to completion at each stage. Instead the learner samples a few examples from the corresponding θ_k , and updates the regression model.⁸

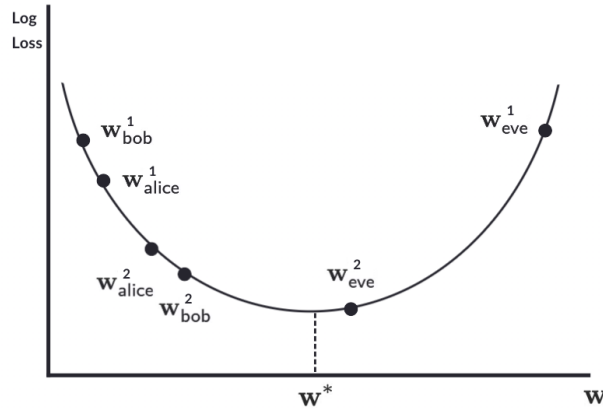


Figure 2.10: QAudit trains one shared instance of logistic regression model from the underlying generative models $\theta_k^1 \dots \theta_k^T$, and updates the model in an online manner over all Agents $k = 1..K$. At stage $t = 1$ in the diagram above, Agents alice, bob and eve all submit generative models $\{\theta_{alice}^1, \theta_{bob}^1, \theta_{eve}^1\}$. Data is sampled from these models and used to train classifiers $\{w_{alice}^1, w_{bob}^1, w_{eve}^1\}$. At stage $t = 2$, the classifiers sample from the generative models again and updated to create versions $\{w_{alice}^2, w_{bob}^2, w_{eve}^2\}$. This process continues for each stage, so that the Agents are essentially racing down the same log-loss hill to see who can reach the bottom point w^* the fastest.

2. Model share mints are issued from every w_k^t . Each mint_k^t is a statement that the underlying generative model θ_k is better than other generative models in the same stage t , and better than generative models in stage $t - 1$. Over the course of one QAudit instance, at least $(T + 1) \times K$ mints will be issued.
3. In order to prevent the classifier model from saturating, QAudit periodically re-initializes all the regression models. This is known as *regime changes*, and is defined in 2.6.8.

The advantages of scoring a regression surrogate as part of the tokenization process instead of the underlying generative model are:

⁸The introduction stated that the overall setup of QAudit_{train} and QAudit_{infer} may superficially resemble generative adversarial networks (GANs) of [GP+14], however the rewards of the generator $p(\phi_{x_\tau} | \phi_{x_\tau}; \theta)$ is not tied to that of the discriminator $p(y | \phi_x; w)$. So online regression is the right comparison here.

1. (*incentive compatibility*): logistic regression have convex log-loss functions as seen in E.3.3. A convex loss-function allows QAudit to place all $(T + 1) \times K$ models onto the same cost curve for comparison. It also ensures the scoring-rule we use to compute the quality of models across versions is incentive compatible, as expressed in D.2. See 2.7 and 2.10 for visual demonstration.
2. (*easier go-to-market*): scoring surrogates allow QAudit markets to operate even if the underlying model is not actively updated. This is an important point, model-trainers may not push checkpoints within every stage t . Moreover, there are many open-source foundational models whose parameters are frozen. Metheus can bootstrap from this reservoir of free models by tokenizing them. This negates the need to launch Metheus with a set of human competitors updating live models, which may be prohibitively expensive to collate.
3. (*broadier use case*): the underlying generative model may not be a model at all, but instead a database of user-generated data. This allows QAudit to run a data market that judges the quality of data as they are streamed into the Metheus data-lake.

Protocol 10 QAudit_{train} training discriminative surrogate subroutine

Require: model parameter θ , noise generator η ,

Require: discriminative model parameter $\mathbf{w}_k^{t-1}$ from last stage

Require: data set $\mathbb{D}_{\text{val}}$, public query set $\mathbb{Q}$, embedding ϕ , number of samples m

```

1:  $\mathbb{D}_{\text{train}} \leftarrow \{\}$ 
2: for  $i \in 1 \dots m$  do  $\triangleright$  build training set
3:    $(\mathbf{z}, \mathbf{x}) \leftarrow (\eta(), \mathbf{p}(\mathbf{x}|\mathbf{x}'; \theta))$  where  $\mathbf{x}' \in \mathbb{Q}$ 
4:    $\mathbb{D}_{\text{train}} \leftarrow [\dots, (\phi_{\mathbf{x}}, 1), (\phi_{\mathbf{z}}, 0)]$ 
5: end for
6: scramble( $\mathbb{D}_{\text{train}}$ )
7:  $\mathbf{w}_k^t \leftarrow \text{trainLogisticRegression}(\mathbb{D}_{\text{train}}, \mathbf{w}_k^{t-1})$   $\triangleright$  update model on  $\mathbb{D}_{\text{train}}$ 
8: let  $\text{logprob}_k^t =$ 

```

$$\sum_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}} \log \mathbf{p}(y | \phi_{\mathbf{x}}; \mathbf{w}_k^t) \text{ where}$$

$$\log \mathbf{p}(y | \phi_{\mathbf{x}}; \mathbf{w}_k^t) = y \cdot \log \sigma(\langle \mathbf{w}_k^t, \phi_{\mathbf{x}} \rangle) + (1 - y) \cdot \log \sigma(1 - \langle \mathbf{w}_k^t, \phi_{\mathbf{x}} \rangle)$$

```

9: return  $(\mathbf{w}_k^t, \text{logprob}_k^t)$ 

```

Commentary on QAudit_{train}:

1. it trains a kernel logistic regression classifier $\mathbf{w}_k^t$ (see E.3.3) by sampling from generative model θ_k^t , or data source $\mathbb{D}_k^t$. This model $\mathbf{w}_k^t$ is updated on every stage t , so that over the T stages QAudit_{train} is doing *online* kernel logistic regression [ZJ+12]. The parameters that control the confidence of QAudit_{train} are sample size m , embedding ϕ , and negative example or noise generator η .
2. (*negative example generator η*): this generator may emit noise $\mathbf{z}$ that is an appropriate control data against the model η . This negative example is labeled with $y = 0$ to form $(\mathbf{z}, 0)$, it is fed to $\mathbf{w}_k^t$ as an example of "bad data." Note the noise generator could in fact be an earlier example of a generative model. Or if the data source $\mathbb{D}_k^t$ is a pool of naturally occurring data within a class, then η could generate images outside of this class. For example if $\mathbb{D}_k^t$ emit images of apples, then η should emit images of oranges.
3. (*sample size m*): controls how many examples QAudit_{train} should sample from θ . The greater than m , the more confident that the classifier is accurately determining how

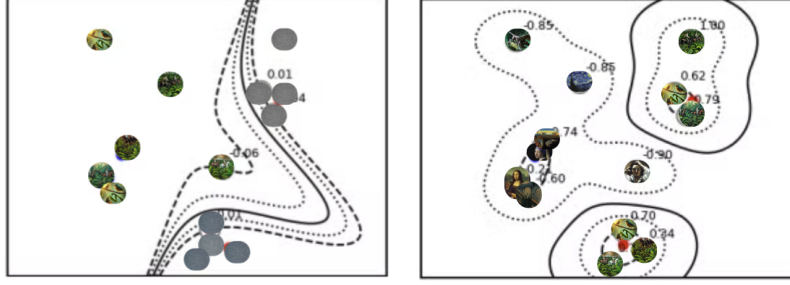


Figure 2.11: Two variations of $\text{QAudit}_{\text{train}}$ under proposal. In the left variation, Agent generative image models θ 's. Images sampled from said models form a training set against a control noise generator η . This dataset is then used to train a discriminative model $p(y|x; \theta)$ that identify noise from image. On the right, images sampled from generative models is trained against true images drawn by real painters. This corresponds to a classifier that can discriminate machine generated images from real ones. Observe that depending on which variation of the tokenization process $\text{QAudit}_{\text{train}}$ uses, the ranking over models differ.

well θ has improved $\mathbf{w}$. However, this also may lead to faster convergence for the classifier, thus leading to lesser improvements during the later times. If the data source is just one datum, as may be the case if the Agent is simply selling a picture or a song, then the noise generator η must only emit one out of class negative example as well.

4. (*embedding ϕ*): from a highly reductionist perspective, machine learning is fundamentally about function search inside a space defined entirely by the data. Thus the manner with which this data is represented determines how the function is constructed, see E.3. The embedding function ϕ ingests raw data and outputs a (potentially high dimensional) representation of this data. They may be drawn from $\mathbb{D}_{\text{val}}$, $\mathbb{D}_{\text{test}}$, θ , or $\mathbb{D}$. The choice of ϕ could significantly affect the outcome of which model is "better."

Protocol 11 $\text{QAudit}_{\text{infer}}$ ensemble inference subroutine

Require: regularization parameter $R \leq K/2$, data set $\mathbb{D}$

Require: price vector for stage t at market close: ρ^t

Require: model parameters $\mathbf{W}^t$

1: $\{\mathbf{w}_r^t\}_{r=1..R} \leftarrow \text{sort}(\mathbf{W}^t, \{\rho_k^t\}_{k=1..K})[1 : R]$ $\triangleright$ select top R models

2: let $\text{logprob}^t = \sum_{(\phi_x, y) \in \mathbb{D}_{\text{test}}} \log p(y|\phi_x; \mathbf{w}^t)$ where

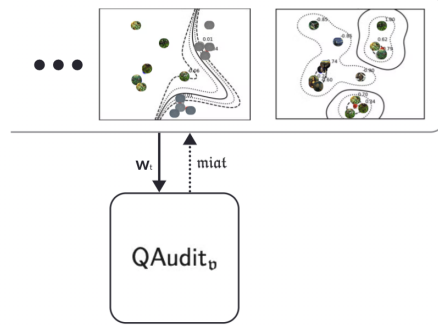
$$\begin{aligned} \log p(y|\phi_x; \mathbf{w}^t) &= \log p(y|\phi_x; \{\mathbf{w}_r^t\}_{r=1..R}) \\ &= y \log p(y=1|\phi_x; \mathbf{w}^t) + (1-y) \log p(y=0|\phi_x; \mathbf{w}^t) \\ &= y \log \left(\sum_r \alpha_r \sigma(\langle \mathbf{w}_r^t, \phi_x \rangle) \right) + (1-y) \log \left(\sum_r \alpha_r \sigma(1 - \langle \mathbf{w}_r^t, \phi_x \rangle) \right) \\ \text{and } \alpha_r &= \frac{\rho_r^t}{\rho_1^t + \dots + \rho_R^t} \end{aligned}$$

3: **return** $(\text{logprob}^t, \{\mathbf{w}_r^t\}_{r=1..R})$

$\text{QAudit}_{\text{infer}}$ selects the winning R classifier models, and use the weighted ensemble of $\mathbf{w}_r^t$'s to predict the label y given some input ϕ_x drawn from the test set $\mathbb{D}_{\text{test}}$. The weight is determined by the final price vector ρ^t at market close on stage t . Thus the Auditors control how each model $\mathbf{w}_r^t$ is weighed.

Remark 2.4.1. (Protocol Composability) QAudit can compose with other machine learning protocols for bespoke tokenization process.

Figure 2.12: Given a fixed QAudit_v wager-elicitation module, we can pair it with any number of QAudit_{train} machine learning modules. The different modules should be developed by the open source community, who will naturally have different metrics as to what a "good" model is. Critically, as different QAudit market instances use the metric modules, the development team behind them are compensated by *miat*.



This arrangement is ideal as there is no one way to judge models, so a "market place of metrics" is the most value-neutral resolution. Additionally, multiple tokenization modules also enable Metheus to launch multiple QAudit markets *per* set of generative models.

2.5 QAudit Elementary Analysis

QAudit is a multi-faceted market whereby Agents are bartering for *miat* with models, while the Auditors are bidding for *miat* with *miat*. In every pair of stages $(t, t + 1)$ where t is odd and $t + 1$ is even, the Agent move first by reporting their model, and then the Auditors move next to wager on the models. The competitive nature of QAudit may be summarized as a game amongst four sets of adversaries:

1. at every stage t , Agents are competing against all other Agents for the top R positions.
2. At every stage t , each Agent is competing against all other $R - 1$ Agents for a share of the bottom $K - R$ Agents' associated wagers.
3. At each stage t , Auditors are competing against other Auditors for 1) the top R slots with the highest wager amount, and 2) a percentage of the wagering amount within each of the R slots from $t + 1$.
4. Over the course of all t stages, every generative model θ_k^t or data source $\mathbb{D}_k^t$ is in a min-max game with a corresponding discriminative classifier w_k^t . In the first case, the generative model seek to spoof the classifier that it is emitting data "sampled from nature," while the classifier is attempting to detect fraudulent model generated data. In the second case, the Agent-provided data base $\mathbb{D}_k^t$ is attempting to confuse the classifier that it is emitting data from the correct class, while the classifier is attempting to detect out of class data.

The QAudit protocol features a symphony of incentives, with many games interleaved concurrently:

1. The sub-protocols QAudit_Θ, QAudit_v, and QAudit_{infer} are running online learning with mixture of experts using "follow the regularized leader" (FTRL) algorithm.
2. QAudit_{train} is running online kernel logistic regression by sampling from models $\{\theta_k^t\}_{k=1..K}$ or data sources $\{\mathbb{D}_k^t\}_{k=1..K}$ over time stages $t = 1...T + 1$.
3. The Dutch auction function v_ρ^t is running ascending price auction for model shares $k = 1...K$ over all stages $t = 1...T + 1$.

In lieu of these forces, we cue the arguments of [CV10] and [ACV12], and analyze QAudit as 1) a prediction market, 2) a game, and 3) a randomized algorithm.

1. In the first view, QAudit has market participants who report information, which must be collated to form a "correct" view of the world.
2. In the second view, QAudit has Auditors that must be incentivized to settle on equilibrium points. These points must be of "positive welfare" from the perspective of Agents.
3. In the third view, QAudit has access to randomness due to unobserved player motivations, and must find the correct solution in "reasonable amount of time."

Now given this perspective, this section characterizes QAudit with these properties:

1. lemma 2.5.1: QAudit reward structure is *incentive compatible* from the perspective of the Agents. Thus reporting the truth is a *pure strategy equilibrium* (lemma 2.5.2).
2. lemma 2.5.3: collusion, defined as Agents who report the best model, and Auditors who wager on the best model, is a *mixed-strategy equilibrium*.
3. lemma 2.5.8: in the event where every Auditor_i accepts the prompted rankings $\mathfrak{s}_r^*$ provided by QAudit instance, then the Auditors will collectively find the correct permutation in $\text{poly}(K!)$ number of wagers.
4. lemma 2.5.10: in the event where some of the Auditors hold private belief differing from the market maker's suggestion, then the idealized distribution $\tilde{\delta}_r$ over all $K!$ rankings degrades by:

$$\|\mathbf{p} - \tilde{\delta}_r\|_1 \leq \frac{2}{1 - \alpha_r} |\Delta \alpha_r| \quad \|\tilde{\delta}_r - \mathbf{p}\|_1 \leq (1 - \alpha_r) \|\Delta \mathbf{e}\|_1 \quad \|\tilde{\delta}_r - \mathbf{p}\|_1 \leq \frac{\alpha_r}{1 - \alpha_r} \|\Delta \mathbf{M}_r\|_\infty.$$

Stating the degradation in the QAudit solution as a function of:

- a) proportion $1 - \alpha_r$ of Auditors who hold differing opinion.
 - b) The Auditors' private preference over rankings denoted $\mathbf{e} + \Delta \mathbf{e}$.
 - c) How aggressively the Auditors wager, as expressed in the PSA walk matrix $\mathbf{M}$.
5. lemma 2.5.11: if Auditors are partitioned into two blocks, so that some of the Auditors accept the prompted permutation, while others hold a different opinion. Then if the proportion of Auditors who hold private belief $\mathfrak{s}_r^*$ is greater than 50%, the likelihood of finding one of the two accepted solution states is greater than $\frac{1}{2}$.

Ultimately, the soundness of QAudit is built upon these fundamental theorems:

1. The QAudit *Consensus Theorem* (2.6.4) states that for every stage $\mathbf{t}$, there exists a *Correlated Equilibrium* (see definition B.2.3) s.t. the market consensus at market close reflects the true permutation $\mathfrak{s}_r^{*\mathbf{t}}$ in expectation, provided the permutation is prompted by QAudit _{$\mathbf{v}$} at market open in every $\mathbf{t}$.
2. The QAudit *Social Consensus Theorem* (2.6.6) states that in the event where the protocol suggests many possible rankings over models, denoted $\mathfrak{s}_1^*, \dots, \mathfrak{s}_m^*$. Then in every protocol stage $\mathbf{t}$, there exists a point in time τ^* :

$$\tau_f - N - K \log(K) \leq \tau^* < \tau_f - 1 - K \log(K),$$

s.t. it is rational for every Auditor to switch to wagering on a single ranking $\mathfrak{s}_r^*$, favoring this ranking over all others.

3. QAudit *is not a Loser's Game* (2.6.10): states that the QAudit sequential parimutuel reward economic structure has positive expected value for some Auditor - if he plays according to rankings $\mathfrak{s}_r^*$ prompted by the protocol.

The rest of this chapter introduces the necessary computational models, definitions, and concepts. It then walks through the lemmas at a high level, all the proofs are found in Appendix C.

Agent-Agent Sub-Game

We can make some assumptions on the behavior of players:

1. **Agents** maximize their earnings on every stage t against other **Agents**, independent of what occurred in the previous or next stages, conditioned on the previous stage $t - 1$ (Markov Independence).
2. **Auditors** are myopic with 3-stage time horizon. They only consider the stages $(t - 1, t, t + 1)$, and seek to maximize profit subject to the cost of acquiring winnings from the next stage $t + 1$, or the mining reward at $T + 1$. Moreover:
 - a) since the **Auditor** cannot determine the amount wagered over the top R models in stage $t + 1$, he can only assume it is some constant.
 - b) Markov independence: the **Auditor** can inspect the model in stage $t - 1$. But **Auditors'** wagers are independent of model performance in stages $t = 1 \dots T - 2$, conditioned on the model performance in the previous stage $t - 1$.

The next two lemmas assume **Agents** submit the discriminative classifier weights $\mathbf{w}$, instead of generative model θ or data set $\mathbb{D}$.

Lemma 2.5.1. *Agent reward $\mathbf{v}_r^t$ is incentive compatible if **Agents** are myopic and Markov. So that if Agent_r has two private models $\mathbf{w}_r^t$ and $\mathbf{v}_r^t$ with:*

$$\prod_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}} p(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t) > \prod_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}} p(y|\phi_{\mathbf{x}}; \mathbf{v}_r^t), \quad (2.1)$$

then the rational Agent_r will reveal the better model $\mathbf{w}_r^t$ under the payment defined in QAudit_v .

Proof. See proof in Appendix C.1. ■

Lemma 2.5.2. *Reporting the truth is a pure strategy equilibrium for every **Agent**. That is to say given two Agent_r and Agent_s , whereby if each **Agent** can report the truth $\mathbf{w}^t$, or a lesser model $\mathbf{v}^t$, then every **Agent** will report $\mathbf{w}^t$ regardless of how the other **Agent** plays.*

Proof. See Appendix C.1. ■

Informally, the reason the **Agent** is incentivized to report his best model is because he is sharing the wagering pool with other winning **Agents**. And his reward is weighed not by the price ρ_r^t of his model as determined by the **Auditors**, but rather by the model's data likelihood as controlled by himself. So the **Agent** may rely on the **Auditors** to make the top R slots, but once he has cleared the hurdle he controls his own rewards. This is the principle of "eat what you bartered with."

Agent-Auditor Sub-Game

This section clarifies the intuition on why **Agents** and **Auditors** should collude to submit and then wager on the best model.

Lemma 2.5.3. *Collusion is a mixed-strategy equilibrium (see definition B.2.2) between **Agent** and **Auditor**. In this case "collusion" informally refers to the phenomenon whereby the **Agents** report their best model in some stage t , and the **Auditors** wager on the best reported model in this stage. Moreover if two models $\mathbf{w}_r$ and $\mathbf{w}_s$ are the best two models, then if expected data likelihood under $\mathbf{w}_r$ is greater than $\mathbf{w}_s$, then **Auditors** should wager more on $\mathbf{w}_r$ than $\mathbf{w}_s$.*

Proof. See Appendix C.2. ■

Informally, collusion occurs because **Agents** and **Auditors** are profiting from different wagering pools, as long as the **Auditor** does not defect. So **Agents** fish from the t th stage pool, while **Auditors** hunt the pool from the $(t + 1)$ th stage. This is critical, **Agents** and **Auditors** are not competing but colluding: their incentives are aligned and their rewards are orthogonal. This is the principle of **"hunt together but eat separately."**

In reality, QAudit is *not* presented as a competition to the **Agents**. Instead, it is a subsidized training environment, whereby in exchange for lower training/server costs, the **Agents** allow their checkpoints to be tokenized and traded. The **Agents** are tangentially aware of this, but not necessarily informed by this competitive dynamic every time they submit a checkpoint. An analogy can be made with FANNG employees, whose stock-based compensation make up a significant portion of their overall package. Technically the FANNG employee is negotiating their bonuses with traders every day on the job. But this underlying reality does not inform their daily behavior. Similarly, the **Agent** should see QAudit as a "far away market." If they do plan against it, ideally they do not think deeper than what is written here.

In light of this knowledge, the **Auditors** should by virtue of the larger Metheus software offering, trust that the **Agents** are not gaming the system, but rather focused on doing good work. This is why there is no **Auditor-Agent** sub-game. That is to say if the **Agents** observe the **Auditors** wager on a certain kind of models in stages $[t - 2, t - 1, t]$, should the **Agents** submit a different model on stage $t + 1$. If the **Agents** allow **Auditors'** behavior to bias their work, then the solution is not to change the incentives of QAudit, but rather to design a better user interface that further abstracts the market away from them.

Auditor-Auditor Sub-Game within every Stage

While the **Agents** are bartering for **miat** with their models, the **Auditors** are buying **miat** (read: money) directly with money. This arrangement may appear simple, however **Auditor-Auditor** dynamic is a complex one: featuring a phase change from cooperative to competitive paradigm depending on market state. So it is not necessarily a given that the **Auditors** will settle on the correct ranking $\mathfrak{s}^*$ over the **Agents'** models. For example, although proof 2.5.3 shows that **Auditors** should prefer the better model reported across **Agents**, should they decide to "play conservatively" by waiting for other **Auditors** to move first, then the market may close before the **Auditors** bid up the best models. We will show that identifying the best models in a timely manner is possible. The rest of this section develops the basic tools and prove some elementary results, and builds towards the fundamental theorem of QAudit stated in the sequel.

First, it is helpful to review how an **Auditor** would make his buy decision if QAudit is a vanilla sequential parimutuel prediction market of Appendix D.2. Similar to QAudit, the market keeps a persistent inventory state $\mathbf{s}^t = (s_1^t, \dots, s_K^t)$ over K **Agents**. Then logarithmic market scoring rule (LMSR) computes a price vector $\boldsymbol{\rho}^t = (\rho_1^t, \dots, \rho_K^t)$ from the shares, where each ρ_k^t is computed with:

$$\rho_k^t = \frac{\exp(s_k^t)}{\exp(s_1^t) + \dots + \exp(s_K^t)} \quad s_k^t \geq 0, \quad 0 \leq \rho_k^t \leq 1.$$

In the vanilla market, the **Auditors** approach the market sequentially and individually, and he is responsible for his own loss and rake the entire winnings if his buy signal improves the market estimate of the probability or price vector $\boldsymbol{\rho}^t$. Thus if the rational **Auditor** _{i} with private estimate $\mathbf{q}_i = (q_{i,1}, \dots, q_{i,K})$ observes some item r is under-priced, then he should report a buy vector $\mathbf{b}_i^t = (0, \dots, b_{i,r}^t, \dots, 0)$ so that the market price ρ_r^t matches his private estimate $q_{i,r}$. That is he reports buy-signal for some share $b_{i,r}^t$ by solving:

$$\rho_r^t = q_{i,r}^t = \frac{\exp(s_r^t) \exp(b_{i,r}^t)}{\exp(s_r^t) \exp(b_{i,r}^t) + \sum_{k \in \{1..K\}/r} \exp(s_k^t)} \implies b_{i,r}^t = \log \left(\frac{q_{i,r}^t \cdot \sum_{k \in \{1..K\}/r} \exp(s_k^t)}{\exp(s_r^t) (1 - q_{i,r}^t)} \right). \quad (2.2)$$

Once he buys $\mathbf{b}_{i,r}^t$ shares, the market price vector is updated to $\boldsymbol{\rho}^{t+1}$. Then the Auditor_i's compensation would be how well his report improved the data likelihood under some data set $\mathbb{D}_{\text{test}}$. So pick some $(\mathbf{x}, y) \in \mathbb{D}_{\text{test}}$, Auditor_i is rewarded with:

$$\log p(y|\mathbf{x}; \boldsymbol{\rho}^{t+1}) - \log p(y|\mathbf{x}; \boldsymbol{\rho}^t),$$

at reward time as eqn D.12 states. Recall section 2.2 lists the shortcomings of this simplistic setup. The QAudit protocol fixes these shortcomings, but introduces additional competitive dynamics. Since the Auditors are competing with other Auditors for winnings, he must modulate the degree with which he bids the entire truth across time increments τ in each stage t . And weighs his private truth against:

1. how likely will his report tip the balance and place the Agent in the top R slots.
2. How likely will this Agent *stay* in the winning slots once there. That is the price of this action relative to risk of buying Agent_r, only to be swamped by other Auditors who buy some Agent_s who subsequently crowd out Agent_r.
3. The size of the wager on stage $t + 1$ that would reward his desired slot at stage t .

Despite this complex dynamic, we show that under certain conditions, reporting the truth is a *Correlated Equilibrium* (see definition B.2.3). However, the price signal $\boldsymbol{\rho}^t$ alone is no longer a fair reflection of the Agents' true beliefs, as they may bid up the price as the market closes. So instead of analyzing the market price vector, we will examine the underlying value of interest: the permutation over K Agents, denoted with $\mathfrak{s}$. The next few definitions will introduce the computational concepts to codify the setting, namely:

1. a rational *audit strategy* corresponding to how the Auditors wager on the Agents.
2. A *permutation state machine* that tracks how the permutation over K Agents change as a response to this strategy while the market is open.

The purpose of this analysis is to state that if every players play the rational audit strategy using a permutation $\mathfrak{s}_r^*$ prompted by QAudit, then QAudit_v will converge onto this best permutation in expectation in every stage t .

Remark 2.5.4. (The Card Shuffle Problem) The reduced Auditor-Auditor game is analogous to the card shuffling problem: given one deck of cards whereby each card correspond to one Agent, and a table of player so that each player is one Auditor. Let each Auditor have some private belief stating that there is some optimal ordering of the cards, corresponding to the best ordering of Agents' models on stage t . And let this private belief be influenced by some ideal permutation provided by the dealer. Now the game commences, and the deck is passed around the table of Auditors, and each Auditor may switch the position of some cards, rearrange the entire deck, or do nothing. Two questions are asked at the end of the game: 1) what is the distribution over all $52!$ possible arrangements of cards, and 2) if some or all of the auditors believe in the optimal prompted arrangement, how many times does the deck have to be passed around to reach this best arrangement. See [LPW17, p. 100] for a variation of this problem.

Definition 2.5.5. (*Audit Strategy*) for some Auditor_i is a sequence of buy vectors:

$$\begin{aligned} \text{audit strategy} &= [\mathbf{b}_{i,\tau_0}^t, \dots, \mathbf{b}_{i,\tau_f}^t] \\ \text{where } \mathbf{b}_{i,\tau}^t &= (\mathbf{b}_{i,1,\tau}^t, \dots, \mathbf{b}_{i,K,\tau}^t) \quad \text{s.t. } \mathbf{b}_{i,k,\tau}^t \geq 0, \end{aligned} \quad (2.3)$$

whereby the value of every buy vector $\mathbf{b}_{i,\tau}^t$ is conditioned on the Auditor's private belief, and the current market state. Note each buy vector $\mathbf{b}_{i,\tau}^t$ is indexed by time increments $\tau_0 \leq \tau \leq \tau_f$ within stage t . Denoting that at time τ within wagering stage t as defined by QAudit_v, Auditor_i buys $\mathbf{b}_{i,r,\tau}^t$ shares of Agent_r's model.⁹

⁹Recall a model share is some nonfungible machine intelligence token mint_i^t .

Normally we do not write the audit strategy in sequence form, but rather in "algorithm" form. So it is akin to an automated trading bot. In particular, **AuditStrat** is the response of interest for the remaining section, we assume it is used by **Auditors** while trading against the market maker **QAudit_v**.

Algorithm 12 **AuditStrat**($\mathbf{s}_r^*, C, d$): Audit Strategy for every **Auditor_i**

Require: QAudit's suggested permutation $\mathbf{s}_r^*$

Require: Auditor_i's fixed private belief $\mathbf{q}_i^t = (q_{i,1}^t, \dots, q_{i,K}^t)$.

Require: Auditor_i's fixed competitive auction parameters $C \geq 0, d \geq 0$.

```

1:  $\mathbf{s}_i^t \leftarrow \text{induceOrdering}(\mathbf{q}_i^t)$  OR  $\mathbf{s}_r^*$  ▷ Auditor can accept or reject protocol suggestion
2: while market open do
3:    $\rho_\tau^t \leftarrow \text{QAudit}_v$ 
4:    $\mathbf{s}_\tau^t \leftarrow \text{induceOrdering}(\rho_\tau^t)$ 
5:   for  $\text{Agent}_r \in \mathbf{s}_\tau^t$  do
6:     if  $r \in \mathbf{s}_i^t[1..R]$  &  $r \leq R$  then ▷ Agentr is already ranked in top R
7:       Flip coin:
8:       if Head then
9:         buy  $C \cdot b_{r,i}^t + d$ 
10:      else
11:        do nothing
12:      end if
13:    else if  $r \in \mathbf{s}_i^t[1..R]$  &  $r > R$  then ▷ Agentr is not yet ranked in top R
14:      buy  $a \cdot b_{r,i}^t$  where  $a \stackrel{\text{iid}}{\sim} p[0, 1]$ 
15:    else
16:      do nothing
17:    end if
18:  end for
19: end while

```

Some commentary on **AuditStrat** follows:

1. The **Auditors** are prompted with a likely outcome $\mathbf{s}_r^*$, computed from the ranking over K models on the validation set. Critically, each **Auditor_i** may accept this suggestion or use their own private belief $\mathbf{q}_i^t$.
2. The **Auditor_i** checks the current market price vector ρ_τ^t , and determine its induces ordering $\mathbf{s}_\tau^t$. If the Auditor believes **Agent_r**'s model should be ranked within R , and his model is already ranked, then the Auditor flips a coin and either buys shares in **Auditor_i** or do nothing. In the case where he buys shares, the Auditor_i may buy shares in excess of what he would normally buy under expression 2.2 using private parameters C and d . This would occur if the Auditor_i believes the respective position to r in stage $t + 1$ will be large, and he bids to claim a larger share of this pot. In other words, the Auditor_i is in *competitive mode*.
3. In the event where **Agent_r**'s model is not yet ranked one of the top $1..R$ under the market's price vector's induced permutation, then the Auditor_i is in risk-averse *cooperation* mode. In this case, he pursues a *mixed strategy* by sampling some constant $0 \leq a \leq 1$ from distribution $p[0, 1]$, and multiplies it with the constant $b_{r,i}^t \geq 0$ as determined by 2.2. The intuition here is that Auditor_i will signal quality to the market by buying some shares, but not so much that he is "sticking his lonesome neck out" for an **Agent_r** whose quality may appear dubious to other Auditors.
4. If the auditor believes **Agent_r** should not be top ranked at all, then he does nothing. In particular, notice the Auditor does not update his private belief $\mathbf{q}_i^t$ as the QAudit_v market progresses.

Now observe as every Auditor_i runs strategy AuditStrat , the market inventory $\mathbf{s}_\tau^t$ updates as they buy shares with $[\mathbf{b}_{i,\tau_1}^t, \mathbf{b}_{i,\tau_2}^t, \dots]$. The corresponding market price vector $\boldsymbol{\rho}_\tau^t$ changes, as does the induced ordering $\mathbf{s}_\tau^t$. So there is a one-to-one "correspondence" indexed by τ :

$$\underbrace{[\mathbf{s}_{\tau_1}^t, \mathbf{s}_{\tau_2}^t, \dots, \mathbf{s}_{\tau_f}^t]}_{\text{share vectors}} \Rightarrow \underbrace{[\boldsymbol{\rho}_{\tau_1}^t, \boldsymbol{\rho}_{\tau_2}^t, \dots, \boldsymbol{\rho}_{\tau_f}^t]}_{\text{price vectors}} \Rightarrow \underbrace{[\mathbf{s}_{\tau_1}^t, \mathbf{s}_{\tau_2}^t, \dots, \mathbf{s}_{\tau_f}^t]}_{\text{permutations}}.$$

The permutation states $\mathbf{s}_\tau^t$ express a *permutation state machine*, whereby the states are permutations, and the state transitions are permutation functions. Moreover, the updates in price space corresponds to a random walk on the permutation graph expressed by a PSA.

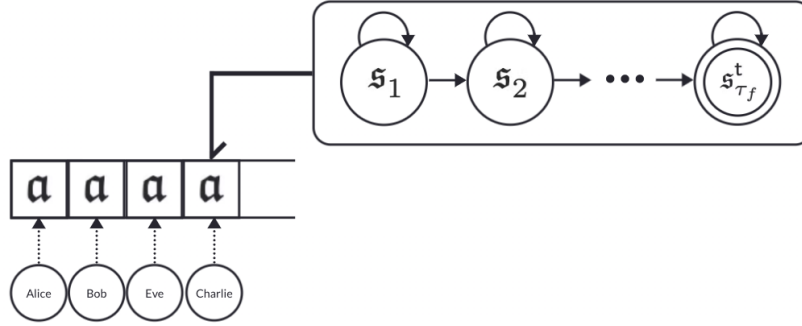


Figure 2.13: The QAudit market can be conceptualized as a permutation state machine (PSA). Each state $\mathbf{s}_\tau, \dots$ in the machine corresponds to a particular ranking of the Agents, and the edges are $\mathbf{a} : \mathbf{s}_\tau \rightarrow \mathbf{s}_{\tau+1}$, which permutes the ranking over Agents. The PSA reads from a tape with symbols $\mathbf{a}$, Auditors such as *alice* and *bob* place the $\mathbf{a}$'s on the tape as they wager in the QAudit market. At market close, the PSA is in some state $\mathbf{s}_{\tau_f}^t$, the top R Agents in this permutation wins round t . Thus we can conceptualize QAudit as a randomized algorithm as defined in B.1.1, so that $\text{QAudit} : \mathbb{S}_K \rightarrow \Delta(\mathbb{S}_K)$ accepts a suggested permutation $\mathbf{s}_r^*$, and output a distribution over the set of permutation $\mathbb{S}_K$. Its source of randomness is the tape filled with $\mathbf{a}$'s written by the Auditors. QAudit's payout mechanisms are designed so that in equilibrium, the likelihood that $\mathbf{s}_{\tau_f}^t = \mathbf{s}_r^*$ is high.

Definition 2.5.6. *Permutation State Automata (PSA) with finite states*, or a permutation state machine expressing permutations over K integers is a four-tuple:

$$\text{PSA} = (\mathbb{S}_K, \mathbb{A}_K, \mathbf{s}_o, \mathbb{S}_K^*),$$

so that:

2.5.6.1. the states are permutations, where every $\mathbf{s}_r \in \mathbb{S}_K$ is some permutation of values $1 \dots K$, so that $|\mathbb{S}_K| = K!$.

2.5.6.2. The edge set $\mathbb{A}_K$, where every edge $\mathbf{a} := (\mathbf{s}_r, \mathbf{s}_s) \in \mathbb{A}_K$ is defined as:

$$\mathbf{a} : \mathbb{S}_K \rightarrow \mathbb{S}_K, \quad \mathbf{a}(\mathbf{s}_r) = \mathbf{s}_s,$$

that permutes some $\mathbf{s}_r \in \mathbb{S}_K$ to create $\mathbf{s}_s \in \mathbb{S}_K$. Observe given K integers, there are $K!$ possible permutation actions, and at most $\frac{K(K-1)}{2}$ state transitions, including the self loop. We denote this self loop, or the identity permutation action with $\mathbf{a}_{id}$.

2.5.6.3. Initial state $\mathbf{s}_o \in \mathbb{S}_K$ that represent the initial permutation.

2.5.6.4. Set of terminal states $\mathbb{S}_K^* \subseteq \mathbb{S}_K$ that is "accepted" under some external criteria.

Note that the state and transitions of the PSA form a graph $G(\mathbb{S}_K, \mathbb{A}_K)$, so normal graph concepts apply here. Finally, the PSA reads an external *state transition tape* whose symbol set is composed of transition actions $\mathbf{a} \in \mathbb{A}_K$, and undertakes the transition according to the $\mathbf{a}$ on this tape. The frequency of said symbols that appear on the tape will correspond to weights or transition probability of the PSA, describing a weighted graph $G(\mathbb{S}_K, \mathbb{A}_K, \mathbf{M})$, with $\mathbf{M}$ as the transition probabilities.

Example 2.5.7. (PSA with two states). A PSA representing permutations over two integers is written:

$$\text{PSA} = \left(\mathbb{S}_K = \{\mathfrak{s}_1, \mathfrak{s}_2\} = \{(1, 2), (2, 1)\}, \mathbb{A}_K = \{(\mathfrak{s}_i, \mathfrak{s}_j) : i, j \in \{1, 2\}\}, \mathfrak{s}_o = (1, 2), \mathbb{S}_K^* = \{(2, 1)\} \right).$$

A "correspondence" between wager vector $\mathbf{b}_{i\tau}^t$ and some transition matrix $\mathbf{a}$ is the transition induced by this wager. For example, given initial market inventory $\mathbf{s}_{\tau_0}^t = (10, 5)$, with implied permutation $\mathfrak{s}_{\tau_0}^t = (1, 2)$. If Auditor_i places the buy vector $\mathbf{b}_{i,\tau_0}^t = (0, 7)$. Then the market inventory state is updated to $\mathbf{s}_{\tau_1}^t = (10, 12)$, with associated updated price or probability vector $\boldsymbol{\rho}_{\tau_1}^t = (0.117, 0.881)$, and implied ordering $\mathfrak{s}_{\tau_1}^t = (2, 1)$. Then $\mathbf{b}_{i,\tau_0}^t$ induces a symbol $\mathbf{a}$ on the *transition tape* so that $\mathbf{a}(\mathfrak{s}_1) = \mathfrak{s}_2$. Note depending on the size of the purchase, a wager vector $\mathbf{b}_{i,\tau_0}^t$ may not change the current market permutation at all, i.e the purchase vector $\mathbf{b}_{i,\tau_0}^t = (0, 4)$ corresponds to the identity permutation $\mathbf{a}_{\text{id}}$.

The PSA abstraction is useful in two ways:

1. PSA separates the permutation graph from the weight of the edges, which is determined by the distribution over symbols $\mathbf{a}$ written onto the tape. Normally, one would place them together in a weighted permutation graph with $G(\mathbb{S}_K, \mathbb{A}_K, \mathbf{M})$, where $\mathbf{M}$ is the transition probabilities, or the "walk matrix." Separating the transition probability from the graph makes it clear that it is a property of the wagering behavior recorded on the tape, not an intrinsic feature of the permutation graph itself. Now we can ask questions such as how changes in audit strategy changes the transition probability, and subsequently the final outcome of QAudit_v. From a computational standpoint, PSA is the same as a weighted graph, so all the classical graph concepts still apply.
2. Alternatively, if we conceptualize QAudit_v as a randomized algorithm, then the permutation matrices on the tape are the random bits, and the Auditors are the source of this randomness. We are interested in determining the likelihood that this randomized algorithm will terminate with the correct answer. In other words, if wagering actions induce a random walk over the PSA permutation graph, then we must show that it converges in reasonable number of wagers.

Notably, on the PSA tape, the distribution over permutation actions $\mathbb{A}_K$ is determined by the Auditors' private belief $\mathfrak{s}_r^*$, the constant C , $\mathbf{d}$, and distribution over $\mathbf{a}$ in line 14 of AuditStrat. However, we do not express an explicit formula for the distribution as a function of $C, \mathbf{d}$ and $\mathbf{a}$. Instead, we speak generally about convergence of the random walk governed by these constants, see B.1.4 and B.1.5 for relevant definitions.

Lemma 2.5.8. *In the event where every Auditor_i accepts the prompted ranking, and hold the corresponding private belief $\mathfrak{s}_r^*$ over a ranking of K models from Agent₁, ..., Agent_K. Then the Auditors will collectively find the correct permutation in $\text{poly}(K!)$ number of wagers. In particular, we have the following bound on hitting time $h_{1,K!}$:*

$$(\mathbf{p}_a^{-1} + \mathbf{p}_a^0 + \mathbf{p}_a^1 + \mathbf{p}_a^2 + \dots + \mathbf{p}_a^{K!-2} + \mathbf{p}_a^{K!-1}) \cdot K! \leq h_{1,K!} \leq \frac{\sum_{k=0}^{K!} \mathbf{p}_a^k}{\mathbf{p}_a \cdot \prod_{k=1}^{K!-2} (1 - \mathbf{p}_{k,k})} \cdot K!. \quad (2.4)$$

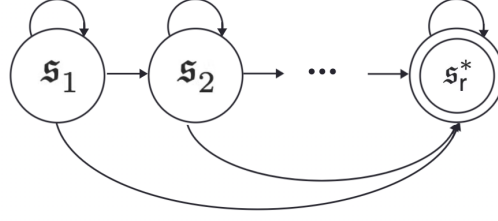


Figure 2.14: When every Auditor accept the prompted permutation s_r^* , the directed permutation graph has a unique sink in s_r^* . So every permutation action $\mathbf{a}$ takes the PSA closer to this sink. Its hitting time is expressed in eqn 2.4.

Proof. See Appendix C.3 ■

Remark 2.5.9. (Realistic wagering behavior) Even though the Auditors converges onto the correct ordering in $\text{poly}(K!)$ number of steps, the $K!$ value is still large. However in reality the LMSR of eqn (2.2) increases the price on each buy signal, as it is proportional to the number of shares outstanding for every Agent_r via: $\rho_r^t \sim \exp(s_r^t)$. The Auditor will weigh this increase in price on a per wager basis, and be less inclined to conservatively increment Agent_r 's position one position at a time. So in reality Auditors will not be crossing the entire span of the PSA graph in $K!$ steps, but make large jumps as his risk profile sees fit.

In the lemma above we assume all the Auditors accept the QAudit prompt and thus share the same private belief over the ranking of players. This is a fair assumption as QAudit publishes the log-likelihood of data for every model $\theta_1^t, \dots, \theta_K^t$ on every stage t , which is a good proxy for final ranking on the test set. The next lemma determines what is likely to occur if the Auditors do not share the same underlying belief. In this case the PSA expresses an undirected graph that does not have a unique sink. Therefore computing hitting time onto any state is vacuous, as the Auditors could always wager to exit this state. Instead we determine the steady state distribution over all states in the PSA graph, and express its gap w.r.t the solution found in the directed PSA. A few more definitions are needed before proceeding with the argument, in particular the idea of representing random walk in matrix form. They are found in B.1.6, B.1.7, and B.1.8.

In the case of the PSA, the probability of each permutation denotes the likelihood that Auditors will converge onto this solution under QAudit wagering. Theorem B.1.8 in the Appendix states that such a steady state must exist and can be found by solving eqn (B.1). Now we reference lemma 2.5.8, and let $\mathbf{M}_r$ be the directed graph's walk matrix. Then following [SW22], we can construct the walk matrix $\mathbf{M}$ of the directed graph of 2.14 with a slight perturbation:

$$\mathbf{M} = (1 - \epsilon)\mathbf{M}_r + \epsilon \frac{\mathbf{e}\mathbf{e}^T}{K!}, \quad \text{where } \epsilon \sim \frac{1}{K!}, \quad \text{and } \tilde{\delta}_r = \mathbf{p}\mathbf{M} \quad \text{for every } \mathbf{p} \in \mathfrak{P}. \quad (2.5)$$

Here $\mathbf{e}$ is the all one's vector of dimension $K! \times 1$; and the uniform random walk matrix $\epsilon \frac{\mathbf{e}\mathbf{e}^T}{K!}$ has dimension $K! \times K!$.¹⁰ This $\mathbf{M}$ is now invertible as there is a $\frac{1}{K!^2}$ probability of a transition from any state onto another. The hitting time onto the sink associated with the PSA parameterized by $\mathbf{M}_r$ is comparable to the mixing time of the undirected graph $\mathbf{M}$. Note we also have the stationary distribution of the PSA, which is approximately the "K-bit" vector $\tilde{\delta}_r = (\delta_1, \dots, \delta_K)$, where $\delta_k = 1$ if $k = r$, and $\delta_k \sim 0$ otherwise. Critically, $\mathbf{M}$ is in fact the Google matrix, where:

1. the Markov transition matrix $\mathbf{M}_r$ describes the internet links.

¹⁰Recall K is the number of Agents participating in an instance of QAudit.

2. $\frac{\mathbf{e}\mathbf{e}^\top}{K!}$ is the "random surfer" behavior that renders $\mathbf{M}$ ergodic.
3. $\tilde{\delta}_r$ is PageRank, or a measure of network centrality of different vertices on the graph.
4. The most likely permutation here $\mathfrak{s}_r^*$ is the top ranked website [BP98].

So we set out to determine how the PSA converges onto the best permutation state $\mathfrak{s}_r^*$ in the setting where many Auditors holding different opinions, and reduced the problem to the sensitivity of network centrality measure on the PSA graph to changes in ϵ .

Lemma 2.5.10. *Suppose there is a block of Auditors who subscribe to the suggested $\mathfrak{s}_r^*$ on stage t , and other Auditors who hold $m - 1$ ($m - 1 < N$) different private beliefs. Let:*

- 2.5.10.1. *the private beliefs $\{\mathfrak{q}_2, \dots, \mathfrak{q}_m\}$ induce a set of permutations $\{\mathfrak{s}_2, \dots, \mathfrak{s}_m\}$;*
- 2.5.10.2. *the proportion of Auditors with each belief $\mathfrak{s}_i$ is $\alpha_r, \dots, \alpha_m$ with $\alpha_r + \sum_{i=2}^m \alpha_i = 1$;*
- 2.5.10.3. *and for now let $\alpha_r \geq \alpha_2 \geq \dots \geq \alpha_m$.*

If we separate each block of Auditors onto their own PSA and use eqn 2.5, then they will induce stationary distributions $\tilde{\delta}_r, \dots, \tilde{\delta}_m$. Now if the Auditors are playing at the same table as is the case in QAudit, and we take the perspective of the largest block of Auditors with belief $\mathfrak{s}_r^$, then the difference between their optimal $\tilde{\delta}_r$ and the stationary distribution is:*

$$\|\mathbf{p} - \tilde{\delta}_r\|_1 \leq \frac{2}{1 - \alpha_r} |\Delta\alpha_r|, \text{ where } \mathbf{p}^\top = \mathbf{p}\mathbf{M}, \mathbf{M} = \sum_i^m \alpha_i \mathbf{M}_i, \text{ and } \|\mathbf{x}\|_1 = \sum_i |x_i|. \quad (2.6)$$

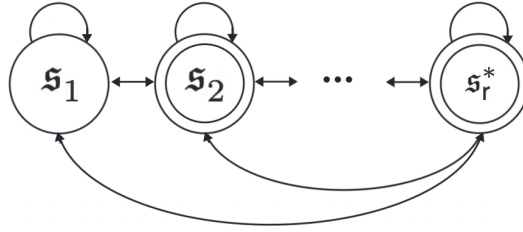


Figure 2.15: When Auditors have different opinions on what the best ranking over Agents are, the permutation graph does not have a unique sink. So we express a stationary distribution over all rankings in $\mathbb{S}_K$ under the walk matrix $\mathbf{M}$ associated with this graph.

Proof. See Appendix C.3. ■

Next, we examine the sensitivity of $\tilde{\delta}$ to changes in $\mathbf{e}$. That is to say instead of uniform preference over K Agents per $\mathbf{e}\mathbf{e}^\top/K!$, suppose we know the Auditors who do not believe in $\mathfrak{s}_r^*$ collectively have some biased preference $\mathbf{e} + \Delta\mathbf{e}$ over the Agents. Then we can build a teleportation matrix with this nonuniform distribution, and by similar reasoning to 2.5.10 find:

$$\|\tilde{\delta} - \mathbf{p}\|_1 \leq (1 - \alpha_r) \|\Delta\mathbf{e}\|_1, \text{ where } \mathbf{p}^\top = \mathbf{p} \left(\alpha_r \mathbf{M}_r + (1 - \alpha_r) \mathbf{e}(\mathbf{e} + \Delta\mathbf{e})^\top \right). \quad (2.7)$$

In the context of PageRank, this vector $\mathbf{e} + \Delta\mathbf{e}$ is the personalization vector. In our context, $\mathbf{e} + \Delta\mathbf{e}$ could be the initial distribution or price vector $\rho_{\tau_0}^t$ computed by QAudit $_{\rho^o}$, which describes how much each model $\mathbf{w}_k^t$ updated from the last stage's model $\mathbf{w}_k^{t-1}$. And, finally we can perturb the value of $\mathbf{M}_r$ itself with $\Delta\mathbf{M}_r$, and find the bound:

$$\|\tilde{\delta} - \mathbf{p}\|_1 \leq \frac{\alpha_r}{1 - \alpha_r} \|\Delta\mathbf{M}_r\|_\infty, \text{ where } \mathbf{p}^\top = \mathbf{p} \left(\alpha_r (\mathbf{M}_r + \Delta\mathbf{M}_r) + (1 - \alpha_r) \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right), \quad (2.8)$$

as seen in [IW06, p. 3]. This perturbation of $\mathbf{M}_r$ correspond to how risk-averse or aggressive the Auditors wager under AuditStrat. Equations 2.7 and 2.8 are the price of anarchy bounds of QAudit. They describe how "bad" does the QAudit solution degrade if the Auditors do not behave.

Now note although lemma 2.5.10 and subsequent bounds convey the l_1 distance between the distribution under $\mathbf{M}_r$ (with belief $\mathfrak{s}_r^*$) and the mixed distributions $\mathbf{M} = \sum_k \alpha_k \mathbf{M}_k$, under m blocks of Auditors. However it does not convey the relative network centrality of permutation $\mathfrak{s}_r^*$ v.s. the other possible rankings. The next lemma considers a very specific case, and examine what will occur if some majority of the Auditors behave.

Lemma 2.5.11. *In the event where the Auditors are partitioned into two blocks, so that some of the Auditors accept the prompted permutation, denoted $\mathfrak{s}_r^*$, and some of which hold a different opinion $\mathfrak{s}_s$. If the proportion of Auditors who hold private belief $\mathfrak{s}_r^*$ is greater than 50%, then the likelihood of finding the state $\mathfrak{s}_r^*$ is greater than landing in some no-man's land state that is neither $\mathfrak{s}_r^*$ nor $\mathfrak{s}_s$. Moreover, under some very mild conditions, the likelihood of finding one of the two accepted solution states may be significantly greater than $\frac{1}{2}$.*

Proof. See Appendix C.3. ■

Remark 2.5.12. (Mixing Time for Random Walk on Symmetric Groups) The PSA construction uses the term "permutation graph" loosely: although the states are labeled with permutation on K integers, this detail does not influence the analysis of the problem. However it is well known that permutations on sets form a symmetric group S_K with $K!$ elements. We can use primitive permutations to generate the group, then study i.e. the mixing time of random walks on this S_K . In particular, the *transposition random walk* describes a process whereby a card is selected from the proverbial deck, and its position is swapped with another card in the same deck. This corresponds to the "slow walk" of lemma 2.5.8. It is known that the mixing time for the transposition random walk on S_K is $K \log K$ to reach uniform randomness, see [Mat20], [Lal18], and [Tan23], and [Nic19] for further discussion.

All in all, we began the subsection with the intuition that Auditors operate in a complex wagering environment. Whereby they would bet in a risk averse manner, relying on the action of other Auditors to move their preferred Agent_r to the front of the pack. Then they would switch to a competitive mode and "double-down" on the winning Agent_r . The above three lemmas shows that under this mixed collaboration-to-competition regime, the Auditors can nonetheless:

1. "hit" the best permutation $\mathfrak{s}_r^*$ in $\text{poly}(K!)$ steps - if every Auditor accepts the recommended ranking $\mathfrak{s}_r^*$.
2. If some Auditors reject $\mathfrak{s}_r^*$, then QAudit_b settles on a steady state distribution over S_K that is of order $|\Delta\alpha_r|$ away from the (near) delta distribution centered in $\mathfrak{s}_r^*$, in $K \log K$ number of wagers.
3. If more than $\frac{1}{2}$ of the Auditors wager according to prompted $\mathfrak{s}_r^*$, then the likelihood of hitting $\mathfrak{s}_r^*$ can be at least $\frac{1}{2}$, if not significantly higher.

Finally, note that even if the Auditors who wager on $\mathfrak{s}_s \neq \mathfrak{s}_r^*$ wins stage t , no Auditor on stage t profits *if* the top models in $\mathfrak{s}_s$ do not out-perform the previous ensemble model at stage $t - 1$. The existence of a hard objective metric during the reward phase prevents Auditors from benefiting by "mooning" a particular model, and using its absolute price as the sole criteria for eventual profit.

2.6 Fundamental Theorems of QAudit

We conclude the discussion on the Auditor-Auditor behavior by lifting concepts of the last section into the terminology of games, so as to:

1. state the existence of a *Correlated Equilibrium* in QAudit for every stage $\mathbf{t}$;
2. determine under what conditions should the **Auditors** change their mind to conform to prevailing market trends.

The correlation setting models strategic interactions of rational agents complemented by a correlation device, that is a mediator who can recommend behavior but cannot enforce it [[Rou14], [FBS20]]. In the classical setting, the mediator exists because the players cannot coordinate to play the best set of actions due to the limitations of their selfish incentives. In the QAudit case, the correct permutation is difficult to identify without side information as the search space is of size $K!$. Thus QAudit recommends the most likely permutation $\mathbf{s}_r^*$ over the K Agents. The Auditors can then wager according to this public ranking.

Fundamental Theorems within Stage $\mathbf{t}$

Definition 2.6.1. (*Auditor Individual Action Space*) Under AuditStrat, Auditors purchase model shares with a sequence of buy vectors via $[\mathbf{b}_{i,\tau_1}^t, \mathbf{b}_{i,\tau_2}^t, \dots]$. Since the PSA interfaces with the Auditors through the tape, let the induced sequence of permutation functions $\mathbf{qauditSeq}_i = [\mathbf{a}_{i,\tau_1}^t, \mathbf{a}_{i,\tau_2}^t, \dots, \mathbf{a}_{i,\tau_t}^t]$ written onto this tape be their actions. That is to say the *action space* is:

$$\mathbb{A}_K^{\tau_t} = \bigtimes_{\tau}^{\tau_t} (\mathbb{A}_K)_{\tau},$$

the τ_t -fold Cartesian product of permutation functions on K integers representing the K Agents. If the Auditor does not play at some time increment τ , it is convenient to say that he places the identity permutation $\mathbf{a}_{id}$ onto the tape instead. Next, define a distribution Ω_i with support on $\mathbb{A}_K^{\tau_t}$, so that some Auditor $_i$ may sample a sequence of actions from $\mathbb{A}_K^{\tau_t}$ according to this Ω_i , written:

$$\mathbf{qauditSeq}_i \stackrel{\text{iid}}{\sim} \Omega_i, \quad \mathbf{qauditSeq}_i \in \mathbb{A}_K^{\tau_t}, \quad \text{where } \mathbf{qauditSeq}_i := [\mathbf{a}_{i,\tau_1}^t, \dots, \mathbf{a}_{i,\tau_t}^t].$$

And observe we use the notation $\mathbf{qauditSeq}_i$ to denote a sequence of permutation functions sampled by Auditor $_i$. If every Auditor $_i$ samples just so and write their permutation symbols onto the PSA tape, then clearly we have $\tau_t \times N$ symbols on the tape over N Auditors at market close. Denote this with:

$$\mathbf{qauditSeq}_{1:N} \stackrel{\text{iid}}{\sim} \Omega, \quad \text{where } \mathbf{qauditSeq}_{1:N} := [\mathbf{a}_{1,\tau_1}^t, \dots, \mathbf{a}_{1,\tau_t}^t, \dots, \mathbf{a}_{N,\tau_1}^t, \dots, \mathbf{a}_{N,\tau_t}^t].$$

And similarly we can write $\mathbf{qauditSeq}_{1:N,-i}$ as the sequence modulo the symbols from Auditor $_i$, this is every player's strategy except Auditor $_i$, so that:

$$\mathbf{qauditSeq}_{1:N} := \mathbf{qauditSeq}_i : \mathbf{qauditSeq}_{1:N,-i}, \quad \text{where } \mathbf{qauditSeq}_{1:N} \in (\mathbb{A}_K^{\tau_t})^N = \bigtimes_i^N (\mathbb{A}_K^{\tau_t})_i.$$

This is one example sequence of the symbols on the tape, highlighting Auditor $_i$'s moves.

Definition 2.6.2. (*Auditor's Collective Action Space*) There exists a natural relationship between $\mathbf{qauditSeq}_{1:N}$ and the walk matrix: the joint distribution $\Omega = \bigtimes_i^N \Omega_i$ over the set of symbols $(\mathbb{A}_K^{\tau_t})^N$ on the tape induces a set of random processes over the PSA. This is expressed as a convex cone of walk matrices:

$$\mathfrak{M}_K = \left\{ \mathbf{M} : \mathbf{M} = \sum_i^m \alpha_i \mathbf{M}_i \text{ where } m \geq 1, \sum_i^m \alpha_i = 1, \mathbf{M}_i \text{ row-stochastic} \right\}.$$

$\mathfrak{M}_K$ is the space of all row-stochastic walk matrices of dimension $K \times K$ closed under convex combination. For example in lemma 2.5.10, we had $\mathbf{M}_r, \mathbf{M} \in \mathfrak{M}_K$. And similarly the uniform walk matrix $\mathbf{ee}^\top / K!$ is also in $\mathfrak{M}_K$, corresponding to a set of Auditors who wagers randomly.

Note the mapping between $(\mathbb{A}_K^T)^N$ and $\mathfrak{M}_K$ is onto, though not necessarily one-to-one. So depending on the proof, it may be convenient to use some $\mathbf{M}_r \in \mathfrak{M}_K$ as the Auditors' "individual action in the context of the collective." This is a way of codifying "herd behavior" that simplifies some argument greatly. Whereas choosing $\mathbf{a}$ is akin to investing in an ad-hoc manner, choosing $\mathbf{M}$ is akin to "buying into an entire future" - the very definition of a lemming.¹¹ Playing with $\mathfrak{M}_K$ entails a hierarchical sampling process, whereby at any point in time τ within stage $\mathbf{t}$, the Auditor can choose any one of $\mathbf{M}_i \in \mathfrak{M}_K$, and then choose some action $\mathbf{a}$ based on $\mathbf{M}_i$ and place the symbol on the PSA tape. Critically, the game state is no longer some permutation $\mathbf{s}$, but rather probability distribution $\mathbf{p}$ over all $K!$ permutations. One can then step through the game states by multiplying any state by $\mathbf{M}_r$ via $\mathbf{p}_{\tau_1}^\top = \mathbf{p}_{\tau_0} \mathbf{M}_r$, thus all prior arguments carry over.

Definition 2.6.3. (*Auditor Utility Function*) We can select any suitable permutation distance function denoted with $x \parallel y$, i.e. Kendall-Tau or Cayley distance. And define the Auditor_i's utility function with:

$$\mathbf{u}_i : \mathbb{A}_K^T \times \mathbb{D} \rightarrow \mathbb{R} \quad \mathbf{u}_i(\text{qauditSeq}_{i:N}, \mathbb{D}_{\text{val}}) = \mathbb{E}[\mathbf{v}_i^\dagger | \mathbb{D}_{\text{val}}] \cdot \frac{1}{1 + (\mathbf{s}_i^\dagger \parallel \mathbf{s}_i^\dagger)},$$

where $\mathbf{s}_i^{*\dagger}$ is the QAudit_v-suggested permutation computed over the validation set $\mathbb{D}_{\text{val}}$ in stage $\mathbf{t}$. Observe this utility function is a product for the *expected reward* $\mathbf{v}_i^\dagger$ metered out by QAudit_v under data distribution, and the permutation distance between the Auditor's ideal permutation $\mathbf{s}_i^\dagger$, and the actual permutation at market close $\mathbf{s}_i^\dagger$. In particular, by maximizing $\mathbf{u}_i$, Auditor_i increases the probability that test-data likelihood under ensemble model in stage $\mathbf{t}$ outperforms that of $\mathbf{t} - 1$, and thus can claim reward from wagers collected in stage $\mathbf{t} + 1$.

Theorem 2.6.4. (*QAudit Protocol Consensus Theorem*) *For every wager stage $\mathbf{t}$, there exists a Correlated Equilibrium (CE) (see definition B.2.3) such that the market consensus at market close reflects the true permutation $\mathbf{s}_r^{*\dagger}$ in expectation, provided the permutation is prompted by QAudit_v at market open in every $\mathbf{t}$.*

Proof. Since this is an existence proof, we submit the randomized audit program AuditStrat($\mathbf{s}_r^*, C, d$) to witness the existence of a viable strategy. Then if we denote qauditSeq_i as a strategy for Auditor_i if he accepts the protocol-prompted $\mathbf{s}_r^{*\dagger}$, and let $\text{qauditSeq}'_i$ be the wagering strategy if he rejects the prompt. Then for every Auditor_i, we can say the following:

$$\begin{aligned} \mathbb{E}_{\mathbb{D} \sim \mathfrak{D}} \left[\mathbb{E}_{\text{qauditSeq}_{1:N} \sim \Omega} \left[\mathbf{u}_i(\text{qauditSeq}_i : \text{qauditSeq}_{1:N, -i}, \mathbb{D}) \middle| \mathbf{s}_r^{*\dagger} \right] \right] &\geq \\ \mathbb{E}_{\text{qauditSeq}_{1:N} \sim \Omega} \left[\mathbf{u}_i(\text{qauditSeq}'_i : \text{qauditSeq}_{1:N, -i}, \mathbb{D}) \middle| \mathbf{s}_r^{*\dagger} \right] &. \end{aligned} \quad (2.9)$$

This is our form of the correlated equilibrium condition, whereby:

2.6.4.1. the outer expectation $\mathbb{E}_{\mathbb{D} \sim \mathfrak{D}}[\cdot]$ is taken w.r.t all samples of test/validation set $\mathbb{D}$ from data distribution $\mathfrak{D}$.

2.6.4.2. The statement before the inequality defines the expected utility value if the Auditor plays according to QAudit-suggested permutation $\mathbf{s}_r^{*\dagger}$.

2.6.4.3. The statement after the inequality defines the expected utility value if the Auditor_i does not play according to $\mathbf{s}_r^{*\dagger}$.

2.6.4.4. both statements are conditioned on $\mathbf{s}_r^{*\dagger}$, the QAudit-suggested permutation.

This statement 2.9 follows from the lemmas of the previous section:

¹¹So the euphemism for lemming is a random walker.

2.6.4.1. by $\text{QAudit}_{\mathbb{D}}$ we know that the test set $\mathbb{D}_{\text{test}}$ and validation set $\mathbb{D}_{\text{val}}$ are drawn from the same distribution, so validation set permutation $\mathfrak{s}_r^{*t}$ is a fair representation of the true permutation $\mathfrak{s}_r^{*t}$ under test set.

2.6.4.2. If every Auditor_i plays according to $\mathfrak{s}_r^{*t}$, then by lemma 2.5.8 there is a random walk s.t. the **Agents** can arrive at the target in polynomial hitting time. By setting the wagering stage to be greater than the hitting time, the PSA will converge on the correct permutation state at market close when **Auditors** wager under AuditStrat .

2.6.4.3. In the event where some portion $(1 - \alpha_r)$ of the **Auditors** reject the suggested permutation $\mathfrak{s}_r^{*t}$ and wager according to $(1 - \alpha_r)(\mathbf{M}_2 + \dots + \mathbf{M}_m)$, then by lemma 2.5.10 the near delta distribution δ centered at $\mathfrak{s}_r^{*t}$ under $\mathbf{M}_r$ degrades according to $\frac{2}{1-\alpha_r}|\Delta\alpha_r|$.

2.6.4.4. In fact, regardless of how some $(1 - \alpha_r)$ portion of **Auditors** who do not believe in $\mathfrak{s}_r^{*t}$ wager, eqn 2.7 shows that the gap between the full consensus case of 2.5.8 and the degraded case is of order $(1 - \alpha_r)\|\Delta\mathbf{e}\|_1$. When α_r is large, this difference is small.

2.6.4.5. Specifically, by lemma 2.5.11, we know that as long as more than $\frac{1}{2}$ of the **Auditors** play according to $\text{qauditSeq}_{1:N}$, then the likelihood of finding $\mathfrak{s}_r^{*t}$ may be far greater than $\frac{1}{2}$, depending on how aggressively the **Auditors** wager.

The three quantities of $|\Delta\alpha_r|$, $\|\Delta\mathbf{e}\|_1$, and $\|\Delta\mathbf{M}_r\|_\infty$ measure the gap between the best possible outcome under data expectation, and the CE outcome. All in all, this proves the existence and stability of correlated equilibrium as defined. ■

Following an earlier comment, it is not a given that the existence of objective metric at reward time would necessarily deter "mooning." The reward phase takes place after the audit phase has ended, and the audit phase alone may last weeks. Concurrently, **Auditors** can sell their positions on mint_k^t in the secondary market. Thus we still need the analysis of theorem 2.6.4 to show that QAudit consensus mechanism deters the **Auditors** from manipulating the market irresponsibly. The QAudit consensus theorem states that unless the **Auditors** in the primary market are 1) "very sure" some other permutation is the correct choice, and/or 2) can convince the secondary buyers that they are sure, then the **Auditors** are better off wagering according to the suggested ranking $\mathfrak{s}_r^{*t}$.

Remark 2.6.5. (Proof Methodology). In the Auditor-Auditor proofs, we assume the existence of some *Homo Economicus* who wagers according to rational self interest, not emotions or loyalty. Normally, analysis that conform to neo-classical orthodoxy is not to be taken seriously. However in the QAudit case there is some justification. It is well known that most trades in DeFi are done by automated algorithms. Thus AuditStrat is not a fictitious rational Auditor, but rather a very real auditing algorithm. In fact, part of Metheus's go-to-market strategy is attracting DeFi "whales" and connecting them with these auditing strategies. This allows us to quickly build market depth and influence market variance. This last point is critical: remember the winning **Agents** are paid by the losing **Auditors**.

The remark above underlines the fact that if the winning **Agents** are to be paid, then there must be variance in how **Auditors** wager. Variance may arise spontaneously as **Auditors** disagree, or it can be constructed by the protocol itself. For example, QAudit may publish many possible final rankings $\mathfrak{s}_1^*, \dots, \mathfrak{s}_m^*$, done on m different sets of validation data of slightly different distributions. And then prompt the **Auditors** with this set, so as to seed different fractions and promote market diversity. Now in every analysis thus far, we assume every Auditor_i choose to accept or reject the market-suggested ranking $\mathfrak{s}_r^*$ at market open within every stage t . And then plays strategy $\text{AuditStrat}(\mathfrak{s}_r^*, C, d)$ without ever updating their private belief w.r.t market condition. In reality people do change their mind, and this can be to our benefit. The next theorem relaxes the non-updating prior constraint, so that the **Auditors** may update their private beliefs per market state. In this case, "momentum trading" helps the market to arrive at one of the possible permutation candidates, with $m - 1$ losers whose wagers will be distributed to the winning **Agents**.

Theorem 2.6.6. (QAudit Social Consensus Theorem) Suppose QAudit seeds the market with candidate final rankings $\mathfrak{s}_1^*, \dots, \mathfrak{s}_m^*$ on the test set in every stage $\mathfrak{t}$, so that the Auditors may play according to any one of the mixed strategies:

$$\mathfrak{M}'_{\mathbf{K}} = \left\{ \text{AuditStrat}(\mathfrak{s}_1^*, C, d), \dots, \text{AuditStrat}(\mathfrak{s}_m^*, C, d) \right\} \implies \left\{ \mathbf{M}_1, \dots, \mathbf{M}_m \right\}.$$

And note that $\mathfrak{M}'_{\mathbf{K}} \subset \mathfrak{M}_{\mathbf{K}}$, and every mixed strategy $\text{AuditStrat}(\mathfrak{s}_s^*, C, d)$ correspond to a unique walk matrix $\mathbf{M}_s$. Then in every stage $\mathfrak{t}$, there exists a point in time $\tau^* \in \{\tau_1, \dots, \tau_{\mathfrak{t}}\}$ s.t. it is rational for every Auditor to switch to a single mixed strategy $\text{AuditStrat}(\mathfrak{s}_r^*, C, d)$, with corresponding game dynamic $\mathbf{M}_r$, favoring one ranking $\mathfrak{s}_r^*$ over all others.

Proof. We use the collective actions space $\mathfrak{M}_{\mathbf{K}}$ of definition 2.6.2. Now the QAudit game can be expanded as a game tree, where the inner/outer leaves are $\mathbf{p}$, or distribution over the set $\mathbb{S}_{\mathbf{K}}$ with $K!$ possible rankings, and the edges are walk matrices $\mathbf{M}_r \in \mathfrak{M}'_{\mathbf{K}}$. That is to say, at any vertex on the tree, the Auditor $_i$ steps to the next vertex using some mixed strategy according to $\text{AuditStrat}(\mathfrak{s}_s^*, C, d) \implies \mathbf{M}_s$, not a weighted sum as was the case in eqn C.4. See [Tad13, p. 91] for more on game trees. The tree is of height $\tau_{\mathfrak{t}}$, and at every inner leaf there are at least m possible expansions or branches, corresponding to the $\geq m$ walk matrices in $\mathfrak{M}_{\mathbf{K}}$. There are at least $m^{\tau_{\mathfrak{t}}}$ possible paths down the tree. Any path down the tree then is of form:

$$\mathbf{p}_{\tau_f}^{\top} = \mathbf{p}_{\tau_o} \left(\underbrace{\mathbf{M}_{1,s} \circ \dots \circ \mathbf{M}_{\tau^*,j}}_{\text{exploration}} \circ \underbrace{\mathbf{M}_{\tau^*+1,r} \circ \dots \circ \mathbf{M}_{\tau_{\mathfrak{t}},r}}_{\text{exploitation}} \right), \text{ where } s, r, j \in \{1 \dots m\}.$$

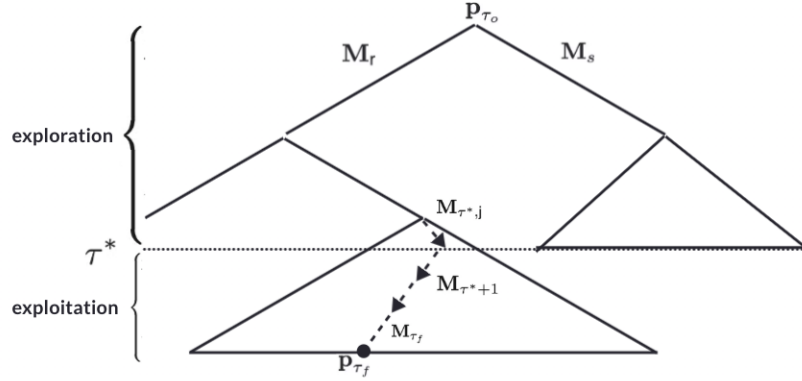


Figure 2.16: In this formulation of QAudit equilibrium condition, the Agents are prompted with many possible permutations and hold heterogeneous beliefs. They are collectively doing m -ary tree search to find the most profitable stationary distribution (binary tree search shown for clarity). Theorem 2.6.6 shows that rational Agents will *explore* any one of the suggested options, playing any $\text{AuditStrat } \mathbf{M}_r, \mathbf{M}_s \in \mathfrak{M}_{\mathbf{K}}$. But beyond $\mathfrak{t}^*$, they will converge onto a consensus, and *exploit* it so that the stationary distribution at market close $\mathbf{p}_{\tau_{\mathfrak{t}}}$ is centered on a unique ranking.

Now recall remark 2.5.12, stating the mixing time of a permutation random walk is of order $K \log(K)$. So at some tree depth $\tau^* < \tau_{\mathfrak{t}}$, the Auditors must make a decision: 1) wager with their existing mixed strategy $\text{AuditStrat}(\mathfrak{s}_s^*, C, d)$, or 2) follow the herd so as to maximize the likelihood that the random walk converges to any $\mathfrak{s}_r^* \in \{\mathfrak{s}_1^*, \dots, \mathfrak{s}_m^*\}$. We can reason as follows:

2.6.6.1. If every Auditor stay with their existing strategy, then under the combined permutation matrix $\mathbf{M}^- = \alpha_1 \mathbf{M}_1 + \dots + \alpha_m \mathbf{M}_m$, the steady state distribution under $\mathbf{M}^-$ will be of order $2|\Delta\alpha_r|$ away from the near-delta distribution $\tilde{\delta}_r$ for every $r = 1 \dots m$ as seen in lemma 2.5.10. Moreover, by 2.5.11 we know that if no block of Auditors have a simple majority, then no permutation $\mathfrak{s}_r^*$ may dominate the others in likelihood.

2.6.6.2. Similar to definition 2.6.3, we can formalize this idea by assigning a utility value $\mathbf{u}_i$ to every terminal leaf in the space of terminal m^{τ_f} outcomes via:

$$\mathbf{u}_i : \mathfrak{P} \rightarrow \mathbb{R}, \text{ where } \mathbf{u}_i(\mathbf{p}) = v, v \in \mathbb{R}, \text{ and } \mathfrak{P} = \{\mathbf{p}(\mathfrak{s}) : \mathfrak{s} \in \mathbb{S}_K\}.$$

Now suppose for some Auditor _{i} ($i = 1 \dots N$), we have an ordering over preferences s.t.:

$$\underbrace{\mathbf{u}_i(\tilde{\delta}_1^*) \geq \dots \geq \mathbf{u}_i(\tilde{\delta}_m^*)}_{\text{preference over prompted permutations}} > \underbrace{\mathbf{u}_i(\mathbf{p}_{m+1}) \geq \dots \geq \mathbf{u}_i(\mathbf{p}_{m^{\tau_f}})}_{\text{preference over unprompted permutations}},$$

$$\text{with } \mathbf{u}_i(\tilde{\delta}_j) > 0, \text{ and } \frac{N-1}{N+1} < \frac{\mathbf{u}_i(\tilde{\delta}_j)}{\mathbf{u}_i(\tilde{\delta}_l)} < \frac{N+1}{N-1} \text{ for every } j, l \in 1 \dots m.$$

Here $\mathbf{p}_{m+l}$ is the distribution, possibly uniform, over permutations not prompted by QAudit; while $\tilde{\delta}_j^*$ is the near-delta distribution centered on prompted permutation $\mathfrak{s}_j^*$. Notably:

- we assume the Auditor strictly prefers outcomes that favor the prompted permutations over the unprompted ones. That is $\mathbf{u}_i(\tilde{\delta}_r^*) > \mathbf{u}_i(\mathbf{p}_j)$ for every $r = 1 \dots m$, and $j = m+1, \dots, m^{\tau_f}$.
- We place a constraint on the relative value of preferences over the prompted permutations. Thus modeling a set of Auditors who are not sure which of the suggested permutation is best - a reasonable assumption given 1) how close the validation sets are to each other, and 2) the assumed nature of DeFi users.

Now this Auditor _{i} may *heuristically* assign the likelihood of achieving any $\tilde{\delta}_j^*$ by the proportion of Auditors who have wagered on it, namely α_j . His expected value for wagering on any outcome is then:

$$\mathbf{E}[\mathbf{u}_i(\tilde{\delta}_j)] = \alpha_j \mathbf{u}_i(\tilde{\delta}_j).$$

In fact, an expected value may be assigned to every one of the m^{τ_f} terminal leaves on the game tree. Now suppose this Auditor _{i} is part of block α_s believing in $\mathfrak{s}_s^*$, and he observes that there is some block α_r betting on $\mathfrak{s}_r^*$, with $\alpha_r > \alpha_s$. Then he may defect if *after* he switches, he achieves $\mathbf{E}[\mathbf{u}_i(\tilde{\delta}_r)] > \mathbf{E}[\mathbf{u}_i(\tilde{\delta}_s)]$. So his hurdle condition for defecting is:

$$\left(\alpha_r + \frac{1}{N}\right) \mathbf{u}_i(\tilde{\delta}_r) > \left(\alpha_s - \frac{1}{N}\right) \mathbf{u}_i(\tilde{\delta}_s) \implies \frac{\alpha_r N + 1}{\alpha_s N - 1} > \frac{\mathbf{u}_i(\tilde{\delta}_s)}{\mathbf{u}_i(\tilde{\delta}_r)}. \quad (2.10)$$

The last statement is true since $\frac{\alpha_r N + 1}{\alpha_s N - 1} > \frac{N+1}{N-1}$ for every $0 < \alpha_r, \alpha_s < 1$. Furthermore, note we have $\frac{\alpha_r N + 1}{\alpha_s N - 1} > \frac{\alpha_j N + 1}{\alpha_s N - 1}$ if $\alpha_r > \alpha_j$. Reasoning like so for every $j \in \{1 \dots m\} \setminus \{r, s\}$, this Auditor _{i} must defect from his $\mathfrak{s}_s^*$ towards the $\mathfrak{s}_r^*$ with the highest α_r , and wager with $\mathbf{M}_r$ from this point thence.

2.6.6.3. All in all, we see that for every time step after τ^* , there exists at least one Auditor _{i} who switches to the highest α_r . Now by setting the threshold time with:

$$\tau_f - N - K \log(K) \leq \tau^* < \tau_f - 1 - K \log(K),$$

then there is enough time for every Auditor _{i} to switch to belief $\mathfrak{s}_r^*$. And the chain still has time to mix and settle on $\tilde{\delta}_r^*$. That is to say after time step $\tau_f - K \log(K)$, we can reuse the argument of eqn C.4 to construct the weighted walk matrix:

$$\mathbf{M}^+ = (1 - \epsilon) \mathbf{M}_r + \epsilon \frac{\mathbf{e} \mathbf{e}^T}{K!}, \text{ with } \epsilon \sim 0,$$

converging to $\tilde{\delta}_r^*$ in $K \log(K)$ time. Furthermore, since this is a game of finite horizon, this sub-game equilibrium is also the game equilibrium. ■

Some final observations:

1. Defecting to the market simple majority at time τ^* is an example of a sub-game equilibrium [Tad13, p. 113]. In QAudit_v prompted by m permutations $\mathfrak{s}_1^*, \dots, \mathfrak{s}_m^*$, with Auditors that can defect to any fraction, there are m sub-game equilibria.
2. In the event where for some Auditor_i his preference is such that $\frac{u_i(\tilde{\delta}_s)}{u_i(\tilde{\delta}_r)} > \frac{N+1}{N-1}$, then this Auditor_i may not defect from his choice of $\mathfrak{s}_s^*$ even in the face of another coalition. So the Auditor_i has to make a decision, either:
 - a) bet with conviction and purchase enough shares to move the market towards $\mathfrak{s}_s^*$, and force other coalitions to respond. This outcome is OK from the perspective of QAudit, as long as $\mathfrak{s}_s^*$ is one of the suggested permutations.
 - b) Sit out the market for the remainder of stage t . We can encode his portion of the bets with the identity matrix $\mathbf{I}$, stating that he does not change the market state. If sufficient number of Agents sit out, then we have a walk matrix of form:

$$\mathbf{M} = (1 - \epsilon - \epsilon')\mathbf{M}_r + \epsilon'\mathbf{I} + \epsilon \frac{\mathbf{e}\mathbf{e}^\top}{K!},$$

which also converges onto $\tilde{\delta}_r$ albeit with a slower mixing time due to $\epsilon'\mathbf{I}$.

3. The proof above assumes $N + K \log(K) < \tau_f$, in the event where τ_f is less than the hurdle, then the Auditors may not have enough time to defect and hit the final permutation $\mathfrak{s}_r^*$. In reality, τ_f expresses the total number of trades within some stage t , so this implies the trading volume in any stage must be greater than $N + K \log(K)$.
4. All the Auditors' wagers before τ^* essentially does not matter, as the Markov chain can converge onto $\tilde{\delta}_r^*$ from any initial distribution $\mathbf{p}_{-}$ at time τ^* . This "burn in" period would ideally feature a diversity of wagers over m possible permutations, the losing wagers associated with the $m - 1$ losing permutations will accrue to the winning Agents at stage t . This benefit Agent retention.
5. Meanwhile, the competitive phase does not have to begin when some specific $\mathfrak{s}_r^*$ is hit, but can instead begin at τ^* when Auditors "sense" the market is about to converge onto one final consensus. A long competitive phase leads to a larger wagering pool that is rewarded to winning Auditors in stage $t - 1$. This promotes Auditor retention.

This concludes the discussions on the fundamental theorems. However, it is important to state that we have just scratched the surface of what can be analyzed within the PSA computational context. In the last two sections, we provided one example of an audit strategy, and use it to argue that the PSA converges onto the best solution. In general, it is far more interesting to construct many variations of audit strategies, game them against each other on the PSA tape, and study the convergence behavior of the induced weighted PSA graph. This thought process will refine the consensus mechanism of the QAudit market maker.

Fundamental Theorem across Stages T

In the last few sections, we proved that:

1. Agent should report the best model they have on hand, subject to some resource considerations.

2. When Auditors act on a strictly individual basis, or the first Auditor who wagers in any given stage t , should wager on the best model submitted in that stage.
3. Auditors may alternate between competing to cooperating with each other. When deciding on what to wager w.r.t market consensus, theorems 2.6.4 and 2.6.6 states that it is in the Auditor's self interest to:
 - a) wager according to the protocol's prompted permutation if one suggestion is given.
 - b) If QAudit suggests more than one permutation, then Auditors should all defect to the market simple majority.

However, all the proofs are equilibrium statements within some stage t . This section extends the discussion to the game over all stages $t = 1 \dots T + 1$, and argue that it is in the interest of the Auditors to continue to wager across all $T + 1$ stages. This is no mere academic point: this section is fundamentally about Auditor retention.

Now recall QAudit's staggered sequential parimutuel incentive structure as seen in table 2.5: it is a zero sum game until the last stage T when the bounty is paid out. That is to say one Auditor's gain on stage t is another Auditor's loss in stage $t + 1$. Clearly, no Auditor would wager in every stage, unless the snake eats its own tail. Instead they would naturally fracture into two cliques: call them Odds and Evens Auditors. The Odds wager on stages $t = 1, 3, 5, \dots$, and the Evens wager on stages $t = 2, 4, 6, \dots$. The immediate question then is: how stable is this game. In fact the problems are more numerous than this. Over horizon $T + 1$ the relevant problems are:

1. The tit-for-tat dynamic between the Odds and Evens cliques may be unstable. As long as both cliques continue to wager, the game continues. However, if just one fraction refuse to play on any stage t , then the other fraction will be burnt as their wagers on stage $t - 1$ would fail to return any revenue. Thus they too would exit the market. They may return one stage before the round T to claim the bounty, but there would be a large gap where the market volume is nil. So the question is how to immediately recover market volume after this defection point.
2. Since total stage T is known a-priori, there exists one clique that has one more stage in the game than the other. For example if T is even, then the Odds has the last stage $T + 1$, and may claim the bounty whose value is known a-priori. Thus there would be a bias in distribution of Agents towards the odd stages, which ironically make the odd stages less profitable. A similar story exists for if T is odd, then there will be more trading volume in the Evens clique.
3. The discriminative models $\mathbf{w}^t$ eventually saturate at some stage t , as their objective functions are convex with unique minimum (see E.3.3). Beyond stage t , the Auditors can no longer profit even if they wager correctly. The Auditors may interpret the market as "tapped out," and leave.
4. The size of the bounty is fixed at the outset. So if the price ascending function v_ρ of protocol QAudit₀ increases its price monotonically over the course of the market, then at some stage t all Agent would refuse to wager as the price floor of the shares are higher than the bounty value. This is an important point: the bounty value is an implicit price ceiling on model shares.

Remark 2.6.7. (The Odds-Evens Dynamic) has precedent in real life. Funds take turns marking up each other's portfolio companies, sending their valuations to the moon. This quid-pro-quo arrangement is profitable, until it isn't. Without someone to pass the inflated hot potato onto, the whole market craters - presently a very tangible situation [Hil24]. In theory, markets have "natural" correction cycles when tides recede and people are shaken out. QAudit preempts this nature with *regime change*.

Definition 2.6.8. (*Regime Change*) is a preemptive hard market reset at some randomly selected stage t^* for some instance of QAudit. The protocol:

2.6.8.1. Resets all discriminative models $\{\mathbf{w}_1^{t^*}, \dots, \mathbf{w}_K^{t^*}\}$, whereby the weights are randomly reinitialized. Recall QAudit judges the quality improvement of $\mathbf{w}_k^{t^*}$ only, so the Agent-submitted generative model $\theta_k^{t^*+1}$ are not reset. In fact, this is a protocol side operation that requires no actions from the Agents at all.

2.6.8.2. Reset Dutch auction parameter $v_\rho^{t^*}$, so that the price floor is once again close to zero. After stage t^* , the v_ρ^t values will increase as before.

Regime change is an automated albeit aggressive policy lever. It solves the aforementioned four problems simultaneously:

1. now the prices of model shares do not increase monotonically beyond the implied price ceiling set by the bounty paid by the PA. Every reset draws new Auditors into the game as the price of mint shares is favorable again.
2. Odds Auditors can no longer benefit if total number of stages T is even. This is because the reset point t^* could be an odd stage itself, rendering the remaining game stages $T - t^*$ an even number. And similarly for Evens if T is odd, and the reset stage is even, making $T - t^*$ odd. Note there is at least one reset per game, so that whether the value of the remaining rounds $T - t^*$ is eventually even or odd, is itself a random variable.
3. The reinitialized models may see dramatic improvement once again as we are training from a lower quality base. This draws new and existing Auditors back into the game.
4. However the Auditors playing at stage $t^* - 1$ may very well be sacrificed, since the market price will be cratering at stage t^* . However the Auditors in stage $t^* + 1$ will essentially receive "free money" at very little cost, as the cost of models $\mathbf{w}_k^{t^*+1}$ will be cheap, while *any* combination of models $\mathbf{w}_k^{t^*+1}$ at stage $t^* + 1$ will defeat the ensemble of reset models at t^* .
5. Critically, as regime change happens randomly, Auditors cannot plan ahead for the change. If they could, no one would wager on round $t^* - 1$. Naturally this negative future expectation would propagate back, and kill all demand for every stage $t \leq t^* - 1$. Randomness is essential.

Now we characterize the min-max game amongst Auditors across T stages. In particular, we will use *regime risk factor* γ to conceptualize the rewards.

Definition 2.6.9. (*Auditor Reward under Regime Change Risk*). Given the last reset occurred at stage t^* (note $t_o = t^*$), the present value of reward to Auditor _{i} under regime change risk factor γ is:

$$\begin{aligned} \text{expected reward} &:= v_t (1 + \gamma)^{-(t-t^*)}, \\ \text{where } v_t &\stackrel{\text{iid}}{\sim} \mathcal{P}[0, s \cdot v_{pa}], \quad s \geq 0, \quad \text{and } \gamma \in (0, 1). \end{aligned} \quad (2.11)$$

There are two values that contribute to reward:

2.6.9.1. (QAudit *reward* v_t). Referencing line 20-21 of QAudit _{v} , v_t is a function of:

- a) which winning models $r = 1 \dots R$ the Auditor _{i} wagered on.
- b) Whether said model ensemble can outperform the previous stage $t - 1$.
- c) Auditor _{i} 's share $\varepsilon_{i,r}^t$ of reward $v_{i,r}^t$ for model $\mathbf{w}_r^t$, under QAudit _{v} , as a function of wagers in stage $t + 1$.

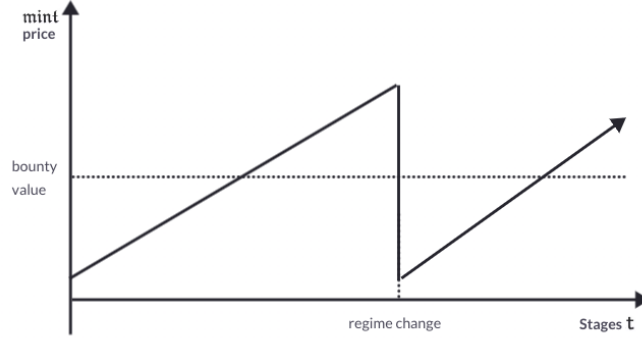


Figure 2.17: Regime change is one of the principle policy levers of QAudit, where all the discriminative models $\mathbf{w}_k$'s are reset, as are the prices of model share mints. This 1) prevents the prices of the model shares from growing beyond the PA-deposited bounty; 2) prevents the discriminative models from saturating; and 3) prevent the Odds or Evens cliques from gaining an advantage because they have one more round.

- d) The value of the Dutch auction factor at next stage v_p^{t+1} .

So v_t is some function of $\sum_{r=1}^R \varepsilon_{i,r}^t v_{i,r}^t$, denominated in *mint*. This complex outcome is then modeled as a r.v. drawn from the interval $[0, s \cdot v_{pa}]$, so that its upper bound $s \cdot v_{pa}$ is a constant multiple of the fixed QAudit bounty.

2.6.9.2. (*Regime risk* ρ) typically called the *discount factor*, it has three interpretations:

- in the case where the game has infinitely number of stages, ρ is a numerical convenience. The present value of an infinite game is $\sum_{t=1}^{\infty} \rho v_t$. So ρ forces this value to converge.
- It models interest rate or time value of money, so that value ρv_t in the future is worth less than value today.
- It models the likelihood that a game will end, so i.e. if $\rho = 0.5$ at some stage t , then the game has a 50% chance of terminating at stage t [Tad13, p. 127].

We use a new interpretation: the likelihood that the game will reset at any stage t , thereby rendering the wagers on stage $t - 1$ worthless. This is a softer version of interpretation (c). That is to say, the probability of regime change at any stage t is:

$$p(\text{regime change at } t) = 1 - (1 + \rho)^{-t}. \quad (2.12)$$

In practice, QAudit will observe the rate of improvement of models $\{\mathbf{w}_1^t, \dots, \mathbf{w}_K^t\}$, and determine if it has slowed down. Thus the value of ρ should differ on every stage t . However in this analysis, we can select some constant value, ie $\rho = 0.05$, stating that in every stage the models improve by 5% on average. So that after $(t - t^*) = 61$ stages, the likelihood that QAudit will regime change is heuristically 95% as we have: $(1 + 0.05)^{-61} = 0.05$. Consequentially, any potential earnings at $t = 61$ is discounted by 95%. Now we can state the expected value for any Auditor $_i$ over some interval between regime changes, given he enters the game on either Evens or Odds.

Theorem 2.6.10. (QAudit is not a Loser's Game) *A loser's game is one whereby the more you play, the more you lose. This lemma states that given regime change occur at some stage t^* , and the Auditor wagers v_{t^*} at that stage. Then provided:*

2.6.10.1. *the per stage earnings Δv of some Auditor_{*i*} is greater than the cumulative regime risk factor up to t^* s.t.:*

$$\Delta v \geq \frac{8v_{t^*}}{\sum_{t=2}^{t^*} (1+r)^{t^*-t}}. \quad (2.13)$$

2.6.10.2. *The Auditor enters the Evens and Odds with equal likelihood.*

2.6.10.3. *The Auditor wins in every stage t except the penultimate regime change stage t^* .*

Then the QAudit sequential parimutuel reward scheme has positive expected value for this Auditor if he plays according to the winning ranking prompted by QAudit.

Proof. See Appendix C.4 ■

The theorem above may appear whimsically named, but it is in fact critical. DeFi markets are highly volatile, and every few years the herd is "cleaned out." This creates a culture of unsustainable harvest, rendering the market unhealthy in the long run. Since Metheus only makes money if the Auditors make money, we are high incentivized to show that QAudit market instances are fair games. In this analysis we assume the Auditor earns some constant Δv on every stage. This is a fair assumption provided:

1. The Agents report models that improve the previous report in every stage.
2. The Auditors wager according to one of the m QAudit-suggested rankings.
3. The wagering stage is sufficiently lengthy, or there is enough market volume for the final ranking to conform to one of the s^* 's.
4. The buy volume is comparable across stages $t = 1 \dots T + 1$.
5. We do not account for the Auditor's wagers on the losing models.

We can relax these assumptions:

1. the earning can be some r.v. ranging over some normalized interval $\Delta v \stackrel{\text{iid}}{\sim} p[-1, 1]$. And find that depending on the "shape" of p , the hurdle set by eqn 2.13 can be either lowered or raised.
2. Additionally, observe that we assume the Auditors choose one of Odds or Evens clique, and plays over all stages. However in real life, the very knowledge of an impending regime change may introduce three additional incentives:
 - a) place downward pressure on volume as we march closer to regime change point t^* , as Auditors whittle out in anticipation of the "hammer."
 - b) Alternatively, the Auditors may be incentivized to draw other Auditors into the game, and thus maintain market volume, so that they may continue to profit until the very end. This is typically what occurs in reality.
 - c) Notably, Auditors may control their risk profile by selling model shares towards the *expected* moment of regime change. Recall QAudit issues a bundle of mint_k^t token for every model w_k^t in every stage t , that is to say the following tokens are in circulation throughout the lifecycle of some QAudit instance:

$$\text{toks} := \left\{ \{(\text{mint}_k^t, n_{1,t})\}_{k=1..K} \right\}_{t=1..T+1} \quad \text{s.t. } |\text{toks}| = \mathcal{O}(K \times (1 + T)).$$

Now let this current moment be $t^* - l$ for any $l \geq 1$, with the expected moment of regime change at t^* . Then an Auditor with $\text{mint}_k^{t^*-l}$ on hand may control his risk of holding model w_k with this trade:

$$\text{sell } \text{mint}_k^{t^*-l}, \text{ and buy any } \text{mint}_k^{t^*-l-n}, \quad \text{s.t. } n \geq 1.$$

The idea is that while the expected value of $\text{mint}_k^{t^*-l-n}$ is some $\Delta v \geq 0$, the value of $\text{mint}_k^{t^*-l}$ may very well be negative. Meanwhile the price of $\text{mint}_k^{t^*-l-n}$ on a per share basis is lower than that of $\text{mint}_k^{t^*-l}$ because of Dutch auction factor v_ρ^t applied over the stages $[t^* - l - n, t^* - l]$. Regime change introduces risks, and thus create endogenous demand for secondary market on **mint** tokens.

- d) The Dutch auction price ascending parameter v_ρ^t and the associated earnings per round Δv_t is assumed to be constant. In reality, the price floor can increase at a geometric rate, which would create additional incentives to trade **mint** tokens in the secondary market.

Remark 2.6.11. (Regime change risk factor and mint-dilution risk) can both be expressed in the value of γ . For example, if an analyst believes the models improve by 5% every stage t , and **mint** inflation rate (read: mining rate) is also 5%. Then he should set $\gamma = 0.10$ to discount the **mint**-denominated earnings over stages $t = 1..T + 1$.

All in all, we see that QAudit protocol endowed with random regime-changes creates a set of tokens with a certain price profile. That is an oscillating curves over model tokens $\text{QAudit}_{\text{toks}}$, so that the price over every set $\{\text{mint}_k^t : t = 1..T + 1\}$ fluctuates over the interval $[0, s \cdot v_{pa}]$, $s > 0$, whereby v_{pa} is the PA-defined bounty value determined a-priori. However, the price should rationally never exceed this v_{pa} ceiling up to some constant, this limits the upside risk for *not* buying **mint** from the perspective of Auditors. The next section will improve the price curve so that as it oscillates up and down, the price overall drifts upward, thus reflecting the shape of the Bitcoin price curve.

2.7 Perpetual QAudit

This section provides one final variation on QAudit: the perpetual QAudit protocol. It addresses the the problem of the implied price ceiling on **mint** tokens. Towards the end of every QAudit competition instance, both the Auditors and PA have the option of depositing more funds into the bounty value v_{pa} . Raising the stakes just so extends both the competition duration, and elevates the price ceiling.

Protocol 13 QAudit_∞ Perpetual QAudit

Require: Bounty v_{pa} , training stage T , qualifiers per stage R

Require: hidden data set $\mathbb{D}$, public query set $\mathbb{Q}$, kernel embedding function ϕ ,

Require: noise generator η , dutch auction function v_ρ^t , permutation distance $a \parallel b$

Require: agent accounts $\{\text{Agent}_k\}_{k=1..K}$, and auditor accounts $\{\text{Auditor}_i\}_{i=1..N}$

```

1:  $v_{pa} \leftarrow \text{PA}_{\text{bounty}}$  ▷ PA deposits bounty
2:  $T^* \leftarrow (T + 1) + T$  ▷ partial iteration of QAudit protocol
3:  $(\Theta, \mathbf{W}, \mathbf{V}, \Delta, \Delta_{\text{Agent}}, \Delta_{\text{Auditor}}) \leftarrow ([], \dots)$  ▷ initialize QAudit results
4: ▷ run QAudit instance for  $T^*$  steps, and pause before bounty is rewarded to accept raise
5: while  $v_{pa} > 0$  do
6:    $(\Theta, \mathbf{W}, \mathbf{V}, \Delta, \Delta_{\text{Agent}}, \Delta_{\text{Auditor}}) \leftarrow \text{QAudit}(v_{pa}, T, R, \eta, \mathbb{D}, \mathbb{Q}, \phi, v_\rho^t, a \parallel b) [1 : T^*]$ 
7:    $v_{pa} \leftarrow v_{pa} + \text{PA}_{\text{bounty}}$  ▷ PA raises bounty
8: end while
9: return  $(\Theta, \mathbf{W}, \mathbf{V}, \Delta, \Delta_{\text{Agent}}, \Delta_{\text{Auditor}})$ 

```

Commentary on QAudit_∞:

1. the perpetual protocol runs the model and wager elicitation steps of QAudit to completion at $T + 1$. And then runs the reward allocation phase up to stage T , pausing right before the bounty v_{pa} is paid to Auditors who wagered on stage $T + 1$. That is to say the submitted models from stages $t = 1 \dots T$ will be judged against the test set, so that the respective market participants will be paid.

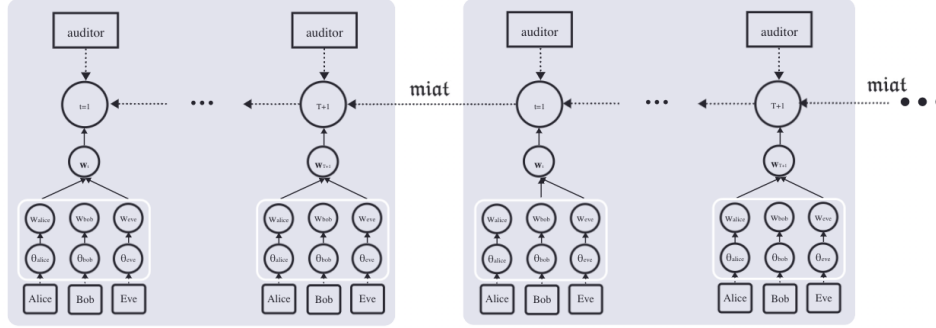


Figure 2.18: In perpetual QAudit, QAudit instances are *conceptually* chained in series. So that the last Auditors at stage $T + 1$ in one market instance are paid by the first Auditors at stage $t = 1$ in the next instance in the chain. In practice, the QAudit protocol simply "rolls over" by allowing the PA and Auditors to raise the bounty value v_{pa} . Thus restarting the market instance but with a higher price ceiling. This creates an overall ascending *mint* price curve as seen in fig 2.19.

2. The principle auditor PA is then given a chance to raise the bounty value. Additionally, any Agents may also join the PA in raising the value, thus becoming a PA themselves. If the bounty value is raised, then a fresh test and validation sets must also be provided.
3. QAudit then runs the competition to (semi)completion for another $T^* = (T + 1) + T$ steps. Each terminal stage T^* then is essentially a market *regime change*.

A few critical observation follows:

1. if the Auditors expect $QAudit_{\infty}$ to operate with infinite horizon as intended, then Folk Theorem suggests the Odds and Evens clique of Auditors should collude to accept the markup on successive stages until the regime change point. After which they would resume colluding as the horizon is once again, infinite.
2. Observe that the bounty value v_{pa} increases on every iteration of the while loop, the price ceiling precipitously elevates in a predictable manner, every $(T + 1) + T$ time steps. This elevation in price ceiling evokes the Bitcoin halving events. Such ruptures are very helpful for market participants, and draws additional players into the market who may trade in an "event-driven" manner.
3. In practice, both potential regime change and $QAudit_{\infty}$'s stake-raising events can become "cultural moments." This will draw in average DeFi enthusiasts, and any spectators in general.

This last point is bears emphasis: the purpose of QAudit is to:

1. build a market for deep learning models at the research and development stage.
2. Maintain liquidity in this market even though most model architectures do not lead to commercial products.

Most Auditors in this market will be retail buyers, they are poor judges of what makes a machine learning model performant. However, they do understand events, and the DeFi crowd tend to trade on momentum. QAudit is specifically designed so that the activities of these buyers may nonetheless incident upon the interest of the machine learning community, which has a different predisposition altogether. The protocol does most of the heavy lifting in terms of determining what is a "good" model. Sophisticated Auditors who disagree with how a

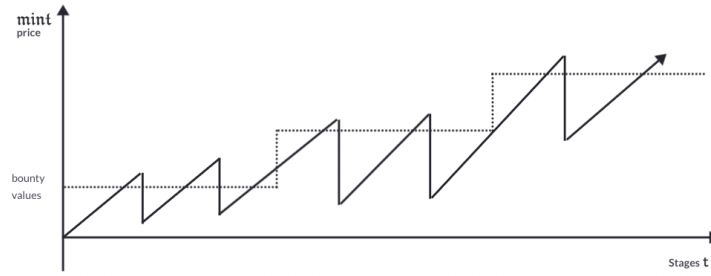


Figure 2.19: Under perpetual QAudit, PA's and Auditors repeatedly buy into the penultimate stage before the bounty is claimed by the last tranche of Auditors. This allows QAudit to lift the price ceiling on mint tokens, and create an overall upward trajectory on mint prices. This cycle of resets leading to greater highs is well known in DeFi sphere, and will attract significant retail attention. This is important: in perpetual QAudit, Auditors are no longer evaluating model performance solely, but instead will speculate on events. Namely the likelihood of regime change on any stage t , and the likelihood that the community will double down on the bounty value.

particular instance of QAudit is measuring the models should develop their own measurement modules, as noted in fig 2.12.

2.8 Conclusion

This chapter specified the Machine Intelligence Quality Audit, or QAudit for short. Our aim is to design a protocol for a more fair machine learning competition, whereby 1) competitors or Agents do not play in winner-take-all games, but rather collude to win or receive runner-up prizes. And 2) a decentralized set of sponsors or Auditors may profit by selecting the best models. They play in a market populated by mint tokens with a well-crafted ascending price curves, featuring regular resets, and overall upward drift in prices. This is comparable to Bitcoin. Many QAudit instances running in parallel allow machine learning teams to tap a wider financial base to carry their training costs, which is substantial. It also allows the broader public to participate the upside of "AI" *before* the models have been commercialized. Notably, we observe that the field of AI has short summers and long winters, and most of the groundbreaking work is done in the winter. QAudit sustains the machine learning scientists through the famine, and we know winter is coming.

Now this idea may sound "good," but it is not without its challenges. AI as a field is saturated with snake oil, in fact it is currently giving DeFi a run for its money. In this cycle generative AI has captured the public imagination, yet very few people truly understand how to judge machine learning models. Thus, QAudit is designed so that if the Auditors just "follow the instructions" and do not YOLO the market, then they can expect to profit. Conversely, QAudit is structured s.t. those who do purposely attempt to Moon the market, as is often the case in DeFi, will find themselves or their friends cleaned out. Thus QAudit is a gentle on-ramp into the Metheus protocol economy. The protocol is:

1. motivated by fundamental principles of machine learning;
2. designed around a set of impartial metrics with well-defined ground truth;
3. the soundness of its incentive structure, or "tokenomics," can be rigorously proven using formal technique around games, markets, and randomized algorithms.

In summary, QAudit is an objective and friendly setting. The next chapter leaves this sterile scene and consider how machine intelligence intersects with the broader society.

CHAPTER 3

Machine Intelligence Fairness Audit

3.1 Introduction

Auditing for algorithmic fairness is a reasonably developed field, both in theory and in practice [MP+24]. It dovetails the proliferation of algorithms in our daily lives, and social justice movements that have permeated all elite institutions. The problem can be summarized as follows: industrial scale generative models memorize the internet, and interpolate where data is missing, thus they reflect the biases and contradictions of the world at large. Suffice it to say, this has many adverse consequences and legal repercussions. **FeaAudit** is designed to resolve the fairness audit problem, while contextualizing the protocol within the needs of **Metheus**. These needs include:

1. (*Stimulate retail interest*) in the **Metheus** ecosystem. Unlike **QAudit** which is a professional product aimed at relatively sophisticated market participants, **FeaAudit** is an easy-to-grasp consumer product that the average person can use. In fact **FeaAudit** is both 1) an additional **miat** token mine, and 2) a data-collection pipeline that is used to improve models from **QAudit**.
2. (*Generate in-ecosystem demand*) for the machine learning models that have passed the **QAudit** inspection of previous chapter. The fairness auditing process itself involves steady interaction with generative models, and constitutes an additional user acquisition channel for partners who host said models.
3. (*Cultivate token sinks*) so that **miat** currency can be cycled out of circulation - while feeding user engagement. In fact, **FeaAudit** will be just one of many auditing protocols that supplements the **QAudit** core. A consortium of such processes will drive in-ecosystem demand for **Metheus**-inspected models, data, and thus promote sustainable **miat** demand.

In the context of machine learning, **FeaAudit** is an example of a *fairness oracle*, or more plainly a data annotation service. This oracle accepts a query response pair $(\mathbf{x}_{\tau-1}, \mathbf{x}_{\tau})$, and ask annotators to label it as either fair or unfair. This labeled dataset is used by downstream algorithms to refine model performance. However, the problem is that in general:

1. people do not respond to surveys willingly.
2. When people do respond, they censor themselves and report socially compliant answers.
3. If the survey occurs in public as is the case in **FeaAudit**, then people may outright lie to signal virtual.

Therefore a good survey product must 1) assure the users that their responses are "masked," and 2) offer a social service that is more interesting than aiding faceless analysts train more HR-friendly generative AI.

Protocol 14 Fairness Audit (FeaAudit)**Require:** generative model $p(\mathbf{x}_t | \mathbf{x}_{t-1}; \theta)$ **Require:** wealth seizure parameter $\xi \in [0.384, 0.8]$ **Require:** witness accounts $\mathbb{W} = \{\text{Wit}_1, \dots\}$ with minimum wallet balance $\mathfrak{v}$

```

1:  $t \leftarrow 0$   $\triangleright$  stage counter
2:  $\mathbb{D} \leftarrow []$   $\triangleright$  init labeled data set
3:  $\text{coin}_p \leftarrow \text{PA}$   $\triangleright$  set privacy parameter
4:  $\text{VA} \leftarrow \text{Wit}_i \in_p \mathbb{W}$   $\triangleright$  protocol selects random witness as vice auditor
5:  $\mathbb{C} \leftarrow \{\{\text{Wit}_k^1\}_{k \geq 1}, \dots, \{\text{Wit}_j^M\}_{j \geq 1}\}$   $\triangleright$  witness voluntarily join Cohorts on rolling basis
6: while  $|\mathbb{C}| > 1$  do
7:    $\mathbf{x}_{t-1} \leftarrow \text{VA}$   $\triangleright$  vice auditor prompts model
8:    $\mathbf{x}_t \leftarrow p(\mathbf{x}_t | \mathbf{x}_{t-1}; \theta)$ 
9:    $\tilde{y} \leftarrow \text{nil}$ 
10:   $t \leftarrow t + 1$ 

11:  Witness testimony phase:
12:  FeaAudit selects  $\text{Wit}_k^m \in_p \mathbb{W}$  and query:
13:    "is response  $\mathbf{x}_t$  to prompt  $\mathbf{x}_{t-1}$  FeaAudit compliant?"
14:     $\text{Wit}_k^m$  responds in private:
15:       $y_k \leftarrow \text{Wit}_k^m$ 
16:      FeaAudit toss bent  $\text{coin}_p$ :
17:      if (toss = Head) then
18:         $\tilde{y} \leftarrow y_k$ 
19:      else (FeaAudit toss1 fair coin)
20:         $\tilde{y} \leftarrow (\text{toss}_1 = \text{Head}) ? \text{Yes} : \text{No}$ 
21:      end if
22:    end private
23:    let  $\mathbf{z} := ((\mathbf{x}_{t-1}, \mathbf{x}_t), \tilde{y})$  & emit( $\mathbf{z}$ )
24:     $\mathbb{D} \leftarrow \mathbb{D} \cup \{\mathbf{z}\}$   $\triangleright$  stream data and store it
25:  end query

26:  PA judgment phase:
27:   $\text{toss}^t \leftarrow \text{nil}$ 
28:  in private:
29:    PA examines response  $\tilde{y}$  and send VA bent  $\text{coin}_q^t$  with hidden bias  $q(\frac{1}{3}, \frac{2}{3})$ 
30:    VA flips bent  $\text{coin}_q^t$  up to 2 times, reveals outcome of last flip with
31:     $\text{toss}^t \leftarrow \text{VA}$ 
32:  end private
33:  if ( $\text{toss}^t = \text{Tail}$ ) then
34:     $\mathfrak{v}_k^t \leftarrow \text{reject}(\text{Wit}_k^m, \xi)$ 
35:     $\text{PA} \leftarrow \text{PA} + \frac{\mathfrak{v}_k^t}{10}$ 
36:     $\text{VA} \leftarrow \text{VA} + \frac{\mathfrak{v}_k^t}{2}$ 
37:    for  $\text{Wit}_j^n \in \mathbb{W}$ , where  $n \neq m$  do
38:       $\text{Wit}_j^n \leftarrow \text{Wit}_j^n + \mathfrak{v}_k^t \cdot \frac{2}{5} \cdot \frac{1}{|\mathbb{W}|}$ 
39:    end for
40:  else
41:     $\mathfrak{v}_{\text{VA}}^t \leftarrow \text{reject}(\text{VA}, \xi)$ 
42:     $\text{PA} \leftarrow \text{PA} + \frac{\mathfrak{v}_{\text{VA}}^t}{10}$ 
43:     $\text{Wit}_k^m \leftarrow \text{Wit}_k^m + \frac{\mathfrak{v}_{\text{VA}}^t}{2}$ 
44:     $\text{VA} \leftarrow \text{Wit}_k^m$ 
45:    for  $\text{Wit}_j^n \in \mathbb{W}$ , where  $n = m$  do
46:       $\text{Wit}_j^n \leftarrow \text{Wit}_j^n + \mathfrak{v}_{\text{VA}}^t \cdot \frac{2}{5} \cdot \frac{1}{|\mathbb{W}|}$ 
47:    end for
48:  end if
49: end while
50: return  $(\mathbb{C}[0], \mathbb{D})$ 

```

FeaAudit Protocol

FeaAudit features this set of players:

1. (PA principle auditor): similar to QAudit, there is a PA in this case who will run an on-chain instance of FeaAudit. The PA must lock up a certain amount of **miat** to launch a FeaAudit instance. When the FeaAudit competition ends, the **miat** is returned in full.
2. (VA vice auditor): works with the PA to judge the Witness's response, and allocate reward or punishment. The VA is not one player, but a set of players that rotate through the role.
3. (Witnesses): examine a set of questions and determine if they are FEE-compliant. Randomly selected Witnesses are rewarded or punished depending on whether the PA and VA agree with their judgment. If the selected Witness refuses to answer a question within sometime frame, then his wealth is seized by default.
4. (Cohorts): are not players but sets of Witnesses. The Cohort labels are oriented around identities such as gender, race, income level, and education. Depending on how a particular instance of FeaAudit protocol is fielded, Witnesses may either choose their Cohort, or be assigned into one by demographic metadata.

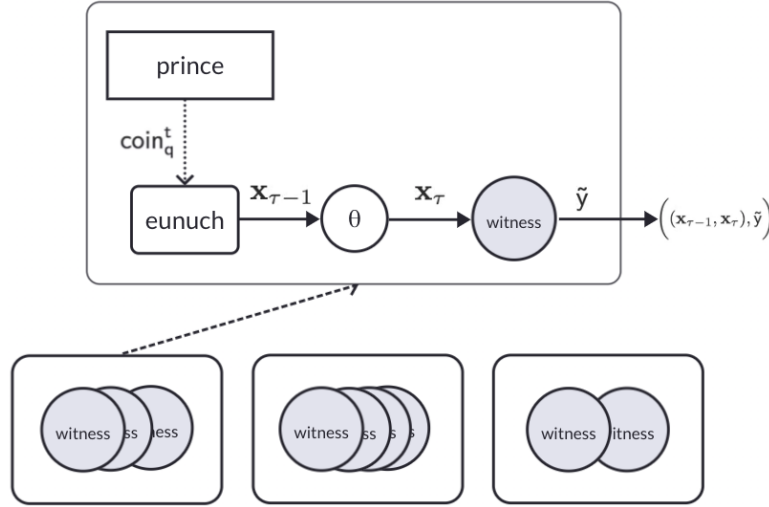


Figure 3.1: In the FeaAudit fairness annotation game, the sitting eunuch or VA queries a generative model θ with input $\mathbf{x}_{t-1}$, and the model outputs response $\mathbf{x}_t$. A random Witness is selected from one of the Cohorts, and asked to judge the fairness of this query response pair $(\mathbf{x}_{t-1}, \mathbf{x}_t)$. The Witness *privately* labels the pair with y , FeaAudit then scrambles the response and publishes masked label $\tilde{y}$. The prince or PA judges the labeled tuple $(\mathbf{x}_{t-1}, \mathbf{x}_t, \tilde{y})$, and *in private* gives bent coin coin_q^t to the eunuch, signaling his judgment. The eunuch flips the coin up to 2 times to reveal the prince's judgment. Depending on how the coin lands, FeaAudit either **reject** the Witness, or the eunuch. If the eunuch is **rejected**, then the Witness becomes the new eunuch. Coin flipping hides all the player's true feeling from each other, thereby rendering the game safe from peer pressure. Unlike QAudit, the losers in FeaAudit cannot rejoin the game on the next stage $t + 1$. FeaAudit continues until only one Cohort survives.

3.2 FeaAudit Elementary Analysis

Through the technical lens, FeaAudit is a fairness oracle equipped with a differential privacy data release mechanism. Here data is collected and released in an on-line manner, while masked under a privacy filter. The fundamental assumption of FeaAudit are as follows:

1. The Witness is a real person. They will examine the randomization technique of line 15-21 of FeaAudit (presented in visual format), and feel sufficiently validated that their true response is hidden behind the coin tosses.
2. The eunuch (VA) acts in a selfish manner to maximize his own earnings from **rejected** Witnesses. Observe in line 28-31 the VA has the option to flip the coin_q^t a second time if the first toss comes up **Head**. We assume:
 - a) if the first toss returns **Head**, then the VA *always* flips the coin a second time. That is even if the Witness is drawn from the same Cohort as him.
 - b) On the other hand, if the first toss comes up **Tail**, then the VA stops here, even if it means **reject** a Witness from his own Cohort.
3. We place no assumptions on the behavior of prince (PA).

Now we show FeaAudit is sound in that it satisfies four desiderata:

1. The Witness is hidden.
2. The Cohort is exposed.
3. The prince PA is hidden.
4. The eunuch VA is exposed.

Observation 3.2.1. *(The Witness is Hidden) The label disclosure mechanism of line 14-22 in FeaAudit is a minor variation of the randomized response mechanism found in [DR14, p. 15]. Its purpose is to provide "plausible deniability" of the Witnesses' response, as a public label of $\tilde{y} = \text{Yes}$ could mean that the Witness finds it fair, or it could be the coin_p landed tails, and a second fair coin was tossed that landed heads. Now we can compute the likelihood of Yes and No. Now given:*

$$\begin{aligned} p(y_k = \text{Yes}) &= r_k & p(y_k = \text{No}) &= 1 - r_k \\ p(\text{coin}_p = \text{Head}) &= p & p(\text{coin}_p = \text{No}) &= 1 - p. \end{aligned}$$

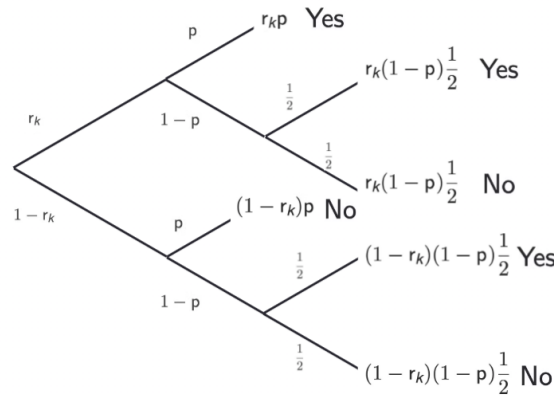


Figure 3.2: Binary tree denoting how the masked label $\tilde{y}$ is generated from true Witness label y . For example, with unobserved probability r_k , the Witness prefers $y = \text{Yes}$. This private preference y is masked with probability $1 - p$. Then a fair coin is tossed to output label **Yes** with likelihood $\frac{1-p}{2}$.

Referencing figure 3.4, the conditional probabilities are:

$$\begin{aligned} p(\tilde{y} = \text{Yes} \mid y = \text{Yes}) &= p + \frac{1}{2}(1 - p) & p(\tilde{y} = \text{Yes} \mid y = \text{No}) &= \frac{1}{2}(1 - p) \\ p(\tilde{y} = \text{No} \mid y = \text{Yes}) &= \frac{1}{2}(1 - p) & p(\tilde{y} = \text{No} \mid y = \text{No}) &= p + \frac{1}{2}(1 - p). \end{aligned} \quad (3.1)$$

Here y_k is the private true label that Wit_k give to the query-response pair $(\mathbf{x}_{\tau-1}, \mathbf{x}_\tau)$, and $\tilde{y}$ is the published label scrambled by FeaAudit. Critically, observe that without knowing r_k , we have $p(\tilde{y} = \text{Yes}) = p(\tilde{y} = \text{No})$ under the privacy mask of coin_p . So one cannot determine the Witness's true preferences.

Observation 3.2.2. (The Cohort is Exposed) This follows immediately from:

$$\frac{p(\tilde{y} = \text{Yes} \mid y = \text{Yes})}{p(\tilde{y} = \text{Yes} \mid y = \text{No})} = \frac{\frac{p+1}{2}}{\frac{1-p}{2}} = \frac{p+1}{1-p}.$$

So FeaAudit is $\ln\left(\frac{p+1}{1-p}, 0\right)$ -differentially private as seen in definition B.1.2.

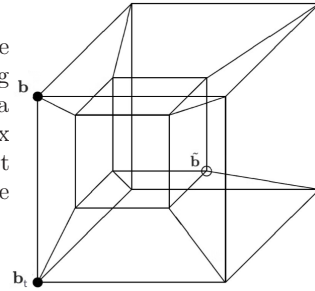
Although the spectators cannot determine any Witness's private preference on an individual basis, the population preference of any Cohort may still determined. Since we have $p(\tilde{y} = \text{Yes} \mid y = \text{Yes}) > p(\tilde{y} = \text{Yes} \mid y = \text{No})$ and so on, as seen in eqn 3.2. The next proposition make this statement precise.

Proposition 3.2.3. (Cohort labels are "sufficiently close") to the true label in every iteration of FeaAudit, provided the PA-set coin_p is defined so that:

$$p \geq 1 - \frac{d}{N}, \quad (3.2)$$

for some PA defined $d \in \{0, 1, 2, \dots\}$.

Figure 3.3: The set of all N -bit strings forms a hypercube with 2^N vertices, whereby we traverse any edge by flipping just one bit. Lines 14-22 of FeaAudit then correspond to a random walk on this hypercube. Starting at the true vertex $\mathbf{b}$, it either flips the t -th bit to reach vertex $\mathbf{b}_t$, or do not flip any bits and stay at $\mathbf{b}$. After N possible flips, we arrive at $\tilde{\mathbf{b}}$.



Proof. Immediately follows from law of large numbers. C.5 goes through the motions. ■

Observation 3.2.4. (The Prince is Protected while the Eunuch is Exposed). At any stage t , the PA reveals his judgment of some Wit_k^m 's evaluation of the model by passing a bent coin_q^t with hidden bias q_t . Now referencing line 28-31 of FeaAudit, the two tosses of the VA's bent coin_q^t creates a new bent coin. Given $p(\text{coin}_q^t = \text{Head}) = q_t$, here are three possible outcomes to VA's random experiment with their associated likelihoods:

Head, Head	: q_t^2
Head, Tail	: $q_t(1 - q_t)$
Tail	: $1 - q_t$.

They correspond to how a rational VA behaves as he selfishly **reject** the Witnesses and appropriate their wealth. For reasons that will be clear later, we call this the Non-Veumann

method, which takes a bent coin and bend its bias towards your favor. The bias of this derived bent coin is then:

$$p(\text{Head}) = q_t^2 \quad p(\text{Tail}) = q_t(1 - q_t) + 1 - q_t = 1 - q_t^2. \quad (3.3)$$

In concrete terms, observe by definition the PA's bent coin coin_q^t must have bias between $q(\frac{1}{3}, \frac{2}{3})$, so that:

$$0.556 \leq p(\text{Tail}) \leq 0.889.$$

The odds are stacked in favor of the VA once he is selected onto the eunuch position. The expected number of stages t^* any VA_k can survive before seeing the elimination sequence of Head, Head is bounded by:

$$\frac{1}{q_u^2} \leq t^* \leq \frac{1}{q_l^2} \implies t^* \in [2.25, 9]. \quad (3.4)$$

So the eunuch's expected tenure at court when flipping the coins to survive exceeds 2 stages. If the eunuch is honest and only tosses each coin_q^t once, then his expected survival time is:

$$\frac{1}{q_u} \leq t^* \leq \frac{1}{q_l} \implies t^* \in [1.5, 3].$$

Whenever Witnesses observe some eunuch VA_k survives more than 3 stages of the game, then they have reason to suspect VA_k is tossing in a self-interest manner. In conclusion, Witnesses cannot determine if the PA is truly displeased with their response, or if it is the self-serving VA-eunuch tossing and raking. Meanwhile the eunuch is exposed for his greed, not his true belief as to whether a label is correct or not.

We built FeaAudit with the intention of protecting the prince PA on a per-Witness basis, so the VA filter offers some protection against the mob. Additionally, the bias of the bent coin_q^t is q_t indexed by t , so it is different on every stage t . However, if the PA demonstrates consistent bias towards a specific direction, then over the long run the Witnesses may be able to determine the PA's secret preference. Once the PA's preference is known, the Witnesses may cheat by testifying in a manner that appeals to the PA's prejudices. Remark 3.3.4 addresses this problem by running many FeaAudit instances in parallel.

Remark 3.2.5. (Alternate randomization technique to protect the PA) We can protect the preference of the PA further if the eunuch VA_k uses the "Von Neumann method" [Mit13]:

- 3.2.5.1. toss coin_q^t twice, if they show Head, Tail or Tail, Head, then disclose the 1st toss;
- 3.2.5.2. if the two tosses come up Head, Head or Tail, Tail, then toss the coin again twice.
- 3.2.5.3. This process repeats until the VA_k sees two tosses of different faces.

This sampling regime creates a fair coin out of a bent coin of any bias q , and thus protects the prince PA's true preference absolutely. However, this may not be ideal as:

- 3.2.5.1. the prince or PA may want to shape public opinion subtly to his own.
- 3.2.5.2. It defeats the purpose of having the PA judge the testimony at all.
- 3.2.5.3. It is too laborious for the VA_k to do.

In general, the randomization technique 28-32 and 14-22 may be an external process that is composed with the main FeaAudit protocol, just as QAudit-machine-learning subroutines can be composed with the main QAudit body.

Now we discuss how players profit from FeaAudit:

1. *The prince PA:* is the most privileged position. His preferences are hidden and he never loses money. Instead he collects $\frac{1}{10}$ of every Witness's or eunuch's funds regardless how the coin tosses land. The PA is in effect taxing the users by running a *fairness farm*, and in doing so helping Metheus tax the users as well.

2. *The Witness*: the witness may earn money passively simply for sitting in one of the Cohorts, and is showered with some fraction of each **rejected** Witness or eunuch's fund. We call this behavior *fairness farming*, and it is the easiest way to enter the Metheus ecosystem. However, once the Witness is called to judge a query-response pair, the odds are less favorable as seen in observation 3.2.4. Nevertheless, it is possible for Witnesses to sit in a Cohort and never be called upon to testify before a query. In fact, FeaAudit rules demands it as by definition, there exists one Cohort whose members have not been all eliminated at end of the game. Knowing this, Witnesses may cheat by shuffling themselves into the largest Cohort. This problem can be easily remedied by capping the size of every Cohort.
3. *The eunuch* VA_k : although the eunuch is exposed, he is compensated with better odds. In *expectation* w.r.t q and ξ , the VA_k may survive long enough to exit the eunuch position with more wealth than he enters with. Specifically, we know from 3.2.4 the survival rate is $0.556 \leq p(\text{Tail}) \leq 0.889$. The next proposition determines under what condition the VA_k may die richer.

Proposition 3.2.6. (*Eunuchs who die old, die rich*). *That is every eunuch VA_k can expect to die wealthier than when he is selected into the position, if FeaAudit set wealth seizure parameter so that:*

$$0.384 < \xi < 0.80. \quad (3.5)$$

Proof. See C.5. ■

Protecting the eunuch's interest is central to the FeaAudit incentive structure, as it prompts the eunuch to *always play selfishly*. In doing so, they can be relied upon to hide the prince's true intentions. The more self-serving the eunuch, the more protected the prince. This is why eunuchs exist. In fact the eunuch's singular greed for *miat* also protects *him*, because the Witnesses can be rest assured that the eunuch believes in nothing - a fact that has many advantages. The next section extends FeaAudit so that Witnesses can also protect their *miat*-downside.

3.3 Tokenizing FeaAudit

A refinement on the baseline game is to tokenize the FeaAudit fairness oracle. Specifically, each Witnesses may sell a *fair and equal annotation token*, or *feat*. Each $feat_k$ is a measure of Wit_k 's earning potential in the FeaAudit game, and is essentially a claim on some portion of the total pot of *miat* in any FeaAudit game. This *feat* is marketed as follows:

1. a Witness Wit_k enters an instance of the FeaAudit game, and deposit the minimum *miat* balance v . Denote this initial wealth with:

$$v_k^o = v. \quad (3.6)$$

Conceptually, Wit_k just purchased a $feat_k$ option with initial payment v .

2. At any stage t , a Wit_k may mint *one* edition of $feat_k$, and sells it to an $Auditor_i$ for some price. In doing so, the $Auditor_i$ has assumed the entire up and downsides of Wit_k . Specifically:

- a) when the Wit_k is airdropped *miat* from **rejected** eunuchs and other Witnesses as seen in line 38 and 46 of FeaAudit, this *miat* is owed to $Auditor_i$. We can denote the deposited balance at this stage t with:

$$v_k^t = v + t\Delta v, \quad (3.7)$$

where the constant Δv is the airdropped *miat* from **rejected** Witnesses. This $t\Delta v$ is the "leverage" gained by simply sitting in the FeaAudit market.

- b) When Wit_k is called to testify the fairness of some query response pair $(\mathbf{x}_{\tau-1}, \mathbf{x}_{\tau})$ and becomes a eunuch VA_k , then the entire upside of VA_k 's earnings is also owed to $Auditor_i$. We can denote VA_k 's current wealth at any stage t with:

$$v_k^t = v + (t^o - 1)\Delta v + \frac{\xi}{2} \sum_{t^o}^t v_i, \text{ where } i \in \{1, \dots\}/k. \quad (3.8)$$

Here the index t^o is when VA_k became a eunuch, and t is the current game stage so that $t^o \leq t$. The Witness is conceptually purchasing his seat as a eunuch with his own money v , and leverage $(t^o - 1)\Delta v$.

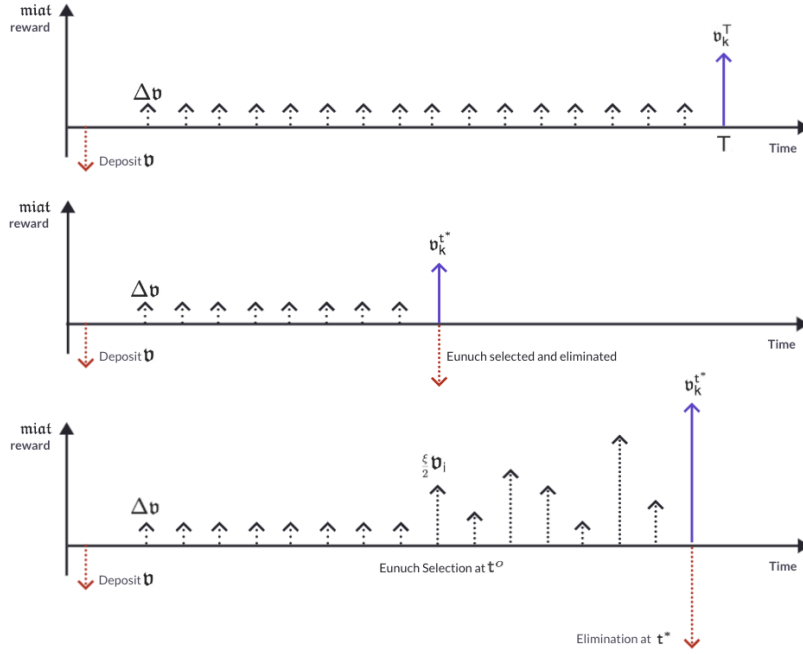


Figure 3.4: Three payout scenarios for FeaAudit. The top graph corresponds to eqn 3.10, where the Witness Wit_k deposits v , sits in the winning Cohort, collect airdrops from other eliminated Witnesses. Notably he is never eliminated himself, so the entire amount v_k^T is given to the $feat_k$ holder. In the middle graph for 3.11, the Witness is chosen to testify, and is immediately eliminated by the sitting eunuch. So the $feat_k$ holder receives $(1 - \xi)v_k^{t*}$. In the bottom graph, the Witness becomes the eunuch, and survives many stages as a eunuch. He earns a variable amount of $\frac{\xi}{2}v$ between t^o and t^* , and is eliminated at t^* . So he returns $(1 - \xi)v_k^{t*}$ to $feat_k$ -holder.

3. When this Wit_k exists the game at stage t^* , whether as a **rejected** Witness or a **rejected** eunuch, the ξv_k^{t*} downside is paid from the **miat** that is otherwise owed to the $feat_k$ owner, namely $Auditor_i$. Each Wit_k 's terminal wealth at elimination is:

$$v_k^T = v + T\Delta v \text{ if does not die,} \quad (3.9)$$

$$v_k^{t*} = v + (t^* - 1)\Delta v \text{ if dies as Witness,} \quad (3.10)$$

$$v_k^{t*} = \underbrace{v + (t^o - 1)\Delta v}_{\text{prefix}} + \underbrace{\frac{\xi}{2} \sum_{t=t^o}^{t^*} v_i}_{\text{suffix}} \text{ if dies as eunuch.} \quad (3.11)$$

So the Auditor_i's "take-home" wealth from feat_k option is $(1 - \xi)v_k^*$.

Now consider how each side would profit:

1. the Auditor_i only pays once: when he buys feat_k . He profits if the price of feat_k is less than $(1 - \xi)v_k^*$, he loses miat otherwise. So Auditors should buy feat_k from Wit_k if he believes this Witness has a "good chance" of either 1) surviving to the end without being called to testify. Or 2) surviving many rounds as a eunuch, which entails two factors:
 - a) Wit_k is drawn from a Cohort whose bias is similar to that of the prince PA.
 - b) When this Wit_k becomes the eunuch VA_k in the future, FeaAudit will draw a runs of Witnesses s.t. 1) their judgments either displeases the PA, and/or 2) the VA_k can continuously flip coin_q^t 's where he does not see the elimination pair of Head, Head. Critically, recall the likelihood of surviving each stage is greater than 55% regardless of how the PA answers.

In sum, Auditors who purchase feat_k are fundamentally speculating on the preference of the prince PA over fairness judgments.

2. The Wit_k pays v upon entry, and his wealth increase on every stage by some Δv simply for sitting in a Cohort. So he may decide at some stage t to sell feat_k and lock in his gains. There are three possible price points he may accept:
 - a) Strictly speaking, Wit_k profits if the price of feat_k is greater than v - the price he paid to enter the game.
 - b) Alternatively he may be greedy, and set the threshold at $v + (t - 1)\Delta v$.
 - c) The most conservative point is $(1 - \xi)(v + (t - 1)\Delta v)$, that is how much Wit_k would be left with if he is called upon to testify, and immediately is immediately rejected. This is effectively a fire sale of his position.

We assume the Witness is irrational, so any price point above is possible. However, the real question is what is the fair market value of this feat_k from the Auditor's perspective. We can express a lower-bound the the value of v_k^* of eqn 3.12 as follows. Given some Wit_k is selling feat_k at stage t , then we know the prefix is bounded by:

$$\alpha v + (t - 1)\Delta v \leq \text{prefix} \leq \alpha v + T\Delta v$$

$$\text{where } \Delta v \geq 0, \alpha \geq 1. \quad (3.12)$$

Here T denotes the end of the FeaAudit game as defined by the while loop, while αv denotes the actual amount the Witness spent to enter the market. So in the worst case, Wit_k is selected for testimony in the next stage, while in the best case, the Wit_k survives until the final stage T . If the Auditor does not expect the Witness to be chosen, then his price quote for feat_k should lie within $(1 - \xi)\text{prefix}$.

Next we assume the Witness will be chosen as a eunuch VA_k at some stage, and wlog call this stage 1. Then we can bound the the suffix of eqn 3.12, that is VA_k's earnings over any game interval.

Proposition 3.3.1. (The VA_k's expected earning) over any interval $[1, t_u^*]$ is bounded with:

$$2\beta v \xi \sum_{t^*=0}^{t_u^*} \binom{t^* + 2}{t^*} \frac{5^{t^*}}{9^{t^*+1}} \leq \mathbf{E} \left[\text{VA}_k \text{'s income on } [1, t_u^*] \right] \leq \frac{\beta v \xi}{2} \sum_{t^*=0}^{t_u^*} \binom{t^* + 2}{t^*} \frac{8^{t^*}}{9^{t^*+1}}. \quad (3.13)$$

Where:

- 3.3.1.1. βv with $\beta \geq 1$ denotes the average wallet balance of Witnesses;
- 3.3.1.2. stage 1 is the moment VA_k becomes a eunuch;
- 3.3.1.3. t_u^* is any value less than the upper bound on the while loop of FeaAudit;
- 3.3.1.4. $t^* \in [1, t_u^*]$ denotes the stage where VA_k is eliminated.
- 3.3.1.5. The expectation is taken w.r.t the coin_q^t tosses of line 29-31.

See fig 3.7 for game tree used to express the bound in eqn 3.14.

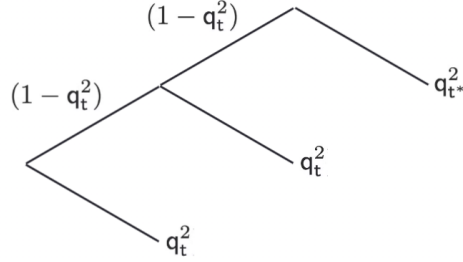


Figure 3.5: At every stage t , the eunuch VA_k dies with likelihood q_t^2 , and continues to next stage $t + 1$ with likelihood $1 - q_t^2$. At the terminal stage t^* , VA_k dies with likelihood $q_{t^*}^2$.

Proof. See C.5. ■

Now we can express the fair price range of $v_k^{t_u^*}$.

1. Using the lower bound in eqn 3.13 we state the range of expected values of $v_k^{t_u^*}$, in the event where the Auditor expects the Witness to be chosen as a eunuch at stage $t + 1$:

$$\alpha v + t \Delta v + 2v \beta \xi \sum_{t^*=t}^{t_u^*} \left(t^* + 2 \right) \frac{5^{t^*}}{9^{t^*+1}} \leq \mathbf{E}[v_k^{t_u^*}] \leq \alpha v + t \Delta v + \frac{\beta v \xi}{2} \sum_{t^*=t}^{t_u^*} \left(t^* + 2 \right) \frac{8^{t^*}}{9^{t^*+1}}. \quad (3.14)$$

The Auditor may now select suitable α , β , ξ , and t_u^* to determine where the asking price of feat_k rests w.r.t this range.

2. We can redefine the lower bound as the event where the Witness is called to testify on stage $t + 1$, and immediately eliminated:

$$\alpha v + t \Delta v \leq \mathbf{E}[v_k^{t_u^*}] \leq \alpha v + t \Delta v + \frac{\beta v \xi}{2} \sum_{t^*=t}^{t_u^*} \left(t^* + 2 \right) \frac{8^{t^*}}{9^{t^*+1}}. \quad (3.15)$$

Depending on the constants, he may pay for any price within this $(1 - \xi)\mathbf{E}[v_k^{t_u^*}]$ interval. It goes without saying that the price in eqn 3.16 should be updated on every stage t to reflect the change in $t \Delta v$. Alternatively, one may set Δv and t^o as random variables and model this expectation.

Remark 3.3.2. (How feat_k is Marketed in Practice). There are two ways to sell feat_k : push and pull. The "pull" model is presented above whereby Witnesses initiate the sale and pull Auditors into the market. In the "push" model which is more natural to how DeFi operates, Auditors initiate the sale by buying say 100 feats at minimum price v , and staking 100 Witnesses into the game. In this case, the Auditors hold a diverse group of Witnesses, and hope at least a few will make it as eunuchs. Meanwhile Witnesses play for free. Moreover, the Auditors and Witnesses may strike some standardized term where by:

3.3.2.1. The **Auditors** receive the taxed portion of their principle in $(1 - \xi)v$, along with the taxed portion of the eunuch's upside upper-bounded by $(1 - \xi)\frac{\beta v \xi}{2} \sum_{t^*=t}^{t_u^*} (t^* + 2) \frac{8^{t^*}}{9^{t^*+1}}$.

3.3.2.2. The **Witnesses** receive the after-tax portion of airdrops from other **rejected** **Witnesses** in $(1 - \xi)t^* \Delta v$.

Under these terms, the **Witnesses** only has upside measured in **miat**, no downside other than time lost. Alternatively, the **Auditor** may want some claw back provisions as downside protection if he stakes αv , so **Witnesses** may receive less than $(1 - \xi)t^* \Delta v$ in payment. Exact details will be at the **Auditors'** discretion.

Remark 3.3.3. (FeaAudit conversion loop)

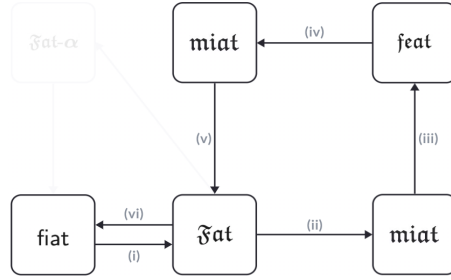


Figure 3.6: When **Witnesses** or **Auditors** buy into the **FeaAudit** annotation game by purchasing **feat**, they undergo the same conversion cycle as fig 2.8, with the commensurate dilution in upside that be recovered by holding **fiat** tokens. Thus **FeaAudit** is both a **miat** token sink and a token source. See fig 4.6 for more information.

Remark 3.3.4. (Scaling FeaAudit). In the basic **FeaAudit** formulation, there is a bottleneck in decision making, as the prince PA must judge every testimony from every **Witness**. Similar to how **QAudit** may be chain in series, **FeaAudit** may be chain in parallel. Whereby i.e. four princes may run four instances of **FeaAudit** together, and **Witnesses** are routed to a random prince in every stage t . Now each prince only has to judge $\frac{1}{4}$ of the **Witnesses** pool, naturally the "tax revenue" is also split by commensurate amount. The **Witness's** testimony is judged by his assigned prince PA, and may become a eunuch to this very PA. Running many instances of **FeaAudit** in parallel protects the prince even more, as routing the **Witness** towards any prince occurs randomly.

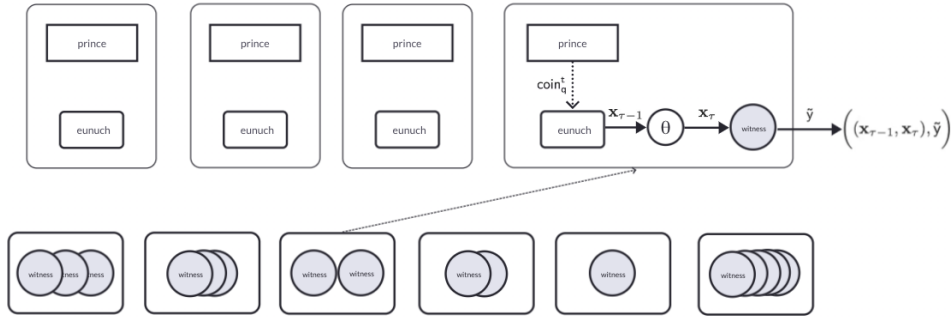


Figure 3.7: **FeaAudit** with four princes, and their respective eunuchs. **Witnesses** are routed to a random princes for testimony.

3.4 Conclusion

This chapter specifies the fair and equal annotation audit or **FeaAudit**. The protocol specifies a family of fairness oracles, or annotator services whereby **Witnesses** are called forth to testify the social compliance of a generative model. The prince or PA then judges the response,

and passes his opinion in secret to the eunuch or the VA. Together they make a decision on whether to reward the Witness or **reject** him. Critically, both the Witness's labels and the prince's judgments are masked by coin tosses. This gives each party plausible deniability. The coin tosses are structured such that downstream algorithms may nonetheless determine population level preferences over Cohorts of Witnesses, perhaps segregated by race, gender, income etc. Thus FeaAudit streams useful labeled data, while satisfying the promises of differential privacy for the Witnesses. Moreover, Witnesses may sell their future earnings with fairness tokens or **feat**. Unlike other DeFi coins whose value is "YOLO or rug," **feat** can be priced via *technical valuation methods*. Its price is a function of the market structure and market constants of FeaAudit, these are known to the buyers/Auditors, and can be determined by eqn 3.16.

Here we used fairness annotation as a framing device to craft a technical narrative. However structurally there is nothing unique about this class of labels. FeaAudit can be easily adapted to any arbitrary labeling task. For example, instead of judging a generative model, a variation of FeaAudit may judge a data set instead. In this case, the Witnesses may be called forth to outline pedestrians in some video. The PA and VA then work in concert to determine if the Witness's labels are "good" by some subjective measure. In fact, a cluster of FeaAudit instances are annotator farms in the Metheus ecosystem, and allow us to generate *labeled* data that improve the quality of models trained in the QAudit instances. Unlike Amazon's Mechanical Turks or other services of similar flavor, the annotators are not paid to label. Instead they are *paying* to label data. From an in-ecosystem currency stand point, the QAudit markets are the **miat** sources, while FeaAudit instances are **miat** sinks. They work in concert to modulate the liquid supply of **miat** across Metheus markets.

PART II

Stablecoin Protocol

Draft: 5th April 2026:Metheus: Ling Xiao

CHAPTER 4

FAT and Stable

4.1 Introduction

The chapter completes the Metheus economy with the Fair, Accountable, and Transparent fiat-pegged stablecoin, or the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin. $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ will be the DeFi currency of interest for most retail users of Metheus. In the previous chapters we saw $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is exchanged for $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ along well-defined exchange rates, and $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ is needed in all the Metheus-related market operations. Thus, the growth of $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ supply and its speculative upside is tied to the overall health of the Metheus economy. The fundamentals of which are:

1. the quality and volume of AI models or training data procured by QAudit markets;
2. the quantity of fairness annotations and labeled data from FeaAudit oracles;
3. the overall rate of transactions relating to $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ -denominated wagers.

Thus $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is the first **entropy-stablecoin** tied to a bundle of essential commodities in the information economy. This is conceptually analogous to the petrodollar, also tied to a critical commodity of the modern world. Unlike the petrodollar however, Metheus **makes wagers, not war**. Because $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is based on a real economy that generates cash flows by pushing in-demand produce, it can be designed with the holder's long-term safety in mind. This means $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is fully fiat-collateralized, and features:

1. no margin calls,
2. no forced liquidations,
3. no funny business of any kind.

In sum: $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is real money, not funny money. The rest of chapter proceeds as follows:

1. section 4.2 outlines the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ issuance and redemption mechanism. In particular:
 - a) primary mint of $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ on the Metheus sidechain;
 - b) and secondary issuance of $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ on the Ethereum mainnet.
2. Section 4.3 expresses the exact relationship between $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ and $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ on a exchange-curve that is piece-wise smooth. And completes the technical narrative of how fluctuations in their relative supply ensure $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ has dividend paying value.
3. Section 4.4 develops a remarkably simple valuation methods for the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin.

4.2 FAT Issuance and Redemption

This sections shows why $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ issuance is the *raison d'être* for the Metheus sidechain.

FAT Retail Issuance

Figure 4.1 diagrams the issuance process, which originates from an off-chain $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ facility and proceeds as follows:

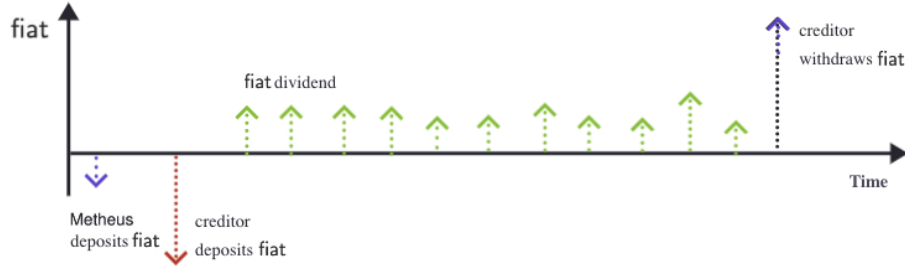


Figure 4.1: Metheus initiates the deposit process by placing some κ fraction of 1 fiat into the pre-designated *off-chain* $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ facility. Then the creditor deposits the remaining $1 - \kappa$ fiat to make 1 fully-collateralized $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin on the Metheus sidechain. This $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ receives pro-rata share of *miat* inflation dividend pool until some predesignated expiration date, at which point the principle of 1 fiat can be withdrawn by $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ holder.

1. Metheus initiates the origination process by depositing κ fraction of 1 fiat into the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ facility, where $0 < \kappa \leq \frac{1}{20}$. This initial Metheus outlay comes from Metheus's operating margins, and is booked as the "cost of deposit acquisition."
2. A creditor approaches Metheus and deposits $1 - \kappa$ in equivalently denominated fiat into the same $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ facility. The creditor receives 1 $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin minted onto the Metheus sidechain. Observe the total $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ supply in the Metheus economy now increases by 1, and there is exactly $1 - \kappa + \kappa = 1$ fiat in the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ facility corresponding to this stablecoin. It is fully collateralized. This $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ can be traded immediately on the sidechain. Now examine this arrangement from the two party's perspective:
 - a) (creditor's perspective): the fair value of this $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is 1 fiat, however he only deposited $1 - \kappa$. So his immediate profit for selling $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ on the sidechain is up to κ . Alternatively he may hold and receive κ in profit at expiration.
 - b) (Metheus's perspective): Metheus just borrowed fiat from the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -creditor at $\frac{1-\kappa}{\kappa}$ leverage. This fund can be reinvested in some low risk instruments to cover operating costs. See fig 4.2 for system value flow diagram.

In this issuance arrangement, the rate of growth in $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is tied to growth in models, data, and annotation in the Metheus ecosystem. So $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ supply is a levered measure of the system capacity to process information, hence the name entropy stablecoin. Finally, observe that since the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -creditor is interfacing with traditional financial (TradFi) systems at the point of issuance, KYC/AML is part of the purchasing process.

3. Alternatively, the creditor may "wrap" this sidechain $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin and move it onto Ethereum mainnet. $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ follows the design pattern of [SCM21], and factors into two "red-black" primitives:
 - a) ($\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ inflation dividend token): this is a claim on the inflation dividend pool. Inflation arise from the conversion process in fig 2.8 and fig 4.6. Where Auditors buy into QAudit/FeaAudit at the current $\text{fiat}:\mathfrak{F}\mathfrak{a}\mathfrak{t}:\text{miat}$ exchange rates, and cash out at higher exchange rates so that their winnings are diluted by increase in *miat* supply. $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ also have a expiration date, i.e. 6 month, until which time $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ will accumulate its pro-rata share of inflation dividends. Each $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ is owed:

$$\text{dividend owed per time period per } \mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha = \frac{\text{fiat denominated inflation pool}}{\text{sply}(\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha)}. \quad (4.1)$$

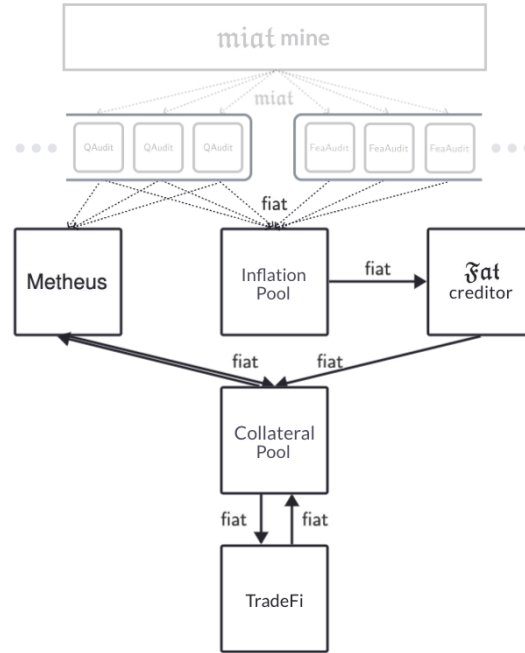


Figure 4.2: In the Metheus operating model, fiat dividend due to *miat*-expansion is placed into the *Inflation Pool*. Metheus and *Fiat*-creditors then place fiat into a joint *Collateral Pool*. This pool invests in *TradeFi* instruments. Returns from this investment are claimed by Metheus, while *miat*-inflation dividend are sent to *Fiat*-creditors.

Where $\text{sply}(\mathfrak{F}\text{at-}\alpha)$ refers to the total supply of *unexpired* outstanding *Fiat- α* . Each time period may last i.e. 7 days, so that every Sunday the entire fiat-denominated-inflation pool is distributed to all the *Fiat- α* holders. After the 6 months expiration date, the accumulation stops, and it is taken out of the payout pool, and the *Fiat- α* supply decreases.

The owner of this expired *Fiat- α* may then burn it at his own convenience. Note this implies *Fiat- α* -supply may grow unbounded. This is OK as tranches of *Fiat- α* will expire at their respective expiration date, so that the supply of un-expired *Fiat- α* that partakes in the dividend pool does not grow at a faster rate than *Fiat* is minted.

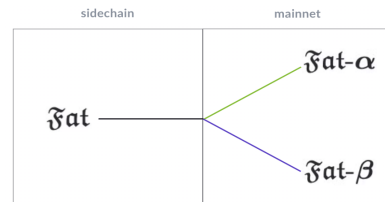


Figure 4.3: *Fiat* issuance proceeds in two steps. 1) first a *Fiat* stablecoin is minted on the Metheus sidechain, where it is fully-collateralized and pegged to 1 fiat of choice. 2) Then this *Fiat* is split into *Fiat- β* , which is redeemable for the 1 fiat underlying. And *Fiat- α* , which claims the *miat*-inflation dividend upside. Both *Fiat- α* and *Fiat- β* are recorded on Ethereum mainnet, they have variable expiration rates and are tradable immediately.

- b) ($\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ *governance stablecoin*): this is a claim on the underlying 1 fiat deposit in the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ facility. In particular, $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ has a some predefined lockup date, i.e. 6 month, after which it can be redeemed in full for the underlying 1 fiat. After redemption, both $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ on mainnet and one $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ on the Metheus sidechain are burnt. So $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ redemption decreases the supply of $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ in the Metheus sidechain, and decreases the exchange rate of $\text{miat}:\mathfrak{F}\mathfrak{a}\mathfrak{t}$ in the QAudit/FeaAudit games. However the corresponding $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ need not be burnt. This implies the instanenous supply of $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ on mainnet differs.

Most importantly, $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ is a *governance token*: $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ holders can determine certain Metheus platform policy decisions, in particular:

- i. $\text{miat}:\mathfrak{F}\mathfrak{a}\mathfrak{t}$ exchange rate floor $\underline{e}$.
- ii. $\text{miat}:\mathfrak{F}\mathfrak{a}\mathfrak{t}$ exchange rate ceiling $\bar{e}$.

Fluctuations in exchange rate significantly affect $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ holders' dividend rewards:

- i. expansion in exchange rate within any competition period $t = 1..T$ increases the amount of fiat-denominated wagers allocated to the inflation-dividend pool of fig 4.2. See fig 4.6 for conversion diagram.
- ii. However, since $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is pegged to 1 fiat of choice, the higher the $\text{miat}:\mathfrak{F}\mathfrak{a}\mathfrak{t}$ exchange rate, the cheaper it is on a cash basis to wager in the Metheus markets. This reduces the wager pool on a fiat-basis, and reduces the fiat-dividend $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ holders receive from the inflation pool.
- iii. $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ -holders must determine the exact balance. See fig 4.7 for visualization of how floor and ceilings affect exchange rate fluctuations.
- iv. Because $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ -holders redeem collateral upon their own volition, unredeemed $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ stablecoins may sit on the book for an excessively long time. Thus the total supply of $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ may approach or exceed that of miat . Metheus use the communally-defined floor $\underline{e}$ to maintain the minimum exchange rate by minting 1 miat per $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ issued.

Finally, observe $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ is a fungible token, its value is 1 fiat modulo time value, and Metheus abide by one $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ one vote. However $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ is a nonfungible token, as its issuance and expiration date affects its value.

FAT Bulk Issuance

In an alternate $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ issuance structure, the **creditor** receives $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ only, and does not move the fiat principle as no $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ has changed hands. The **creditor** in-effect enters a swap structure, whereby he sets aside a tranche of capital, and trades interest rate accrued on capital (in red in fig 4.4) with variable miat -inflation dividend accrued to the the $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ he possesses.

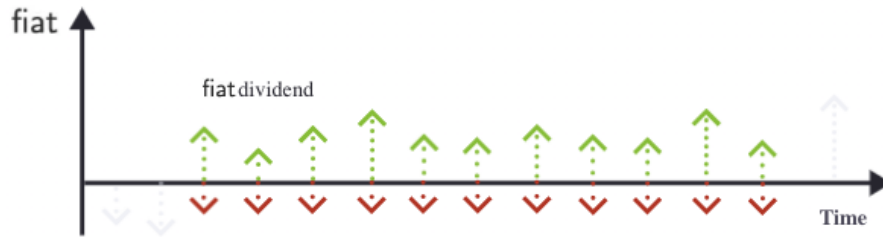


Figure 4.4: In $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ bulk issuance, the **creditor** swaps fiat due to interest on principle he has set aside, against the fiat dividend from $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ he receives. Notably, $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ is exchanged at no cost, and there is no redemption process as the **creditor** does not have $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ on hand.

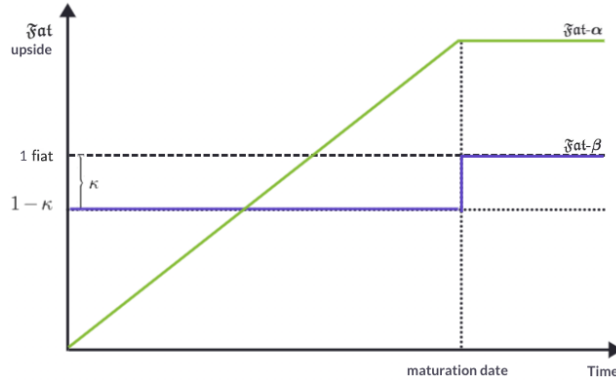


Figure 4.5: $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ upside scenario, where green is $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ and blue is $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ upside (best seen in color). In this case, the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -creditor buys $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ in the sidechain for $1 - \kappa$ fiat, and holds both primitives to expiration. He sees 1) a fixed upside of κ for $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ redemption from Metheus; and 2) receives rolling $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ -inflation dividend from $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ until expiration date. In this figure they expire on the same day. In reality this need not be the case.

As the section header suggests, $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ bulk issuance is not a retail product. It is available only through private placement.

Qualitative Stakeholder Discussion

$\mathfrak{F}\mathfrak{a}\mathfrak{t}$ holders are exposed to the upside of Metheus economy in $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$, whilst carrying very little downside in holding $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$. Moreover, by setting κ to be a function of interest rate in fiat-collateral of choice, the creditor's downside is at most zero. Once a creditor mints the $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha/\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ pair, he can sell both, hold both, or sell one whilst holding the other. Qualitatively speaking, these actions imply the following:

1. (Hold both $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$). In holding $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ until expiration, the creditor is delaying his profit of κ until redemption. In holding $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$, he is long the Metheus economy, and is rewarded with $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ -inflation dividend until expiration.
2. (Hold $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$, sell $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$). The creditor is long Metheus as before by holding $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$. However by selling $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ for up to 1 fiat, he is pocketing the $\leq \kappa$ profit now.
3. (Hold $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$, sell $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$). The creditor delays the κ profit until expiration. But is pulling the upside from $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ to the present day by selling. This could occur if he expects 1) $\mathfrak{Q}\mathfrak{A}\mathfrak{u}\mathfrak{d}\mathfrak{i}\mathfrak{t}/\mathfrak{F}\mathfrak{e}\mathfrak{a}\mathfrak{A}\mathfrak{u}\mathfrak{d}\mathfrak{i}\mathfrak{t}$ operations to enter a downturn, or 2) a deluge of $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ buyers will enter the market and dilute the $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ inflation pool.
4. (Sell both $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$). The creditor pockets the κ profit today, and also pulls the $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ upside to the present day.

In sum, the creditor who initiates $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ issuance has minimum upside of κ . Auditors should hold $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ as a way to hedge inflation risk as they partake in the Metheus economy. Steps (vii) and (viii) in fig 4.6 are pivotal aspects of the overall Metheus tokenomic package. **In Metheus, auditors are creditors.** This aligns the incentives end-to-end. So that the Auditors not only rank the Agents in "good faith" in any particular $\mathfrak{Q}\mathfrak{A}\mathfrak{u}\mathfrak{d}\mathfrak{i}\mathfrak{t}$ market, they must also long the aggregate Metheus economy overall, if they wish to realize the full extent of their wagers.

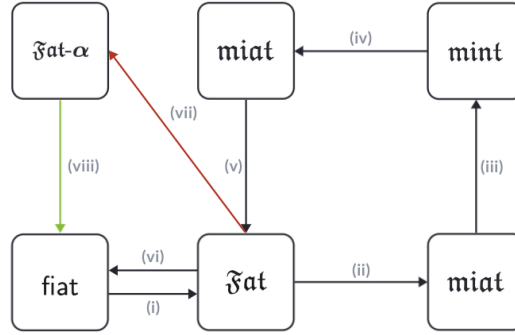


Figure 4.6: In this sequel to fig 2.8, the Auditor enters QAudit and sees his earnings diluted by *miat* inflation in steps (i) - (vi). In particular, value leakage occurs in step (v) when *miat* is converted back to *FAT* at the prevailing and higher exchange rate. This value is accrued to some tranche of *FAT-α* as dividend in step (vii). If the Auditor holds this tranche of *FAT-α*, then he sees his fiat upside returned to him in step (viii). Moreover, Auditors may also hold *FAT-α* to hedge their overall audit risk. So that even if they lose some particular stage *t*, they can nonetheless see some upside in inflation dividend.

4.3 FAT Exchange Rate

Since Metheus initiates the *FAT* issuance by depositing some amount κ fiat, the rate at which *FAT* is minted is fundamentally tied to the operating profit of Metheus. That is, the higher the QAudit/FeaAudit activity, the higher the *miat* supply to support the transaction volume, the greater the inflation and the higher the operating profit. The converse spiral is also true. And because *FAT-α*/*FAT-β* pairs mature in regular intervals, the supply of *FAT* is self-correcting. However, it is possible for the supply of *FAT* and *miat* to approach each other, or even for *FAT* to exceed *miat* supply. This is an adverse event that breaks the Metheus operating model.

Thus, Metheus features a *piece-wise exchange-rate bonding function* whereby within a certain exchange rate envelop, the *miat*:*FAT* exchange rate floats. However below this threshold $\frac{\text{miat}}{\text{FAT}} \leq \underline{e}$, one *miat* is minted for every *FAT* issued, so that the exchange rate invariant $\frac{\text{miat}}{\text{FAT}} > \underline{e}$ is maintained. Similarly, if the *miat*:*FAT* exchange rate drifts above the upper bound with $\frac{\text{miat}}{\text{FAT}} > \bar{e}$, then any *miat* in the global pool is burnt. This exchange rate floor is set by the holders of *FAT-β*, and constitutes the primary reason one would hold *FAT-β* instead of selling it to pocket the arbitrage profit. Now observe, Auditors that become *FAT*-creditors determine how much of their upside is taken by inflation. Naively, they would drive the floor $\underline{e}$ to be as low as possible. This would make sense if he:

1. expects to wager on the winning models in every stage in QAudit;
2. and/or expects to always stake the best eunuchs in every FeaAudit fairness pool.

However, if he expects to lose sometimes as FeaAudit and QAudit are designed to do, then he is better off setting a higher exchange rate floor. Critically: while inflation reduces the upside of Auditors, it also acts as *insurance* for the same Auditors *if* they hold *FAT-α* tokens, and receive inflation dividends from activities platform wide. Additionally, Auditors holding *FAT-α* tokens are also incentivized to bring other Auditors into the Metheus ecosystem, as their wagers also accrue to dividend pool. Thus *FAT-α* is both an Auditor-acquisition and Auditor-retention instrument.

The last point bears repetition, *FAT-α* tokens are *calls* for the underlying Metheus economy for retail buyers. However, if the *FAT-α* holder is also an Auditor, then he is buying *puts* on his own activity as an Auditor. Each Auditor must determine this exact balance of conflicting interests, the point of intersection is his ideal exchange rate floor $e_{m:f}$. The Metheus governance process collates the private floors due to each creditor through a voting

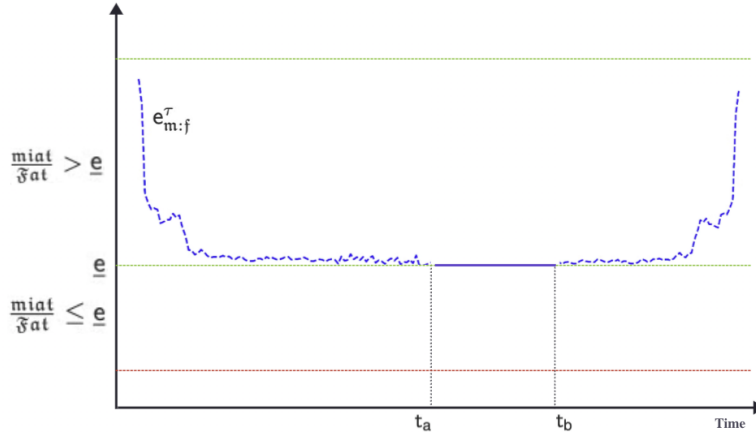


Figure 4.7: An exchange rate evolution scenario whereby $\text{miat}:\text{fiat}$ exchange rate $e_{m:f}^{\tau}$ transitions from floating to fixed, and back to floating rate again. Between the time interval $[0, t_a]$, the $\text{miat}:\text{ƒat}$ exchange rate is above floor $\underline{e}$. It precipitously falls towards value $\underline{e}$ as more ƒat are issued. On the interval $[t_a, t_b]$, the exchange rate is fixed at $\underline{e}$ as miat is mined in proportion to ƒat issuance. Beyond t_b , relative miat supply grows as $\text{QAudit}/\text{FeaAudit}$ activity recovers, so the exchange rate is allowed to fluctuate per platform activity. Normally, this exchange rate floor $\underline{e}$ is set by $\text{ƒat}-\beta$ governance token holders. However, there is an exchange rate "hard-deck" defined by Metheus (red) that $\text{ƒat}-\beta$ holders cannot broach when voting. For example, this hard-deck may be 10:1, stating that the lowest exchange rate is 10 miat per ƒat stable token. Finally, there is also an exchange rate ceiling $\bar{e}$ (green) set by $\text{ƒat}-\beta$ holders, beyond which point miat tokens in the unclaimed global pool are burnt.

process, and sets the value of $e_{m:f}$ accordingly to provide collectively-defined level of ƒat social security.

4.4 FAT Valuation

This section provides a remarkably simple valuation framework for ƒat , $\text{ƒat}-\alpha$, and $\text{ƒat}-\beta$ tokens. We take inspiration from quantitative finance, but adapt them to use basic techniques that is suitable for the underlying Metheus economy.

ƒat Valuation

Recall the creditor purchases ƒat for $1 - \kappa$, with the remaining κ contributed from Metheus operating margins. Rationally, if the dividend value of ƒat is zero, then no buyer would pay more than 1 fiat it light of its fixed collateral value. So 1 fiat is the lower bound on the price ceiling. Now we argue the price floor is 1 fiat using the adage that "the value of a thing is its arbitrage revalue." Under this conceptual trading path, creditor can profit on the Metheus side chain immediately by i.e. selling ƒat for $1 - \frac{\kappa}{2}$, and pocketing $\frac{\kappa}{2}$ in profit. Such arbitrage trades would continue until all profits disappear. Thus ƒat is worth at least 1 fiat collateral of choice on the side chain. Since trading fee is low on the Metheus side chain, a rational trader who purchased ƒat with the intention of selling it should sell ƒat on the side chain to pocket the upside. This sidechain arbitrage action maintain the ƒat price floor.

ƒat-β Valuation

On the Ethereum mainnet, the margin of κ is too low on account of mainnet transaction costs. However if the creditor hold $\text{ƒat}-\beta$ en bulk, then he can pursue arbitrage opportunity

as above, until the the revalued price of $\mathfrak{F}\text{at-}\beta$ is close to 1 fiat. Most importantly, $\mathfrak{F}\text{at-}\beta$ can be redeemed for exactly 1 fiat, and can never de-peg modulo TradFi sanction risks.

$\mathfrak{F}\text{at-}\alpha$ Valuation

This is the token of interest for an alternative valuation method. Examining the green curve in fig 4.5, it is clear that $\mathfrak{F}\text{at-}\alpha$ is in essence a call on the miat -inflation pool. The volume of this pool is a wholistic measure of "temperature" in the Metheus economy. So it appears reasonable to use Black-Scholes to price $\mathfrak{F}\text{at-}\alpha$. However we will pursue a different path, this is sensible for a few reasons:

1. (*Audience background*): Black-Scholes was developed by people from the physical sciences. They work in continuous space-time, and express stock-price movement with continuous random walk (Brownian motion) [Shr10]. This monograph's target readers are programmers. They work in discrete time, are more comfortable with tossing coins, traversing upside-down trees, and use simple sums and vanilla integrals.
2. (*System Dynamics*): Black-Scholes places very little assumption on the underlying system model. Metheus's underlying economy is specified exactly in this monograph, so we can and should state the value of $\mathfrak{F}\text{at-}\alpha$ as a function of these parameters.
3. (*Practical Considerations*): in reality Black-Scholes is not used to price options anyways. Instead the market prices options, while Black-Scholes is used to compute the implied volatility with the price as an input. ¹

In valuing $\mathfrak{F}\text{at-}\alpha$, there are a few assumptions about how the Metheus economy operates:

1. Users cannot burn miat on an ad-hoc basis. This is can be set in the miat -contract, so that miat cannot be sent to the burn address $0x00\ldots$ except by the Metheus admin.
2. Metheus burns miat in an ad-hoc manner, which is not modeled in this section. In practice, burn events will be signaled well ahead of time, so that $\mathfrak{F}\text{at-}\alpha$ buyers are aware of its influence on miat -inflation dividends.
3. The dividend payment period of eqn 4.1 is:
 - a) roughly in sync with audit period T of QAudit .
 - b) Greater than the rate at which Witnesses are rejected in FeaAudit .
 - c) Within this duration T , the **Auditors** and **Agents**/Witnesses can only liquidate their position at an exchange rate $e_{m:f}^{t^f}$ that is no lesser than the exchange rate $e_{m:f}^{t^o}$ they bought into the market. That is:

$$e_{m:f}^{t^o} \leq e_{m:f}^{t^f}, \text{ where } t^o < t^f. \quad (4.2)$$

This market constraint prevents the unfortunate scenario where the **Auditors** buys in at 10:1 $\text{miat}:\mathfrak{F}\text{at}$ exchange rate, and liquidate at 8:1 exchange rate, and discover that there is not enough money in the fiat-denominated wager pool.

For example, if $\mathfrak{F}\text{at-}\alpha$ expires in 26 weeks, and each dividend payment period is 7 days, then the $\mathfrak{F}\text{at-}\alpha$ -creditor is paid 26 times from the miat -inflation pool (see fig 4.8). Recall in each payment period, the entire inflation pool is distributed pro-rata to every $\mathfrak{F}\text{at-}\alpha$ holder. Given this constraint, we can set T to be 7 days, so that at the end of every week the **Agent**-submitted models are tested on the hidden test set, and **Auditors'** fiat wagers are distributed according to miat inflation. This fast turnover is ideal:

¹That is except in illustrious shops such as Long-Term Capital Management, which despite its name blew up in short order.

- a) faster cash turnaround allow **Auditors** to liquidate their positions, pocket their winnings, and possibly buy back into the market. This increases emotional engagement and increases retention.
- b) Faster rate of payout allow **Metheus** to flood the **miat**-inflation pool with fiat, so that **fat-α** holders are paid out each week. In the unthinkable arrangement where i.e. T in **QAudit** is 52 weeks, then **fat-α** holders may never see a cash return on their holdings.
- c) Regular payouts significantly simplify the valuation model we present below.

Naturally, **QAudit** markets do not last 7 days only, recall in 2.7 both the **PA** and **Auditor** have a chance to raise the bounty value and reset the **QAudit** instance.

All in all, these assumptions will be reified as constraints into the **Metheus** ecosystem calendar. Now we will proceed with a step by step valuation instruction, building towards eqn 4.15. The **fat-α** valuation method is a brute-force sampling regime. It samples all "value-generating-paths" the **Metheus** market can take over each dividend period, and compute their expected value. This program serves as both:

1. a valuation model for **fat-α**;
2. a system dynamics model for the entire **Metheus** economy.

The system model is framed by four hyper-parameters: p , q , μ , and σ , along with proper support over said distributions. The meanings of these parameters are expounded on at length in the main body. The hierarchical sampling process used to compute **fat-α** intrinsic value is summarized in fig 4.9. Now we name the elementary variables and/or entities that underlie the **Metheus** economy:

1. (time step τ): the *discrete* time step τ is an elementary input into the **fat-α** valuation model. Its numerical value should be selected based on the frequency of audit transactions in the **Metheus** ecosystem. For example, if every 10 minutes an **Auditor** or **Witness** approaches **QAudit** and **FeaAudit** respectively, then $\tau = 10$ minutes. Thus τ is a measure of system frequency or hertz.
2. (dividend period $[\tau_0, \tau_f]$): indexed by time step τ . Given $\tau = 10$ minutes, and every dividend period is 7 days, then $\tau_f = 7 \cdot 24 \cdot 60 / 10 = 1008$. And we have a sequence of time step indices for *every* dividend period:

$$[\tau_0, \tau_f] = [\tau_0, \tau_1, \tau_2, \dots, \tau_{1007}], \text{ where } \tau_f = \tau_{1007}. \quad (4.3)$$

Following the example above, if **fat-α** expires in 6 months, then there are 26 such $[\tau_0, \tau_f]$ intervals. The valuation model will estimate the dividend accrued in every step τ of every dividend interval.

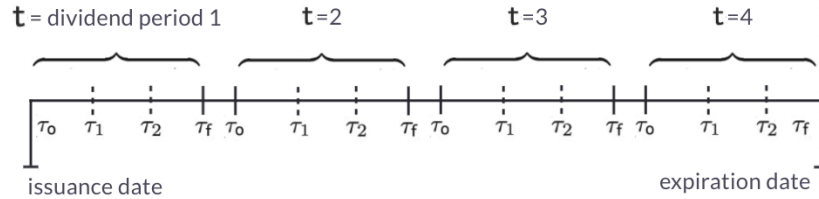


Figure 4.8: In this simple example, **fat-α** expires after 4 dividend periods. Each period has 4 time steps: $[\tau_0, \tau_1, \tau_2, \tau_f]$. This valuation method estimates the **miat** and **fiat** emission from each time step τ , and compute their expected value.

3. (*exchange rate* $e_{m:f}^\tau$): the $\text{miat}:\mathfrak{F}\text{at}$ exchange rate as a function of time-step τ . As miat is mined, the exchange rate $e_{m:f}^\tau$ increases. Similarly, as $\mathfrak{F}\text{at}$ is issued, $e_{m:f}^\tau$ decreases; as $\mathfrak{F}\text{at}$ expires, $e_{m:f}^\tau$ increases. Thus $e_{m:f}^\tau$ is not an independent r.v. in the system, but rather a consequence of change in supply of miat and $\mathfrak{F}\text{at}$.
4. (*AuditUnit*): this is an abstraction of both *FeaAudit* and *QAudit* market protocols of the last two chapters. That is:

$$\text{AuditUnit} \triangleq \text{QAudit} \mid \text{FeaAudit}.$$

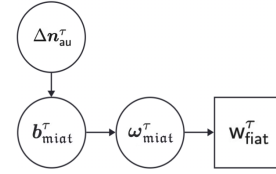
In the context of $\mathfrak{F}\text{at}-\alpha$, each *AuditUnit* instance is in essence a vending machine. At each time step τ , Auditor put in fiat, and receive some amount of fiat back. Concurrently, this *AuditUnit* vending machine has miat flow characterized by figure 4.10.



Figure 4.10: The *AuditUnit* abstraction and its interaction with the miat and fiat collateral pool. 1) On top, the *AuditUnit* accepts fiat from Auditors, a portion of which is cycled into the fiat collateral pool due to mined miat . This positive fiat outflow increases overall miat supply, and increases the fiat-denominated miat -inflation pool. On the bottom, a similar story with fiat, except net miat -flow is zero, so overall miat supply does not change. Thus no fiat is placed into the inflation dividend pool.

Every *AuditUnit* is associated with three random variables (r.v.): 1) b_{miat}^τ , 2) $\omega_{\text{miat}}^\tau$, and 3) w_{fiat}^τ . They are sampled in a hierarchical process in the order with which they are listed.² At every time step τ , the sampling process proceeds as follows:

Figure 4.11: On the time interval $[\tau_0, \tau_f]$, a three step sampling process proceeds as follows: 1) first the multinomial r.v. $\Delta n_{\text{au}}^\tau$ denoting the fluctuations in the number of *AuditUnit* is sampled. 2) Then the Bernoulli r.v. b_{miat}^τ is sampled, stating whether the Auditor is purchasing newly mined miat . 3) Next $\omega_{\text{miat}}^\tau$ is sampled from the truncated normal distribution for the Auditor's miat -denominated order volume. Finally the sampled $\omega_{\text{miat}}^\tau$ value converts to w_{fiat}^τ at the current exchange rate $e_{m:f}^\tau$.



- a) (*Bernoulli r.v.* $b_{\text{miat}}^\tau \stackrel{q}{\sim} [0, 1]$) modeling the likelihood that the Auditor who approached this *AuditUnit* does not have sufficient funds in his wallet, so that he buys miat to wager. Define:

$$\mathbb{p}(b_{\text{miat}}^\tau = 1) = q. \quad (4.4)$$

If we sample $b_{\text{miat}}^\tau = 0$ then the program stops here. However if $b_{\text{miat}}^\tau = 1$, then the Auditor does not have sufficient miat . Metheus mines miat and sell it to this Auditor. Consequently, this valuation program proceeds to the next step.

- b) ($\omega_{\text{miat}}^\tau$ sampled from the truncated normal distribution) with support on $\mathbb{R}^+ \cup \{0\}$:

$$\omega_{\text{miat}}^\tau \sim \mathcal{N}(\mu, \sigma, [0, L]), \quad (4.5)$$

where L is some suitably large number, μ and σ are analyst-chosen values based on historical wagering patterns. This r.v. defines how much miat the Auditor

²This sampling process resembles latent Dirichlet allocation process for topic modeling in classical natural language processing models.

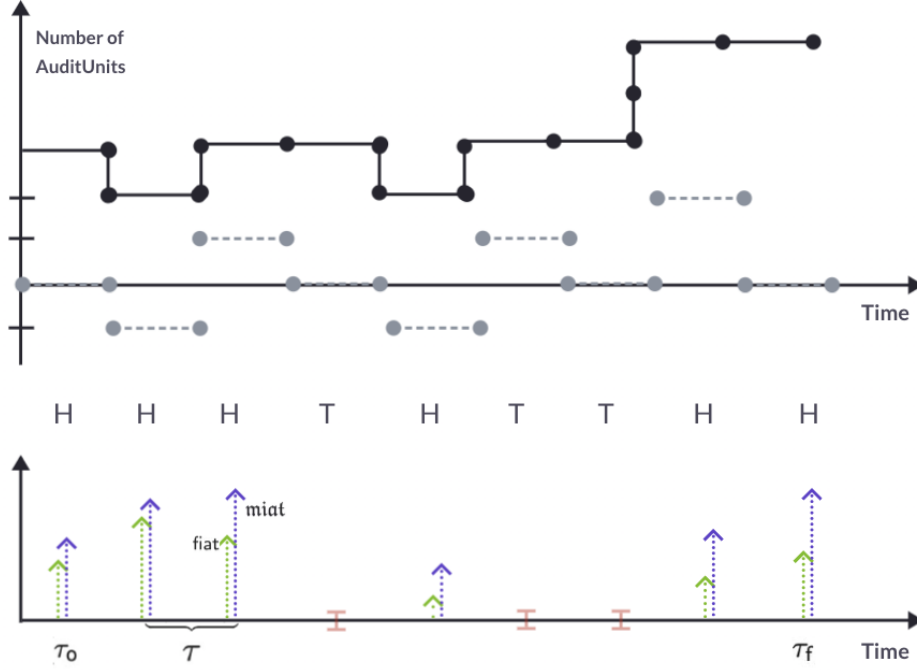


Figure 4.9: $\mathfrak{F}\text{at-}\alpha$ is valued by computing its *expected* fiat-denominated dividend yield over all dividend periods. The fixed duration of each period is denoted with $[\tau_0, \tau_f]$. In every $\tau \in [\tau_0, \tau_f]$, an Auditor approaches the AuditUnit and wagers some amount of **miat** in the market at the current **miat**: $\mathfrak{F}\text{at}$ exchange rate. This diagram shows the sampling process for *one* dividend paying period.

- Top: the valuation program first generates an example path of AuditUnits. Denote the number of AuditUnit at time τ with n_{au}^τ , then the delta in this value is modeled with multinomial r.v. $\Delta n_{\text{au}}^\tau$. The program samples $\Delta n_{\text{au}}^\tau$ $\tau_f - 1$ times (gray dotted lines) to construct the number of AuditUnits over time interval $[\tau_0, \tau_f]$ (discrete black trajectory). For example if $n_{\text{au}}^\tau = 10$, then there are 10 AuditUnits operating at time step τ .
- Middle: τ_f coins with bias q are tossed, expressing the likelihood that the Auditor who approached the AuditUnit has sufficient **miat** on hand. Denote this Bernoulli r.v. with b_{miat}^τ , so that $b_{\text{miat}}^\tau = \text{H}$ implies that the Auditor has to buy **miat** with **fiat**, thus increasing the total **miat** supply and contributing to the inflation pool. If $b_{\text{miat}}^\tau = \text{T}$, then no **miat** is mined and no fiat spent.
- Bottom: At every step τ where $b_{\text{miat}}^\tau = \text{H}$, sample the Auditor's **miat**-denominated buy order from the truncated normal distribution with $\omega_{\text{miat}}^\tau \sim \mathcal{N}(\mu, \sigma, [0, L])$. Finish by computing the fiat amount using the current exchange rate $e_{\text{m:f}}^\tau$ updated at each time step τ .

The process above generates the expected **miat** mined and fiat spent *per* future path within one dividend period. The program then sums over all future paths to compute the expected value of $\mathfrak{F}\text{at-}\alpha$ for one dividend paying period. Repeat for N dividend periods and we have $\mathbf{E}[\mathfrak{F}\text{at-}\alpha]$, the intrinsic value of $\mathfrak{F}\text{at-}\alpha$.

will wager, and $\{0\}$ in the domain expresses an Auditor who declines to wager. Additionally, the initial **miat** value is a constant denoted with ω_{miat}^0 , it is the global supply of **miat** on the Metheus sidechain at the beginning of dividend period τ_0 . If we sample $\omega_{\text{miat}}^\tau > 0$, then we can proceed to the final step.

- c) (*Induced fiat value w_{fiat}^τ*): the generated value $\omega_{\text{miat}}^\tau$ converts to w_{fiat}^τ value at the current exchange rate with:

$$w_{\text{fiat}}^\tau = \frac{\omega_{\text{miat}}^\tau}{e_{\text{m:f}}^\tau}, \quad \text{where } e_{\text{m:f}}^\tau = \frac{\text{sply}(\text{miat}, \tau)}{\text{sply}(\mathfrak{F}\text{at}-\beta, \tau)}. \quad (4.6)$$

w_{fiat}^τ is the Auditor's fiat-denominated spend at time τ on one AuditUnit. Note since $e_{\text{m:f}}^\tau > 1$ by construction (see fig 4.7), $w_{\text{fiat}}^\tau < \omega_{\text{miat}}^\tau$ for every τ .

- d) (Number of audit units n_{au}^τ): the Metheus economy is populated by AuditUnits, they are the elementary settings of production. When PAs initiate new QAudit or FeaAudit markets, n_{au} increases. When markets wind down and proceeds are distributed, n_{au} decreases. We model the change in number of AuditUnit with the multinomial r.v. $\Delta n_{\text{au}}^\tau$. It takes on discrete value that the analyst may select depending on his expectation of Metheus market conditions. So the range of n_{au} is a "model hyper-parameter." For example, $\Delta n_{\text{au}}^\tau$ may range over $[-1, 0, 1]$, stating that at any time step τ , the number of AuditUnits may decrease by one, remain constant, or increase by 1. Then the analyst may assign discrete probability for every $\Delta n_{\text{au}}^\tau \in [-1, 0, 1]$ with:

$$\mathbf{p}(\Delta n_{\text{au}}^\tau = -1) = \mathbf{p}_{-1} \quad \mathbf{p}(\Delta n_{\text{au}}^\tau = 0) = \mathbf{p}_0 \quad \mathbf{p}(\Delta n_{\text{au}}^\tau = 1) = \mathbf{p}_1. \quad (4.7)$$

This is effectively a three-sided dice tossed i.i.d. In the simplest case, the distribution may be uniform with $\mathbf{p}(\Delta n_{\text{au}}^\tau) = \frac{1}{3}$ w.r.t the example above. The initial number of AuditUnits in every dividend period is denoted n_{au}^0 .

Now we sample one value generating trajectory for the first dividend period:

1. given initial value n_{au}^0 at the moment of $\mathfrak{F}\text{at}-\alpha$ issuance, generate a n_{au} trajectory for the first dividend period. This is done by tossing the Δn_{au} dice $\tau_f - 1$ times for this dividend period, generating the time indexed integer sequence $[n_{\text{au}}]$:

$$[n_{\text{au}}] = [n_{\text{au}}^{\tau_0}, n_{\text{au}}^{\tau_0+1}, n_{\text{au}}^{\tau_0+2}, \dots, n_{\text{au}}^{\tau_f-1}, n_{\text{au}}^{\tau_f}],$$

where $n_{\text{au}}^{\tau+1} = n_{\text{au}}^\tau + \Delta n_{\text{au}}^\tau$.

(4.8)

By the i.i.d assumption, the likelihood of this specific trajectory is:

$$\mathbf{p}([n_{\text{au}}]) = \prod_{\tau=\tau_0}^{\tau_f-1} \mathbf{p}(\Delta n_{\text{au}}^\tau). \quad (4.9)$$

Observe that by modeling $\Delta n_{\text{au}}^\tau$ instead of n_{au}^τ , we simplify the random experiment. See fig 4.9 or figure below for visuals of this trajectory.

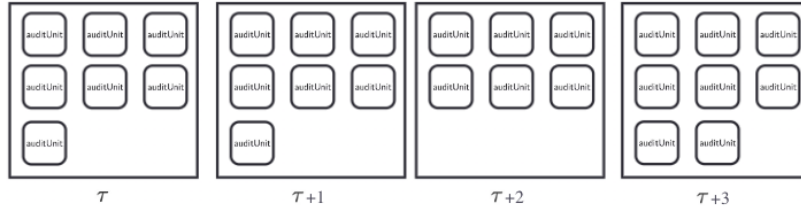


Figure 4.12: The fundamental value drivers in the Metheus are AuditUnits. This valuation method generates possible futures in (de)growth of AuditUnits over dividend period $[\tau_0, \tau_f]$, and estimate the **miat**/fiat expenditure in each unit.

2. Next for every **AuditUnit** in the trajectory $[n_{au}]$, toss the q -biased coin τ_f times to generate a bit string of length at most τ_f :

$$[b_{miat}] = [b_{miat}^{\tau_o}, \dots, b_{miat}^{\tau_f}], \text{ where } b_{miat}^{\tau} \stackrel{d}{\sim} [0, 1]. \quad (4.10)$$

In fig 4.9, one such τ_f -bit string is shown as HHHT... Observe that if an **AuditUnit** was added *within* the $[\tau_o, \tau_f]$ interval, then the length of its bit string $[b_{miat}]$ is lesser than τ_f . We solve this problem by padding it with 0's so that $|[b_{miat}]| = \tau_f$. The interpretation is that there is a phantom **AuditUnit** that the **Auditors** cannot approach, until it "comes online." The probability of any *singular* τ_f -bit string with $0 \leq h \leq \tau_f$ heads is then:

$$p([b_{miat}]) = q^h(1-q)^{\tau_f-h}. \quad (4.11)$$

3. Next for every $\tau \in [\tau_o, \tau_f]$ and each **AuditUnit**, compute the expected value of ω_{miat}^{τ} according to eqn 4.5. That is for every **AuditUnit**_k:

$$[E_{q,\mu,\sigma}[\omega_{miat}]]_k := [E_{q,\mu,\sigma}[\omega_{miat}^{\tau_o}], E_{q,\mu,\sigma}[\omega_{miat}^{\tau_o+1}], \dots, E_{q,\mu,\sigma}[\omega_{miat}^{\tau_f-1}], E_{q,\mu,\sigma}[\omega_{miat}^{\tau_f}]],$$

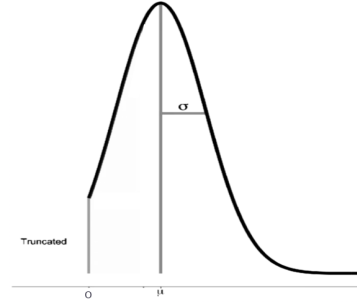
where $E_{q,\mu,\sigma}[\omega_{miat}^{\tau}] = b_{miat}^{\tau} \int_{\omega_{miat} \in [0,L]} \omega_{miat} f(\omega_{miat}; \mu, \sigma, [0, L]) d\omega_{miat}$. (4.12)

Here $f(\omega_{miat}; \mu, \sigma, [0, L])$ is the truncated normal distribution. Each $E_{q,\mu,\sigma}[\omega_{miat}^{\tau}]$ corresponds to a blue arrows in fig 4.9. Repeating this sampling process for every **AuditUnit**_k yields set of sequences:

$$\left\{ [E_{q,\mu,\sigma}[\omega_{miat}]]_1, \dots, [E_{q,\mu,\sigma}[\omega_{miat}]]_k, \dots, [E_{q,\mu,\sigma}[\omega_{miat}]]_{n_{au}^{\tau}} \right\},$$

This is the **miat** mint events on every **AuditUnits** over dividend interval $[\tau_o, \tau_f]$.

Figure 4.13: For every $\tau \in [\tau_o, \tau_f]$ at each **AuditUnit**, the expected value of **miat**-denominated order $E_{q,\mu,\sigma}[\omega_{miat}^{\tau}]$, is calculated by integrating the truncated normal distribution with mean μ , and variance σ^2 . In this model, every **AuditUnit** at each time τ has the same parameters. In a refinement, these values may be a function of each **AuditUnit** and time step.



4. Given this sequence of **miat** mint events, we can express the induced sequence of **fiat** spent across every **AuditUnit** over the dividend period $[\tau_o, \tau_f]$:

$$\left\{ [E_{q,\mu,\sigma}[w_{fiat}]]_1, \dots, [E_{q,\mu,\sigma}[w_{fiat}]]_k, \dots, [E_{q,\mu,\sigma}[w_{fiat}]]_{n_{au}^{\tau}} \right\},$$

where $[E_{q,\mu,\sigma}[w_{fiat}]]_k := [E_{q,\mu,\sigma}[w_{fiat}^{\tau_o}], \dots, E_{q,\mu,\sigma}[w_{fiat}^{\tau_f}]]$,

and $E_{q,\mu,\sigma}[w_{fiat}^{\tau}] = \frac{E_{q,\mu,\sigma}[\omega_{miat}^{\tau}]}{e_{m:f}^{\tau}}$. (4.13)

Note we convert from **miat** to **fiat** using instananeous exchange rate τ with:

$$e_{m:f}^{\tau} = \frac{\omega_{miat}^o + \sum_k^{n_{au}^{\tau}} E_{q,\mu,\sigma}[\omega_{miat}^{\tau_o}] + \dots + E_{q,\mu,\sigma}[\omega_{miat}^{\tau}]}{\text{sply}(\mathfrak{F}at-\beta + 1, \tau)}. \quad (4.14)$$

This is the sum over all the **miat** that has been mined across all **AuditUnits** thus far at time τ , divided by the initial supply of **fat-β** plus one. This $e_{m:f}^\tau$ value should then be bounded by the exchange rate floor and ceiling $(\underline{e}, \bar{e})$.

5. Then compute the amount of **fiat** allocated to the inflation pool at the end of the 1st dividend period τ_f with:

$$\bar{\varrho}^1 = \mathbf{E}_{q,\mu,\sigma} \left[\text{sply}(w_{\text{fiat}}^{\tau_f}) \right] = \left(1 - \frac{e_{m:f}^{\tau_0}}{e_{m:f}^{\tau_f}} \right) \cdot \sum_i^{n_{\text{au}}} \sum_{\tau=\tau_0}^{\tau_f} \mathbf{E}_{q,\mu,\sigma} [w_{\text{fiat}}^\tau], \quad \text{with } e_{m:f}^{\tau_0} \leq e_{m:f}^{\tau_f}.$$

At the end of the first dividend period τ_f , **AuditUnits** liquidates the **Auditors'** positions, whence $e_{m:f}^{\tau_0}/e_{m:f}^{\tau_f}$ fraction of the **fiat**-denominated wagering pool is inflated away into the dividend pool. Note without inflation, $\bar{\varrho}^1 = 0$.

6. Now repeat steps (1) - (5) above over all future trajectories on $[n_{\text{au}}^\tau]$, $[b_{\text{miat}}^\tau]$, and $[\omega_{\text{miat}}^\tau]$ sequences over $[\tau_0, \tau_f]$. Observe:
 - a) $\Delta n_{\text{au}} \in [1..m]$, so there are m^{τ_f-1} possible sequences of $[n_{\text{au}}]$.
 - b) $b_{\text{miat}} \in [0, 1]$, so there are 2^{τ_f} τ_f -bit-string of form $[b_{\text{miat}}]$ for every sequence $[n_{\text{au}}]$.
 - c) The values of $\omega_{\text{miat}}^\tau$ are the same in every trajectory since we are using the expected value $\mathbf{E}_{q,\mu,\sigma} [\omega_{\text{miat}}^\tau]$.

Therefore the valuation program considers $m^{\tau_f-1} \times 2^{\tau_f}$ paths of arriving at cumulative dividend pool denominated in **fiat**, at time $t = 1$:

$$\{ \bar{\varrho}_1^t, \dots, \bar{\varrho}_{m^{\tau_f-1} \times 2^{\tau_f}}^t \}.$$

Define $(\Delta n_{\text{au}})_r$ as r th value of $[1..m]$, then the expected value of dividends over all trajectories is written:

$$\begin{aligned} \mathbf{E}_{p,q,\mu,\sigma} [\bar{\varrho}] &= \left[\sum_p \sum_{h=0}^{\tau_f} \binom{\tau_f}{h} q^h (1-q)^{\tau_f-h} \bar{\varrho}_i^t \right] \quad \text{where } \bar{\varrho}_i^t \in \{ \bar{\varrho}_1^t, \dots, \bar{\varrho}_{m^{\tau_f-1} \times 2^{\tau_f}}^t \}, \\ \text{and } \left[\sum_p \right] &:= \sum_{(\Delta n_{\text{au}})_1=0}^{\tau_f-1} \dots \sum_{(\Delta n_{\text{au}})_m=0}^{\tau_f-1} \frac{(\tau_f-1)!}{(\Delta n_{\text{au}})_1! \dots (\Delta n_{\text{au}})_m!} p \left([n_{\text{au}}] \right). \end{aligned} \quad (4.15)$$

7. Now this expected dividend pool is allocated pro-rata to every **fat-α** holder at $t = 1$:

$$\mathbf{E}_{p,q,\mu,\sigma} [\varrho^t] = \frac{\mathbf{E}_{p,q,\mu,\sigma} [\bar{\varrho}^t]}{\text{sply}(\text{fat-}\alpha, t) + 1}.$$

This is the expected dividend value due to one additional **fat-α** purchased by the analyst at the end of the 1st dividend period τ_f , *if* this **fat-α** is minted from one additional **fat** token issued.

8. The quantity $\mathbf{E}_{p,q,\mu,\sigma} [\varrho^t]$ is the expected value of dividend paid out to **fat-α** holder in the 1st period, over all possible futures on interval $[\tau_0, \tau_f]$. Now we compute the next

period, updating the initial conditions as the average of the previous sample paths:

$$\begin{aligned}
\left(w_{\text{fiat}}^{\tau_0}\right)^{t+1} &= 0 & \left(n_{\text{au}}^0\right)^{t+1} &= \left[\sum_p\right] n_{\text{au}}^{\tau_f} \\
\left(\omega_{\text{miat}}^0\right)^{t+1} &= \omega_{\text{miat}}^0 + \left[\sum_p\right] \sum_{h=0}^{\tau_f} \binom{\tau_f}{h} q^h (1-q)^{\tau_f-h} \bar{\omega}_{\text{miat}} \\
\text{where } \bar{\omega}_{\text{miat}} &= \sum_i^{n_{\text{au}}} \sum_{\tau=\tau_0}^{\tau_f} \mathbf{E}_{q,\mu,\sigma} \left[\omega_{\text{miat}}^{\tau} \right] \\
\left(e_{\text{m:f}}^{\tau_0}\right)^{t+1} &= \frac{\left(\omega_{\text{miat}}^0\right)^{t+1}}{\text{sply}(\mathfrak{fat}-\beta+1, \tau_0)}. \tag{4.16}
\end{aligned}$$

Here $w_{\text{fiat}}^{\tau_0} = 0$ as by construction, the entire dividend pool is paid out. Repeating steps (1) - (7), the program arrives at $\mathbf{E}_{p,q,\mu,\sigma}[\varrho^{t+1}]$ in the next period.

9. Iterating steps (1) - (8) over all dividend periods, with appropriately updated initial conditions $n_{\text{au}}^{\tau_0}$, $\omega_{\text{miat}}^{\tau_0}$, and $e_{\text{m:f}}^{\tau_0}$. We arrive at the expected value of $\mathfrak{fat}-\alpha$ with:

$$\mathbf{E}_{p,q,\mu,\sigma}[\mathfrak{fat}-\alpha] = \sum_{t=1}^N \mathbf{E}_{p,q,\mu,\sigma}[\varrho^t] \tag{4.17}$$

where N is the total number of dividend periods for this particular $\mathfrak{fat}-\alpha$ token, and the analyst should discount each $\mathbf{E}_{p,q,\mu,\sigma}[\text{dividend}_t]$ factor with hurdle rate of choice.

Some final commentaries on this valuation method:

1. as promised, the intrinsic value of $\mathfrak{fat}-\alpha$ is a statement of the system dynamics of the Metheus economy. Baked within steps (1) - (9) above are all the constraints of the FeaAudit and QAudit market protocols. In particular:
 - a) the regular liquidation of wagering positions;
 - b) the frequency with which Auditors approach the market;
 - c) the protocol's floating exchange rate.
2. In this valuation model, we do not differentiate between AuditUnits or Auditors' behaviors, this is why μ , σ , and q are the same across all units. In reality, some AuditUnits will see more activity than others. So their respective parameters may differ. However this will also make the valuation model more computationally complex.
3. Because all the values are fixed a-priori, the values of eqn 4.15 and eqns 4.16 do not need to be "sampled." Instead their value can be computed quickly and exactly. However if the analyst model changes in parameters, then the framework requires true sampling.
4. Instead of resetting the initial condition as 4.16 suggests, it is possible to express the entire trajectory of $[n_{\text{au}}]$, $[b_{\text{miat}}]$, $[\omega_{\text{miat}}]$, and $[w_{\text{fiat}}]$ over all dividend paying periods. So that each trajectory is of length $N \times (\tau_f - \tau_0)$, and i.e. the number of trajectories in $[b_{\text{miat}}]$ is $2^{N \times (\tau_f - \tau_0)}$. Then the valuation program would walk down the N dividend paying periods, and compute the dividend owed in that interval.
5. This model assumes Δn_{au} , b_{miat} , and ω_{miat} are independent. In practice they do depend on each other in ways that is difficult to express, so the principle of independence minimizes the number of assumptions the model has to make.
6. What is not modeled in this method is the increase in supply of $\mathfrak{fat}-\beta$. The analyst may find their own model for increase in supply per market conditions, and enter into the last expression in eqn 4.16 in $\text{sply}(\mathfrak{fat}-\beta, \tau_0)$.

7. Estimated $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ value may differ greatly depending on the choice of:

- a) the hyper-parameter set $\{\mu, \sigma, \mathfrak{q}, \mathfrak{p}\}$;
- b) the support of $\mathfrak{p}$ in $\Delta \mathfrak{n}_{\text{au}} \in [1 \dots m]$;
- c) the range $[0, L)$ the truncated normal distribution is defined over.

Large range of estimated values is ideal as generous $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ spreads stimulate trading activity. Recall $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ is a retail product, it gives the average folk a chance to speculate on the future of the AI economy.

Remark 4.4.1. (Alternative $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ mechanism). Instead of regular $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ -inflation dividend payouts, Metheus can withhold distribution, and become a $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ market-maker of last resort. Thus Metheus is both the $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ issuer and a $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\text{Amm}$ with this arrangement:

4.4.1.1. issue $\mathfrak{F}\mathfrak{a}\mathfrak{t}$, $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ as before. In particular, $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ still has voting rights, and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ has an expiration date attached as fig 4.8 shows.

4.4.1.2. Collect fiat-denominated inflation into the inflation pool, but do not distribute.

4.4.1.3. At any point in time τ before the $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ expiration date, its holder may approach the $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ Amm and sell it to Metheus at this price:

$$\text{price}(\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha) = \frac{\text{sply}(\text{fiat inflation pool}, \tau)}{\text{sply}(\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha, \tau)}.$$

That is to say the Amm price of $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ is the pro-rata distribution rate. The fiat value of $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ in the dividend vs. buy-back scenarios are comparable modulo discount factor. Hence the valuation method developed before still applies.

4.5 Conclusion

This chapter specifies the Fair, Accountable, and Transparent stablecoin, or $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin. It is fully collateralized by fiat of choice, whereby both Metheus and the $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\text{creditor}$ contribute to the collateral pool. This arrangement allows $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ -holders to buy 1 unit of fiat at a discounted value, and gives Metheus a persistent source of leverage. Furthermore, $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ factors into $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ expressing the $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}$ -inflation dividend, and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ claiming the underlying collateral of exactly 1 fiat. In an ideal scenario, Metheus-Auditors are also $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\text{creditors}$. This aligns the incentive end-to-end, so that Auditors are committed to the platform's long term success, and receive commensurate $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ upside from value-generating activities ecosystem wide.

Additionally $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ is a governance token, its holders define the $\mathfrak{m}\mathfrak{i}\mathfrak{a}\mathfrak{t}:\mathfrak{F}\mathfrak{a}\mathfrak{t}$ exchange rate envelope. The Metheus operating model hinges on fluctuations in this exchange rate, so $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ holders directly control how much of the Auditors' collective wagers are returned to the winners, vs. allocation to $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ social security. The chapter proceeds to value $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ and $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ as a function of the underlying economy. It argues the arbitrage revalue of $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\beta$ tokens is 1 fiat even though its initial sale price is less than one. Then we express the intrinsic value of $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ as a consequence of Metheus's underlying system dynamics. The $\mathfrak{F}\mathfrak{a}\mathfrak{t}\text{-}\alpha$ valuation framework is the system model.

4.6 Postscript

In so far as we know, $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ is the first **entropy stablecoin**. Its rate of issuance and $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ upside is a direct function of the underlying machine-intelligence economy. This point is critical, whereas most stablecoins are based on pure speculation, or just passively accept deposits, the $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ stablecoin mediates a true economy. "Auditing" then is a wholistic way to measure the myriads of achievements within this sector. $\mathfrak{F}\mathfrak{a}\mathfrak{t}$ holders are not only believers of machine-intelligence as a transformational computing paradigm, they are in fact pushing it

forward by broadening the horizon of what is possible. Together we invest in more models, more data, more talent, and more mechanisms of governing the future of this technology.

Draft: 5th April 2026:Metheus: Ling Xiao

Appendices

Draft: 5th April 2026:Metheus: Ling Xiao

APPENDIX A

Monograph Notation

Elementary Notions

Notation	Description
Amm	Automated market maker
creditor	buyer of $\mathfrak{F}at$ stablecoins
$Agent_k$	the k th Agent
$Auditor_r$	the r th Auditor
PA	the principle Auditor
Wit_k	the k th Witness
VA	the vice Auditor or eunuch
Cohort	a set of Witnesses marked by demographic data
AuditUnit	either an instance of FeaAudit or QAudit market protocol
τ	one time increment within one stage, i.e. 10 minutes
t	one stage, may be i.e. 24 hours
T	total number of stages, may be i.e. 7 days
fiat	any fiat of choice
miat	the <i>Machine Intelligence Audit Token</i> , a ERC20 token on the Metheus sidechain
$mint_k^t$	the <i>Machine Intelligence Nonfungible Token</i> for the k th model at stage t , a ERC1155 nonfungible token on the Metheus side chain
$feat_k^t$	the <i>Fairness, Equality, and Equity token</i> for the k th Witness at stage t a ERC721 nonfungible token on the Metheus side chain
$\mathfrak{F}at$	the <i>Fair Accountable Transparent stablecoin</i> , pegged to 1 fiat of choice
$\mathfrak{F}at-\beta$	the $\mathfrak{F}at$ ERC20 stablecoin pegged to 1 fiat of choice on the Ethereum mainnet
$\mathfrak{F}at-\alpha$	the $\mathfrak{F}at$ high-variance ERC721 nonfungible token on the Ethereum mainnet
κ	Metheus's contribution into the fiat-denominated $\mathfrak{F}at$ collateral pool

Table A.1: Notation used for elementary market entities in Metheus.

FeaAudit Notation

Notation	Description
v_k^t	Wit_k 's or VA_k 's miat-denominated wealth at stage t
$\xi \in (0, 1)$	wealth seizure parameter

Table A.2: Notation used for all FeaAudit market entities in Metheus.

QAudit Notation

Notation	Description
v_{pa}	PA-defined bounty value
v_k^t	reward given to Agent_k or Auditor_k at stage t
ρ_k^t	regret taken from Auditor_k at stage t
s^t	QAudit Amm's share vector $(s_1^t, \dots, s_K^t)$ at stage t
b_k^t	Auditor_k 's buy vector $(b_1^t, \dots, b_K^t)$ at stage t
ρ	price vector $(\rho_1^t, \dots, \rho_K^t)$ of all models at stage t
v_ρ^t	Dutch price ascending price factor at stage t applied to all models
$\gamma \in (0, 1)$	regime change risk which functions as a discount factor
$\mathbb{D}_k^t$	generic source of data (i.e. model θ or dataset $\mathbb{D}$) reported by Agent_k at stage t
$\mathbb{D}_{\text{train}}$	public training set of form $\{(\mathbf{x}, y)_1, \dots\}$
$\mathbb{D}_{\text{test}}$	hidden test set used for final judgment of models
$\mathbb{D}_{\text{val}}$	public validation set for initial ranking of models
η	noise function that emits data for discriminative model

Table A.3: Notation used for all QAudit market entities in Metheus.

Fiat Notation

Notation	Description
$e_{m:f}^\tau$	miat:fiat exchange rate at time step τ
b_{miat}^τ	r.v. denoting whether some Auditor is purchasing miat with fiat
$\omega_{\text{miat}}^\tau$	r.v denoting the amount of miat spent by Auditor
ω_{miat}^0	constant denoting the initial supply of miat at time τ_0
w_{fiat}^τ	r.v. denoting the amount of fiat spent by Auditor
n_{au}^τ	r.v. denoting the total number of AuditUnit at time τ
n_{au}^0	constant denoting the initial number of AuditUnit at time τ
ϱ^t	expected value of the global fiat-denominated inflation
ϱ^t	dividend pool at dividend period t
ϱ^t	expected dividend value per share of Fiat stablecoin at dividend period t

Table A.4: Notation used for all Fiat-related market entities in Metheus.

Statistics Notation

Notation	Description
$y \in \{0, 1\}$	binary label on $\mathbf{x}$
$\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{V}$	some arbitrarily-valued vector input in some space $\mathbb{V}$
$\phi_{\mathbf{x}} \in \mathbb{R}^{d_0}$	real-valued embedding of $\mathbf{x}$ mapped onto feature space of dimension d_0 by the function ϕ
$\theta_k^t \in \mathbb{R}^{d \times n \times h}$	a generative model of form $\mathbf{p}(\phi_{\mathbf{x}_\tau} \phi_{\mathbf{x}_{\tau-1}}; \theta_k^t)$ submitted by the k th Agent at stage t , with input $\phi_{\mathbf{x}_{\tau-1}}$, and generated output $\phi_{\mathbf{x}_\tau}$
$\mathbf{w}_k^t \in \mathbb{R}^{d_0 \times n}$	parameterizes discriminative model of form $\mathbf{p}(y \phi_{\mathbf{x}}; \mathbf{w}_k^t)$ from modeled trained on the k th Agent_k 's submission at stage t
$p_k^t, q_r^t \in [0, 1]$	scalar probability values submitted by k th or r th Agent at stage t
$\mathbf{p} \in [0, 1]^{ \Omega }$	probability vector over set abstract sample space Ω
$\mathbf{p} : \mathbb{R}^{d_0} \rightarrow [0, 1]$	probability function of form $\mathbf{p}(\phi_{\mathbf{x}})$
coin_p	Bernoulli r.v. with bias p

Table A.5: Notation used for all statistics and machine learning related entities.

APPENDIX B

Definitions

All book definitions are found here.

B.1 Randomized Algorithms

Definition B.1.1. (*Randomized Algorithm*). A randomized algorithm $\mathcal{A}$ with domain $\mathbb{A}$ and range $\mathbb{B}$ is associated with a mapping $M : \mathbb{A} \rightarrow \Delta(\mathbb{B})$. On input $\mathbf{a} \in \mathbb{A}$, the algorithm $\mathcal{A}$ outputs $\mathcal{A}(\mathbf{a}) = \mathbf{b}$ with probability $(M(\mathbf{a}))_{\mathbf{b}}$ for every $\mathbf{b} \in \mathbb{B}$. The probability space is over the coin flips of algorithm $\mathcal{A}$ [DR14, p. 16]. Here the *probability simplex* on discrete set $\mathbb{B}$, denoted $\Delta(\mathbb{B})$ is defined with:

$$\Delta(\mathbb{B}) = \left\{ \mathbf{x} \in \mathbb{R}^{|\mathbb{B}|} : x_i \geq 0 \text{ for every } i \text{ and } \sum_{i=1}^{|\mathbb{B}|} x_i = 1 \right\}.$$

Definition B.1.2. (*Differential Privacy*). A randomized algorithm $\mathcal{A}$ with domain $\mathbb{N}^{\mathcal{X}}$ is (ϵ, δ) -differentially private if for all $\mathbb{S} \subseteq \text{Range}(\mathcal{A})$ and for all $\mathbf{x}, \mathbf{y} \in \mathbb{N}^{\mathcal{X}}$ s.t. $\|\mathbf{x} - \mathbf{y}\|_1 \leq 1$:

$$\mathbf{p}(\mathcal{A}(\mathbf{x}) \in \mathbb{S}) \leq \exp(\epsilon) \mathbf{p}(\mathcal{A}(\mathbf{y}) \in \mathbb{S}) + \delta,$$

where the probability space is over the coin flips of the mechanism $\mathcal{A}$. If $\delta = 0$, then we say $\mathcal{A}$ is ϵ -differentially private [DR14, p. 17].

Definition B.1.3. (*Chernoff Bound for Binomial Distribution*). Let $X \sim \text{Bin}(N, \mathbf{p})$ and let $\mu = \mathbf{E}[X]$. For any $0 < \delta < 1$:

B.1.3.1. Upper tail bound:

$$\mathbf{p}(X \geq (1 + \delta)\mu) \leq \exp\left(-\frac{\delta^2 \mu}{3}\right).$$

B.1.3.2. Lower tail bound:

$$\mathbf{p}(X \leq (1 - \delta)\mu) \leq \exp\left(-\frac{\delta^2 \mu}{2}\right).$$

Definition B.1.4. (*Random Walk and Markov Chain*) A random walk is a process for traversing a graph where at every step we follow an outgoing edge chosen uniformly at random. A Markov chain is similar except the outgoing edge is chosen according to an arbitrary fixed distribution. An important property of a Markov chain is the memorylessness property: the future behavior of a Markov chain depends only on its current state, and not on how it arrived at the present state. So that given r.v. X indexed by time τ , we have:

$$\mathbf{p}(X = \mathbf{x}_{\tau+1} | \mathbf{x}_0, \dots, \mathbf{x}_\tau) = \mathbf{p}(X = \mathbf{x}_{\tau+1} | \mathbf{x}_\tau).$$

[MR95, p. 129].

Definition B.1.5. (*Hitting time*) The hitting time h_{uv} (sometimes called the mean first passage time) is the expected number of steps in a random walk over graph with vertices u and v that starts at u and ends upon first reaching v [MR95, p. 133].

Definition B.1.6. (*Markov Chain Transition Matrix*) Given r.v. X_τ indexed by time τ denoting the probability that the current state is at $\mathfrak{s}_\tau$ at step τ , and a probability vector where:

$$\mathbf{p}_\tau = ([\mathbf{p}_\tau]_1, \dots, [\mathbf{p}_\tau]_r, \dots, [\mathbf{p}_\tau]_K) \quad \mathbf{p}(X_\tau = \mathfrak{s}_r) = [\mathbf{p}_\tau]_r \quad \sum_{r=1}^K [\mathbf{p}_\tau]_r = 1.$$

We can associate it with a state transition matrix $\mathbf{M}$, or the "walk matrix." The Walk matrix has dimension $K \times K$, so that every entry $[\mathbf{M}]_{rs}$ is the probability of transition from state $\mathfrak{s}_r$ to state $\mathfrak{s}_s$:

$$[\mathbf{M}]_{rs} = \mathbf{p}(X_{\tau+1} = \mathfrak{s}_s | X_\tau = \mathfrak{s}_r).$$

Additionally, the walk matrix $\mathbf{M}$ is row-stochastic in that the sum of entries in every row r must be 1, corresponding to the likelihood of exiting state $\mathfrak{s}_r$ in the Markov chain. Given this matrix, we can write one step of the random walk with $\mathbf{p}_{\tau+1} = \mathbf{p}_\tau \mathbf{M}$, and n steps of the walk with: $\mathbf{p}_n = \mathbf{p}_o \mathbf{M}^n$.

Definition B.1.7. (*Stationary Distribution*) for the Markov chain with transition matrix $\mathbf{M}$ is written: $\mathbf{p} = \mathbf{p} \mathbf{M}^n$, this is the steady state of the function $\mathbf{M}$ [MR95, p. 132].

Theorem B.1.8. (*Fundamental Theorem of Markov Chains*) An irreducible, finite, and ergodic Markov chain has properties:

B.1.8.1. all states are ergodic, meaning it is ergodic.

B.1.8.2. There is a unique stationary distribution $\mathbf{p}$ s.t.

$$\mathbf{p}^\top = \mathbf{p} \mathbf{M}, \tag{B.1}$$

where $\mathbf{M}$ is the walk matrix, and observe that $\mathbf{p}$ is the principle eigenvector of $\mathbf{M}$ with eigenvalue $\lambda_1 = 1$.

B.1.8.3. The hitting time of any state $\mathfrak{s}_r$ is $h_{sr} = \frac{1}{[\mathbf{p}^*]_r}$.

B.1.8.4. let $N(r, \tau)$ be the number of times the Markov chain visit state r in τ steps, then:

$$\lim_{\tau \rightarrow \infty} \frac{N(r, \tau)}{\tau} = [\mathbf{p}^*]_r.$$

Finally, we know that the first eigenvalue of $\mathbf{M}$ λ_1 is always equal to 1, and the second eigenvalue λ_2 determines the mixing time, or the rate at which repeated applications of $\mathbf{M}$ at any $\mathbf{p}$ converges to the stationary distribution [MR95, p. 132].

B.2 Game Theory

The follow set of equilibrium definitions are drawn from [Rou14], and are oriented around the notion of *potential games*.

Definition B.2.1. (*Pure Nash Equilibria*) A strategy profile $\mathbf{s}$ of a cost-minimization game is a *pure Nash equilibrium* (PNE) if for every player $i \in \{1, \dots, K\}$ and every unilateral deviation $\mathbf{s}_i \in \mathbb{S}_i$:

$$\mathbf{C}_i(\mathbf{s}) \leq \mathbf{C}_i(\mathbf{s}'_i, \mathbf{s}_{-i}).$$

Definition B.2.2. (*Mixed Nash Equilibria*) Distributions $\mathbf{p}_1, \dots, \mathbf{p}_K$ over strategy sets $\mathbb{S}_1, \dots, \mathbb{S}_K$ of a cost-minimization game constitutes a *Mixed Nash Equilibria* (MNE) if for every player $i \in \{1, \dots, K\}$ and every unilateral deviation $\mathbf{s}_i \in \mathbb{S}_i$:

$$\mathbf{E}_{\mathbf{s} \sim \mathbf{q}} [\mathbf{C}_i(\mathbf{s})] \leq \mathbf{E}_{\mathbf{s} \sim \mathbf{q}} [\mathbf{C}_i(\mathbf{s}'_i, \mathbf{s}_{-i})],$$

where $\mathbf{q} := \mathbf{p}_1 \times \dots \times \mathbf{p}_K$.

Definition B.2.3. (*Correlated Nash Equilibria*) A distributions $\mathbf{q}$ on the set $\mathbb{S}_1, \dots, \mathbb{S}_K$ of outcomes of a cost-minimization game is a *Correlated Nash Equilibria* (CE) if for every player $i \in \{1, \dots, K\}$ and every unilateral deviation $\mathbf{s}_i \in \mathbb{S}_i$:

$$\mathbf{E}_{\mathbf{s} \sim \mathbf{q}} [\mathbf{C}_i(\mathbf{s}) | \mathbf{s}_i] \leq \mathbf{E}_{\mathbf{s} \sim \mathbf{q}} [\mathbf{C}_i(\mathbf{s}'_i, \mathbf{s}_{-i}) | \mathbf{s}_i].$$

Definition B.2.4. (*Coarse Correlated Equilibria*) A distributions $\mathbf{q}$ on the set $\mathbb{S}_1, \dots, \mathbb{S}_K$ of outcomes of a cost-minimization game is a *Coarse Correlated Equilibria* (CCE) if for every player $i \in \{1, \dots, K\}$ and every unilateral deviation $\mathbf{s}_i \in \mathbb{S}_i$:

$$\mathbf{E}_{\mathbf{s} \sim \mathbf{q}} [\mathbf{C}_i(\mathbf{s})] \leq \mathbf{E}_{\mathbf{s} \sim \mathbf{q}} [\mathbf{C}_i(\mathbf{s}'_i, \mathbf{s}_{-i})].$$

Definition B.2.5. (*Price of Anarchy*) of a game w.r.t social function f is:

$$\frac{\min_{\mathbf{s}, NE} f(\mathbf{s})}{\text{OPT}} \text{ for utilities} \quad \text{and} \quad \frac{\max_{\mathbf{s}, NE} f(\mathbf{s})}{\text{OPT}} \text{ for costs.}$$

NE is Nash Equilibrium. The sum of social welfare is written:

$$f(\mathbf{s}) = \sum_i \mathbf{u}_i(\mathbf{s}).$$

And the optimal solution is:

$$\text{OPT} = \max_{\mathbf{s} \in \mathbb{S}} f(\mathbf{s}).$$

Here $\mathbf{s} = (\mathbf{s}_1, \dots, \mathbf{s}_n)$ denotes the joint vector of strategies selected by n players in the space of $\mathbb{S}_1 \times \dots \times \mathbb{S}_n$ actions. With game utility/cost function $\mathbf{u}_i : \mathbb{S} \rightarrow \mathbb{R}$ for every player i , at any joint action $\mathbf{s} \in \mathbb{S}$.

APPENDIX C

Proofs

This section provides all proofs from the main body.

C.1 Agents-Agents Sub-Game

Proof. for lemma 2.5.1. Recall Agent_r 's objective is to report one of the top R models that can participate in rewards, and maximize his profit with:

$$\arg \max_{\mathbf{w}} (\mathbf{v}_r^t - \text{cost}_r^t) = \arg \max_{\mathbf{w}} (\beta_r \cdot \mathbf{v}^t - \text{cost}_r^t)$$

where $\beta_r = \frac{e^{\log \text{prob}_r^t}}{e^{\log \text{prob}_1^t} + \dots + e^{\log \text{prob}_R^t}}$ and $e^{\log \text{prob}_r^t} = \prod_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}} p(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t).$

The value $\mathbf{v}^t$ is some constant that is wagered on all the losing models in this current stage t , and cost_r^t is some fixed cost of creating any model $\mathbf{w}$. Setting $\mathbf{q} = \exp(\log \text{prob}_r^t)$, the expression above can be rewritten as:

$$\arg \max_{\mathbf{w}} \frac{\mathbf{q}}{\mathbf{q} + C} \cdot \mathbf{v}^t - \text{cost}_r^t \quad \text{where} \quad C = \sum_{\mathbf{s} \in \{1..R\}/\{r\}} \exp(\log \text{prob}_{\mathbf{s}}^t) \quad (\text{C.1})$$

It is self evident that Agent_r 's solution is to maximize data-likelihood $\mathbf{q}$, if neither cost_r^t nor $\mathbf{v}^t$ depend on the values of $\mathbf{w}_r^t$.

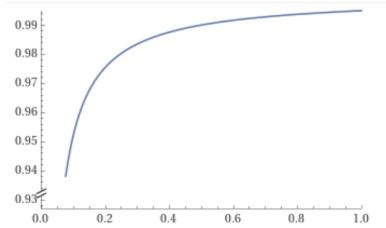


Figure C.1: Expression C.1 plotted with $C = 0.005$, $\mathbf{v}^t = 1$, and $\text{cost}_r^t = 0$. Note the expression is monotonically increasing in $\mathbf{q} = \exp(\log \text{prob}_r^t)$, stating Agent_r can maximize his share of winning wagers by maximizing data likelihood.

Moreover, we can expand C.1 modulo the cost factor into a training objective and determine how the Agent should behave with this new objective:

$$\begin{aligned} \arg \max_{\mathbf{w}} \frac{\mathbf{q}}{C + \mathbf{q}} \cdot \mathbf{v}^t &= \arg \max_{\mathbf{w}} \left(\frac{1}{\frac{C}{\mathbf{q}} + 1} \right)^{-1} \cdot \mathbf{v}^t \\ &\xrightarrow{\log} \arg \max_{\mathbf{w}} \left(\log \mathbf{v}^t - \log \left(1 + \frac{C}{\prod p(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t)} \right) \right). \end{aligned}$$

Note $\log \mathbf{v}^t$ is a constant ≥ 0 , and as the data likelihood term $\prod p(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t)$ increases with better $\mathbf{w}$, the overall value of the objective function increases. That is to say Agent_r should

select $\mathbf{w}_r^t$ over $\mathbf{v}_r^t$ as:

$$\log \mathbf{v}^t - \log(1 + C \cdot (\prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t))^{-1}) > \log \mathbf{v}^t - \log(1 + C \cdot (\prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{v}_r^t))^{-1}),$$

per assumption of expression 2.1. Alternatively, we can find a more familiar expression:

$$\begin{aligned} & \arg \max_{\mathbf{w}} \frac{q}{C+q} \cdot \mathbf{v}^t \\ & \arg \max_{\mathbf{w}} \prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t) \cdot (C + \prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t))^{-1} \cdot v \\ & \arg \max_{\mathbf{w}} \underbrace{\sum \log \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t)}_{\text{maximize data likelihood}} - \underbrace{\log(C + \prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t))}_{\text{bias term}} + \log \mathbf{v}^t \\ & \stackrel{|\mathbb{D}| \rightarrow \infty}{\sim} \arg \max_{\mathbf{w}} \sum \log \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t) - \underbrace{\log C' + \log \mathbf{v}^t}_{\text{some constant to Agent}_t}. \end{aligned}$$

And see that the first term is the classical training objective of maximizing data likelihood as seen in remark E.3.4. While the second term is a bias term whereby the data likelihood under the Agent_t 's model appears again. Notably, for very large values of $|\mathbb{D}|$, we must have $\log(C + \prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t)) < 0$, as $0 < C, q \ll 1$. Moreover, at large $|\mathbb{D}|$ and sufficiently large R , the term $\prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t)$ grows vanishingly small compared to the term C , so that the entire tail end of the expression is just a constant from the perspective of Agent_t . This suggests the reward $\mathbf{v}_r^t$ does not affect the goal of maximizing data-likelihood in the limit. ■

Proof. for 2.5.2. We assume both Agent_r and Agent_s expects to be one of the winners, that is with $r, s \in \{1, \dots, R\}$. Then Agent_r 's winning wager split is of form:

$$\beta_r^* = \frac{\prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{w}_r^t)}{e^{\log \text{prob}_1^t} + \dots + e^{\log \text{prob}_R^t}} \quad \beta_r = \frac{\prod \mathbf{p}(y|\phi_{\mathbf{x}}; \mathbf{v}_r^t)}{e^{\log \text{prob}_1^t} + \dots + e^{\log \text{prob}_R^t}},$$

and similarly for β_s^* and β_s . Then we have the payoff table:

		Agent_s	
		$\mathbf{w}_s^t$	$\mathbf{v}_s^t$
Agent_r	$\mathbf{w}_r^t$	$(\beta_r^* \mathbf{v}^t, \beta_s^* \mathbf{v}^t)$	$(\beta_r^* \mathbf{v}^t, \beta_s \mathbf{v}^t)$
	$\mathbf{v}_r^t$	$(\beta_r \mathbf{v}^t, \beta_s^* \mathbf{v}^t)$	$(\beta_r \mathbf{v}^t, \beta_s \mathbf{v}^t)$

Figure C.2: A two player sub-game, with winning Agent_r competing against Agent_s for a portion of the reward $\mathbf{v}^t$. Note $\beta_r^* > \beta_r$, and $\beta_s^* > \beta_s$, given these two fixed permutations, any other permutation over the four β 's are allowed.

Now referencing fig C.2, take the perspective of Agent_r and consider how his reports rank w.r.t Agent_s :

1. $\beta_r^* > \beta_r > \beta_s^* > \beta_s$: clearly we have $\beta_r^* \mathbf{v}^t > \beta_r \mathbf{v}^t$, so Agent_r reports $\mathbf{w}_r^t$.
2. $\beta_r^* > \beta_s^* > \beta_r > \beta_s$: it is still the case that $\beta_r^* \mathbf{v}^t > \beta_r \mathbf{v}^t$, so Agent_r reports $\mathbf{w}_r^t$.
3. $\beta_s^* > \beta_r^* > \beta_r > \beta_s$: in this case if Agent_s reports $\mathbf{w}_s^t$, then he takes a larger share of the rewards. But for Agent_r we still have $\beta_r^* \mathbf{v}^t > \beta_r \mathbf{v}^t$, so Agent_r reports $\mathbf{w}_r^t$.

A similar argument can proceed with locations of β_s . In sum, regardless of how the various β_r and β_s are ordered with respect to each other, reporting the truth $\mathbf{w}_r^t$ is the best response for Agent_r . Thus reporting the truth is a *pure strategy equilibrium* in the sub-game amongst winning Agents within each stage t . ■

A few caveats to the above lemmas follows:

1. it is only valid if we assume cost_r^t is constant and does not change regardless of how well the Agent trains his model. If the cost is some function of the Bregman divergence between the Agent's current model and his previous submission, i.e.: $vD(\mathbf{w}_r^{t-1} || \mathbf{w}_r^t)$. Then an Agent's optimal report is a convex combination of his last report and the best current report, as seen in eqn D.10. In this case, the Agent should maintain a distribution over acceptable $\mathbf{w}$'s, ranging over model parameters that is no worse than the last reported parameter $\mathbf{w}^{t-1}$, and sample from this distribution. Thus reporting a better model than last stage is a *mixed strategy Nash Equilibrium*.
2. Note we assume the Agent does not know other Agents' report, hence the constant value C . This is a fair assumption as the submission stage is a closed one, and the stage t 's models are not revealed until wagering time at $t + 1$.
3. In reality, each Agent submit generative model or data source, not a discriminative model $\mathbf{w}$. So they will report their honest θ or $\mathbb{D}$ if they are confident that the corresponding classifier model trained by QAudit is a "fair" reflection of the original reports.

Remark C.1.1. We can also set C and $\mathbf{v}^t$ to be some random variable as they are unknowns from the perspective of the Agent. From a Bayesian perspective, this would make sense if we have some prior estimates of C and $\mathbf{v}^t$, and then can update them based on observations. But the Agent cannot observe other Agents' reports until the market opens on stage $t + 1$, and similarly for Auditors' wagers. Thus setting C and $\mathbf{v}^t$ as r.v. would be an unhelpful formality.

C.2 Agents-Auditors Sub-Game

Proof. for lemma 2.5.3. We prove the case where $R \geq 2$. Given K Agents reporting classifier models $\mathbf{w}_r^t$ with convex cost function, and set of Auditors: $\{\text{Auditor}_i\}_{i=1..N}$. Now suppose for some stage t the two best models $\{\mathbf{w}_r^t, \mathbf{w}_s^t\}$ are submitted by two Agents: $\{\text{Agent}_r, \text{Agent}_s\}$. Recall in QAudit the Agents move first with the intention of competing against other Agents. And as C.2 shows, reporting their true $\mathbf{w}_k^t$ is each Agent's dominant strategy. At reward time, QAudit_v builds ensemble model $\mathbf{w}^{t-1}$, and some $\mathbf{w}^t$ is built from $\{\mathbf{w}_r^t, \mathbf{w}_s^t\}$. They could rank in one of two ways:

$$\mathbf{E}_{\mathbb{D} \sim \mathbf{p}} \left[\prod_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}} p(y | \phi_{\mathbf{x}}; \mathbf{w}^{t-1}) \right] > \text{ or } < \mathbf{E}_{\mathbb{D} \sim \mathbf{p}} \left[\prod_{(\phi_{\mathbf{x}}, y) \in \mathbb{D}} p(y | \phi_{\mathbf{x}}; \mathbf{w}^t) \right].$$

Here the expectation is with respect to the population distribution $\mathbf{p}$ from which the test set $\mathbb{D}$ is drawn. Now if the Auditor expects the data likelihood under $\mathbf{w}^{t-1}$ is greater than the current stage's ensemble model, then he would not wager at all. In the second case

where data likelihood under $\mathbf{w}^t$ is greater, then we will show that 1) the Auditor is better off wagering on the winning Agents than some losing Agent_k , and 2) the Auditor should wager *more* on the better winning Agent measured by data likelihood. Suppose the Auditor_i observes from the evaluation set $\mathbb{D}_{\text{val}} \exp(\log \text{prob}_r^t) > \exp(\log \text{prob}_s^t)$, and some Agent_k whose model is worse than both r and s . Furthermore, suppose the amount wagered on the losing Agents is v^t , and the winning pot in stage $t + 1$ is v^{t+1} . Then:

1. Auditor should not wager on Agent_k : wlog we select Agent_r . And it is clear from C.3 that Auditor_i 's dominant strategy is to select $\mathbf{w}_r^t$ over $\mathbf{w}_k^t$. From Agent_r 's perspective, in the event where his wager would have made a difference and placed Agent_r into the top R slot, then Agent_r is compensated with ϱ_r^t instead of winning wager.

Auditor _i			
	wager $\mathbf{w}_r^t$ wager $\mathbf{w}_k^t$		
$\mathbf{w}_r^t$	<table border="1" style="width: 100%; height: 100%;"> <tr> <td style="width: 50%; text-align: center; vertical-align: middle;">$(\beta_r v^t, \alpha_r v_l^{t+1})$</td><td style="width: 50%; text-align: center; vertical-align: middle;">$(\beta_r v^t \text{ or } \varrho_r^t, 0)$</td></tr> </table>	$(\beta_r v^t, \alpha_r v_l^{t+1})$	$(\beta_r v^t \text{ or } \varrho_r^t, 0)$
$(\beta_r v^t, \alpha_r v_l^{t+1})$	$(\beta_r v^t \text{ or } \varrho_r^t, 0)$		

Figure C.3: A two player sub-game showing Auditor_i should not defect from Agent_r . Here β_r is defined as above, and $\alpha_r = \rho_r^t / \sum_{s=1}^R \rho_s^t$.

2. Auditor_i may benefit when wagering more on Agent_r 's model than that of Agent_s . The inequality here is the regret factor ϱ_i^t . In the event where Agent_r does better both in hindsight and as stated on the evaluation set $\mathbb{D}_{\text{val}}$, then $\text{QAudit}_{\varrho_v}$ allocates some portion of Auditor_i's winnings back to Agent_r for lost compensation.

Auditor _i			
	wager $\mathbf{w}_r^t$ wager $\mathbf{w}_s^t$		
$\mathbf{w}_r^t$	<table border="1" style="width: 100%; height: 100%;"> <tr> <td style="width: 50%; text-align: center; vertical-align: middle;">$(\beta_r v^t, \alpha_r v_l^{t+1})$</td><td style="width: 50%; text-align: center; vertical-align: middle;">$(\beta_r v^t + \varrho_i^t \text{ or } \varrho_r^t, \alpha'_r v_l^{t+1} - \varrho_i^t)$</td></tr> </table>	$(\beta_r v^t, \alpha_r v_l^{t+1})$	$(\beta_r v^t + \varrho_i^t \text{ or } \varrho_r^t, \alpha'_r v_l^{t+1} - \varrho_i^t)$
$(\beta_r v^t, \alpha_r v_l^{t+1})$	$(\beta_r v^t + \varrho_i^t \text{ or } \varrho_r^t, \alpha'_r v_l^{t+1} - \varrho_i^t)$		

Figure C.4: A two player sub-game showing Auditor_i should wager more on Agent_r than Agent_s if data-likelihood is greater under $\mathbf{w}_r^t$. Here α_r and α'_r differ by the number of shares purchased for $\mathbf{w}_r^t$, with $\alpha_r > \alpha'_r$. The value v_l^{t+1} refers to the corresponding betting pool for slot l in stage $t + 1$.

Now consider Auditor_i's total payout over both models $\mathbf{w}_r^t$ and $\mathbf{w}_s^t$, for defecting from Agent_r to Agent_s . Holding all other bids constant, we have:

$$\underbrace{\frac{s_{r,i}^t}{s_r^t} v_a^{t+1} + \frac{s_{s,i}^t}{s_s^t} v_b^{t+1}}_{\text{lhs}} \stackrel{?}{\geq} \underbrace{\frac{s_{r,i}^{t'}}{s_r^{t'}} v_b^{t+1} + \frac{s_{s,i}^{t'}}{s_s^{t'}} v_a^{t+1}}_{\text{rhs}} - \varrho_i^t \quad \text{where}$$

$$\varrho_i^t = \frac{s_{i,r}^t}{s_r^t} \cdot D(\mathbf{w}^t || \mathbf{w}_*^t)$$

$$s_{r,i}^t > s_{r,i}^{t'}, \quad s_{s,i}^t < s_{s,i}^{t'}, \quad s_s^t > s_s^{t'} \quad s_r^t < s_r^{t'}.$$

Where $s_{r,i}^t$ is the number of shares Auditor_i purchased when not defecting from Agent_r , $s_{r,i}^{t'}$ is what the Auditor_i purchased when he did defect, v_a^{t+1}, v_b^{t+1} are the total wagers for the respective top and 2nd slots in stage $t+1$, and regret ϱ_i^t is Auditor_i 's regret w.r.t $\mathbf{w}_*^t$, or the best ensemble model.

Depending on the size of the Bregman divergence term $D(\mathbf{w}^t || \mathbf{w}_*^t)$, the *lhs* may or may not be greater than the *rhs*. Thus there are scenarios whereby Auditor_i benefit from wagering more on Agent_s over the better ranked Agent_r under log-likelihood of the evaluation set. This scenario could arise if the Auditor believes $\mathbf{w}_s^t$ is sufficiently underpriced, and is willing to take the Bregman-regret hair-cut. In other words, Auditors should maintain a distribution over the top ranked Agents, perhaps informed by price of said Agents, and select from this distribution when wagering. All in all, under QAudit Auditor-Agent "collusion" is a *mixed strategy equilibrium*. ■

C.3 Auditors-Auditors Sub-Game within every Stage

Proof. for lemma 2.5.8 In this case we let $n = K!$, and select some suitable order of $\mathfrak{s}$'s so we can define a PSA of form:

$$\text{PSA} = \left(\mathbb{S}_K = \{\mathfrak{s}_1, \dots, \mathfrak{s}_n\}, \mathbb{A}_K = \{(\mathfrak{s}_r, \mathfrak{s}_s) : \mathfrak{s}_r, \mathfrak{s}_s \in \mathbb{S}_K, r \leq s\}, \mathfrak{s}_o = \mathfrak{s}_1, \mathbb{S}_K^* = \{\mathfrak{s}_n\} \right),$$

expressing a directed permutation graph with a single terminal state, or an "absorbing chain" [SW22, p. 24]. Let $\mathbb{S}_K$ be a lexicographic ordering of the states, so that $\mathfrak{s}_1 = (1, 2, \dots, K)$, and $\mathfrak{s}_n = (K, K-1, \dots, 2, 1)$. In the worst case, we have $\mathfrak{s}_o = \mathfrak{s}_1$ and $\mathfrak{s}_r^* = \mathfrak{s}_n$, and the Auditors must cross the entire width of the PSA graph to find $\mathfrak{s}_r^*$. For example if $K = 3$, with $\mathfrak{s}_o = (1, 2, 3)$ and $\mathfrak{s}_r^* = (3, 2, 1)$, then the longest path under neighbor transposition shuffle is: $[(1, 2, 3), (2, 1, 3), (2, 3, 1), (3, 2, 1)]$. And note that may be many paths of this length.

The expected time it takes for the Auditors to cross the graph by random walking is hitting time, denoted $h_{1n} := h_{o*}$. This value can be computed by first expanding the process via a reverse random walk, and then bounding it from above. Recall AuditStrat endows the directed PSA graph with a set of transition probabilities:

$$\begin{aligned} \text{state}_n : & \quad p_{nn} = 1 \\ \text{state}_{n-1} : & \quad p_{n-1,n-1} + p_a = 1 \\ \text{state}_{n-2} : & \quad p_{n-2,n-2} + p_a + p_{n-2,n} = 1 \\ \text{state}_{n-3} : & \quad p_{n-3,n-3} + p_a + p_{n-3,n-1} + p_{n-3,n} = 1 \\ \dots & \\ \text{state}_1 : & \quad p_{1,1} + p_a + p_{1,3} + \dots + p_{1,n} = 1. \end{aligned}$$

Where every $0 \leq p_{kk} < 1$ for $k = 1 \dots n$ is the probability of a self transition. While p_a is the probability of neighbor transposition. Note we assume that p_a is the same for every vertex in $\mathbb{S}_K$. And $p_{k,k+l}$ for $l = 2 \dots n - k$ is the probability of jumping l states in the PSA. Finally note we have $p_{n-2,1} = 1 - p_{n-2,n-2} - p_a$, etc. In real life, this transition probability table may be constructed by collecting data from how Auditors wager. Here we can proceed with the reverse random walk using the variable names:

1. $h_{n,n}$: this is the sink, so the hitting time is 0 as the random walk has terminated.

2. $h_{n-1,n}$: this is the expected number of steps to transition from the penultimate state to the final state, written:

$$\begin{aligned} h_{n-1,n} &= (1 - p_a)(1 + h_{n-1,n}) + p_a \\ h_{n-1,n} &= \frac{1}{p_a}. \end{aligned}$$

3. $h_{n-2,n}$: this is the expected number of steps to transition from the 2nd to last state to the final state:

$$\begin{aligned} h_{n-2,n} &= p_{n-2,n-2}(1 + h_{n-2,n}) + p_a(1 + h_{n-1,n}) + 1 \cdot (1 - p_a - p_{n-2,n-2}) \\ h_{n-2,n} &= \frac{2}{1 - p_{n-2,n-2}}. \end{aligned}$$

4. $h_{n-3,n}$: as before we have:

$$\begin{aligned} h_{n-3,n} &= p_{n-3,n-3}(1 + h_{n-3,n}) + p_a(1 + h_{n-2,n}) + p_{n-3,n-1}(1 + h_{n-1,n}) \\ &\quad + (1 - p_a - p_{n-3,n-3} - p_{n-3,n-1}) \\ h_{n-3,n} &= \frac{2p_a^2 + p_{n-3,n-1}(1 - p_{n-2,n-2}) + p_a(1 - p_{n-2,n-2})}{p_a(1 - p_{n-2,n-2})(1 - p_{n-3,n-3})} \\ &\stackrel{(1)}{\leq} \frac{2p_a^2 + p_a + p_{n-3,n-1}}{p_a(1 - p_{n-2,n-2})(1 - p_{n-3,n-3})}, \end{aligned}$$

where going into (1) we use $1 - p \leq 1$.

5. $h_{n-4,n}$, we have:

$$\begin{aligned} h_{n-4,n} &= p_{n-4,n-4}(1 + h_{n-4,n}) + p_a(1 + h_{n-3,n}) + p_{n-4,n-2}(1 + h_{n-2,n}) + p_{n-4,n-1}(1 + h_{n-1,n}) \\ &\quad + 1 - p_a - p_{n-4,n-4} - p_{n-4,n-2} - p_{n-4,n-1} \\ h_{n-4,n}(1 - p_{n-4,n-4}) &\leq \underbrace{\frac{p_a(2p_a^2 + p_a + p_{n-3,n-1})}{p_a(1 - p_{n-2,n-2})(1 - p_{n-3,n-3})}}_A + \underbrace{\frac{2 \cdot p_{n-4,n-2}}{1 - p_{n-2,n-2}}}_B + \underbrace{\frac{p_{n-4,n-1}}{p_a} + 1}_C \end{aligned}$$

In the numerator of A, B, C, and the 1 factor we have:

$$\begin{aligned} A: \quad & p_a(2p_a^2 + p_a + p_{n-3,n-1}) && \leq 2p_a^3 + p_a^2 + p_a \\ B: \quad & 2 \cdot p_{n-4,n-2}p_a(1 - p_{n-3,n-3}) && \leq 2p_a \\ C: \quad & p_{n-4,n-1}(1 - p_{n-2,n-2})(1 - p_{n-3,n-3}) && \leq 1 \\ 1: \quad & p_a(1 - p_{n-2,n-2})(1 - p_{n-3,n-3}) && \leq p_a. \end{aligned}$$

So the sum of numerators is:

$$(2p_a^3 + p_a^2 + p_a) + 2p_a + p_a + 1 \leq 4(p_a^3 + p_a^2 + p_a + 1).$$

Thus the hitting time is bounded from above with:

$$h_{n-4,n} \leq \frac{4(p_a^3 + p_a^2 + p_a + 1)}{p_a(1 - p_{n-2,n-2})(1 - p_{n-3,n-3})(1 - p_{n-4,n-4})}.$$

In general, we must have:

$$h_{n-(n-1),n} \leq \frac{(n)(p_a^n + p_a^{n-1} + \dots + p_a + 1)}{p_a \prod_{k=1}^{n-2} (1 - p_{k,k})} \implies$$

$$h_{1,K!} \leq \frac{\sum_{k=0}^{K!} p_a^k}{p_a \cdot \prod_{k=1}^{K!-2} (1 - p_{k,k})} \cdot K!. \quad (C.2)$$

All in all, in the event where all the **Auditors** agree on the best permutation $\mathfrak{s}_r^*$, the hitting time, or number of wagers required to permute $\mathfrak{s}_o$ onto $\mathfrak{s}_{K!}$, is $\text{poly}(K!)$ as desired. Moreover, observe in the event where the **Auditors** buy enough shares on each wager to rearrange the permutation when it is not at $\mathfrak{s}_r^*$, then we have the self transition probability $p_{k,k} \sim 0$, so that eqn (C.2) can be bounded "reasonably" from below by:

$$h_{1,K!} \geq (p_a^{-1} + p_a^0 + p_a^1 + p_a^2 + \dots + p_a^{K!-2} + p_a^{K!-1}) \cdot K!, \quad (C.3)$$

where each p_a is the likelihood of transitioning to the next permutation in lexicographic order. Note the h.o.t are vanishing small, we have i.e $h_{1,K!} \geq (1 + p_a^{-1} + p_a + p_a^2 + p_a^3) \cdot K!$ ■

Proof. for lemma 2.5.10. In this case, we are working with the PSA expressing an undirected permutation graph:

$$\text{PSA} = \left(\mathbb{S}_K = \{\mathfrak{s}_1, \dots, \mathfrak{s}_{K!}\}, \mathbb{A}_K = \{(\mathfrak{s}_r, \mathfrak{s}_s) : \mathfrak{s}_r, \mathfrak{s}_s \in \mathbb{S}_K\}, \mathfrak{s}_o = *, \mathbb{S}_K^* = \{\mathfrak{s}_r^*, \dots, \mathfrak{s}_m^*\} \right).$$

This undirected graph can be represented with the walk matrix $\mathbf{M}$. And by assumption it is the weighted sum of m directed graphs: $\mathbf{M} = \alpha_r \mathbf{M}_r + \dots + \alpha_m \mathbf{M}_m$, rewritten:

$$\mathbf{M} = \alpha_r \mathbf{M}_r + (1 - \alpha_r) \tilde{\mathbf{M}}, \text{ where } \tilde{\mathbf{M}} = \alpha_2 \mathbf{M}_2 + \dots + \alpha_m \mathbf{M}_m.$$

Now for this proof, we are taking the *subjective* perspective of QAudit whose public ranking is $\mathfrak{s}_r^*$. So the protocol does not know other **Auditors'** private beliefs, it only observe that "sometimes" the PSA jumps to some random state $\mathfrak{s}_r^*$ that it would not have taken. Thus we can simplify $\tilde{\mathbf{M}}$ to the uniform random walk matrix $\frac{\mathbf{e}\mathbf{e}^\top}{K!}$, stating that with probability $1/K$, the PSA jumps to some random permutation state, written:

$$\mathbf{M} \sim \alpha_r \mathbf{M}_r + (1 - \alpha_r) \frac{\mathbf{e}\mathbf{e}^\top}{K!}. \quad (C.4)$$

Now the sensitivity of the stationary distribution $\mathbf{p}$ under $\mathbf{M}$ as a function of α_r can be computed. This argument is from [LM98, p. 362] and [CM99], and proceeds as follows:

$$\begin{aligned} \mathbf{p}^\top &= \mathbf{p} \mathbf{M}_r = \mathbf{p} \left(\alpha_r \mathbf{M}_r + (1 - \alpha_r) \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) & \Rightarrow \\ \mathbf{p}^\top (\mathbf{I} - \alpha_r \mathbf{M}_r) &= \mathbf{p} \frac{\mathbf{e}\mathbf{e}^\top}{K!} - \alpha_r \mathbf{p} \frac{\mathbf{e}\mathbf{e}^\top}{K!} & \Rightarrow \\ \frac{\partial}{\partial \alpha_r} \left(\mathbf{p}^\top (\mathbf{I} - \alpha_r \mathbf{M}_r) \right) &= \frac{\partial}{\partial \alpha_r} \left(\mathbf{p} \frac{\mathbf{e}\mathbf{e}^\top}{K!} - \alpha_r \mathbf{p} \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) & \Rightarrow \\ \frac{\partial \mathbf{p}^\top}{\partial \alpha_r} (\mathbf{I} - \alpha_r \mathbf{M}_r) + \mathbf{p} \frac{\partial}{\partial \alpha_r} (\mathbf{I} - \alpha_r \mathbf{M}_r) &= \frac{\partial}{\partial \alpha_r} \left(\mathbf{p} \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) - \frac{\partial}{\partial \alpha_r} \left(\alpha_r \mathbf{p} \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) & \Rightarrow \\ \frac{\partial \mathbf{p}^\top}{\partial \alpha_r} (\mathbf{I} - \alpha_r \mathbf{M}_r) - \mathbf{p} \mathbf{M}_r &= 0 - \mathbf{p} \frac{\mathbf{e}\mathbf{e}^\top}{K!} & \Rightarrow \\ \frac{\partial \mathbf{p}^\top}{\partial \alpha_r} &= \mathbf{p} \left(\mathbf{M}_r - \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) (\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1}. \end{aligned}$$

Where going into the last line, we know the inverse of $\mathbf{I} - \alpha_r \mathbf{M}_r$ must exist as it is non-singular. Now we bound the last line, consider some value at index i in $\frac{\partial \mathbf{p}}{\partial \alpha_r}$:

$$\left(\frac{\partial \mathbf{p}^\top}{\partial \alpha_r} \right)_i = \mathbf{p} \left(\mathbf{M}_r - \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) (\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1} \mathbf{b}_i,$$

where $\mathbf{b}_i$ is the n -bit vector with value 1 at index i , and zeros everywhere else. Using the identity $|\mathbf{x}^\top \mathbf{y}| \leq \|\mathbf{x}\|_1 \frac{y_{max} - y_{min}}{2}$, and setting $\mathbf{y} = (\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1} \mathbf{b}_i$, we have:

$$\begin{aligned} \left| \left(\frac{\partial \mathbf{p}^\top}{\partial \alpha_r} \right)_i \right| &\leq \left\| \mathbf{p} \left(\mathbf{M}_r - \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) \right\|_1 \cdot \frac{y_{max} - y_{min}}{2} \\ &\stackrel{(1)}{\leq} 2 \cdot \frac{y_{max} - y_{min}}{2} \\ &\leq y_{max} - y_{min}, \end{aligned}$$

where (1) follows from $\|\mathbf{p}(\mathbf{M}_r - \frac{\mathbf{e}\mathbf{e}^\top}{K!})\|_1 \leq 2$ as the product is akin to a probability vector. Next bound y_{max} by:

$$y_{max} \stackrel{(1)}{\leq} \|(\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1} \mathbf{b}_i\|_\infty \stackrel{(2)}{\leq} \|(\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1} \mathbf{e}\|_\infty \stackrel{(3)}{\leq} \frac{1}{1 - \alpha_r},$$

where (1) follows from the fact that the infinity norm is defined with $\|\mathbf{x}\|_\infty = \max_i |x_i|$. (2) follows from the fact that $\mathbf{y} = (\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1} \mathbf{b}_i \leq (\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1} \mathbf{e}$, as $\mathbf{e}$ is the all ones vector. And (3) follows from evaluating $(\mathbf{I} - \alpha_r \mathbf{M}_r)\mathbf{e}$. For example in the $n \times n$ case we have:

$$\begin{aligned} (\mathbf{I} - \alpha_r \mathbf{M}_r)\mathbf{e} &= \left(\begin{bmatrix} 1 & & 0 \\ & \ddots & \\ 0 & & 1 \end{bmatrix} - \begin{bmatrix} \alpha_r p_{11} & \dots & \alpha_r p_{1n} \\ \vdots & \ddots & \vdots \\ \alpha_r p_{n1} & \dots & \alpha_r p_{nn} \end{bmatrix} \right) (1, \dots, 1)^\top \\ &= \left(1 - \alpha_r p_{11} - \dots - \alpha_r p_{1n}, \dots, 1 - \alpha_r p_{n1} - \dots - \alpha_r p_{nn} \right)^\top. \end{aligned}$$

Note every row sum yields: $1 - \alpha_r p_{r1} - \dots - \alpha_r p_{rn} = 1 - \alpha_r (\sum_{i=1}^n p_{ri}) = 1 - \alpha_r$. So the maximum value can be bounded with:

$$((\mathbf{I} - \alpha_r \mathbf{M}_r)\mathbf{e})_i = 1 - \alpha_r \implies ((\mathbf{I} - \alpha_r \mathbf{M}_r)^{-1} \mathbf{e})_i = \frac{1}{1 - \alpha_r}.$$

Similarly, we know $y_{min} \geq 0$. Putting it all together and we have:

$$\left| \left(\frac{\partial \mathbf{p}^\top}{\partial \alpha_r} \right)_i \right| \leq y_{max} \leq \frac{1}{1 - \alpha_r} \stackrel{(1)}{\implies} \frac{\partial \mathbf{p}^\top}{\partial \alpha_r} \leq \frac{2}{1 - \alpha_r},$$

where (1) follows from the fact that $\mathbf{p}$ is a probability vector. Next we discretize this statement by perturbing α_r with some $\Delta\alpha_r$ so that: $0 \leq \alpha_r + \Delta\alpha_r < 1$, and define the original and perturbed stationary distribution with:

$$\mathbf{p}_1^\top = \mathbf{p}_1 \left(\alpha_r \mathbf{M}_r + (1 - \alpha_r) \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right) \quad \mathbf{p}_2^\top = \mathbf{p}_2 \left((\alpha_r + \Delta\alpha_r) \mathbf{M}_r + (1 - \alpha_r - \Delta\alpha_r) \frac{\mathbf{e}\mathbf{e}^\top}{K!} \right).$$

Then by common sense and [IW06, p. 6]:

$$\|\mathbf{p}_1 - \mathbf{p}_2\|_1 \leq \frac{2}{1 - \alpha_r} |\Delta\alpha_r|.$$

Finally, referencing eqn 2.5, set $\alpha_r = 1 - \epsilon$, and write $\mathbf{p}^\top = \mathbf{p}((\alpha_r - \Delta\alpha_r)\mathbf{M}_r + (1 - \alpha_r + \Delta\alpha_r)\frac{\mathbf{e}\mathbf{e}^\top}{K!})$, with $0 \leq \Delta\alpha_r < 1 - \epsilon$. Then we have:

$$\|\tilde{\delta}_r - \mathbf{p}\|_1 \leq \frac{2}{1 - \alpha_r} |\Delta\alpha_r| = \frac{2|\Delta\alpha_r|}{\epsilon},$$

as eqn 2.6 states. ■

Proof. for lemma 2.5.11. Consider a partition over Auditors where some portion p have private belief q_r^t , and $(1-p)$ believes in q_s^t :

$$\begin{cases} p & \text{play suggested action } s_r^* \\ 1-p & \text{hold private belief } q_s^t \text{ that induces } s_s. \end{cases} \quad 0 < p < 1.$$

Given these parameters, we can simplify the PSA graph into just three states so that s_c is the "super-state" composed of every non- r, s states: $s_c := \{s_i : i = 1 \dots K, i \neq r, s\}$. Call this s_c "no-man's land." Then the PSA is of form:

$$\text{PSA} = \left(\mathbb{S}_K = \{s_r^*, s_s, s_c\}, \quad \mathbb{A}_K = \{(s_i, s_j) : s_i, s_j \in \mathbb{S}_K\}, \quad s_o = *, \quad \mathbb{S}_K^* = \{s_s, s_r^*\} \right).$$

Notice the initial state s_o is arbitrary as there is a unique steady state distribution over states regardless of initialization point. Additionally, we assume:

1. QAudit samples Auditors with uniform randomness, so that the likelihood of sampling any Auditor holding belief s_r^* is their population proportion p .
2. if the PSA is at the intermediate state s_c , then the PSA will transition with likelihood:

$$p\left(X_{\tau+1}|X_\tau = s_c\right) = \begin{cases} 2q & \text{remain in state } s_c \\ p-q & \text{transition to state } s_r^* \\ 1-p-q & \text{transition to state } s_s \end{cases} \quad 0 \leq q < \frac{1}{2}.$$

3. The Auditors jump from state s_r^* to s_s with probability ϵ , and vice versa.

All in all, we can define state transition matrix $\mathbf{M}$ with steady state probability vector $\mathbf{p} = [p(s_c^*), p(s_r), p(s_s)]$,¹ so that $\mathbf{p}^T \mathbf{e} = 1$, and $\mathbf{p} = \mathbf{pM}$ per eqn (B.1):

$$[p(s_c^*), p(s_r), p(s_s)]^T = [p(s_c^*), p(s_r), p(s_s)] \begin{bmatrix} 2q & p-q & 1-p-q \\ 1-p-\epsilon & p & \epsilon \\ p-\epsilon & \epsilon & 1-p \end{bmatrix},$$

and note that $\mathbf{M}$ is row-stochastic. Solving this expression, we find a solution set of steady state distributions that is a function of p and q :

1. (case $p = \frac{1}{2}$): in this case we also have: $0 < \epsilon < \frac{1}{2}$, and $0 < q \leq \frac{1}{3}$:

$$p(s_c) = \frac{1-2\epsilon}{3-4q-2\epsilon} \quad p(s_r^*) = \frac{1-2q}{3-4q-2\epsilon} \quad p(s_s) = \frac{1-2q}{3-4q-2\epsilon}.$$

Unsurprisingly, when the Auditors' opinion are divided evenly, then the likelihood of finding s_r^* is just as likely as finding s_s . Moreover for $\epsilon > 1/3$, the likelihood of landing in no-man's land is greater than finding either one of the two private permutations. At $\epsilon = 1/3$, the distribution over the states is uniform.

2. (case $\frac{1}{2} < p < 1$, $0 < \epsilon < p$, $0 < q \leq \frac{1}{3}$):

$$\begin{aligned} p(s_c) &= \frac{p-p^2-\epsilon^2}{p^2-p-q(2\epsilon+1)-\epsilon^2+\epsilon+1} & p(s_r^*) &= \frac{p^2-p(q+\epsilon)-q\epsilon+\epsilon}{p^2-p-q(2\epsilon+1)-\epsilon^2+\epsilon+1} \\ p(s_s) &= \frac{p^2+p(q+\epsilon-2)-q(1+\epsilon)+1}{p^2-p-q(2\epsilon+1)-\epsilon^2+\epsilon+1} \end{aligned}$$

This is the case where more Auditors believe in s_r^* than s_s . The solution space can be partitioned in according to additional conditions:

¹Note $\mathbf{p}$ is not the same as the price vector ρ^t , which expresses the likelihood that any Agent_K will finish first on stage t .

a) If we require the likelihood of $p(s_r^*) > p(s_c)$, then:

$$\begin{aligned}
 & p(s_r^*) > p(s_c) \implies \\
 & \underbrace{\frac{1}{2} < p \leq \frac{2}{3}, 0 < q < 2p - 1, 0 < \epsilon < p}_{(1)} \text{ or} \\
 & \underbrace{\frac{1}{2} < p \leq \frac{2}{3}, 2p - 1 < q \leq \frac{1}{3}, \frac{1}{2}(\sqrt{6pq - 7p^2 + 2p + p^2 - 2q + 1} + p + q - 1) < \epsilon < p}_{(2)} \text{ or} \\
 & \underbrace{\frac{2}{3} < p \leq 1, 0 < q \leq \frac{1}{3}, 0 < \epsilon < p}_{(3)}.
 \end{aligned}$$

In conditions (1) and (2) the proportion p of Auditors holding belief s_r^* may be less than $2/3$, provided the probability of transitioning out of no-man's land is better than $1/3$. In condition (3), whereby more than $2/3$ of the Auditors hold belief s_r^* , then no additional constraints is placed on q and ϵ above the assumptions of the current solution.

b) In general, it is useful to define an arbitrary minimum values of $p(s_r^*)$, and determine what the value of p , q , and ϵ must be to achieve the desired threshold. For example with numerical constraints $p(s_r^*) \geq 0.8$, we have many solutions:

$$\begin{aligned}
 & p(s_r^*) > 0.8 \implies \\
 & \underbrace{0.615 < p \leq 0.8, \frac{4 - 5p}{4 - 2p} < q < \frac{1}{3}, l_\epsilon < \epsilon < p}_{(1)} \\
 & \underbrace{0.8 < p \leq 0.829, 0 < q < \frac{1}{3}, l_\epsilon < \epsilon < p, \text{ or}}_{(2)} \\
 & \underbrace{0.829 < p \leq 0.840, 0 < q < \frac{p^2 + 4p - 4}{5p - 4}, l_\epsilon < \epsilon < p}_{(3)}, \\
 & \text{where } l_\epsilon = \frac{1}{8}(\sqrt{9p^2 + 50pq - 74p + 9q^2 - 58p + 65 + 5p - 3q - 1}).
 \end{aligned}$$

Notably, in (1) we see it is possible to achieve more than 80% likelihood of landing on s_r^* when only 62% of the Auditors hold this opinion, provided the likelihood of remaining in s_c no-man's land is less than $2/3$, and ϵ is larger than some lower bound l_{eps} .

Next the proportion p is allowed to be larger than 0.8, and the likelihood of staying in no-man's land is also allowed to be greater.

3. (case $\frac{1}{2} < p < \frac{1-q}{2q}$, $0 < \epsilon < p$, $\frac{1}{3} < q \leq \frac{1}{2}$), alternatively we can have $0 < \epsilon < \frac{1}{2}(\sqrt{4p^2 - 4p + 4q^2 - 8q + 5} - 2q + 1)$:

$$\begin{aligned}
 p(s_c) &= \frac{p - p^2 - \epsilon^2}{p^2 - p - q(2\epsilon + 1) - \epsilon^2 + \epsilon + 1} & p(s_r^*) &= \frac{p^2 - p(q + \epsilon) - q\epsilon + \epsilon}{p^2 - p - q(2\epsilon + 1) - \epsilon^2 + \epsilon + 1} \\
 p(s_s) &= \frac{p^2 + p(q + \epsilon - 2) - q(\epsilon + 1) + 1}{p^2 - p - q(2\epsilon + 1) - \epsilon^2 + \epsilon + 1}.
 \end{aligned}$$

Similar to the case above, we can place conditions on the desired value of $p(s_r^*)$:

$$p(s_r^*) > 0.8 \implies$$

$$\underbrace{0.5 < p \leq 0.615, \frac{5p-4}{2p-4} < q < \frac{1}{2p+1}, l_\epsilon < \epsilon < p}_{(1)}$$

$$\underbrace{0.615 < p \leq 0.840, \frac{1}{3} < q < \frac{1}{2p+1}, l_\epsilon < \epsilon < p \text{ or}}_{(2)}$$

$$\underbrace{0.842 < p \leq 0.957, \frac{1}{3} < q < \frac{1}{2p+1}, l_\epsilon < \epsilon < p,}_{(3)}$$

$$\text{where } l_\epsilon = \frac{1}{8}(\sqrt{9p^2 + 50pq - 74p + 9q^2 - 58p + 65} + 5p - 3q - 1).$$

We can achieve the desired outcome of $p(s_r^*)$ with a slightly majority value in p as seen in case (1), provided the likelihood of staying in s_e is lesser than $2/3$.²

Now since the label of s_r^* is arbitrary, the same analysis applies to s_s if that is the prompted ranking. All in all, this means that if the proportion of Auditors who hold private belief s_r^* is greater than 50%, then the likelihood of finding the solution s_r^* can be greater than 50% under some very mild conditions. ■

C.4 Fundamental Theorem across Stages

Proof. for theorem 2.6.10. In this proof, we select any Auditor_i, and fix v_t^i across the the Auditors so that $v_t^i = v_t$. And note we have switched the position of the index from the conventional notation so that:

$$v_t^i := v_t^t.$$

This avoid confusion later. Next pick some interval over stages $[t^*, t^* + \Delta t^*]$, where 1) t^* is the last regime change point, and 2) the future regime change stage $t^* + \Delta t^*$ can be either odd or even. Then we can compute the present value of a sequence of wagers over the stage interval. We make two assumptions before proceeding:

1. because of the Dutch auction factor, the price of $\min_k^t$ shares increase on every stage, and *assuming* the Auditors are wagering in a tit-for-tat manner, then we can write:

$$v_{t+1} = v_t + \Delta v_t, \text{ where } \Delta v_t > 0, \text{ and } t \in [t^*, t_e^*]. \quad (C.5)$$

Stating that the Auditor_i and Auditor_j are dutifully marking up each other's wagers on every stage in accordance to Dutch auction factor v_p^t . And by assumption Auditor_i is winning in every stage t .

2. Suppose $t^* + \Delta t^*$ will take value according to:

$$t^* + \Delta t^* := \begin{cases} \text{even} & \text{with probability } p \\ \text{odd} & \text{with probability } 1 - p. \end{cases}$$

3. For each scenario above, Auditor_i joins Odds and Evens clique with probability:

$$\text{Auditor}_i \text{ plays} := \begin{cases} \text{Evens} & \text{with probability } q \\ \text{Odds} & \text{with probability } 1 - q. \end{cases}$$

Note that p and q are independent as Auditors do not know a-priori whether $t^* + \Delta t^*$ is odd or even.

²All of the arithmetic here is done by wolframalpha.com.

Then we have the game tree whereby for any Auditor_i, his expected value for playing is:

$$\mathbb{E}[x] = p(qv_{ee} + (1-q)v_{eo}) + (1-p)(qv_{oe} + (1-q)v_{oo}).$$

Where v_{ee} is the discounted value of the game to Auditor_i if $t^* + \Delta t^*$ is even, and he joins the Evens clique; v_{eo} is the discounted value if $t^* + \Delta t^*$ is even, and the Auditor_i joins the Odds clique, and so on. We may compute x as follows:

1. Let $t_e^* = t^* + \Delta t^*$ be even, then if Auditor_i enter the game on the Odds stages, his profits are:

$$\begin{aligned} v_{eo} &\stackrel{(1)}{=} - \underbrace{\left(v_{t^*+1} \cdot (1+r)^{-1} + v_{t^*+3} \cdot (1+r)^{-2} + \dots + v_{t_e^*-1} \cdot (1+r)^{-(t_e^*-1)} \right)}_{\text{Auditor}_i\text{'s wagers on odd stages}} \\ &\quad + \underbrace{\left(v_{t^*+2} \cdot (1+r)^{-1} + v_{t^*+4} \cdot (1+r)^{-2} + \dots + v_{t_e^*-2} \cdot (1+r)^{-(t_e^*-2)} \right)}_{\text{Auditor}_j\text{'s wagers on even stages}} \\ &\stackrel{(2)}{\geq} \left(-v_{t^*+1} + \underbrace{v_{t^*+1} + \Delta v_{t^*+1}}_{\text{Auditor}_j\text{'s wager } v_{t^*+2}} \right) \frac{1}{(1+r)^{t^*+1}} + \dots \\ &\quad + \left(-v_{t_e^*-3} + \underbrace{v_{t_e^*-3} + \Delta v_{t_e^*-3}}_{\text{Auditor}_j\text{'s wager } v_{t_e^*-2}} \right) \frac{1}{(1+r)^{t_e^*-3}} - v_{t_e^*-1} \frac{1}{(1+r)^{t_e^*-1}}. \end{aligned}$$

Now we define the profit hurdle with:

$$\begin{aligned} &\stackrel{(3)}{\Rightarrow} \underbrace{\frac{\Delta v_{t^*+1}}{(1+r)^{t^*+1}} + \dots + \frac{\Delta v_{t_e^*-3}}{(1+r)^{t_e^*-3}}}_{\text{Auditor}_i\text{'s revenue assuming he wins every round}} > \underbrace{\frac{v_{t_e^*-1}}{(1+r)^{t_e^*-1}}}_{\text{Auditor}_i\text{'s spending}} \\ &\stackrel{(4)}{\Rightarrow} \frac{\Delta v}{(1+r)^{t^*+1}} + \dots + \frac{\Delta v}{(1+r)^{t_e^*-3}} > \frac{v_{t_e^*-1}}{(1+r)^{t_e^*-1}} \end{aligned} \quad (C.6)$$

Where we reason as follows:

- a) going into (1) we note that the Evens Auditor would not wager on round $t_e^* = t^* + \Delta t^*$, since he can see that the models have been reset. So the Odds wager one more round than the Evens.
- b) Going into (2), we substitute eqn (C.5) and rewrite the Evens Auditor_j's wager as a function of the Odds Auditor_i's wager on the previous stage. And use the inequality:

$$-\frac{v_{t^*+1}}{(1+r)^{t^*+1}} + \frac{v_{t^*+1} + \Delta v_{t^*+1}}{(1+r)^{t^*+2}} \geq \frac{-v_{t^*+1} + v_{t^*+1} + \Delta v_{t^*+1}}{(1+r)^{t^*+1}},$$

to pull the discount factor backward one step on $v_{t^*+2} = v_{t^*+1} + \Delta v_{t^*+1}$. And observe that if the model improvement between consecutive stages t is sufficiently small, then r is also small, so the two values above are approximately equal.

- c) Going into (3), we assume Auditor_i wins every round, and compute his discounted revenue. This assumption is fair as the previous theorems showed that if the Auditors just follow the QAudit prompt, then they will find the best permutation over Agents. Now because of 1) the sequential parimutuel payout structure, and 2) the Dutch auction price ascending factor v_ρ^t , there is some earnings Δv_t for every round t . That is except the penultimate round $t_e^* - 1$, when the Evens Auditor_j

refuse to wager due to regime change. Note as $(1 + \gamma)^t$ is monotonically decreasing with:

$$(1 + \gamma)^{-(t_e^*-1)} < (1 + \gamma)^{-(t_e^*-2)} < \dots < (1 + \gamma)^{-(t^*+1)},$$

we know that $(1 + \gamma)^{-(t_e^*-1)}$ must be the smallest value.

d) Going into (4), we know by construction of $\mathbf{v}_\rho^t$ that $\Delta \mathbf{v}_t$ must be the same for every t on a per share basis assuming both initialization price vector $\mathbf{v}_\rho^o$ and purchasing volume are constant across stages. Then we know $\Delta \mathbf{v}_{t^*+l} = \Delta \mathbf{v}$ for every $l = 1, \dots, t_e^* - 1$.

e) All in all, we see that the drivers of profit risk are:

- i. *Regime change risk* γ : this is a function of the underlying improvement in models over consecutive stages.
- ii. *Price at $t_e^* - 1$* : the ratio $\mathbf{v}_{t_e^*-1}/\Delta \mathbf{v}$ determines the amount of loss Auditor_i takes at the moment of regime change. In particular, the break-even point is:

$$\Delta t^* \Delta \mathbf{v} = \mathbf{v}_{t_e^*-1}.$$

This suggests that if regime change is frequent, the Dutch auction factor $\mathbf{v}_\rho^t$ must be aggressive.

2. Alternatively, suppose t_e^* is even as before, and Auditor_i choose to wager with the Evens, then his profits are:

$$\begin{aligned} v_{ee} &\stackrel{(1)}{=} \underbrace{\left(\mathbf{v}_{t^*+1} \cdot (1 + \gamma)^{-1} + \mathbf{v}_{t^*+3} \cdot (1 + \gamma)^{-2} + \dots + \mathbf{v}_{t_e^*-1} \cdot (1 + \gamma)^{-(t_e^*-1)} \right)}_{\text{Auditor}_j \text{'s wagers on odd stages}} \\ &\quad - \underbrace{\left(\mathbf{v}_{t^*+2} \cdot (1 + \gamma)^{-2} + \mathbf{v}_{t^*+4} \cdot (1 + \gamma)^{-4} + \dots + \mathbf{v}_{t_e^*-2} \cdot (1 + \gamma)^{-(t_e^*-2)} \right)}_{\text{Auditor}_i \text{'s wagers on even stages}} \\ &\stackrel{(2)}{\geq} \frac{-\mathbf{v}_{t^*+2} + \mathbf{v}_{t^*+2} + \Delta \mathbf{v}_{t^*+2}}{(1 + \gamma)^{t^*+2}} + \dots + \frac{-\mathbf{v}_{t_e^*-2} + \mathbf{v}_{t_e^*-2} + \Delta \mathbf{v}_{t_e^*-2}}{(1 + \gamma)^{t_e^*-2}} \\ &\stackrel{(3)}{\implies} \frac{\Delta \mathbf{v}_{t^*+2}}{(1 + \gamma)^{t^*+2}} + \frac{\Delta \mathbf{v}_{t^*+4}}{(1 + \gamma)^{t^*+4}} + \dots + \frac{\Delta \mathbf{v}_{t_e^*-2}}{(1 + \gamma)^{t_e^*-2}} \geq 0 \\ &\stackrel{(4)}{\implies} \frac{\Delta \mathbf{v}}{(1 + \gamma)^{t^*+2}} + \frac{\Delta \mathbf{v}}{(1 + \gamma)^{t^*+4}} + \dots + \frac{\Delta \mathbf{v}}{(1 + \gamma)^{t_e^*-2}} \geq 0. \end{aligned} \tag{C.7}$$

The reasoning are similar to the Odds case:

- a) going into (1) we simply flipped the sign and the index i, j from above. Note in this case, the Evens does not start playing until $t^* + 2$, and plays until stage $t_e^* - 2$, as they observe the regime change in stage t_e^* .
- b) Going into (2), we substitute eqn (C.5) and proceed with the same inequality as before.
- c) Going into (3), we note that the profit is always at least zero, provided that the Evens Auditor_i's ensemble model always defeats the previous stages' models. This means if Auditor_i choose correctly and plays the Evens, then he never loses money if t_e^* is even.
- d) Going into (4), we use the same reasoning around $\Delta \mathbf{v}$ as before.

3. Reasoning just so, and let $t_o^* = t^* + \Delta t^*$ be odd, so we have interval $[t^*, t_o^*]$. Then we find v_{oo} 's expected value is no less than zero:

$$\frac{\Delta v}{(1+r)^{t^*+1}} + \frac{\Delta v}{(1+r)^{t^*+3}} + \dots + \frac{\Delta v}{(1+r)^{t_o^*-2}} \geq 0. \quad (C.8)$$

4. And similarly t_o^* is odd as before, then v_{oe} must meet hurdle:

$$\frac{\Delta v}{(1+r)^{t^*+2}} + \frac{\Delta v}{(1+r)^{t^*+4}} + \dots + \frac{\Delta v}{(1+r)^{t_o^*-3}} \geq \frac{v_{t_o^*-1}}{(1+r)^{t_o^*-1}}. \quad (C.9)$$

Consequently, Auditor_i's expected value discounted by regime risk is:

$$\begin{aligned} \mathbf{E}[x] = & p \cdot \Delta v \cdot \left(\frac{1-q}{(1+r)^{t^*+1}} + \frac{q}{(1+r)^{t^*+2}} + \dots + \frac{1-q}{(1+r)^{t_o^*-3}} + \frac{q}{(1+r)^{t_o^*-2}} - \frac{(1-q)v_{t_o^*-1}/\Delta v}{(1+r)^{t_o^*-1}} \right) \\ & + (1-p) \cdot \Delta v \cdot \left(\frac{q}{(1+r)^{t^*+1}} + \frac{1-q}{(1+r)^{t^*+2}} + \dots + \frac{q}{(1+r)^{t_o^*-3}} + \frac{1-q}{(1+r)^{t_o^*-2}} - \frac{qv_{t_o^*-1}/\Delta v}{(1+r)^{t_o^*-1}} \right). \end{aligned}$$

So we have:

$$\begin{aligned} \mathbf{E}[x] = & p \Delta v \sum_{t=1}^{t_o^*} \frac{(1-q)(t \bmod 2) + q(1+t \bmod 2)}{(1+r)^{t^*+t}} + (1-p) \Delta v \sum_{t=1}^{t_o^*} \frac{q(t \bmod 2) + (1-q)(1+t \bmod 2)}{(1+r)^{t^*+t}} \\ & \underbrace{\hspace{15em}}_{\text{Auditor}_i\text{'s earnings until regime change}} \\ & - \underbrace{\left(\frac{(1-q)v_{t_o^*-1}}{(1+r)^{t_o^*-1}} - \frac{qv_{t_o^*-1}}{(1+r)^{t_o^*-1}} \right)}_{\text{Auditor}_i\text{'s loss on regime change}} \quad (C.10) \end{aligned}$$

We can leave this expression in unreduced form as $t_o^* \neq t_e^*$. And observe that the regime change risk $-(1+r)^{-(t_e^*-1)}$ and $-(1+r)^{-(t_o^*-1)}$ are vanishingly small, if $t_e^* - t^*$ and $t_o^* - t^*$ are sufficiently large. Now in the special case where:

1. $t_o^* = t_e^* + 1$,
2. $q = \frac{1}{2}$, so $q = 1 - q$,

then we can simplify the expression further. Consider the sum:

$$p \cdot \frac{1-q}{(1+r)^{t^*+1}} + (1-p) \cdot \frac{q}{(1+r)^{t^*+1}} = \frac{pq}{(1+r)^{t^*+1}}.$$

Proceeding as above for every term in C.10, we have:

$$\begin{aligned} \frac{1}{\Delta v} \cdot \mathbf{E}[x] = & \frac{pq}{(1+r)^{t^*+1}} + \frac{pq}{(1+r)^{t^*+2}} + \dots + \frac{pq}{(1+r)^{t_o^*-3}} + \frac{pq}{(1+r)^{t_o^*-2}} + \frac{pq}{(1+r)^{t_o^*-1}} \\ & - \frac{v_{t_o^*-1}}{\Delta v} \cdot \frac{pq}{(1+r)^{t_o^*-1}} - \frac{v_{t_e^*}}{\Delta v} \cdot \frac{(1-p)q}{(1+r)^{t_e^*}} \\ \stackrel{(1)}{\leq} & - \frac{v_{t_e^*}}{\Delta v} \cdot \left(\frac{1}{(1+r)^{t_e^*-1}} + \frac{1}{(1+r)^{t_e^*}} \right) + \sum_{t=1}^{t_e^*-1} \frac{pq}{(1+r)^{t^*+t}} \\ \stackrel{(2)}{\leq} & - \frac{2v_{t_e^*}/\Delta v}{(1+r)^{t_e^*-1}} + \sum_{t=1}^{t_e^*-1} \frac{pq}{(1+r)^{t^*+t}} \end{aligned}$$

$$\begin{aligned}
 & \xrightarrow{(3)} \sum_{t=1}^{t_e^*-1} \frac{pq}{(1+r)^{t^*+t}} \geq \frac{2v_{t_e^*}/\Delta v}{(1+r)^{t_e^*-1}} \\
 & \xrightarrow{(4)} \sum_{t=1}^{t_e^*-1} \frac{p}{(1+r)^{t^*+t}} \geq \frac{4v_{t_e^*}}{\Delta v} \frac{1}{(1+r)^{t_e^*-1}} \\
 & \xrightarrow{(5)} \\
 & v_{t_e^*} \leq \Delta v \frac{p}{4} \sum_{t=2}^{t_e^*} (1+r)^{t_e^*-t} \quad \text{or} \quad \frac{v_{t_e^*}}{\Delta v} \leq \frac{p}{4} \sum_{t=2}^{t_e^*} (1+r)^{t_e^*-t} \quad \text{or} \\
 & \Delta v \geq \frac{4v_{t_e^*}}{p} \cdot \frac{1}{\sum_{t=2}^{t_e^*} (1+r)^{t_e^*-t}} \xrightarrow{(6)} \frac{8v_{t_e^*}}{\sum_{t=2}^{t_e^*} (1+r)^{t_e^*-t}} \quad \text{if } p = \frac{1}{2}. \tag{C.11}
 \end{aligned}$$

We reason as follows:

1. going into (1) we use the fact that $p, 1-p, q \leq 1$, and $v_{t_e^*-1} \leq v_{t_e^*}$ given price markup Δv_t on every stage t .
2. Going into (2), we observe:

$$\frac{2}{(1+r)^{t_e^*-1}} \geq \frac{1}{(1+r)^{t_e^*}} + \frac{1}{(1+r)^{t_e^*-1}}.$$

3. Going into (3), we note that $pq = \frac{p}{2}$.
4. Going into (4, 5), we reduce every term in the sum with:

$$\frac{(1+r)^{t_e^*-1}}{(1+r)^l} = (1+r)^{t_e^*-1-l}$$

for every $l = 1..t_e^* - 1$. And then set $\mathbf{E}[x] \geq 0$ to solve for the conditions on $v_{t_e^*}$ and Δv .

5. Going into (6), we set $p = \frac{1}{2}$ to arrive at eqn 2.13

In fact, we can reduce C.11 further by setting $v_{t_e^*} = \epsilon + (t_e^* - 1)\Delta v$, where $v_{t_e^*} := \epsilon$. And then we have:

$$\begin{aligned}
 \Delta v & \geq \frac{8(\epsilon + (t_e^* - 1)\Delta v)}{\sum_{t=2}^{t_e^*} (1+r)^{t_e^*-t}} \implies \\
 \Delta v & \geq \frac{8\epsilon}{(\sum_{t=2}^{t_e^*} (1+r)^{t_e^*-t}) - 8(t_e^* - 1)} = \frac{\epsilon}{\max\left[1, (\sum_{t=2}^{t_e^*} (1+r)^{t_e^*-t}) - 8(t_e^* - 1)\right]}.
 \end{aligned}$$

Where going into the equal sign, we set $8\epsilon = \epsilon$. A similar analysis can be done for $t_e^* = t_o^* + 1$. All in all, we can see that as Δt^* increases, it becomes more feasible to satisfy conditions C.11, for every value of r . For example, for $\Delta t^* = 100$, $\epsilon = 1$, and $r = 0.05$, we have $\Delta v \geq 0.000515$. Thus QAudit must balance Dutch auction ascending factor v_ρ^t with the frequency of regime change to maintain Auditor retention. Now given conditions C.11 are met, and as the choice of interval $[t^*, t^* + \Delta t^*]$ is arbitrary, we know that the multi-stage QAudit game has positive expected value for every Auditor's coin toss decision to play Odds or Evens. ■

C.5 FeaAudit

Proof. for proposition 3.2.3. Suppose some Cohort^n has N Witnesses. And suppose every Witness is called, then their testimony before the PA is an bit string of length N . The set of all possible N -bit strings form a boolean-cube with 2^N vertices, where the edges correspond to flipping a single bit. We conceptualize a random experiment as follows:

1. There is a true label N -bit string:

$$\mathbf{b} = [y_1, \dots, y_t, \dots, y_N],$$

corresponding to some vertex on the hypercube. Where the elements of $\mathbf{b}$ are arranged in chronological order w.r.t the while loop of **FeaAudit**.

2. At some stage t , the Witness named Wit_t^n is called to testify, his true preference is y_t as seen in $\mathbf{b}$.
3. The **FeaAudit** protocol then flips this y with probability $1 - p$, as seen in eqn 3.2. So the probability of flipping a bit is:

$$p(\tilde{y} = \text{Yes} \mid y = \text{No}) + p(\tilde{y} = \text{No} \mid y = \text{Yes}) = \frac{1}{2}(1 - p) + \frac{1}{2}(1 - p) = 1 - p.$$

Critically, the flips are independent. This creates a N -bit string where the masked bit $\tilde{y}_t$ is flipped with likelihood $1 - p$.

4. After all N Witnesses have testified, we have the corrupted string:

$$\tilde{\mathbf{b}} = [\tilde{y}_1, \dots, \tilde{y}_t, \dots, \tilde{y}_N], \text{ where } \tilde{y}_t = y_t \text{ with probability } p.$$

Now we have to determine 1) the *expected* distance between $\tilde{\mathbf{b}}$ and $\mathbf{b}$ measured in edge traversals; and 2) the likelihood that $\tilde{\mathbf{b}}$ and $\mathbf{b}$ differ by more than some amount so as to render the collected data "too noisy" to use. Denoting distance with: $X = \|\tilde{\mathbf{b}} - \mathbf{b}\|_1$, so that the distance is x when x bits have been flipped. Then the expected value of this distance is:

$$\begin{aligned} \mathbf{E}[X] &= \sum_{x=0}^N x \binom{N}{x} p^{N-x} (1-p)^x \stackrel{(1)}{=} \sum_{x=0}^N N \binom{N-1}{x-1} p^{N-x} (1-p)^x \\ &= N \sum_{x=0}^{N-1} \binom{N-1}{x} p^{N-x-1} (1-p)^{x+1} \stackrel{(2)}{=} N(1-p) \sum_{x=0}^{N-1} \binom{N-1}{x} p^{N-x-1} (1-p)^x \\ &= N(1-p). \end{aligned}$$

Where going into (1), we use $x \binom{N}{x} = N \binom{N-1}{x-1}$, going into (2) the sum is 1. And the expectation is w.r.t the coin tosses of the first coin_p , and the second fair coin with $p(\text{Head}) = \frac{1}{2}$. Now we require this distance to be less than some d where $d = 0, 1, 2, \dots$:

$$N(1-p) \leq d \implies p \geq 1 - \frac{d}{N}.$$

Finally using Chernoff bound of def B.1.3, we can determine the likelihood that X is more than $(1 \pm \delta)$ away from $\mathbf{E}[X] = d$. For every $\delta \in (0, 1)$:

$$p(X \geq (1 + \delta)d) \leq \exp\left(-\frac{\delta^2 d}{3}\right).$$

Stating the likelihood of encountering any $\tilde{\mathbf{b}}$ generated by **FeaAudit** that strays too far from $\mathbf{E}[X]$ decays exponentially in d . So the overall N -bit string summarizing any Cohort 's preference reflects the actual value $\mathbf{b}$ if the PA sets coin_p so that $p \geq 1 - \frac{d}{N}$. ■

Proof. for proposition 3.2.6. Observe that the PA's coin_q^t has bias $(q_l, q_u) = (\frac{1}{3}, \frac{2}{3})$. Referencing eqn 3.4, we have:

$$q_l^2 \leq p(\text{Head}) \leq q_u^2, \text{ for every } t,$$

And by 3.5 the eunuch's survival time at court is between $t^* \in [2.25, 9]$ stages. Now suppose VA_k enters the eunuch position with initial wealth v_k , and setting the condition so that he can *expect* to leave the eunuch seat with more wealth than when he started. Then we have this condition on the earning sequence:

$$\begin{aligned} (1 - \xi) \left(v_k + \frac{\xi}{2} \sum_{i=0}^{t^*} v_i \right) &\geq v_k \text{ where } i \in [1, \dots] & (C.12) \\ \implies v_k &\geq \frac{1 - \xi}{2} \sum_{i=0}^{t^*} v_i \xrightarrow{(1)} v_k \geq \frac{1 - \xi}{2} \left(\frac{1}{q^2} + 1 \right) v \\ \implies \xi &\geq 1 - 2 \frac{v_k}{v} \frac{q^2}{1 + q^2} \xrightarrow{(2)} \xi \geq 1 - \frac{2q^2}{1 + q^2} \\ \implies \xi &> 0.384 \text{ if } q = \frac{2}{3}, \text{ or } \xi < 0.8 \text{ if } q = \frac{1}{3} \end{aligned}$$

Where:

1. going into (1) we use the fact that $t^* = \frac{1}{q^2}$ in expectation, and note we add one extra round as the VA_k is given *miat* as soon as he is selected into the position. And that the minimum wallet balance to enter *FeaAudit* must be v .
2. Going into (2), we use set $v_k = v$ and note that $-v_k/v \leq -1$.

In the expression, we assume the VA_k enters the eunuch position with the minimum required funds v . In the event where the VA_k 's wealth is $\alpha \cdot v$ for $\alpha \geq 1$, and the average Witness wealth is $\beta \cdot v$, then we should set ξ to:

$$\xi \sim 1 - \frac{2q^2\alpha}{(1 + q^2)\beta}, \text{ where } \alpha, \beta \geq 1. \quad (C.13)$$

In a production *FeaAudit* protocol, it may be advisable to adjust ξ dynamically based on how much funds are in the players' wallet. ■

Proof. for proposition 3.3.1. Given some interval $t^* \in [1, t_u^*]$, where t^* denotes the stage where VA_k flips a *Head, Head* sequence and is thus eliminated. Then we know this corresponds to a binary tree of depth t_u^* , where in every tree-depth $t \in [1, t_u^*]$, the left branch terminates on $p(\text{Head, Head}) = q_t^2$, while the right branch continues with $p(\text{Head, Tail or Tail}) = 1 - q_t^2$. The probability of terminating on any stage t^* is then:

$$p(\text{rejected on stage } t^*) = q_{t^*}^2 \prod_{t=1}^{t^*-1} (1 - q_t^2).$$

Using eqn C.12 for VA_k 's wealth accumulation for terminating on stage t^* modulo the $v + t\Delta v$ prefix.

The VA's expected value over the coin tosses with support on interval $[1, t_u^*]$ is:

$$\begin{aligned}
\mathbf{E}\left[\text{VA}_k\text{'s income on } [1, t_u^*]\right] &= \sum_{t^*=1}^{t_u^*} \underbrace{q_{t^*}^2 \prod_{t=1}^{t^*-1} (1 - q_t^2)}_{p(\text{rejected on stage } t^*)} \cdot \underbrace{\frac{\xi}{2} \sum_{i=0}^{t^*} v_i}_{\text{income if rejected on } t^*} \\
&\stackrel{(1)}{\geq} \sum_{t^*=1}^{t_u^*} \frac{2^2}{3^2} \left(\frac{5}{9}\right)^{t^*-1} \frac{\xi}{2} (t^* + 1) \beta v \\
&\stackrel{(2)}{\geq} 2\beta v \xi \cdot \sum_{t^*=0}^{t_u^*} \left(t^* + 2\right) \frac{5^{t^*}}{9^{t^*+1}}.
\end{aligned}$$

Where going into (1), we set the average Witnesses' holding to $v_i = \beta v$ for $\beta \geq 1$. Then we let $q_{t^*} = \frac{2}{3}$ and note that:

$$p\left(\text{rejected on stage } t^* \mid q_{t^*} = \frac{2}{3}\right) > p\left(\text{rejected on stage } t^* \mid q_{t^*} = \frac{1}{3}\right).$$

Going into (2), we shift the index so that $t^* = 0$. Similarly, we can find an upper bound with:

$$\begin{aligned}
\mathbf{E}\left[\text{VA}_k\text{'s income on } [1, t_u^*]\right] &\stackrel{(1)}{\leq} \sum_{t^*=1}^{t_u^*} \frac{1^2}{3^2} \left(\frac{8}{9}\right)^{t^*-1} \frac{\xi}{2} (t^* + 1) \beta v \\
&\leq \frac{\beta v \xi}{2} \sum_{t^*=0}^{t_u^*} \left(t^* + 2\right) \frac{8^{t^*}}{9^{t^*+1}},
\end{aligned}$$

where going into (1) we set $q_{t^*} = \frac{1}{3}$. ■

APPENDIX D

Sequential Parimutuel Prediction Markets

D.1 Introduction

The heart of QAudit is the prediction market, designed to *elicit truthful report* of information from agents, and compensate agents in accordance to the quality of their report. In this regard, a prediction or information market is similar to a stock market. One common design pattern is to enumerate the outcome set $\Omega = \{.., \omega_i, ..\}$ and offer a security for each outcome ω_i , so that the price reflects the likelihood of the outcome in Ω . For example, in a *permutation betting* prediction market, a designer may elicit the ranking of a three-way horse race between horses $\{1, 2, 3\}$ from the betters. There are $3! = 6$ possible outcomes:

$$\Omega = \{(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)\}.$$

The most expressive market maker would offer one Arrow-Debreu security (binary option) ω_i for each outcome, so that the traders holding the option ω_i receive \$1.00 if the horses finish the order specified by ω_i , and receive nothing otherwise. Each ω_i would be priced by some pricing function **score** so that the price is reflection of the probability $\mathbf{p}$ of the event occurring:

$$\sum_{\omega_i \in \Omega} \text{score}(\omega_i) = 1 \quad 0 \leq \text{score}(\omega_i) \leq 1. \quad (\text{D.1})$$

A betting market whereby every natural outcome is expressed in a binary option is *complete* [For+07]. Clearly as size of Ω grows, it may be difficult to market every security. Thus it is prudent to express a smaller set of events with $\Omega_a = \{1, 2, 3\}$ stating which horse will finish first, or $\Omega_b = \{1, 2, 3\}$ stating which horse will finish second. The event set Ω_a is called the *winning track*. A pricing function **score** that elicits the truthful likelihood estimation of events from rational betters is called a *proper scoring rule* [[Han02],[Han03],[ACV12],[GR07],[FPW23]]. Proper scoring rules ensure the agents reporting the information can only profit if they reveal their private truth. That is to say the proper scoring rule is *incentive compatible*: it aligns the agent's interest with those of the market designer. Definition D.1.1 restates this notion in more formal terms.

Definition D.1.1 (Proper Scoring Rule). Let there be a random variable ranging over event space $\Omega = \{\omega\}_n$, let the probability vector $\mathbf{p} = (\mathbf{p}_1, .., \mathbf{p}_\omega, .., \mathbf{p}_n)$ where $\sum_{\omega} \mathbf{p}_\omega = 1$ be a distribution over the set Ω , and let $\mathfrak{P}$ be the set of all possible probability distributions over Ω , ie $\mathbf{p}, \mathbf{r} \in \mathfrak{P}$. A scoring rule:

$$\text{score} : \mathfrak{P} \times \Omega \rightarrow \mathbb{R}$$

maps some probability distribution $\mathbf{p} \in \mathfrak{P}$ and event $\omega \in \Omega$ onto the scalar $\text{score}(\mathbf{p}, \omega)$. For example, if a person reports his probability distribution as $\mathbf{r} = (r_1, .., r_\omega, .., r_n)$ and event ω is

realized, then his reward would be $\text{score}(\mathbf{r}, \omega)$. A scoring rule is proper if telling the truth maximizes the score. That is to say score must satisfy the incentive compatibility constraint:

$$\mathbf{p}^* = \arg \max_{\mathbf{r}} \sum_{\omega \in \Omega} p_{\omega} \text{score}(\mathbf{r}, \omega), \quad (\text{D.2})$$

and a rational participation constraint:

$$\sum_{\omega} p_{\omega} \text{score}(\mathbf{p}, \omega) \geq 0.$$

Moreover, if an expert reports his probability as $\mathbf{r}$, then the scoring rule score is *strictly proper* if reporting $\mathbf{r} = \mathbf{p}$ uniquely maximizes the payout to the expert [GR07].

The next two properties a market designer should satisfy is *budget balance* and *optimal opinion pooling*, incidentally they are mutually reinforcing, and the solution is the *market scoring rule* [Han02]. A market scoring rule is akin to the proper scoring rule, except the payout score is no longer a stateless function, but is instead a function of both the event reported ω_i , and the shared underlying market state $\mathbf{p}$: $\text{score}(\mathbf{p}, \omega_i)$. This underlying market state $\mathbf{p}$ is updated each time an agent buys from the market, and represents the market consensus for a distribution over event space Ω . A sequential parimutuel market works as follows:

1. The agents approach the market sequentially, and report their hidden estimation of likelihood by buying Arrow-Debreu options.
2. The Amm keeps a shared likelihood estimate across reports, reflecting the latest market consensus on the likelihood of an event. The market scoring rule incorporates the Agent's report and updates the likelihood. See table D.1.

Sequential parimutuel market: Agent report $\mathbf{p}$

Stage t	Agent wagers $\mathbf{v}_{k,i}^t$	Agent reports $\mathbf{p}$	reward $\mathbf{v}_{\text{Agent}}^t$
0	0	$\mathbf{p}_{\text{Amm}}^0 \rightarrow \log 0.5$	0
1	$\mathbf{v}_{\text{alice}}^1$	$\mathbf{p}_{\text{alice}}^1 \rightarrow \log 0.4$	0
2	$\mathbf{v}_{\text{bob}}^2$	$\mathbf{p}_{\text{bob}}^2 \rightarrow \log 0.7$	$\mathbf{v}_{\text{eve}}^2$
3	$\mathbf{v}_{\text{eve}}^3$	$\mathbf{p}_{\text{eve}}^3 \rightarrow \log 0.9$	$\mathbf{v}_{\text{pa}}$

Table D.1: In sequential prediction market over $t = 3$ rounds, the market keeps a persistent state $\mathbf{p}$ denoting the likelihood an event will occur (i.e. this event may be whether it will rain on Sunday). Each Agent_k approaches the Amm and reports his subjective probability $\mathbf{p}_k^t$. The Agents deposit some wager w_k^t as they report to signal their degree of confidence. The Amm then updates the market likelihood to reflect the latest consensus. At test time, nature reveals that it does rain on Sunday. Thus every Agent_k who improved the market's estimate is rewarded with the next Agent_s 's wager, denoted in green. Agents who degraded the market's estimate lose their wager. If the last Agent reporting improves the market's estimate, then she is rewarded by the bounty $\mathbf{v}_{\text{pa}}$. Thus the market achieves budget balance, and the most the Amm has to pay is the last Agent to report.

A sequential parimutuel prediction market with shared market scoring rule satisfies the two properties:

1. (*optimal opinion pooling*) If the agent holds a private belief for event ω_i , and judge the market price/likelihood of ω_i 's as low, then he will buy the under-priced option,

thus elevating its price. Many agents may buy option ω_i until its likelihood under the market scoring rule `score` matches their subjective beliefs. As the likelihood or price of ω_i increases, all other events in Ω will see decrease in likelihood because of constraint D.1 [CV10]. Market scoring rules can pool opinions in both thin markets and thick markets.

- a) In the thin market, trading volume is low and the agents may not buy every option ω_i on offer, thus rendering the likelihood of some events under-estimated by thin market activity. Market scoring rule ensures that when the likelihood of one event rises, the likelihood of other events fall. Thus the buy-signal propagates to all events.
- b) In the thick market, many agents are buying options. The market scoring rule collates their buy signal using the same principle, thus constructing a single source of truth for what the "market believe will occur."

Thus optimal opinion pooling is achieved in all market conditions. The most well-known example of market scoring rule is the *logarithmic market scoring rule* (LMSR), which is used in **QAudit** [Han02].

2. (*budget balance*) This implies the agents are mostly taking winnings from each other, and the market designer does not have to pay every agent's profit. This is possible because each agent is paying for the previous agent's score. This is the *sequential parimutuel* aspect of the prediction market. Assuming the final observed outcome is ω^* , and the agents report distributions $\{\mathbf{p}_o, \dots, \mathbf{p}_T\}$ over t rounds, then the payment and payout process can be written algebraically:

$$\begin{aligned} \text{market payout} &= \text{score}(\mathbf{p}_T, \omega^*) - \text{score}(\mathbf{p}_{T-1}, \omega^*) + \text{score}(\mathbf{p}_{T-1}, \omega^*) - \dots - \text{score}(\mathbf{p}_o, \omega^*) \\ &= \text{score}(\mathbf{p}_T, \omega^*) - \text{score}(\mathbf{p}_o, \omega^*). \end{aligned}$$

Thus the market designer is only responsible for paying the difference between the initial report $\mathbf{p}_o$, and the final report $\mathbf{p}_T$. All other agents are paying for each other, this is "budget balance." Overall, the market designer's worst case loss is:

$$\max \text{payout} = \max_{\omega^* \in \Omega} \max_{\mathbf{p}_T \in \mathfrak{P}} \left(\text{score}(\mathbf{p}_T, \omega^*) - \text{score}(\mathbf{p}_o, \omega^*) \right). \quad (\text{D.3})$$

Normally, the market designer would be responsible for paying the value of D.3, thus the market designer is also a *market maker*. However **QAudit** is a decentralized auditing protocol, and Metheus merely runs the software. Instead any principle auditor launching an instance of **QAudit** must do so with fiat-denominated bounty computed according to D.3. Thus the PA is the market maker.

In the example given thus far, we assume the market's underlying state is the set of events $\Omega = \{\omega_1, \dots\}$ and a full distribution $\mathbf{p}$ over events Ω . The market charges the agent to update the shared global market state $\mathbf{p}$. There are many other formats of information that can be elicited from agents, amongst which are distribution parameters and raw data.

1. In machine learning the distribution $\mathbf{p}$ is typically parameterized by some $\mathbf{w}$. We observe r.v. drawn according to distribution $\mathbf{p}$ with $\mathbf{x} \in \mathfrak{p}$, and learn a discriminative model with $\mathbf{p}(y|\mathbf{x}; \mathbf{w})$, denoting the likelihood that the data $\mathbf{x}$ has label y under parameter $\mathbf{w}$. As long as the model $\mathbf{w}$ is parameterizing a distribution in the exponential family (see Appendix F) with convex log-partition function, then it is possible to build a market scoring rule that is incentive compatible [[ACV12], [AK+14],[CV10]]. The reason is that such models are typically trained by maximizing the log-likelihood of data, which is the same as minimizing the Kullback-Leibler divergence between current

model parameter and the optimal parameter (see remark E.3.4). Thus the log-partition function is a proxy for the cost function. In fact, the logarithmic market scoring rule reduces to the Bregman divergence (or the Kullback-Leibler divergence in primal form [WJ08]) between the two distributions on this log-partition function [AK+14]. See table D.2 below.

Agent reports classifier $\mathbf{w}$		
Wager $\mathbf{v}_t^k$	Data transformation	Amm shared state
$\mathbf{v}_1^{\text{alice}}$	$\mathbf{w}_{\text{alice}}^1 \rightarrow \log \mathbf{p}_{\text{alice}}^1$	latest reported $\mathbf{w}$

Table D.2: If the **Agent** reports some $\mathbf{w}^t$ at round t parameterizing a logistic regression model, then it is possible to compute a likelihood $\mathbf{p} = \mathbf{p}(y = 1|\mathbf{x}; \mathbf{w}^t) = \sigma(\mathbf{w}^t \mathbf{x})$ given data point $(\mathbf{x}, y)$. If the model is drawn from the exponential family, then its computed probability $\mathbf{p}$ is s.t budget balance can be achieved in expectation. The **Amm**'s shared state is the latest $\mathbf{w}^t$.

2. In the event where the elicited model θ does not parameterize an exponential family distribution, as is the case when $\mathbf{p}(\mathbf{x}; \theta)$ is a deep artificial neural network (ANN), then we must score a convex surrogate instead. Often times, the most commercially interesting ANNs such as large language models (LLMs) and image generation models do not output a probability at all, instead they output real values conditioned on inputs. LLM for example is of form $\mathbf{p}(\mathbf{x}_t|\mathbf{x}_{t-1}; \theta)$, that is to say the model is parameterized by θ , and receive prompts $\mathbf{x}_{t-1}$ and generate response $\mathbf{x}_t$. Thus the surrogacy must score the likelihood of response given query. The surrogacy may be a logistic regression model that determines the quality of the generated response.

Agent reports generative model θ		
Wager $\mathbf{v}_t^k$	Data transformation	Amm shared state
$\mathbf{v}_1^{\text{alice}}$	$\theta_{\text{alice}}^1 \rightarrow \mathbf{w}_{\text{alice}}^1 \rightarrow \log \mathbf{p}_{\text{alice}}^1$	$\mathbf{w}$ trained on θ

Table D.3: When **Agents** report some generative model θ or raw data $\mathbb{D}$, then **QAudit** samples from the submission and build a classifier $\mathbf{w}$, and scores the quality of this classifier at discriminating "good" vs. "bad" data. The **Amm**'s shared state is the $\mathbf{w}^t$ trained from the latest submitted θ^t .

3. **Agents** can also report raw data. Similar to the previous point, raw data does not have some cost function attached, but it can be used to train a convex surrogate for scoring. A **QAudit** that is eliciting data from agents is in fact running a data market and buying information from users. This is an important aspect of the Metheus economy, and significantly expand the surface area of who can participate in the games. That is most people cannot train machine learning models, but most people do have access to some data.
4. Finally, **Agents** may not report a distribution at all, but rather buy shares of some underlying event. This is the most natural case for **Agents** who perceive the prediction market as a stock market. In this case the **Amm** keeps an underlying state that is the total shares owed to all **Agent**, denoted with $\mathbf{s}$. This market inventory can be turned into a proper distribution $\mathbf{p}$ with the LMSR in example D.1.2.

Agent reports buy vector $\mathbf{b}$		
Wager $\mathbf{v}_t^k$	Data transformation	Amm shared state
$\mathbf{v}_1^{\text{alice}}$	$\mathbf{b}_{\text{alice}}^1 \rightarrow \mathbf{p}_{\text{alice}}^1$	$\mathbf{s} = (\mathbf{s}_1^t, \dots, \mathbf{s}_K^t)$

Table D.4: When Agents report buy vector for shares, the market consensus likelihood is updated by logarithmic market scoring rule.

Finally, we draw a connection between sequential prediction markets and online learning. Since the prediction market is collating predictions from experts sequentially, it is in effect an online learning algorithm leveraging a mixture of experts to arrive at some final prediction [[ACV12], [Rak09]]. Thus the QAudit auditing process is fundamentally oriented around monetizing online learning from mixture of experts, where every Agent is an expert. See [Rak09] and [CL06] for further discussion.

Example D.1.2. (*Logarithmic market scoring rule: LMSR*) QAudit use LMSR as a pricing function [[CA10],[ACV12],[CP07]]. Defined with:

$$\rho_\omega = \text{score}(\omega; \mathbf{s}) = \frac{\exp(\mathbf{s}_\omega/b)}{\sum_{\nu \in \Omega} \exp(\mathbf{s}_\nu/b)}.$$

And observe $0 \leq \rho_\omega \leq 1$. Now let $b = 1$ and $\Omega = \{a, b\}$, and setup an automated market maker (Amm) that issues Arrow-Debreu securities so that the market pays out \$1 if ω occurs. Then initialize the Amm with state:

$$\mathbf{s}^0 = (10, 10).$$

The initial market cost is $\text{cost}(\mathbf{s}^0) = \log(e^{10} + e^{10}) = \4.64 . And its price per option is:

$$\rho_a = \text{score}(a; \mathbf{s}^0) = \frac{e^{10}}{e^{10} + e^{10}} = \$0.50 \quad \rho_b = \text{score}(b; \mathbf{s}^0) = \frac{e^{10}}{e^{10} + e^{10}} = \$0.50.$$

Now suppose Alice places buys 3 shares of a and one share of b , so that is her order is $\mathbf{b}_{\text{alice}} = (3, 1)$. According to the posted price, she is charged:

$$\left\langle \mathbf{b}_{\text{alice}}, \nabla \text{cost}(\mathbf{s}^0) \right\rangle = (3, 1)^T (0.50, 0.50) = \$2.00.$$

Now the market state is:

$$\mathbf{s}^1 = \mathbf{s}^0 + \mathbf{b}_{\text{alice}} = (13, 11),$$

where we use element wise addition. And the prices at $\mathbf{s}^1$ becomes:

$$\rho_a = \text{score}(a; \mathbf{s}^1) = \frac{e^{13}}{e^{13} + e^{11}} = \$0.881 \quad \rho_b = \text{score}(b; \mathbf{s}^1) = \frac{e^{11}}{e^{13} + e^{11}} = \$0.119.$$

And note how purchasing more shares of a increases the price of a and decreases the price of b . Finally let $\omega^* = a$, then the payout outcome vector is $(1, 0)$, and Alice's payout is:

$$\left\langle (1, 0), \mathbf{b}_{\text{alice}} \right\rangle = (1, 0)^T (3, 1) = \$3.00.$$

Her profit is $\$3.00 - \$2.00 = \$1.00$. In the event where the payout vector is $(0, 1)$, her payout is:

$$\left\langle (0, 1), \mathbf{b}_{\text{alice}} \right\rangle = (0, 1)^T (3, 1) = \$1.00.$$

So her profit is $\$1.00 - \$2.00 = -\$1.00$. Next suppose bob posts buy order $\mathbf{b}_{bob} = (-2, 5)$, stating he shorts two shares of a and longs 5 shares of b , then he is charged:

$$(-2, 5)^\top (0.881, 0.119) = -\$1.166,$$

so in this case the market pays Bob for his order. The market state becomes $\mathbf{s}^2 = (11, 16)$ and updates prices to:

$$\mathbf{p}(\mathbf{a}; \mathbf{s}^2) = \frac{e^{11}}{e^{16} + e^{11}} = \$0.007 \quad \mathbf{p}(\mathbf{b}; \mathbf{s}^2) = \frac{e^{16}}{e^{16} + e^{11}} = \$0.993.$$

Finally suppose $\omega^* = a$ so that the payout vector is $(1, 0)$, then Bob's payout is $(1, 0)^\top (-2, 1) = -\$2.00$. Bob's loss on this trade is $\$1.166 - \$2.00 = -\$0.834$, and he has to pay from his balance cover this bet. If $\omega^* = b$, then Bobs' payout is $(0, 1)^\top \cdot (-2, 1) = \1.00 , and he gains $\$2.166$ on this trade. If Bob is the last agent to trade with the protocol, then the Amm has to pay Bob the $\$1.00$, this is Bob's mining reward.

D.2 Exponential Family Sequential Prediction Market

In this case, the distribution cannot be used in downstream applications. Instead the agents share *one* underlying model θ that parameterizes the distribution generating the data. In lieu of issuing Arrow-Debreu options for every event $\omega \in \Omega$, the Amm sells permissions to update the shared model.

Algorithm 15 Exponential Family Sequential Prediction Market

Require: bounty $\mathbf{v}_{pa} \geq 0$, instance of the exponential family with (φ, A)

Require: private dataset $\mathbb{D} = \mathbb{D}_{train} \cup \mathbb{D}_{test} = \{\mathbf{x}_{train,i}\} \cup \{\mathbf{x}_{test,i}\}$

Require: reveals training set $\mathbb{D}_{train} = \{\mathbf{x}_{train,i}\}$

Require: each Agent k has balance a_k so that

$a_k \leftarrow 0$ for $k \in [1 \dots K]$

$\theta_0 \leftarrow \tau \mathbf{1}$

$\Theta_0 \leftarrow [\theta_0]$

$b_{Amm} \leftarrow \mathbf{v}_{pa}$

for $k \in [1 \dots K]$ **do**

 Agent $_k$ query Amm state and training set: $(\theta_{k-1}, \{\mathbf{x}_{train,i}\})$

 Agent $_k$ submit updated model θ_k to market and receive price:

$\rho_k = \mathbf{mc}(\rho(\cdot, \theta_k, \theta_{k-1}; \varphi, A), \{\mathbf{x}_{train,i}\})$

 Agent $_k$ is charged ρ_k

 Amm updates state with:

$\theta_{k-1} \leftarrow \theta_k$

$\Theta_{k-1} \leftarrow \Theta_{k-1} \cup [\theta_k]$

$b_{Amm} \leftarrow b_{Amm} + \rho_k$

end for

Reveals test set $\mathbb{D}_{test}$

for $k \in [1 \dots K]$ **do**

$a_k \leftarrow \sum_{\mathbf{x}_{test,i} \in \mathbb{D}_{test}} \text{score}(\mathbf{x}_{test,i}, \theta_k, \theta_{k-1}; \varphi, A)$

end for

For example the data may be generated by a Bernoulli distribution over $\{0, 1\}$, in which case the market state is the likelihood the proverbial coin will land head; or the data may be generated by a Gaussian distribution, whereby the parameters are sample mean and variance. Following the convention of [AK+14], we call this type of market the *exponential family prediction market*. The exponential family is of form:

$$\mathbf{p}(\mathbf{x}; \theta) = \exp(\langle \varphi(\mathbf{x}), \theta \rangle - A(\theta)) \quad \text{where } A(\theta) = \log \int_{\Omega^n} \exp(\langle \varphi(\mathbf{x}), \theta \rangle) \nu d(\mathbf{x}).$$

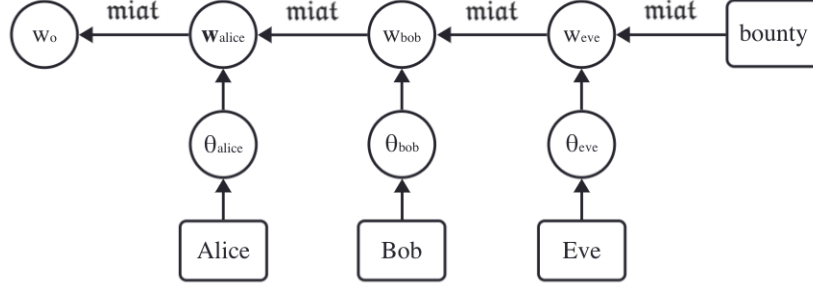


Figure D.1: In the exponential family sequential prediction market, the protocol keeps a common state $\mathbf{w}$, which parametrizes a discriminative model $\mathbf{p}(y|\mathbf{x}; \mathbf{w})$. Agents such as Alice approach the market and submit their generative model θ_{alice} . The protocol then trains a discriminative model $\mathbf{w}_{\text{alice}}$ from this θ_{alice} , and updates the protocol state $\mathbf{w}$.

Where φ is the sufficient statistic associated with data sampled from distribution over Ω^n , $\mathbf{A}$ is the log partition function, and θ is the *canonical parameter* of the distribution. The incentives in this market are of flavor:

$$\rho(\mathbf{x}, \theta_k, \theta_{k-1}; \varphi, A) = \mathbb{E}_{\mathbf{p}(\cdot; \theta_{k-1})} [\text{score}(\mathbf{x}, \theta_k, \theta_{k-1}; \varphi, A)] = -D(\theta_{k-1} || \theta_k) \quad (\text{D.4})$$

$$\text{score}(\mathbf{x}, \theta_k, \theta_{k-1}; \varphi, A) = \varphi(\mathbf{x})^\top (\theta_k - \theta_{k-1}) + A(\theta_{k-1}) - A(\theta_k) \quad (\text{D.5})$$

Where $\mathbf{x}$ is the data point sampled according to distribution $\mathbf{p}$. The vector θ_k is the k th submission of model parameters. $D(\cdot || \cdot)$ is the Bregman divergence between two distributions parameterized by θ_k and θ_{k-1} , and A is the log partition function. Note these are class level definitions using canonical parameterization of the exponential family. It is also possible to rewrite the price and rewards in the mean forms, though we omit them until needed.

The expression $\mathbf{mc}(\text{score}(\cdot), \{\mathbf{x}_{\text{train},i}\})$ refers to some sampling function using examples $\mathbf{x}_{\text{train},i}$ to estimate the KL divergence (read: *score*). If the KL divergence has closed form expression where mean parameters are sufficient, then $\mathbf{mc}$ is the identity function:

$$\mathbf{mc}(\text{score}(\cdot), \{\mathbf{x}_{\text{train},i}\}) = \text{score}(\cdot).$$

The Exponential Family Market Maker: Part I

A basic understanding of exponential family distribution is needed to understand this market. The best source is [WJ08], there is brief introduction in Appendix F that situates the topic within prediction markets. In particular, the pricing mechanism in a sequential prediction market involves comparing the quality of the current prediction against the previous one, in the context of the final outcome at market closing. Thus the motivating question here is how to price the difference in quality between two models of the exponential family, and how various representations of exponential family can be used in said pricing. This section derives the ρ and *score* function from LMSR. But instead of using the prior formulation where the entire distribution $\mathbf{p}$ is reported, [AK+14] showed it is possible to arrive at a proper scoring rule, with an incentive compatible measure of agent reports - without recording the full distribution as market state.

Recall the log scoring rule parameterized by the full distribution is $\text{score}(\mathbf{p}, \omega) = \log(\mathbf{p}_\omega)$. Now instead of eliciting the full distribution $\mathbf{p}$, only the canonical parameter θ is reported. Then the scoring rule can be rewritten as:

$$\text{score}(\theta, \mathbf{x}) = \log(\mathbf{p}(\mathbf{x}; \theta)),$$

for some realization of the vector $\mathbf{x} \in \Omega^n$, drawn according to $\mathbf{p} \in \mathfrak{P}$ with parameter $\boldsymbol{\theta}$. This form is only usable if the canonical representation satisfies the same property as eliciting the full distribution. In particular, the canonical form must satisfy the incentive capability condition: given the agent's truthful $\boldsymbol{\theta}_1$, and some $\boldsymbol{\theta}_2 \neq \boldsymbol{\theta}_1$, with $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2 \in \Theta$, score is incentive compatible if:

$$\mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta}_1)}[\log(\mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_1))] - \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta}_1)}[\log(\mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_2))] \geq 0. \quad (\text{D.6})$$

It turns out, this is true for LMSR only if $\mathbf{p}$ is in the exponential family.

Theorem D.2.1. *Take the log scoring rule $\text{score}(\boldsymbol{\theta}, \mathbf{x}) = \log(\mathbf{p}(\mathbf{x}; \boldsymbol{\theta}))$ defined on densities parameterized by $\boldsymbol{\theta} \in \Theta$, it is proper if the densities are in the exponential family.*

Proof. Going in the forward direction, assume $\mathbf{p}$ is in the exponential family, then $\log(\mathbf{p}(\mathbf{x}; \boldsymbol{\theta})) = \boldsymbol{\varphi}(\mathbf{x})^\top \boldsymbol{\theta} - \mathbf{A}(\boldsymbol{\theta})$. And we show the expected value of reporting the truth is no lesser than the expected value of reporting a lie. Let $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2 \in \Theta$ be the agent's true belief and lie respectively, then:

$$\begin{aligned} & \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta}_1)}[\log(\mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_1))] - \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta}_1)}[\log(\mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_2))] \\ &= \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta}_1)} \left[\boldsymbol{\varphi}(\mathbf{x})^\top \boldsymbol{\theta}_1 - \mathbf{A}(\boldsymbol{\theta}_1) - (\boldsymbol{\varphi}(\mathbf{x})^\top \boldsymbol{\theta}_2 - \mathbf{A}(\boldsymbol{\theta}_2)) \right] \\ &= \int_{\Omega^n} \mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_1) \left(\mathbf{A}(\boldsymbol{\theta}_2) - \mathbf{A}(\boldsymbol{\theta}_1) + (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^\top \boldsymbol{\varphi}(\mathbf{x}) \right) \nu(d\mathbf{x}) \\ &= \mathbf{A}(\boldsymbol{\theta}_2) - \mathbf{A}(\boldsymbol{\theta}_1) + (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^\top \int_{\Omega^n} \mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_1) \boldsymbol{\varphi}(\mathbf{x}) \nu(d\mathbf{x}) \\ &= \mathbf{A}(\boldsymbol{\theta}_2) - \mathbf{A}(\boldsymbol{\theta}_1) - \nabla \mathbf{A}(\boldsymbol{\theta}_1)^\top (\boldsymbol{\theta}_2 - \boldsymbol{\theta}_1). \end{aligned} \quad (\text{D.7})$$

Where going into the last line we use the moment matching condition to set:

$$\mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta}_1)}[\boldsymbol{\varphi}(\mathbf{x})] = \int_{\Omega^n} \mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_1) \boldsymbol{\varphi}(\mathbf{x}) \nu(d\mathbf{x}) = \boldsymbol{\mu}_1 = \nabla \mathbf{A}(\boldsymbol{\theta}_1).$$

Now note because $\mathbf{p}$ is in the exponential family with convex $\mathbf{A}$, the value of eqn (D.7) is ≥ 0 by definition, thus proving incentive compatibility condition. Furthermore, the gap between telling the truth and telling a lie is the Bregman divergence between the two $\boldsymbol{\theta}$'s on $\mathbf{A}(\boldsymbol{\theta})$. Now going in the opposite direction, assume the scoring rule is proper, then it must be convex. That is to say for some strictly convex function G , Bregman divergence must hold:

$$\text{score}(\mathbf{p}, \omega) = G(\mathbf{p}) - \nabla G(\mathbf{p})^\top (\mathbf{p} - f(\omega)).$$

Setting $\mathbf{p} = \boldsymbol{\mu}$, $\omega = \mathbf{x}$, $G = \mathbf{A}^*$, and $f = \boldsymbol{\varphi}$, we use the dual form of the scoring rule $\log(\mathbf{p}(\mathbf{x}, \boldsymbol{\mu}))$:

$$\begin{aligned} \log(\mathbf{p}(\mathbf{x}, \boldsymbol{\mu})) &= \mathbf{A}^*(\boldsymbol{\mu}) - \nabla \mathbf{A}^*(\boldsymbol{\mu})^\top (\boldsymbol{\mu} - \boldsymbol{\varphi}(\mathbf{x})) \\ &= \boldsymbol{\mu}^\top \boldsymbol{\theta}_\mu - \mathbf{A}(\boldsymbol{\theta}_\mu) - \boldsymbol{\theta}_\mu^\top (\boldsymbol{\mu} - \boldsymbol{\varphi}(\mathbf{x})) \\ &= \boldsymbol{\varphi}(\mathbf{x})^\top \boldsymbol{\theta}_\mu - \mathbf{A}(\boldsymbol{\theta}_\mu). \end{aligned}$$

Where going into the second line, we use the conjugate definition of $\mathbf{A}^*(\boldsymbol{\mu})$, and the fact that $\nabla \mathbf{A}^*(\boldsymbol{\mu}) = \boldsymbol{\theta}_\mu$ by condition (3) of the properties of $\mathbf{A}^*$ (see Appendix F [AK+14].) ■

Informally, there is a correspondence between incentive compatibility of LMSR, and maximum likelihood estimation in models based on exponential family distributions:

1. every proper scoring rule can be described with an equivalent convex function;
2. the exponential family's log partition function $\mathbf{A}(\boldsymbol{\theta})$ is convex;

3. the gradient of the log partition function is the distribution's expected value;
4. during model training, maximizing the likelihood of data under model θ is equivalent to minimizing the KL-divergence of the true θ^* and model θ on the $A(\theta)$ surface;
5. in LMSR, the agent maximizes his expected payoff by minimizing the KL divergence between his reported value and private value.

Now we can derive the reward score and price ρ functions for each $\mathbf{x}$:

$$\begin{aligned} \text{score}(\mathbf{x}, \theta_k, \theta_{k-1}; \varphi, A) &= \log(p(\mathbf{x}; \theta_k)) - \log(p(\mathbf{x}; \theta_{k-1})) \\ &= \left(\varphi(\mathbf{x})^\top \theta_k - A(\theta_k) \right) - \left(\varphi(\mathbf{x})^\top \theta_{k-1} - A(\theta_{k-1}) \right) \\ &= \varphi(\mathbf{x})^\top (\theta_k - \theta_{k-1}) + A(\theta_{k-1}) - A(\theta_k). \end{aligned}$$

Similarly, ρ is the expected value of the reward under the market's current $\mathbf{p}$:

$$\begin{aligned} p(\mathbf{x}, \theta_k, \theta_{k-1}; \varphi, A) &= \mathbf{E}_{p(\cdot; \theta_{k-1})} \left[\log(p(\mathbf{x}; \theta_k)) - \log(p(\mathbf{x}; \theta_{k-1})) \right] \\ &= \int_{\Omega^n} p(\mathbf{x}; \theta_{k-1}) \left(\varphi(\mathbf{x})^\top (\theta_k - \theta_{k-1}) + A(\theta_{k-1}) - A(\theta_k) \right) \nu(d\mathbf{x}) \\ &= A(\theta_{k-1}) - A(\theta_k) + (\theta_k - \theta_{k-1})^\top \int_{\Omega^n} p(\mathbf{x}; \theta_{k-1}) \varphi(\mathbf{x}) \nu(d\mathbf{x}) \\ &= - \left(A(\theta_k) - A(\theta_{k-1}) - \nabla A(\theta_{k-1})^\top (\theta_k - \theta_{k-1}) \right) \\ &= -D(\theta_{k-1} || \theta_k). \end{aligned}$$

Where going into the second to last line, we use the moment matching condition of exponential family to set

$$\int_{\Omega^n} p(\mathbf{x}; \theta_{k-1}) \varphi(\mathbf{x}) \nu(d\mathbf{x}) = \mathbf{E}_{p(\cdot; \theta_{k-1})} [\varphi(\mathbf{x})] = \nabla A(\theta_{k-1}).$$

The function ρ is the Bregman divergence between two distribution parameterized by θ (see figure ??). This makes intuitive sense, Agent_k is paying to update the model θ , and similar to previous protocols, the more he changes the distribution the more he pays. Given the score function, the market maker's payout under Protocol 16 is:

$$\begin{aligned} &\sum_{k=1}^K \varphi(\mathbf{x})^\top (\theta_k - \theta_{k-1}) + A(\theta_{k-1}) - A(\theta_k) \\ &= \left(\varphi(\mathbf{x})^\top (\theta_1 - \theta_0) + A(\theta_0) - A(\theta_1) \right) + \left(\varphi(\mathbf{x})^\top (\theta_2 - \theta_1) + A(\theta_1) - A(\theta_2) \right) + \dots \\ &= \varphi(\mathbf{x})^\top (\theta_K - \theta_0) + A(\theta_0) - A(\theta_K). \end{aligned} \tag{D.8}$$

That is to say the Amm's loss depend only on the quality of the last model vs the initial model parameter θ_0 . Thus the parimutuel nature of LMSR is preserved. Note eqn D.8 is in fact the KL divergence between θ_k and θ_0 : $D(p(\mathbf{x}; \theta_0) || p(\mathbf{x}; \theta_K))$. The market maker's worst case loss is then:

$$\sup_{\mathbf{x} \in \Omega^n} \sup_{\theta_K \in \Theta} \left(\varphi(\mathbf{x})^\top (\theta_K - \theta_0) + A(\theta_0) - A(\theta_K) \right). \tag{D.9}$$

Where $\mathbf{x}$ refers to the worst draw of test sample the Amm could encounter, and $\theta_K \in \Theta$ refers to the best performing model parameter from $\Theta = \{\theta \in \mathbb{R}^d : A(\theta) < \infty\}$. In

production, this would be the maximum mining reward. Note we can decrease this worst case loss by initializing some θ_0 that is already pre-trained. In other words, instances of Protocol 16 launched to improve pre-trained models have lower mining rewards.

Now observe the pricing function poses an odd quandary for the agent: he can minimize the price ρ for an update by minimizing the divergence between his reported θ_k , and the market θ_{k-1} . Therefore even if he believes the market state is wrong, he is incentivized to *not* report his true θ should he deem the write-privilege too expensive. However, reporting a conservative (read: *less risky*) θ may also lead to lower profit. Specifically, the agent's profit for some test sample $\mathbf{x}_{test}$ is:

$$\begin{aligned} & \underbrace{\left(\varphi(\mathbf{x}_{test})^\top (\theta_k - \theta_{k-1}) + \mathbf{A}(\theta_{k-1}) - \mathbf{A}(\theta_k) \right)}_{\text{payout}} - \underbrace{\left(\mathbf{A}(\theta_k) - \mathbf{A}(\theta_{k-1}) - \nabla \mathbf{A}(\theta_{k-1})^\top (\theta_k - \theta_{k-1}) \right)}_{\text{price}} \\ &= \left(\varphi(\mathbf{x}_{test}) + \nabla \mathbf{A}(\theta_{k-1}) \right)^\top (\theta_k - \theta_{k-1}) + \left(2\mathbf{A}(\theta_{k-1}) - 2\mathbf{A}(\theta_k) \right) \end{aligned}$$

And we see that reporting a $\theta_k \sim \theta_{k-1}$ also degrades his reward, so there is a trade off here that can be expressed. Assume the agent has some exponential utility function U :

$$U_a(w) = -\frac{1}{a} e^{-aw}.$$

Where w is the agent's wealth, and a is the agent's risk aversion parameters. Observe as a increases the agent's risk aversion increases. And when $a \rightarrow 0$, the agent approaches risk neutrality. [AK+14] showed that given a LMSR market with market state θ_{k-1} , an agent with private belief θ_k^* and risk aversion a_k has optimal state update:

$$\theta_k = \frac{1}{1 + a_k} \theta_k^* + \frac{a_k}{1 + a_k} \theta_{k-1}. \quad (\text{D.10})$$

In particular, note that a risk neutral agent with $a_k = 0$ reports the truth: $\theta_k = \theta_k^*$. Where as a risk averse agent with large a_k will only incrementally update the model state so that $\theta_k \sim \theta_{k-1}$. Now suppose each agent k has risk aversion parameter a_k , private truth θ_k^* , and updates the model by $\tilde{\delta}_k$, then after K updates the final market state will be:

$$\theta_K = \theta_0 + \sum_{k=1}^K \tilde{\delta}_k = \frac{\theta_0 + \sum_{k=1}^K \frac{\theta_k^*}{a_k}}{1 + \sum_{k=1}^K \frac{1}{a_k}}.$$

So that the market equilibrium state is a convex combination of initial market state, and all the agent's beliefs weighed by their risk aversion [AK+14].

Exponential Family Market Maker: Part II

The previous section characterized log scoring rule using the exponential family, and showed that the LMSR properties are preserved. The canonical form reveals clear expression of incentive compatibility, but the mean parameterization is more natural in computational settings. This section explores LMSR's dual cost function representation. Specifically, the relationship between the cost function cost for generalized LMSR, and the log partition function $\mathbf{A}$ in the exponential family. Recall in the discrete case the cost function analogue to the log scoring rule is $\text{cost}(\mathbf{s}) = b \log \sum_{\omega \in \Omega} e^{s_\omega/b}$. The continuous version of the log market scoring rule cost is:

$$\mathbf{A}(\theta) = \log \int_{\Omega^n} \exp(\langle \mathbf{x}, \theta \rangle) \nu(d\mathbf{x}).$$

This is in fact the log partition function. Now recall the probability or price quote in cost function based markets is $\nabla \text{cost}(\theta) = \nabla \mathbf{A}(\theta)$. And by the moment matching conditions

of exponential family, the Jacobian of $\mathbf{A}$ (see F.1 and F.3 is the distribution's mean, vis $\nabla \mathbf{A}(\boldsymbol{\theta}) = \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta})}[\boldsymbol{\varphi}(\mathbf{x})] = \boldsymbol{\mu}$.

Hence there is a correspondence between eliciting the mean of the distribution and pricing the agent's contribution. However numerically speaking it is unclear how to use the elicited $\boldsymbol{\mu}$ in practice. In the case of Bernoulli variable on $\{0, 1\}$, the mean is some value in $[0, 1]$, which is easily interpreted as both likelihood and price. However in multivariate Gaussian the reported mean is $\Lambda = (\boldsymbol{\mu}, \boldsymbol{\sigma})$, which cannot be used as the price. Instead, Protocol 16 use the reported means to build the model $\mathbf{p}(\mathbf{x}; \boldsymbol{\mu})$, and then computes the likelihood of data under the model $\mathbf{p}$. This dual market with state $\boldsymbol{\mu}$ has incentives:

$$\begin{aligned} \rho(\boldsymbol{\mu}_k, \boldsymbol{\mu}_{k-1}; \mathbf{A}^*) &= -\left(\mathbf{A}^*(\boldsymbol{\mu}_k) - \mathbf{A}^*(\boldsymbol{\mu}_{k-1}) - \nabla \mathbf{A}^*(\boldsymbol{\mu}_{k-1})^\top (\boldsymbol{\mu}_k - \boldsymbol{\mu}_{k-1})\right) \\ &= -D(\boldsymbol{\mu}_{k-1} \parallel \boldsymbol{\mu}_k) \\ &= -\int_{\Omega^n} \mathbf{p}(\mathbf{x}; \boldsymbol{\mu}_{k-1}) \log \frac{\mathbf{p}(\mathbf{x}; \boldsymbol{\mu}_{k-1})}{\mathbf{p}(\mathbf{x}; \boldsymbol{\mu}_k)} \nu(d\mathbf{x}) \end{aligned} \quad (\text{D.11})$$

$$\begin{aligned} \text{score}(\mathbf{x}, \boldsymbol{\mu}_k, \boldsymbol{\mu}_{k-1}) &= \log(\mathbf{p}(\mathbf{x}; \boldsymbol{\mu}_k)) - \log(\mathbf{p}(\mathbf{x}; \boldsymbol{\mu}_{k-1})) \\ &= \log \frac{\mathbf{p}(\mathbf{x}; \boldsymbol{\mu}_k)}{\mathbf{p}(\mathbf{x}; \boldsymbol{\mu}_{k-1})}. \end{aligned} \quad (\text{D.12})$$

Note the input to the `score` function is run on just one example of $\mathbf{x}$. Where as in familiar machine learning setting, one normally receive a test set of form $\mathbb{D}_{\text{test}} = \{.. \mathbf{x}_{test,i} ..\}$ with many examples. The Amm can simply run `score` for every $\mathbf{x}_{test,i}$ in the test set, and sum the payouts/losses with: $\sum_s \text{score}(\mathbf{x}_{test,i}, \boldsymbol{\mu}_k, \boldsymbol{\mu}_{k-1})$ for every agent k . However this undermines the the max loss guarantee of expression (D.9) whereby we assume only one final event is revealed and scored. This contradiction can be resolved by either scaling the price by a factor of up to $|\mathbb{D}_{\text{test}}|$, or scale the final reward down by the same factor. The same measure can extend to models in production, where the model developer and data providers used to train the model are paid on a per-correct inference regime. What follows are a few examples where the distribution and the KL divergence can be written in closed form, so the sampling function `mc` in Protocol 16 is just the identity function that outputs the `score` function.

Example D.2.2 (Bernoulli bit). There is some Bernoulli variable on $\{1, 0\}$ with sufficient statistic $\varphi(x) = x$. The Amm elicits the likelihood of 1's. It initializes the market with $\mu_0 = 0.5$. The Amm sources data set and reveals training set. `alice` approaches the market, counts the training bits, and reports $\hat{\mu}_{\text{alice}} = 0.6$. She is charged:

$$\rho(\mu_{\text{alice}}, \mu_0) = 0.5 \cdot \ln \frac{0.5}{0.6} + 0.5 \cdot \ln \frac{0.5}{0.4} = \$0.0204.$$

The market ends and reveals test set $\{x_{test,i}\} = \{0, 0, 1, 1, 1\}$. `alice` is paid for every bit she guesses correctly and penalized for the wrong bits:

$$\ln \left(\frac{0.4^2 \cdot 0.6^3}{0.5^5} \right) = 2 \cdot \ln \frac{0.4}{0.5} + 3 \cdot \ln \frac{0.6}{0.5} = \$0.10068.$$

We can scale up the each agent's ρ up to a factor of 5 to recover the Amm max loss guarantee.

The next two examples demonstrates two concepts:

1. how to write reward and price functions for conditional distributions;
2. sampling the data set to estimate KL divergence;

Example D.2.3. (*Logistic Regression*) In this case we are working with the conditional distribution. The data set is of form $\{.., (\mathbf{x}_i, y_i), ..\}$, where $\mathbf{x} \in \Omega^n$, and $y_i \in \{1, 0\}$. The goal is to predict $\mathbf{p}(y|\mathbf{x}; \mathbf{w})$. Here we have some model $z = \mathbf{w}^\top \mathbf{x} + b$, where $\mathbf{w}$ is the weights vector,

and b is the bias term. We can lift both $\mathbf{w}$ and $\mathbf{x}$ into affine space and write $z = (\mathbf{w} \ b)^\top (\mathbf{x} \ 1)$ for cleaner notation, from now on it is understood that $\mathbf{w} = (\mathbf{w} \ b)$ and $\mathbf{x} = (\mathbf{x} \ 1)$. Next z is turned into a probability with $\sigma(z) = \frac{1}{1 + \exp(-z)}$, so that the Bernoulli distribution of y conditioned on $\mathbf{x}$ is:

$$p(y = 1 | \mathbf{x}; \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}} \quad p(y = 0 | \mathbf{x}; \mathbf{w}) = \sigma(1 - \mathbf{w}^\top \mathbf{x}) = \frac{e^{-\mathbf{w}^\top \mathbf{x}}}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}.$$

Given some model parameterized by $\mathbf{w}$, the likelihood of some sample $(\mathbf{x}_i, y_i)$ is:

$$\log(p(y_i | \mathbf{x}_i; \mathbf{w})) = y_i \log(\sigma(\mathbf{w}^\top \mathbf{x}_i)) + (1 - y_i) \log(\sigma(1 - \mathbf{w}^\top \mathbf{x}_i)). \quad (\text{D.13})$$

And note that when $y_i = 0$, the expression above reduces to $(1 - y_i) \log(\sigma(1 - \mathbf{w}^\top \mathbf{x}_i))$. When $y_i = 1$, expression D.13 becomes $\log(\sigma(\mathbf{w}^\top \mathbf{x}_i))$. The training objective given data set $\mathbb{D}_{\text{train}} = \{.., (\mathbf{x}_i, y_i), ..\}$ is:

$$\begin{aligned} \arg \min_{\mathbf{w} \in \mathcal{M}} \text{loss}(\mathbf{w}, \mathbf{w}_0) \quad \text{where} \\ \text{loss}(\mathbf{w}, \mathbf{w}_0) = \sum_{(\mathbf{x}_i, y_i) \in \mathbb{D}_{\text{train}}} -y_i \log(\sigma(\mathbf{w}^\top \mathbf{x}_i)) - (1 - y_i) \log(\sigma(1 - \mathbf{w}^\top \mathbf{x}_i)). \end{aligned}$$

It can be shown that this expression is convex in $\mathbf{w}^\top \mathbf{x}$ [WJ08]. Now given some data point $(\mathbf{x}_i, y_i)$, the reward of updating the model from $\mathbf{v}$ to $\mathbf{w}$ is:

$$\begin{aligned} \text{score}((\mathbf{x}_i, y_i), \mathbf{w}, \mathbf{v}) &= \log(p(y_i | \mathbf{x}_i; \mathbf{w})) - \log(p(y_i | \mathbf{x}_i; \mathbf{v})) \\ &= y_i \cdot \log(\sigma(\mathbf{w}^\top \mathbf{x}_i)) + (1 - y_i) \cdot \log(\sigma(1 - \mathbf{w}^\top \mathbf{x}_i)) \\ &\quad - y_i \cdot \log(\sigma(\mathbf{v}^\top \mathbf{x}_i)) - (1 - y_i) \cdot \log(\sigma(1 - \mathbf{v}^\top \mathbf{x}_i)). \end{aligned} \quad (\text{D.14})$$

Alternatively we can write eqn (D.14) as:

$$\text{score}((\mathbf{x}_i, y_i), \mathbf{w}, \mathbf{v}) = \begin{cases} \log(\sigma(1 - \mathbf{w}^\top \mathbf{x}_i)) - \log(\sigma(1 - \mathbf{v}^\top \mathbf{x}_i)) & (y_i = 0) \\ \log(\sigma(\mathbf{w}^\top \mathbf{x}_i)) - \log(\sigma(\mathbf{v}^\top \mathbf{x}_i)) & (y_i = 1) \end{cases}$$

It can be shown that when $\mathbf{w}$ classifies $\mathbf{x}_i$ more accurately than $\mathbf{v}$, then expression (D.14) is > 0 , and it is < 0 otherwise. Notice the Agents are only graded on the part of the log-likelihood that estimates the probability of the given outcome y_i .

Now the market initializes $\mathbf{v} = \mathbf{0}$, so that $p(y = 1 | \mathbf{x}; \mathbf{v}) = \frac{1}{2}$. Then it publishes the training set $\mathbb{D}_{\text{train}} = \{..(\mathbf{x}_s, y_i)...\}$. Alice queries the training data, and submit $\mathbf{w}$. Her price evaluated at any given $(\mathbf{x}_i, y_i)$ in the training data is:

$$\begin{aligned} -\rho((\mathbf{x}_i, y_i), \mathbf{w}, \mathbf{v}) &= D(p(y_i | \mathbf{x}_i; \mathbf{v}) || p(y_i | \mathbf{x}_i; \mathbf{w})) \\ &= \mathbf{1}_{y_i, 1} \cdot \frac{1}{1 + \exp(-\mathbf{v}^\top \mathbf{x}_i)} \cdot \ln \left(\frac{1 + \exp(-\mathbf{w}^\top \mathbf{x}_i)}{1 + \exp(-\mathbf{v}^\top \mathbf{x}_i)} \right) \\ &\quad + \mathbf{1}_{y_i, 0} \cdot \frac{\exp(-\mathbf{v}^\top \mathbf{x}_i)}{1 + \exp(-\mathbf{v}^\top \mathbf{x}_i)} \cdot \ln \left(\frac{1 + \exp(-\mathbf{v}^\top \mathbf{x}_i)}{\exp(-\mathbf{v}^\top \mathbf{x}_i)} \cdot \frac{\exp(-\mathbf{w}^\top \mathbf{x}_i)}{1 + \exp(-\mathbf{w}^\top \mathbf{x}_i)} \right). \end{aligned}$$

Where $\mathbf{1}_{y_i, 1} = 1$ if $y_i = 1$, and $\mathbf{1}_{y_i, 1} = 0$ otherwise. The price for the update is then computed over subset of the training set, named $\mathbb{D}_{\text{train}}$:

$$\mathbf{mc}(\rho(\cdot, \mathbf{w}, \mathbf{v}), \mathbb{D}_{\text{train}}) = \frac{1}{|\mathbb{D}_{\text{train}}|} \sum_{(\mathbf{x}_i, y_i) \in \mathbb{D}_{\text{train}}} D(p(y_i | \mathbf{x}_i; \mathbf{v}) || p(y_i | \mathbf{x}_i; \mathbf{w})).$$

Here $\mathbf{mc}$ uses the average KL divergence over all point-wise estimates on the training data. This average is the unbiased estimate of the true KL divergence. This expression is what Alice will pay to update the model parameter from $\mathbf{v}$ to $\mathbf{w}$.

APPENDIX E

Machine Learning

E.1 Introduction

This chapter offers a brief introduction to the field of machine learning with a *narrow* pedagogical approach. In this view, the workhorse of machine learning is kernel regression. Here the data points are projected onto some "suitable space", and a surface is drawn to fit the points via function search. The next few sections make this intuition precise by:

1. expressing the properties of this Hilbert space $\mathcal{H}_\kappa$;
2. explore elementary regression techniques in $\mathcal{H}_\kappa$;
3. build towards feed-forward deep neural networks in context of this view.

This discourse is suitable for understanding QAudit, which is concerned with how to measure different models in a principled manner. Thus, many subjects are completely absent in this chapter. They include:

1. a basic survey into what machine learning is: how to train models, cross validation, and bias-variance trade-offs, etc.
2. A tour through the different machine learning "settings", i.e. generative vs discriminative models, reinforcement learning, transfer learning, and classical methods.
3. Any discussion of optimization techniques as seen in [BV04] and [GB16].
4. Practical concerns such as how to prepare data, train LLMs, or prompt them.

Also missing is a basic introduction into probability, although we do provide a primer on the exponential family distribution in Appendix F. All in all, this chapter is suitable for readers who already understand machine learning, and wish to judge whether **Metheus** takes the long view on the subject matter.

E.2 Hilbert Space

Definition

The nature Hilbert space $\mathcal{H}$ is defined by constraining the behavior of its *norm* and *distance metric*. Roughly speaking, distance in any well-formed $\mathcal{H}$ must correspond to intuitive sense of how the real 3D space we live in behaves. In particular, this intuition will translate to invariants around triangles and parallelograms.

Definition E.2.1. (Hilbert Space $\mathcal{H}$) Let $\mathbb{V}$ be a vector space. A norm is a mapping $\|\cdot\| : \mathbb{V} \rightarrow [0, \infty)$ that satisfies:

$$\begin{aligned} \|\mathbf{x}\| &\geq 0 \text{ for every } \mathbf{x} \in \mathbb{V} && \text{(nonnegative)} \\ \|\mathbf{x}\| &= 0 \text{ iff } \mathbf{x} = 0 && \text{(strictly positive)} \\ \|\mathbf{x} + \mathbf{y}\| &\leq \|\mathbf{x}\| + \|\mathbf{y}\| && \text{(triangle inequality)} \\ \|a\mathbf{x}\| &= a\|\mathbf{x}\| \text{ for every } a \in \mathbb{R} && \text{(homogeneous).} \end{aligned}$$

In Hilbert spaces, the norm is expressed as the *inner product*, that is a mapping $\langle \cdot, \cdot \rangle : \mathbb{V} \times \mathbb{V} \rightarrow \mathbb{R}$ which satisfies for every $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{V}$, and $a, b \in \mathbb{R}$:

$$\begin{aligned} \langle \mathbf{x}, \mathbf{x} \rangle &\geq 0 && \text{(nonnegative)} \\ \langle \mathbf{x}, \mathbf{x} \rangle &= 0 \text{ iff } \mathbf{x} = 0 && \text{(strictly positive)} \\ \langle \mathbf{x}, a\mathbf{y} + b\mathbf{z} \rangle &= a\langle \mathbf{x}, \mathbf{y} \rangle + b\langle \mathbf{x}, \mathbf{z} \rangle && \text{(linear in the 2nd argument)} \\ \langle \mathbf{x}, \mathbf{y} \rangle &= \langle \mathbf{y}, \mathbf{x} \rangle && \text{(Hermitian symmetric).} \end{aligned}$$

Vectors $\mathbf{x}$ and $\mathbf{y}$ are *orthogonal* if $\langle \mathbf{x}, \mathbf{y} \rangle = 0$. Now define:

$$\|\mathbf{x}\|_2 = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} \quad \text{(norm)} \quad (\text{E.1})$$

$$\|\mathbf{x}_1 - \mathbf{x}_2\|_2 = \sqrt{\langle \mathbf{x}_1 - \mathbf{x}_2, \mathbf{x}_1 - \mathbf{x}_2 \rangle} \quad \text{(distance).} \quad (\text{E.2})$$

And observe how the definition uses the L^2 norm, which resembles the Pythagorean's theorem. Finally, every Hilbert space must be complete. A space is complete if every *Cauchy sequence* converges to a limit, a sequence $\mathbf{x}_1, \mathbf{x}_2, \dots$ is Cauchy if $\|\mathbf{x}_m - \mathbf{x}_n\|_2 \rightarrow 0$ as $m, n \rightarrow \infty$. A complete normed space is known as a *Banach space*¹. A *Hilbert Space $\mathcal{H}$* then is a complete normed space where the norm is the L^2 inner product defined in eqn E.1 [[Bea73], [Axl20]].

The Reproducing-Kernel Hilbert Space (RKHS)

A RKHS is a class of smooth functions defined by the *Mercer Kernel* κ . Once a RKHS is defined, a learner can conduct "function search" in this space according to some cost criteria. For example, a reproducing kernel space could be defined by some kernel κ with a data set, and a learner would search this space for some linear function f that maximally separates the points. Mercer kernels are used when a learner wish to classify the data using a linear function, but the data is not separable by it (see fig (E.1)). One solution is to "lift" the data to a higher dimension using a kernel κ , and search for a linear separator in the new space (so called "feature space") induced by κ (see fig (E.2)).

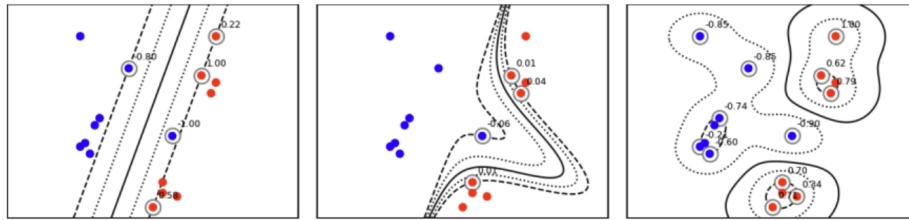


Figure E.1: Left: original data space where a linear classifier cannot shatter the data set, as evidence by the points circled (best seen in color). Center: a polynomial kernel that describes a quadratic separator, which is linear in feature space, but is nonlinear when projected back into data space. Right: radial basis function (RBF) kernel that is even more expressive: the classification boundaries become discrete islands when projected into data space [Van23].

¹Note all Hilbert spaces are Banach spaces, but the converse is not true.

Remark E.2.2. Linear solutions are "nice" because they are less expensive to train, and have convex loss functions, which ensures the solution they find is optimal. Another avenue is to use nonlinear expressions to separate the data points, as is the case with artificial neural networks do (ANN). ANNs and kernel-based methods are different ways of addressing the same problem: classifying points that cannot be shattered by linear separators. However where as the kernels κ have to be hand designed, ANN learn the feature space directly from data. Every mapping is still some polynomial, however the learned polynomial are so non-intuitive no person could have derived it from either principle or experience. Nonetheless, understanding Kernels inform the understanding of ANNs.

Definition E.2.3. (Mercer Kernel κ). A Mercer kernel is a continuous function κ that is *symmetric* and *positive semidefinite*, that is for every $f \in \mathcal{H}$, and $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{X}$:

$$\begin{aligned} \kappa : \mathbb{X} \times \mathbb{X} &\rightarrow \mathbb{R} \\ \kappa(\mathbf{x}_1, \mathbf{x}_2) &= \kappa(\mathbf{x}_2, \mathbf{x}_1) && \text{(symmetric)} \\ \int_{\mathbb{X}} \int_{\mathbb{X}} \kappa(\mathbf{x}_1, \mathbf{x}_2) f(\mathbf{x}_1) f(\mathbf{x}_2) d\mathbf{x}_1 d\mathbf{x}_2 &\geq 0 && \text{(positive semidefinite).} \end{aligned}$$

These two conditions ensure κ is a proper distance, or "similarity" relation. An example of κ is the Gaussian radial basis function (RBF) kernel:

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\frac{\|\mathbf{x}_1 - \mathbf{x}_2\|_2^2}{2\sigma^2}\right) \quad \text{(RBF kernel).} \quad (\text{E.3})$$

If we partially apply κ so that one of the argument is fixed, then this partially applied function is the **feature map**:²

$$\begin{aligned} \phi_{\mathbf{x}_1} : \mathbb{X} &\rightarrow \mathbb{R} \\ \phi_{\mathbf{x}_1} &:= \kappa(\mathbf{x}_1, \cdot). \end{aligned}$$

And notice the feature map is a vector, and the notation $\phi_{\mathbf{x}}$ evokes the basis of eqn (??). The set of feature vectors will span all possible functions in the Kernel space. For example in the RBF kernel, if we let $\mathbf{x}_1 = 3$, then we have: $\phi_3(\mathbf{x}_2) = \exp\left(-\frac{\|3 - \mathbf{x}_2\|_2^2}{2\sigma^2}\right)$. The inner product of two partially applied kernels is defined with:

$$\langle \phi_{\mathbf{x}_1}, \phi_{\mathbf{x}_2} \rangle = \underbrace{\langle \kappa(\mathbf{x}_1, \cdot), \kappa(\mathbf{x}_2, \cdot) \rangle}_{\text{reproducing property}} = \kappa(\mathbf{x}_1, \mathbf{x}_2) = \phi_{\mathbf{x}_1}^\top \phi_{\mathbf{x}_2}. \quad (\text{E.4})$$

The result will be a scalar expressing similarity of $\mathbf{x}_1$ to $\mathbf{x}_2$ in the space defined by kernel κ . Observe this expression uses the reproducing property of κ , whereby when two partially applied functions are applied to each other, it is just the inner product [[Was19], [SC04]].

E.3 Learning in Hilbert Space

Note that it is not the case where given some data set, and arbitrary training objective, the solution to this objective must be found in the $\mathcal{H}_\kappa$. The next theorem shows which kind of objective functions yield solutions in the RKHS defined by the kernel.

Theorem E.3.1. (The Representer Theorem). Let the data set be $\{(\mathbf{x}_i, y_i) : i = 1..m\}$. Let κ be a reproducing kernel function, so that there is a RKHS $\mathcal{H}_\kappa$ spanned by the κ -embedding of data set: $\{\kappa(\mathbf{x}_i, \cdot) : i = 1..m\}$ (see fig (E.3)). Now suppose the data is generated by function

²Sometimes this notation $\phi(\mathbf{x}_1)$ is used to denote $\phi_{\mathbf{x}_1}$. In this case $\phi(\mathbf{x}_1)(\mathbf{x}_2) = \langle \phi_{\mathbf{x}_1}, \phi_{\mathbf{x}_2} \rangle$.

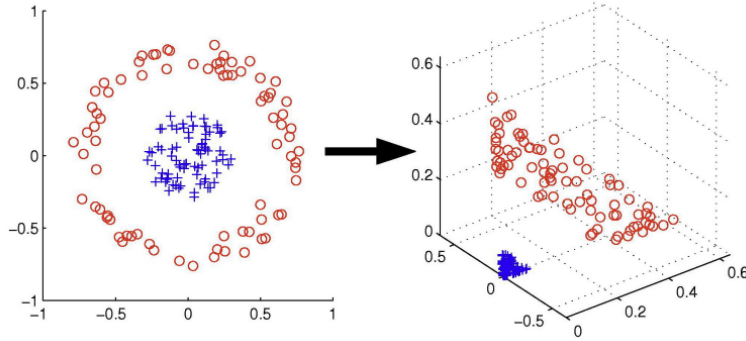


Figure E.2: Left: Given m data points $\mathbf{X} \in \mathbb{R}^{m \times 2}$, where each data point $\mathbf{x} = (a_1, a_2) \in \mathbb{R}^2$, $\mathbf{X}$ is not separable by linear classifiers. Right: the data set is "lifted" into a higher dimension by the kernel, where each point in this new space is defined $\kappa(\mathbf{x}, \cdot) = \phi_{\mathbf{x}} = (a_1^2, a_2^2, \sqrt{2} a_1 a_2)$. This set of points is now separated by a hyperplane in $\mathbb{R}^3$ [TJ04].

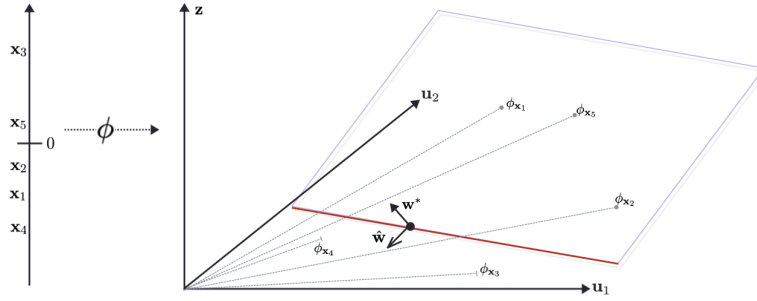


Figure E.3: The labeled data set $\{(\mathbf{x}_1, 1), (\mathbf{x}_2, 1), (\mathbf{x}_3, -1), (\mathbf{x}_4, -1), (\mathbf{x}_5, 1)\}$ is not separable in the original data space in $\mathbb{R}^2$. They are projected onto Hilbert space $\mathcal{H}_\kappa$ in $\mathbb{R}^2$, with kernel κ and feature map ϕ . $\mathcal{H}_\kappa$ is spanned by the kernelized data $\{\phi_{\mathbf{x}_i} = \kappa(\mathbf{x}_i, \cdot)\}_{i=1}^5$, or the orthonormal basis $\{\mathbf{u}_1, \mathbf{u}_2\}$. $\mathcal{H}_\kappa$ rests in ambient space in $\mathbb{R}^3$ spanned by $\{\mathbf{u}, \mathbf{u}_2, \mathbf{z}\}$. Representer theorem states that if the learning objective is defined as eqn (E.5), then any solution $\mathbf{w}^* = \hat{\mathbf{w}} + \mathbf{w}_\perp$ in $\mathbb{R}^3$ has a component $\hat{\mathbf{w}}$ lying in $\mathcal{H}_\kappa$ (in $\mathbb{R}^2$), which also shatters the points. That is $\hat{\mathbf{w}}$ is spanned by the feature vectors with: $\hat{\mathbf{w}} = \sum_{i=1}^5 \alpha_i \phi_{\mathbf{x}_i}$.

$y_i = f(\mathbf{x}_i) = \mathbf{x}_i^\top \mathbf{w}$. Define regularizer Γ that is non-decreasing, and estimator $y_i = \mathbf{w}^\top \mathbf{x}_i$. If the original objective is of form:

$$\arg \min_{\mathbf{w}} \text{loss}(\mathbf{w}) \quad \text{where} \quad \text{loss}(\mathbf{w}) = \underbrace{\sum_{i=1}^m (y_i - \langle \mathbf{w}, \mathbf{x}_i \rangle)^2}_{\text{prediction loss}} + \underbrace{\Gamma(\|\mathbf{w}\|_2^2)}_{\text{regularizer}}. \quad (\text{E.5})$$

Then a solution $\hat{\mathbf{w}}$ lies in $\mathcal{H}_\kappa$, that is $\hat{\mathbf{w}}$ can be described as a linear combination of the kernelized data set with:

$$\hat{\mathbf{w}} = \sum_{i=1}^m \alpha_i \kappa(\mathbf{x}_i, \cdot). \quad (\text{E.6})$$

Proof. Suppose there is some solution $\mathbf{w}^* = \hat{\mathbf{w}} + \mathbf{w}_\perp$, where $\hat{\mathbf{w}}$ lies in $\mathcal{H}_\kappa$, and $\mathbf{w}_\perp$ lies in

$\mathcal{H}_\kappa^\perp$, that is the space orthogonal to $\mathcal{H}_\kappa$.³ First we show $\hat{\mathbf{w}}$ minimizes the regularizer:

$$\|\mathbf{w}^*\|_2^2 = \|\hat{\mathbf{w}}\|_2^2 + \|\mathbf{w}_\perp\|_2^2 \geq \|\hat{\mathbf{w}}\|_2^2.$$

And since Γ is non decreasing, we know:

$$\Gamma(\|\mathbf{w}^*\|_2^2) = \Gamma(\|\hat{\mathbf{w}}\|_2^2 + \|\mathbf{w}_\perp\|_2^2) \geq \Gamma(\|\hat{\mathbf{w}}\|_2^2).$$

This means the regularizer Γ can be minimized by the $\hat{\mathbf{w}}$ lying in $\mathcal{H}_\kappa$. Now we show the classification loss only depend on the component lying on the subspace $\mathcal{H}_\kappa$. For every $\mathbf{x}_i$ evaluated in the kernalized space:

$$\langle \mathbf{w}^*, \kappa(\mathbf{x}_i, \cdot) \rangle = \langle \hat{\mathbf{w}}, \kappa(\mathbf{x}_i, \cdot) \rangle + \underbrace{\langle \mathbf{w}_\perp, \kappa(\mathbf{x}_i, \cdot) \rangle}_{0 \text{ by def}} = \langle \hat{\mathbf{w}}, \kappa(\mathbf{x}_i, \cdot) \rangle.$$

So the part of $\mathbf{w}^*$ that is orthogonal to $\mathcal{H}_\kappa$ does not influence the optimization objective. Hence we know $\hat{\mathbf{w}} = \sum_{i=1}^m \alpha_i \kappa(\mathbf{x}_i, \cdot)$ [Bar08]. ■

Since support vector machines (SVM) and Ridge regression have the loss function stated above, they can be *kernalized*. That is to say it is possible to learn an SVM classifier or run kernalized ridge regression, in the feature space defined by κ . The following example demonstrates a simple kernel regression, and introduces the concept of Kernel matrix or Gram matrix.

Example E.3.2. (Kernalized Ridge Regression)

Given a set of data of form $\{(\mathbf{x}_i, y_i) : i = 1..m\}$, where each $\mathbf{x} \in \mathbb{R}^{d_0 \times 1}$, and the label vector $\mathbf{y} = (y_1, \dots, y_m) \in \mathbb{R}^{1 \times m}$. Assume the data is generated by some function $y_i = f(\mathbf{x}_i) = \mathbf{w}^\top \mathbf{x}_i$, then $\mathbf{w}$ is estimated by minimizing the regularized *quadratic loss* in the data space with:

$$loss(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^m (y_i - \langle \mathbf{w}, \mathbf{x}_i \rangle)^2 + \lambda \|\mathbf{w}\|_2^2$$

In the kernalized space, this loss becomes:

$$\begin{aligned} loss(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^m (y_i - \langle \mathbf{w}, \phi_{\mathbf{x}_i} \rangle)^2 + \lambda \|\mathbf{w}\|_2^2 \\ &\stackrel{(1)}{=} \underbrace{\frac{1}{2} \sum_{i=1}^m \left(y_i - \left\langle \sum_{j=1}^m \alpha_j \phi_{\mathbf{x}_j}, \phi_{\mathbf{x}_i} \right\rangle \right)^2}_{\text{prediction loss}} + \underbrace{\lambda \|\mathbf{w}\|_2^2}_{\text{regularization}} \end{aligned}$$

Where going into (1) we use the fact that $\mathbf{w}$ is spanned by the feature vectors via: $\mathbf{w} = \sum_{j=1}^m \alpha_j \phi_{\mathbf{x}_j}$. Now rewrite "prediction loss" for $\mathbf{x}_i$ w.r.t all feature basis with:

$$\sum_{j=1}^m \alpha_j \langle \phi_{\mathbf{x}_j}, \phi_{\mathbf{x}_i} \rangle = (\alpha_1, \dots, \alpha_m) (\kappa(\mathbf{x}_1, \mathbf{x}_i), \dots, \kappa(\mathbf{x}_m, \mathbf{x}_i))^\top = \langle \boldsymbol{\alpha}, \mathbf{k}_i \rangle$$

We can place all $\mathbf{k}_i$'s into an $m \times m$ matrix with:

$$\mathbf{K} = [\mathbf{k}_1, \dots, \mathbf{k}_m] = \begin{bmatrix} \kappa(\mathbf{x}_1, \mathbf{x}_1) & \dots & \kappa(\mathbf{x}_1, \mathbf{x}_m) \\ \vdots & \ddots & \vdots \\ \kappa(\mathbf{x}_m, \mathbf{x}_1) & \dots & \kappa(\mathbf{x}_m, \mathbf{x}_m) \end{bmatrix} \quad (\text{E.7})$$

³I.e.: $\mathbf{w}^* = (a, b, c)$, $\hat{\mathbf{w}} = (a, b, 0)$, and $\mathbf{w}_\perp = (0, 0, c)$. And with direct sum we have: $\mathbf{w}^* = \hat{\mathbf{w}} + \mathbf{w}_\perp$.

This $\mathbf{K}$ is known as the Gram matrix, the *Kernel matrix*. It is the principle object of study in all Kernel methods (more on this later). The loss can be rewritten in vector notation using $\mathbf{K}$:

$$\frac{1}{2} \sum_{i=1}^m \left(y_i - \left\langle \sum_{j=1}^m \alpha_j \phi_{\mathbf{x}_j}, \phi_{\mathbf{x}_i} \right\rangle \right)^2 = \frac{1}{2} \|\mathbf{y} - \mathbf{K}\boldsymbol{\alpha}\|_2^2 = \frac{1}{2} (\mathbf{y} - \mathbf{K}\boldsymbol{\alpha})^\top (\mathbf{y} - \mathbf{K}\boldsymbol{\alpha}).$$

And similarly for the regularizer:

$$\begin{aligned} \|\mathbf{w}\|_2^2 &= \langle \mathbf{w}, \mathbf{w} \rangle = \left\langle \sum_{i=1}^m \alpha_i \phi_{\mathbf{x}_i}, \sum_{j=1}^m \alpha_j \phi_{\mathbf{x}_j} \right\rangle \\ &= \sum_{i,j} \alpha_i \alpha_j \langle \kappa(\mathbf{x}_i, \cdot), \kappa(\mathbf{x}_j, \cdot) \rangle = \sum_{i,j} \alpha_i \alpha_j \kappa(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha}. \end{aligned}$$

And the objective becomes:

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathcal{H}_\kappa} \frac{1}{2} \underbrace{(\mathbf{y} - \mathbf{K}\boldsymbol{\alpha})^\top (\mathbf{y} - \mathbf{K}\boldsymbol{\alpha})}_{\text{loss function}} + \underbrace{\lambda \boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha}}_{\text{regularization}} \quad (\text{E.8})$$

Solving for $\boldsymbol{\alpha}$ and we have:

$$\mathbf{w}^* = \sum_i \alpha_i \phi_{\mathbf{x}_i} = \mathbf{K}(\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y} \quad \text{where } \boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}. \quad (\text{E.9})$$

And note that $\mathbf{w}^*$ is in the span of data set in feature space. And $\boldsymbol{\alpha}$ is informed by the Kernel matrix $\mathbf{K}$, and biased by the regularization constant λ . Moreover, the regularization is added to the diagonal element of $\mathbf{K}$ with $\mathbf{K} + \lambda \mathbf{I}$. In the case of classification via i.e. SVMs, adding λ 's to the diagonal terms creates a greater margin on the classification boundary. Finally, observe the feature vectors $\phi_{\mathbf{x}_i}$ are not needed for regression, all necessary information about the data is stored in the Gram matrix. This makes $\mathbf{K}$ the principle object of study across Kernel methods [[Wel19], [Sch00], [Rad+22a]].

Kernel matrix $\mathbf{K}$

The $m \times m$ square matrix $\mathbf{K}$ in eqn (E.7) is the central "*data structure*" of interest in any kernel-based method. Each one of its ij th entries is defined with the inner product of the embedding of the i th data point with that of the j th one:

$$\mathbf{K}_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi_{\mathbf{x}_i}, \phi_{\mathbf{x}_j} \rangle \quad \forall 1 \leq i, j \leq m. \quad (\text{E.10})$$

Some basic observations follows.

1. $\mathbf{K}$ is an *information bottleneck*, interfacing between data on one side, and learning algorithm on the other. Many of the information is lost to the learning algorithms, i.e. the Kernel is invariant to where the origin is placed in $\mathcal{H}_\kappa$ [SC04, p. 69].
2. Since κ is a Mercer kernel by definition, $\mathbf{K}$ is symmetric and positive semidefinite:

$$\begin{aligned} \mathbf{K} &= \mathbf{K}^\top && (\text{symmetric}) \\ \mathbf{x}^\top \mathbf{K} \mathbf{x} &\succeq 0 && (\text{positive semidefinite}). \end{aligned}$$

Notice the two ways of defining symmetric and nonnegative conditions are analogous.

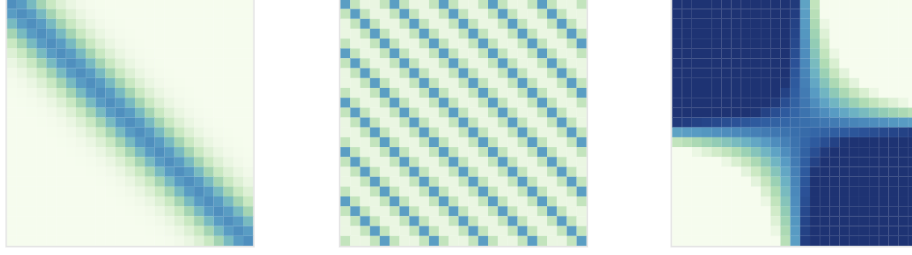


Figure E.4: Placing existing kernels κ into polynomials or exponential functions create new valid kernels. These kernels induce different kernel matrices $\mathbf{K}$, with different similarity scores between any pair of $\mathbf{x}_1$ and $\mathbf{x}_2$.

A) RBF/Gaussian kernel $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\frac{\|\mathbf{x}_1 - \mathbf{x}_2\|_2^2}{2\sigma^2}\right)$.

B) periodic kernel $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \sigma^2 \exp\left(-\frac{2\sin^2(\pi|\mathbf{x}_1 - \mathbf{x}_2|/p)}{2\sigma^2}\right)$.

C) linear kernel $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \sigma_1^2 + \sigma_2^2(\mathbf{x}_1 - b)(\mathbf{x}_2 - b)$.

Darker colors indicate higher similarity scores, notice in each case the diagonal represent self similarity. In every case, the variance parameter σ determines the average distance away from the function's mean when running regression on $\mathbf{K}$ [GKD19].

3. *Kernel trick*: this is an optimization choice, where the entire Gram matrix $\mathbf{K}$ is pre-computed and stored for reference. For example, if κ is the RBF kernel defined in eqn (E.3), then its feature vector at each $\mathbf{x}_i$ is:

$$\phi_{\mathbf{x}_i} = \kappa(\mathbf{x}_i, x) = \exp\left(-\frac{\|\mathbf{x}_i - x\|_2^2}{2\sigma^2}\right).$$

Now because $e^x = \sum_k \frac{1}{k!} x^k$ is a polynomial of an infinite degree (or a very high degree on a computer), this feature vector is potentially very large degree d . Thus if the dimensionality of the feature vector d is much larger than the dimensionality of the data m , then it makes sense to pre-compute the Gram matrix of size $m \times m$. So that given $\mathbf{x}_i, \mathbf{x}_j \in \{(\mathbf{x}_i, y_i) : i = 1..m\}$, the fully computed kernel is just a scalar:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma^2}\right) = \exp\left(-\frac{\mathbf{x}_i^\top \mathbf{x}_j}{2\sigma^2}\right).$$

And the $d \times 1$ feature vector $\phi_{\mathbf{x}_i}$ is not needed, this is the trick. In the case where the $m^2 < m \times d$, computing the Gram matrix is efficient. However in the case where $m^2 > m \times d$, then pre-computing the Gram matrix could be very expensive space wise [[Sch00], [Rad+22a]].⁴

4. *Eigenvalue decomposition of $\mathbf{K}$* : since $\mathbf{K}$ is square, it can be decomposed into:

$$\mathbf{K} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top = \sum_{i=1}^m \lambda_i \mathbf{v}_i \mathbf{v}_i^\top, \quad (\text{E.11})$$

⁴For the programming inclined, the feature vector $\kappa(\mathbf{x}_i, \cdot) = \phi_{\mathbf{x}_i}$ is similar to partially evaluated function in Lazy programming languages, where i.e. summation over a list `sum [1..d]` is only evaluated "half way." So that the spine of the list is opened, but the summation has not been carried through. Depending on the size of d , this could cause a space leak. Where as the fully applied kernel $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ is equivalent to forcing the summation to finish with the summed number.

where λ_i and $\mathbf{v}_i$ are eigenvalues and eigenvectors of $\mathbf{K}$. The set of $\mathbf{v}_i$'s define the feature space spanned by the training set of $\mathbf{x}$'s. In fig (E.3), the basis set $\{\mathbf{u}_1, \mathbf{u}_2\}$ can be found by eigenvalue decomposition.

Example E.3.3. (Kernel Logistic Regression) We can apply the kernel closure properties in this *classification* model.⁵ Given data set of m labeled points: $\mathbf{X} \in \mathbb{R}^{m \times d_0}$, with $\mathbf{x} \in \mathbb{R}^{d_0}$, and labels $\mathbf{y} \in \{0, 1\}^m$. The model estimates the likelihood of label y given $\mathbf{x}$ with:

$$\begin{aligned} p(y = 1 | \mathbf{x}; \mathbf{w}) &= \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}, \\ p(y = 0 | \mathbf{x}; \mathbf{w}) &= \sigma(1 - \mathbf{w}^\top \mathbf{x}) = \frac{\exp(-\mathbf{w}^\top \mathbf{x})}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}. \end{aligned} \quad (\text{E.12})$$

Where $\sigma(\cdot)$ is the *sigmoid activation function*, defined as: $\sigma(x) = 1/(1 + e^{-x})$, and the input to $\sigma(\cdot)$ is often called the *logit*. As an aside, the bias term w_o is represented by lifting the data into affine space with: $\mathbf{w} = (\mathbf{w}, w_o)$ and $\mathbf{x} = (\mathbf{x}, 1)$, so that $\mathbf{w}^\top \mathbf{x} = \mathbf{w}^\top \mathbf{x} + w_o$.

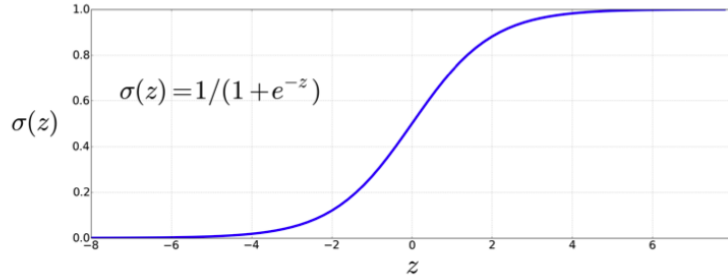


Figure E.5: The sigmoid activation function maps real values onto probabilities. In this case, the logit is $z = \langle \mathbf{w}, \mathbf{x} \rangle$. Note for a function to be convex, the epigraph of the function must be above the tangent plane supporting every point in this function. Hence $\sigma(\cdot)$ is not a convex function. However, the overall learning objective can still be convex, provided the relationship between log-loss and the model parameter $\mathbf{w}$ is appropriately expressed [JM24].

The model parameters can be found by maximum likelihood estimation (MLE), where the learning objective find the $\mathbf{w}$ that maximizes the likelihood of the training data. So given any example $(\mathbf{x}_i, y_i)$, its likelihood under eqn (E.12) is:

$$\begin{aligned} p(y_i | \mathbf{x}_i; \mathbf{w}) &= p(y = 1 | \mathbf{x}; \mathbf{w})^{y_i} \cdot p(y = 0 | \mathbf{x}; \mathbf{w})^{(1-y_i)} \\ &= (\sigma(\mathbf{w}^\top \mathbf{x}))^{y_i} \cdot (\sigma(1 - \mathbf{w}^\top \mathbf{x}))^{(1-y_i)}. \end{aligned}$$

Where y_i is the given label or ground truth, while $p(y_i | \mathbf{x}_i; \mathbf{w})$ is the likelihood of the label y_i occurring given the value of $\mathbf{x}_i$, under the model parameterized by $\mathbf{w}$. Observe how if $y_i = 1$, then the likelihood reduces to $p(y_i | \mathbf{x}_i; \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x})$, and if $y_i = 0$, then the expression is: $p(y_i | \mathbf{x}_i; \mathbf{w}) = \sigma(1 - \mathbf{w}^\top \mathbf{x})$. The log-likelihood of a singled labeled data pair is then:

$$\log(p(y_i | \mathbf{x}_i; \mathbf{w})) = y_i \log(\sigma(\mathbf{w}^\top \mathbf{x})) + (1 - y_i) \log(\sigma(1 - \mathbf{w}^\top \mathbf{x})).$$

Log likelihood is used to address the numerical underflow problem. Since the likelihood of the entire data set would grow vanishingly small, if the *data set* likelihood is expressed

⁵Even though this is called "regression."

with: $\prod_i p(y_{i=1}|\mathbf{x}_i; \mathbf{w})^{y_i} p(y_{i=0}|\mathbf{x}_i; \mathbf{w})^{1-y_i}$. Thus, the objective is to maximize the log-likelihood of data, or minimize the negative loss over all m examples with:

$$\log(\text{loss}(\mathbf{w})) = - \sum_{i=1}^m \left(y_i \log(\sigma(\mathbf{w}^\top \mathbf{x}_i)) + (1 - y_i) \log(\sigma(1 - \mathbf{w}^\top \mathbf{x}_i)) \right). \quad (\text{E.13})$$

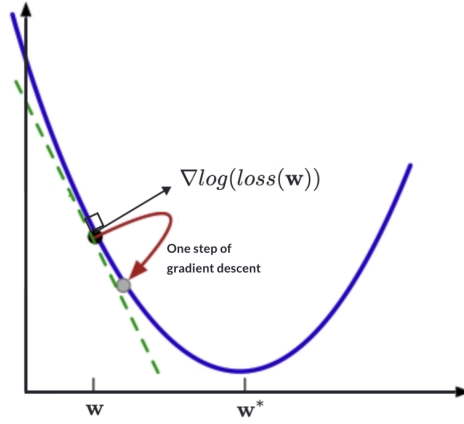


Figure E.6: In gradient descent, there is a convex relationship between the log-loss and model parameter $\mathbf{w}$. At any $\mathbf{w}$, we can compute the gradient or slope of the curve with $\nabla \log(\text{loss}(\mathbf{w}))$. Because the curve is convex, there is a unique optimal solution at $\mathbf{w}^*$. The solver can find this solution provided the step size α is sufficiently small [JM24].

And note that because log is a convex function, its convex combination is convex. However unlike ridge regression, this objective does not admit a closed form solution. Instead gradient descent is used to update the $\mathbf{w}$ with:

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \alpha \nabla \log(\text{loss}(\mathbf{w})),$$

where $\alpha \in (0, 1)$ is the learning rate. Next we can kernelize eqn (E.12) with:

$$\begin{aligned} p(y = 1|\mathbf{x}; \mathbf{w}) &= \sigma(\mathbf{w}^\top \phi_{\mathbf{x}}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \phi_{\mathbf{x}})}, \\ p(y = 0|\mathbf{x}; \mathbf{w}) &= \sigma(1 - \mathbf{w}^\top \phi_{\mathbf{x}}) = \frac{\exp(-\mathbf{w}^\top \phi_{\mathbf{x}})}{1 + \exp(-\mathbf{w}^\top \phi_{\mathbf{x}})}. \end{aligned} \quad (\text{E.14})$$

And note that by the closure properties defined above, $p(y|\phi_{\mathbf{x}}; \mathbf{w})$ is a valid kernel. Where given data point $\mathbf{x}_j$, and kernel matrix $\mathbf{K}$ with entries $\kappa(\mathbf{x}_i, \mathbf{x}_j)$, we have, i.e.:

$$p(y_j = 1|\mathbf{x}_j; \mathbf{w}) = \frac{1}{1 + \exp\left(-\sum_{i=1}^m \alpha_i \kappa(\mathbf{x}_i, \mathbf{x}_j)\right)}.$$

So that the kernel matrix $\mathbf{K}$ is used here during training and inference. The loss function then becomes:

$$\log(\text{loss}(\mathbf{w})) = - \sum_{i=1}^m \left(y_i \log(\sigma(\mathbf{w}^\top \phi_{\mathbf{x}_i})) + (1 - y_i) \log(\sigma(1 - \mathbf{w}^\top \phi_{\mathbf{x}_i})) \right). \quad (\text{E.15})$$

This *loss* can be optimized by the same numerical method outlined above [JM24].

Remark E.3.4. (Maximizing likelihood vs minimizing KL divergence) In the MLE objective above, we maximize the likelihood of the model given data. So that given current model settings $\mathbf{p}(y|\mathbf{x}; \mathbf{w})$, and optimal solution $\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)$, we can write:

$$\begin{aligned} \arg \max_{\mathbf{w}} \prod_{i=1}^m \mathbf{p}(y_i|\mathbf{x}_i; \mathbf{w}) \\ \implies \arg \max_{\mathbf{w}} \frac{m}{m} \cdot \sum_{i=1}^m \log(\mathbf{p}(y_i|\mathbf{x}_i; \mathbf{w})) \xrightarrow{p} m \cdot \mathbf{E}_{\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)} [\log(\mathbf{p}(y|\mathbf{x}; \mathbf{w}))] \\ \implies - \arg \min_{\mathbf{w}} \mathbf{E}_{\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)} \left[\log \left(\frac{1}{\mathbf{p}(y|\mathbf{x}; \mathbf{w})} \right) \right]. \end{aligned}$$

That is the empirical likelihood converges in probability to the expected population likelihood, and note going into the last line, minimizing the negative log-likelihood is what the optimizer solves. Now recall the KL-divergence expression of eqn (F.6), given current model settings $\mathbf{p}(y|\mathbf{x}; \mathbf{w})$, and optimal solution $\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)$, their divergence is:

$$\begin{aligned} \arg \min_{\mathbf{w}} D(\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*) || \mathbf{p}(y|\mathbf{x}; \mathbf{w})) &= \mathbf{E}_{\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)} \left[\log \left(\frac{\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)}{\mathbf{p}(y|\mathbf{x}; \mathbf{w})} \right) \right] \\ &= \mathbf{E}_{\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)} \left[\log \mathbf{p}(y|\mathbf{x}; \mathbf{w}^*) - \log \mathbf{p}(y|\mathbf{x}; \mathbf{w}) \right] \\ &\stackrel{(1)}{\sim} \mathbf{E}_{\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)} \left[- \log \mathbf{p}(y|\mathbf{x}; \mathbf{w}) \right] \\ &= \mathbf{E}_{\mathbf{p}(y|\mathbf{x}; \mathbf{w}^*)} \left[\log \left(\frac{1}{\mathbf{p}(y|\mathbf{x}; \mathbf{w})} \right) \right]. \end{aligned}$$

Where going into (1), we use the fact that the term on left is a constant. So we see that maximizing the log-likelihood of data is the same as minimizing the KL-divergence between the optimal model parameter $\mathbf{w}^*$ and the current weights $\mathbf{w}$.

The $\sigma(\cdot)$ function in kernalized logistic regression is an example of an *activation function*. Neural networks are built from repeated application of $\sigma(\mathbf{w}^\top \mathbf{x})$ units. In fact, if we let the k th dimension of $\mathbf{w} \in \mathbb{R}^{k \times m}$ to go to infinity, then training this neural network with infinitely wide $\mathbf{w}$ is equivalent to training a two-layered logistic regression model.

E.4 Neural Tangent Kernel

This section continues the discussion of model selection in function space, and applies it to deep artificial neural networks (ANNs). In particular, we will show the equivalence between deep learning and kernel regression when the width of the intermediate layers go to infinity. This "Extra-wideness plays a crucial role in the proof: it is shown that as width increases, training causes increasingly smaller changes (in a proportionate sense) in the parameters. [AD+19]" This chapter will retrace many of these arguments:

1. the section "Neural Networks" will give a broad overview of ANNs, and their advantage over traditional kernel methods.
2. "Neural Network Gaussian Process" pose deep learning in two layered feed-forward networks as Gaussian process regression.
3. The last two sections will derive neural tangent kernels for multiplayer feed forward, and recurrent neural networks. Thus concluding the justification for ANN-derived kernel work units in the Metheus network.

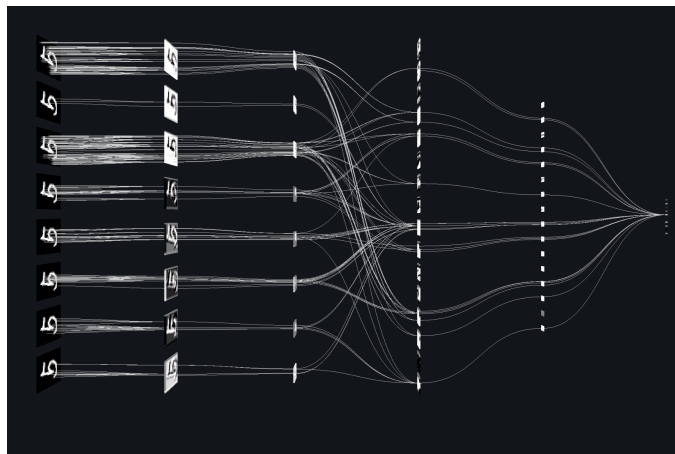


Figure E.7: Obligatory image of a neural network. This image is depicting MNIST classification using convolutional neural nets. Aside from that, it is unclear what else this picture is suppose to communicate. Most diagrams of neural networks are like this, albeit less artistic. This only deepens the mystique that the next few sections will dispel.

Neural Networks

Kernel based methods capture nonlinear relationship amongst input and output data, while maintaining the convexity of the loss function. However, there are some shortcomings when choosing the embedding function ϕ . One can either 1) opt for a very expressive infinite-order polynomial such as the Gaussian Kernel, or 2) manually engineer ϕ . The problem with the first approach is that overtly expressive kernels can over fit the data. In the latter approach, the statistician would need years of domain experience to construct a clever kernel. The deep learning approach is to learn the embedding alongside all model parameters, in other words learn the "basis functions" in an adaptive manner. For example in the kernel regression case of E.3.2, the model trained on m data points was of form:

$$y = f(\mathbf{x}; \mathbf{w}) = \langle \mathbf{w}, \phi_{\mathbf{x}} \rangle = \left\langle \sum_{i=1}^m \alpha_i \kappa(\mathbf{x}_i, \cdot), \kappa(\mathbf{x}, \cdot) \right\rangle = \left\langle \sum_{i=1}^m \alpha_i \sigma(\mathbf{B}\mathbf{x}_i), \sigma(\mathbf{B}\mathbf{x}) \right\rangle = \mathbf{A}\sigma(\mathbf{B}\mathbf{x}).$$

where we use the expression $\sigma(\mathbf{B}\mathbf{x}_i)$ to express the partially applied kernel function at the data point $\mathbf{x}_i$. In the classical kernel case, $\mathbf{B}$ is some user defined block of polynomial functions. The activation function $\sigma(\cdot)$ is some nonlinear function, i.e. e^x , whose parameters are not updated during learning. This $\sigma(\cdot)$ serves as a decision function: i.e., it may average the signal going into a vertex on the computational graph, or it may take the maximum signal and squash its value so that it is between $[0, 1]$. In the ANN case, $\mathbf{B}$ is learned alongside the regression coefficients $\mathbf{A}$. That is $\mathbf{A}$ and $\mathbf{B}$ are functions of the data $(\mathbf{X}, \mathbf{y})$, and of each other. This interdependence renders the optimization problem non-convex. In the general L -layer-deep ANN, we put all the parameters in $\mathbf{w} = (\mathbf{A}, \mathbf{B}^1, \dots, \mathbf{B}^L)$, and the model is:

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{A}\sigma(\mathbf{B}^L(\dots\sigma(\mathbf{B}^1(\mathbf{x}))\dots)),$$

with L blocks of polynomials $\mathbf{B}^l$ transforming the input from the previous "layer," followed by nonlinear activation function $\sigma(\cdot)$ [BMR21]. In order to learn the parameters, the optimizer repeats two tasks many times over for every l th layer:

1. construct a "function space" from "data" $\phi_{\mathbf{x}_i}^{l-1}$ supplied by the $(l-1)$ th layer. This space is spanned by the set of basis vectors $\{\phi_{\mathbf{x}_i}^{l-1}\}$, where each basis vector is constructed with $\phi_{\mathbf{x}_i}^{l-1} = \sigma(\mathbf{B}^{l-1}\mathbf{x}_i^{l-1})$.

2. Learn a set of polynomials $\mathbf{B}^l$ according to some decision criteria. This block will define a new function space, spanned by $\phi_{\mathbf{x}_i}^l = \sigma(\mathbf{B}^l \mathbf{x}_i^l)$ for the next layer $l + 1$.

Each repetition of the above process constructs one layer in the deep ANN. At each layer, the "data" is supplied from the previous layer. So that at the bottom layer the "data" is just the $(\mathbf{X}, \mathbf{y})$ pairs sampled from nature, in the intermediate layer the data is output from models of the previous layer. Back-propagation makes end-to-end optimization of the model possible, so that every basis functions at each layer is learned together, along with the parameters of the final classification or regression objective parameterized by $\mathbf{A}$.

Now depending on the nature of data, its distribution can be summarized by two kinds of ANN architectures: feed-forward and recurrent neural nets.

1. The example shown in fig (E.7) is a *feed-forward networks*. They are used for data with global dependence, usually paired with global feature extractors such as convolution filters. Convolutional neural nets (CNNs) are the workhorse in this setting. In general, any data that can be "construed" as images with global information dependencies admit some solution with CNNs. This certainly applies to visual inputs of every kind, but it also find application in Alpha-Go, where the entire go-game-board is represented by the output from a CNN [SS+17].
2. *Recurrent neural networks (RNNs)* are nonlinear auto-regressive model used for time-series data. Dialogue systems with sentence generation is the canonical use case, trajectory generation for robotic arms moving through space is also a valid setting, as are path prediction for pedestrians crossing the road in a self-driving car application.

The last example is what makes deep ANNs compelling: the same architecture used for sentence parsing can also be used to evaluate pedestrians at a stop sign [[VS+17],[YBS22]]. Although the "depth" of deep ANNs capture the imagination of the public, most of the intuition for ANNs occur when the *width* of the intermediate layers are taken to infinity [Bel21]. In fact, it is known that arbitrarily wide ANN can approximate any function:

"...standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, multilayer feedforward networks are a class of universal approximators [HSW89]."

Thus the logical point to start is to express a simple two-layered feed-forward neural networks with infinitely wide bottom layer. At initialization, this infinitely wide ANN is equivalent to Gaussian process regression, whereby the bottom layer forms a kernel, while the top layer perform linear summation of kernalized data [Nea96]. Furthermore:

"...the evolution of an ANN during training can also be described by a kernel: ... the Neural Tangent Kernel (NTK). This kernel is central to describe the generalization features of ANNs. While the NTK is random at initialization and varies during training, in the infinite-width limit it converges to an explicit limiting kernel and it stays constant during training. This makes it possible to study the training of ANNs in function space instead of parameter space. Convergence of the training can then be related to the positive-definiteness of the limiting NTK [JG+18]."

Most saliently, the training of ANN follows a descent along the kernel gradient w.r.t the NTK, which is convex. *That is to say that training some ANNs under specific conditions is equivalent to (NTK) kernel regression.* This analysis generalize to feed-forward networks of arbitrary depth, and recurrent neural networks as well. In practice ANNs do not have

infinite width, and yet, there is "excellent empirical agreement between the predictions of [infinitely wide ANNs] and those of the linearized version even for finite practically-sized networks" [LX+19]. The next three sections expounds on this point. The arguments are drawn from [JG+18] and [LZB21], but follows the gentler approach of [Rad+22b], [Rad+22d], and [Rad+22c].

Feed-forward Neural Networks as Gaussian Process

(Setting) Similar to least squares shown in example E.3.2, the data is assumed to be generated by $y = f(\mathbf{x}; \mathbf{w}) = \langle \mathbf{w}, \mathbf{x} \rangle + \epsilon$, where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. In the ANN setting, we gather all the parameters in $\mathbf{w} = (\mathbf{A}, \mathbf{B})$, and define the network $f(\mathbf{x}; \mathbf{w})$ with:

$$\begin{aligned} f : \mathbb{R}^{d_0} &\rightarrow \mathbb{R} \\ f(\mathbf{x}; \mathbf{w}) &= \frac{1}{\sqrt{d_1}} \mathbf{A} \sigma(\mathbf{B} \mathbf{x}) \quad \text{for } \mathbf{y} \in \mathbb{R} \quad \mathbf{x} \in \mathbb{R}^{d_0} \quad \text{with } \mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ \text{where } \mathbf{A} &\in \mathbb{R}^{1 \times d_1} \quad \mathbf{B} \in \mathbb{R}^{d_1 \times d_0} \quad \sigma(\cdot) : \mathbb{R} \rightarrow \mathbb{R}. \end{aligned} \quad (\text{E.16})$$

Some clarifications:

1. The activation function $\sigma(\cdot)$ is applied element wise.
2. The $\frac{1}{\sqrt{d_1}}$ is a numerical convenience that we will explain later. Note we can always just multiply the final values of $\mathbf{A}$ by $1/\sqrt{d_1}$, and thus remove this constant term from the expression
3. The vectors $\mathbf{x}$ are normalized, and are drawn i.i.d from the surface of a sphere with dimension $d_0 - 1$, see fig (E.8). In practice this is not a stringent assumption as the data can always be zero-meaned and normalized to be between 0 and 1.

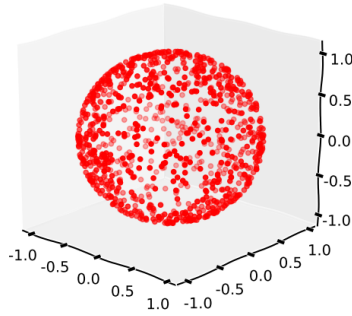


Figure E.8: The data are drawn $\mathbf{x} \stackrel{\text{iid}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I})$, where $\mathbf{0} \in \mathbb{R}^{d_0-1}$, and $\mathbf{I} \in \mathbb{R}^{d_0-1 \times d_0-1}$. That is the sphere is lying in ambient space with dimension d_0 , but the distribution has support in $d_0 - 1$. This is a very common assumption since for high values of d_0 , most of the volume of the sphere are on the surface, not the interior. So a randomly generated set of vectors will be on the surface.

Now given data set:

$$(\mathbf{X} \in \mathbb{R}^{m \times d_0}, \mathbf{y} \in \mathbb{R}^{m \times 1}),$$

the model is trained by minimizing the least squares error using gradient descent:

$$\text{loss}(\mathbf{A}, \mathbf{B}) = \sum_{i=1}^m (y_i - f(\mathbf{x}_i))^2 = \sum_{i=1}^m (y_i - \mathbf{A} \sigma(\mathbf{B} \mathbf{x}_i))^2. \quad (\text{E.17})$$

Note this objective is not convex as $\mathbf{A}$ and $\mathbf{B}$ are both optimized. That is given f initialized at $(\mathbf{A}^0, \hat{\mathbf{B}})$, the two matrices are updated in parallel via gradient descent with:

$$\begin{aligned} \mathbf{A}^{t+1} &\leftarrow \mathbf{A}^t - \eta \nabla \text{loss}_{\mathbf{A}}(\mathbf{A}^t, \mathbf{B}^t) \\ \mathbf{B}^{t+1} &\leftarrow \mathbf{B}^t - \eta \nabla \text{loss}_{\mathbf{B}}(\mathbf{A}^t, \mathbf{B}^t). \end{aligned}$$

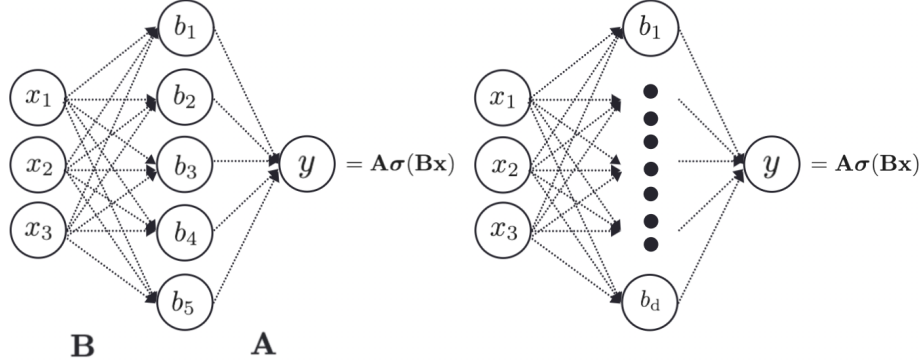


Figure E.9: A two layer neural network $y = \mathbf{A}\sigma(\mathbf{B}\mathbf{x})$, where $\mathbf{x} = (x_1, x_2, x_3) \in \mathbb{R}^3$. Left: the feature matrix where $d_1 = 5$, so it has dimension $\mathbf{B} \in \mathbb{R}^{3 \times 5}$, thus mapping $\mathbf{x}$ to some vector in $\mathbb{R}^5$. The output matrix has dimension $\mathbf{A} \in \mathbb{R}^{5 \times 1}$, and maps the inputs $\mathbf{x}$ to some label y . Right: in the limit, the d_1 dimension of $\mathbf{B} \in \mathbb{R}^{3 \times d_1}$ grows to infinity. Now consider a setup where the entries of the infinite dimensional $\mathbf{B}$ are fixed by sampling via $\mathbf{B}_{ij} \sim \mathcal{N}(0, 1)$. Given fixed $\hat{\mathbf{B}}$, if we just train $\mathbf{A}$ using least squares loss, then this setting is the same as kernel regression, where $\sigma(\hat{\mathbf{B}}\mathbf{x})$ is the kernel. This can be generalized to neural nets of arbitrary depth, so that training the k th layer of a neural network, whilst fixing the bottom $k - 1$ layers, is just regression. Kernel regression can be posed in a non-parametric manner using the tools of Gaussian process, this motivates the derivation of neural tangent kernels for different activation functions [Rad+22b].

The next proposition will recover convexity by simplifying the problem. It states that if the values of the feature matrix $\mathbf{B}$ are randomly initialized and then frozen, then eqn (E.17) reduces to kernel regression.

Proposition E.4.1. (*Neural Network Gaussian Process - NNGP*) Given the setting above, suppose the learner initializes the values of $\mathbf{B}$ by sampling its entries i.i.d from $\mathcal{N}(\mathbf{0}, \mathbf{I})$, and then freezes the weights of this $\hat{\mathbf{B}}$. Then training with the loss by varying $\mathbf{A}$ while fixing $\hat{\mathbf{B}}$:

$$\text{loss}(\mathbf{A}) = \sum_{i=1}^m (y_i - \mathbf{A}\sigma(\hat{\mathbf{B}}\mathbf{x}_i))^2, \quad (\text{E.18})$$

is equivalent to kernalized linear regression. Here the basis functions of the feature space $\mathcal{H}_{\kappa}$ is defined by $\phi_{\mathbf{x}_i} = \sigma(\hat{\mathbf{B}}\mathbf{x}_i)$, for every $\mathbf{x}_i$ in the training set. In particular, as the width of $\mathbf{B}$ approaches infinity, that is $d_1 \rightarrow \infty$, we have:

E.4.1.1. the solution to regression is given by:

$$f(\mathbf{x}; \mathbf{w}) \sim \sum_{i=1}^{d_1} \alpha_i \sigma(\langle \hat{\mathbf{b}}_i, \mathbf{x} \rangle). \quad (\text{E.19})$$

E.4.1.2. The kernel $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ is defined as:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \left\langle \frac{1}{\sqrt{d_1}} \phi_{\mathbf{x}_i}, \frac{1}{\sqrt{d_1}} \phi_{\mathbf{x}_j} \right\rangle. \quad (\text{E.20})$$

Moreover, the value of this kernel is a r.v., by definition. The source of randomness are the data drawn by $\mathbf{x}_i, \mathbf{x}_j \stackrel{\text{iid}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I})$, and entries of $\mathbf{B}$ populated according to $(\hat{\mathbf{B}})_{ij} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1)$.

Thus the kernel values are drawn according to $\kappa(\mathbf{x}_i, \mathbf{x}_j) \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \Lambda)$: centered at zero, with covariance:

$$\Lambda = \begin{bmatrix} \langle \mathbf{x}_i, \mathbf{x}_i \rangle & \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ \langle \mathbf{x}_j, \mathbf{x}_i \rangle & \langle \mathbf{x}_j, \mathbf{x}_j \rangle \end{bmatrix}.$$

A diagram of the setting is provided in fig (E.9).

Proof. The expected value of the function $\mathbf{E}[f(\mathbf{x}; \mathbf{w})]$ is a straight forward derivation. We can write the terms of $\mathbf{A}$ and $\hat{\mathbf{B}}$ as:

$$\mathbf{A} = (\alpha_1, \dots, \alpha_{d_1}) \quad \hat{\mathbf{B}} = \begin{bmatrix} \hat{b}_{11} & \dots & \hat{b}_{1d_0} \\ \vdots & \ddots & \vdots \\ \hat{b}_{d_1 1} & \dots & \hat{b}_{d_1 d_0} \end{bmatrix} = \begin{bmatrix} \text{---} \hat{\mathbf{b}}_1 \text{---} \\ \vdots \\ \text{---} \hat{\mathbf{b}}_{d_1} \text{---} \end{bmatrix},$$

where $\mathbf{b}_r \in \mathbb{R}^{1 \times d_0}$ are the r th row vectors for $r = 1, \dots, d_1$. Then for some $\mathbf{x} = (x_1, \dots, x_{d_0})$, we can rewrite the ANN with:

$$\begin{aligned} f(\mathbf{x}; \mathbf{w}) &= \frac{1}{\sqrt{d_1}} \mathbf{A} \boldsymbol{\sigma}(\hat{\mathbf{B}} \mathbf{x}) \\ &= \frac{1}{\sqrt{d_1}} \mathbf{A} \boldsymbol{\sigma} \left(\begin{bmatrix} \text{---} \hat{\mathbf{b}}_1 \text{---} \\ \vdots \\ \text{---} \hat{\mathbf{b}}_{d_1} \text{---} \end{bmatrix} \mathbf{x}^\top \right) = \frac{1}{\sqrt{d_1}} (\alpha_1, \dots, \alpha_{d_1}) \cdot \begin{bmatrix} \boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_1, \mathbf{x} \rangle) \\ \vdots \\ \boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_{d_1}, \mathbf{x} \rangle) \end{bmatrix} \\ &= \frac{1}{\sqrt{d_1}} \sum_{r=1}^{d_1} \alpha_r \boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_r, \mathbf{x} \rangle). \end{aligned} \quad (\text{E.21})$$

Some observations:

1. The model $f(\mathbf{x}; \mathbf{w})$ is a *linear* summation of the featurized data, this is a consequence of fixing the feature matrix to $\hat{\mathbf{B}}$.
2. The value $\mathbf{x}$ is featurized with the function $\boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_r, \mathbf{x} \rangle)$, with randomly initialized $\hat{\mathbf{b}}_r$. If the $\mathbf{b}_r$ is constructed from the training set instead, then we would recover the kernel expression of eqn (E.3.2).
3. The sum is over d_1 , so that as $d_1 \rightarrow \infty$, $\mathbf{E}[f(\mathbf{x}; \mathbf{w})]$ is Gaussian by definition, see fig (??).

Now given $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^{d_0}$, we use the definition of the kernel from eqn (E.10) to write its expected value as $d_1 \rightarrow \infty$:

$$\begin{aligned} \kappa(\mathbf{x}_i, \mathbf{x}_j) &= \left\langle \frac{1}{\sqrt{d_1}} \phi_{\mathbf{x}_i}, \frac{1}{\sqrt{d_1}} \phi_{\mathbf{x}_j} \right\rangle \\ &= \left\langle \frac{1}{\sqrt{d_1}} \boldsymbol{\sigma}(\hat{\mathbf{B}} \mathbf{x}_i), \frac{1}{\sqrt{d_1}} \boldsymbol{\sigma}(\hat{\mathbf{B}} \mathbf{x}_j) \right\rangle \\ &= \frac{1}{\sqrt{d_1}} \boldsymbol{\sigma} \left(\begin{bmatrix} \text{---} \hat{\mathbf{b}}_1 \text{---} \\ \vdots \\ \text{---} \hat{\mathbf{b}}_{d_1} \text{---} \end{bmatrix} \mathbf{x}_i^\top \right)^\top \cdot \frac{1}{\sqrt{d_1}} \boldsymbol{\sigma} \left(\begin{bmatrix} \text{---} \hat{\mathbf{b}}_1 \text{---} \\ \vdots \\ \text{---} \hat{\mathbf{b}}_{d_1} \text{---} \end{bmatrix} \mathbf{x}_j^\top \right) \\ &= \frac{1}{d_1} \left[\boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_1, \mathbf{x}_i \rangle), \dots, \boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_{d_1}, \mathbf{x}_i \rangle) \right] \cdot \begin{bmatrix} \boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_1, \mathbf{x}_j \rangle) \\ \vdots \\ \boldsymbol{\sigma}(\langle \hat{\mathbf{b}}_{d_1}, \mathbf{x}_j \rangle) \end{bmatrix} \end{aligned}$$

$$\begin{aligned}
&= \frac{1}{d_1} \sum_{r=1}^{d_1} \sigma(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle) \cdot \sigma(\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) \\
&\stackrel{(1)}{=} \mathbf{E}_{\hat{\mathbf{B}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\langle \sigma(\hat{\mathbf{B}} \mathbf{x}_i), \sigma(\hat{\mathbf{B}} \mathbf{x}_j) \right\rangle \right].
\end{aligned} \tag{E.22}$$

Whereby going into (1), we take the limit $d_1 \rightarrow \infty$ and use the law of large numbers to rewrite the sum as converging to the expectation of the inner product in probability. That is:

$$\lim_{d_1 \rightarrow \infty} \frac{1}{d_1} \sum_{r=1}^{d_1} \sigma(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle) \cdot \sigma(\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) \xrightarrow{p} \mathbf{E}_{\hat{\mathbf{b}}_r \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\langle \sigma(\hat{\mathbf{B}} \mathbf{x}_i), \sigma(\hat{\mathbf{B}} \mathbf{x}_j) \right\rangle \right].$$

Now first observe the expected value of $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ is zero, since the r cth entry of feature matrix is drawn with $\hat{\mathbf{b}}_{rc} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1)$, and $\mathbf{x}_i \stackrel{\text{iid}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I})$ by assumption. So the product of two centered Gaussian is another centered Gaussian. Next note the expectation is taken w.r.t the random variables $\hat{\mathbf{B}}_{rc}$, however we want to write the value of the kernel $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ as drawn from the joint distribution over the pair of random variables $(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle)$:

$$\begin{aligned}
&\mathbf{E}_{(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) \sim \mathcal{N}(\mathbf{0}, \Lambda)} \left[\left\langle \sigma(\hat{\mathbf{B}} \mathbf{x}_i), \sigma(\hat{\mathbf{B}} \mathbf{x}_j) \right\rangle \right] \\
&\Lambda = \begin{bmatrix} \text{cov}(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle) & \text{cov}(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) \\ \text{cov}(\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle) & \text{cov}(\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) \end{bmatrix}.
\end{aligned}$$

The mean is once again $\mathbf{E}[\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle] = \mathbf{E}[\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle] = 0$. Now consider their covariance, fixing two constants $\mathbf{x}_i, \mathbf{x}_j$ we can compute their covariance with:

$$\begin{aligned}
\text{cov}(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) &= \mathbf{E}_{\hat{\mathbf{b}}_r \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle - \mathbf{E}[\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle] \right] \cdot \mathbf{E}_{\hat{\mathbf{b}}_r \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle - \mathbf{E}[\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle] \right] \\
&\stackrel{(1)}{=} \mathbf{E}_{\hat{\mathbf{b}}_r} \left[\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle \right] \cdot \mathbf{E}_{\hat{\mathbf{b}}_r} \left[\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle \right] \stackrel{(2)}{=} \mathbf{E}_{\hat{\mathbf{b}}_r} \left[\sum_{c=1}^{d_0} \hat{b}_{rc} x_{ic} \right] \cdot \mathbf{E}_{\hat{\mathbf{b}}_r} \left[\sum_{c=1}^{d_0} \hat{b}_{rc} x_{jc} \right] \\
&\stackrel{(3)}{=} \sum_{c=1}^{d_0} x_{ic} \cdot \mathbf{E}_{\hat{\mathbf{b}}_r} [\hat{b}_{rc}] \cdot \sum_{c=1}^{d_0} x_{jc} \cdot \mathbf{E}_{\hat{\mathbf{b}}_r} [\hat{b}_{rc}] \stackrel{(4)}{=} \sum_{c=1}^{d_0} x_{ic} \cdot x_{jc} \cdot (\mathbf{E}_{\hat{\mathbf{b}}_r} [\hat{b}_{rc}])^2 \\
&\stackrel{(5)}{=} \mathbf{E}_{\hat{\mathbf{b}}_r} \left[\sum_{c=1}^{d_0} x_{ic} \cdot x_{jc} \cdot \hat{b}_{rc}^2 \right] \stackrel{(6)}{=} \langle \mathbf{x}_i, \mathbf{x}_j \rangle.
\end{aligned}$$

Where going into (1), we use the fact that $\mathbf{E}_{\hat{\mathbf{b}}_r \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle] = 0$ for every $\mathbf{x}_i$. Going into (2) uses the definition of inner product by writing $\mathbf{x}_i = (x_{i1}, \dots, x_{ic}, \dots, x_{id_0})$, and $\hat{\mathbf{b}}_r = (\hat{b}_{r1}, \dots, \hat{b}_{rc}, \dots, \hat{b}_{rd_0})$. (3) is by linearity of expectation, and the fact that x_{ic}, x_{jc} does not depend on the expectation. Going into (4) we use the fact that the cross product terms $\sum_{c,d=1}^{d_0} x_{ic} x_{jd} \mathbf{E}[\hat{b}_{rc}] \mathbf{E}[\hat{b}_{rd}] = 0$, since the b_{rc} and b_{rd} terms are independent. In (5) we use the linearity of expectations and the fact that $\mathbf{E}_{\hat{\mathbf{b}}_r} [x_{ic}] = x_{ic}$ for $\mathbf{x}_i$ and $\mathbf{x}_j$. In (6) we use the definition of the inner product to rewrite the sum. A similar argument holds for covariance over $\text{cov}(\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle)$, etc. Thus we have:

$$\begin{aligned}
\kappa(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{E}_{(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) \sim \mathcal{N}(\mathbf{0}, \Lambda)} \left[\left\langle \sigma(\mathbf{B} \mathbf{x}_i), \sigma(\mathbf{B} \mathbf{x}_j) \right\rangle \right] \\
&\Lambda = \begin{bmatrix} \langle \mathbf{x}_i, \mathbf{x}_i \rangle & \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ \langle \mathbf{x}_j, \mathbf{x}_i \rangle & \langle \mathbf{x}_j, \mathbf{x}_j \rangle \end{bmatrix},
\end{aligned} \tag{E.23}$$

with $\phi_{\mathbf{x}_i} = \sigma(\mathbf{B} \mathbf{x}_i)$ as desired [Rad+22b]. $\blacksquare$

(Activation Functions) Now we will find an explicit expression for the expectation of eqn (E.23) for specific activation functions.

1. As a warm up, consider the activation $\sigma(x) = e^{ix}$, for some imaginary number i and $x \in \mathbb{R}$. The inner product is defined as:

$$\left\langle \{a_n\}_{n \geq 0}, \{b_n\}_{n \geq 0} \right\rangle = \sum_{n \geq 0} a_n \overline{b_n}.$$

Thus an ANN using e^{ix} as the activation function has kernel:

$$\begin{aligned} \kappa(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{E}_{\langle \mathbf{b}_r, \mathbf{x}_i \rangle, \langle \mathbf{b}_r, \mathbf{x}_j \rangle \sim \mathcal{N}(\mathbf{0}, \Lambda)} \left[\left\langle \sigma(\mathbf{B}\mathbf{x}_i), \overline{\sigma(\mathbf{B}\mathbf{x}_j)} \right\rangle \right] \\ &= \mathbf{E}_{\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle \sim \mathcal{N}(\mathbf{0}, \Lambda)} \left[\exp \left(i \cdot (\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle - \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle) \right) \right] \\ &= \prod_{c=1}^{d_0} \mathbf{E}_{\hat{b}_{rc} \sim \mathcal{N}(0,1)} \left[\exp \left(i b_{rc} \cdot (x_{ic} - x_{jc}) \right) \right] \\ &= \prod_{c=1}^{d_0} \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp \left(i b_{rc} \cdot (x_{ic} - x_{jc}) \right) \exp \left(\frac{b_{rc}^2}{2} \right) d\nu(b_{rc}) \\ &= \prod_{c=1}^{d_0} \exp \left(-\frac{(x_{ic} - x_{jc})^2}{2} \right) \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp \left(\frac{(b_{rc} - i(x_{ic} - x_{jc}))^2}{2} \right) d\nu(b_{rc}) \\ &= \prod_{c=1}^{d_0} \exp \left(-\frac{(x_{ic} - x_{jc})^2}{2} \right) = \exp \left(-\frac{1}{2} \|\mathbf{x}_i - \mathbf{x}_j\|_2^2 \right). \end{aligned}$$

This is just the Gaussian kernel where the length parameter is set to 1 [Rad+22b].

2. *ReLU activation*: this is the most common activation today, defined with:

$$\text{ReLU}(x) = \sigma(x) = \max(0, x). \quad (\text{E.24})$$

Applying the definition of expectation, the kernel value becomes:

$$\begin{aligned} \kappa(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{E}_{\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle \sim \mathcal{N}(\mathbf{0}, \Lambda)} \left[\left\langle \sigma(\mathbf{B}\mathbf{x}_i), \sigma(\mathbf{B}\mathbf{x}_j) \right\rangle \right] \\ &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \left\langle \sigma(\mathbf{B}\mathbf{x}_i), \sigma(\mathbf{B}\mathbf{x}_j) \right\rangle \mathbf{p} \left(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle : \mathbf{0}, \Lambda \right) d(\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle) d(\langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle), \end{aligned}$$

where the $\mathbf{p}(\cdot)$ is the Gaussian distribution of eqn (??). The complete calculation can be found in [Rad+22b]. The expectation of the kernel reduces to:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{\pi} \left(\langle \mathbf{x}_i, \mathbf{x}_j \rangle \left(\pi - \cos^{-1} \frac{\langle \mathbf{x}_i, \mathbf{x}_j \rangle}{\|\mathbf{x}_i\|_2 \|\mathbf{x}_j\|_2} \right) + \sqrt{\|\mathbf{x}_i\|_2^2 \|\mathbf{x}_j\|_2^2 - \langle \mathbf{x}_i, \mathbf{x}_j \rangle^2} \right), \quad (\text{E.25})$$

where $\|\mathbf{x}\|_2^2 = \langle \mathbf{x}, \mathbf{x} \rangle$. This is a periodic kernel with some variable shift. And note the values of the feature matrix $\mathbf{B}$ do not appear because we fixed $\hat{\mathbf{B}}$, so that $\text{cov}(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$.

3. *Dual ReLU activation*: recall in our case the data points lie on the surface of the unit sphere, so that their squared norm is 1: $\|\mathbf{x}\|_2^2 = \langle \mathbf{x}, \mathbf{x} \rangle = 1$. One can define a dual activation function $\tilde{\sigma}(\cdot) : [-1, 1] \rightarrow \mathbb{R}$, so that eqn (E.23) above simplifies to:

$$\begin{aligned} \kappa(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{E}_{\langle \hat{\mathbf{b}}_r, \mathbf{x}_i \rangle, \langle \hat{\mathbf{b}}_r, \mathbf{x}_j \rangle \sim \mathcal{N}(\mathbf{0}, \Lambda)} \left[\left\langle \tilde{\sigma}(\mathbf{B}\mathbf{x}_i), \tilde{\sigma}(\mathbf{B}\mathbf{x}_j) \right\rangle \right] \stackrel{(1)}{=} \tilde{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j) \\ \Lambda &= \begin{bmatrix} 1 & \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ \langle \mathbf{x}_j, \mathbf{x}_i \rangle & 1 \end{bmatrix}. \end{aligned} \quad (\text{E.26})$$

Where going into (1), we exchange the application the dual activation with the inner product with the following identity.

Proposition E.4.2. (*Hermite Polynomials*) Define the Hermite polynomials $\{h_i\}_{i \geq 0}$ as the orthonormal basis for the Hilbert space $L^2(\cdot)$ (see eqn (??)), with inner product $\langle f, g \rangle = \int_{\mathbb{R}} f(x)g(x)e^{-x^2/2}dx$, constructed by performing Gram-Schmidt orthogonalization on $L^2(\cdot)$ using the polynomials $\{x^i\}_{i \geq 0}$. If $\sigma(\mathbf{x}) = h_n(\mathbf{x})$ where $\mathbf{x}$ are on the surface of a unit sphere, then:

$$\ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j)^n \quad (\text{E.27})$$

for $\mathbf{x}_i^\top \mathbf{x}_j \in [-1, 1]$.

The proof is given in [ODo21, p. 336], and also be found in [Rad+22b]. And so the kernel in eqn (E.25) becomes:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{\pi} \left(\langle \mathbf{x}_i, \mathbf{x}_j \rangle \left(\pi - \cos^{-1} \langle \mathbf{x}_i, \mathbf{x}_j \rangle \right) + \sqrt{1 - \langle \mathbf{x}_i, \mathbf{x}_j \rangle^2} \right), \quad (\text{E.28})$$

and we see the substitution of $\langle \mathbf{x}, \mathbf{x} \rangle = 1$ [Rad+22b].

Remark E.4.3. (Double Descent Phenomenon and Over Parameterization) The most noteworthy aspect of running NNGP regression with ever-wider width of $\hat{\mathbf{B}}$ is that it captures the double-descent phenomenon frequently associated with deep ANNs.

That is the test error of the model continue to decrease long after training error have flat lined *if* the mode is sufficiently wide. This flies in the face of both conventional wisdom and traditional optimization theory, which states that high capacity model tend to over fit data by simply interpolating the training points, and therefore cannot generalize well (fig (??)). In ANNs, interpolation leads to generalization (see fig E.10).

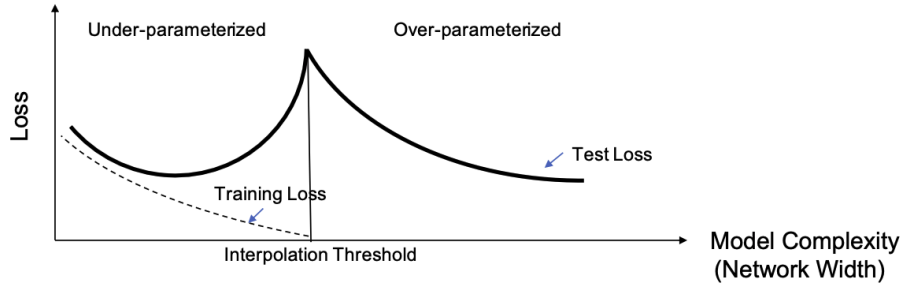


Figure E.10: Classical machine learning theory is allergic to the notion of interpolation, roughly defined as selecting a model whereby the number of parameters is equal or greater than the number of data points. For the model in eqn (E.18), interpolation threshold occurs at $d_1 = m$ for m training examples. Beyond this threshold, the model simply interpolates the points, which leads to over fitting. In the ANN setting however, the number of parameters may be greater than the number of data points. Moreover, as the width of the ANN expand beyond that of the interpolation threshold, test error continues to decrease. The test error is maximal at the threshold, a phenomenon that can be remedied by regularization via early stopping. This suggests that not only do large ANNs interpolate data, *interpolation leads to generalization* [[Bel21, p. 18],[Rad+22b]].

Remark E.4.4. (ANN Gaussian Process Regression) While kernalized least squares regression is an example of a parametric method, we can also use eqn (E.28) in non-parametric methods

such as $\mathcal{GP}$ regression. Unlike parametric methods, $\mathcal{GP}$ regression not only predicts the mean $y = \mathbf{E}[f(\mathbf{x}; \mathbf{w})]$ value, but also outputs an uncertainty around the prediction. This is accomplished by sampling from the space of all functions at the test point $\mathbf{x}_t$ after posterior updates on the training data. In particular, the mean of the sampled functions is the least squares solution, while the variance around the sample is determined by the kernel matrix built from training and test data. Notation wise, given test data $\mathbf{x}_t$, and training data $(\mathbf{X}, \mathbf{y})$, the expected value of y is written:

$$f(\mathbf{x}_t; \mathbf{w}) \mid \mathbf{X}, \mathbf{y}, \mathbf{x}_t \sim \mathcal{GP}\left(\mathbf{E}[f(\mathbf{x}; \mathbf{w})], \kappa(\mathbf{x}_i, \mathbf{x}_t)\right) \quad \mathbf{x}_i \in \mathbf{X}.$$

Here $\kappa(\cdot, \cdot)$ is the kernel function that defines the "uncertainty" of $\mathcal{GP}$ regression [RW06, p. 13].⁶ Now given training set $(\mathbf{X}, \mathbf{y})$ and test set $\mathbf{X}_t$, so that all $\mathbf{x}$'s are properly normalized and zero-meaned, we can construct kernel matrices from the data with:

$$\begin{aligned} \mathbf{K}_{\mathcal{GP}} &= \begin{bmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^\top & \mathbf{K}_{**} \end{bmatrix} + \sigma^2 \mathbf{I} \\ \mathbf{K}_{ij} &= \kappa(\mathbf{x}_i, \mathbf{x}_j), \quad \mathbf{x}_i, \mathbf{x}_j \in \mathbf{X} \\ (\mathbf{K}_*)_{ij} &= \kappa(\mathbf{x}_i, \mathbf{x}_j), \quad \mathbf{x}_i \in \mathbf{X}, \mathbf{x}_j \in \mathbf{X}_t, \quad (\mathbf{K}_{**})_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j), \quad \mathbf{x}_i \in \mathbf{X}_t, \mathbf{x}_j \in \mathbf{X}_t. \end{aligned}$$

Here the ij th entry of every matrix is computed with the kernel κ defined in eqn (E.28). The constant σ is some normalization factor so that all the matrices are invertible. Next we can define the regression weights with $\mathbf{A} = (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}$ per eqn (E.9). So the neural network $\mathcal{GP}$ regression is written:

$$\begin{aligned} f(\mathbf{X}_t) \mid \mathbf{X}, \mathbf{y}, \mathbf{X}_t &\sim \mathcal{GP}(\boldsymbol{\mu}_p, \boldsymbol{\Sigma}_p) \\ \boldsymbol{\mu}_p &= \mathbf{K}_*^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y} \quad \boldsymbol{\Sigma}_p = (\mathbf{K}_{**} + \sigma^2 \mathbf{I}) - \mathbf{K}_*^\top (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{K}_*. \end{aligned} \quad (\text{E.29})$$

Observe no training is required here, all that is needed is data fusion using the mean and kernel matrix expressions derived in Appendix ???. In particular, the kernel function κ is defined entirely by eqn (E.28), and is constant during the equivalent ANN training path that would occur via gradient descent [LB18].

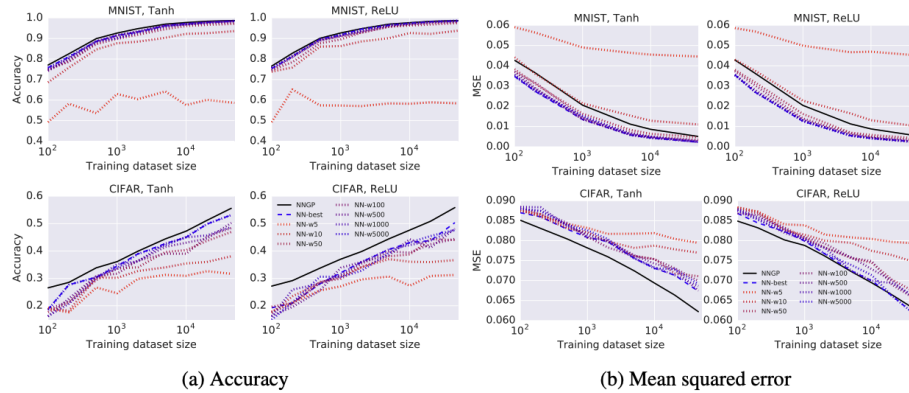


Figure E.11: Test accuracy and mean squared error on MNIST and CIFAR-10 dataset are shown for the best performing NNGP and best performing ANN for given width. ‘NN-best’ denotes the best performing (on the validation set) neural network across all widths and trials. $\mathcal{GP}$ regression often outperforms finite width networks. And as the width of ANNs increase, their performance approaches that of $\mathcal{GP}$ regression [LB18].

⁶ $\mathcal{GP}$ regression is a large topic, a primer can be found in Appendix ???. In particular, subsection "One Loop in Gaussian Process Regression" shows how to update regression mean and variance after data fusion.

Remark E.4.5. (Alternate derivation of kernel function) In eqn (E.20), the function κ is written using the definition of the kernel via eqn (E.10). There is an alternate derivation of the kernel from the expression of the ANN itself. Given the function $f(x; \mathbf{w}) = \mathbf{A}\boldsymbol{\sigma}(\hat{\mathbf{B}}x)$ with fixed $\hat{\mathbf{B}}$. And for simplicity sake, assume $x \in \mathbb{R}$, $\mathbf{X} \in \mathbb{R}^{1 \times m}$, $\mathbf{B} \in \mathbb{R}^{1 \times 2}$, and $\mathbf{A} \in \mathbb{R}^{2 \times 1}$. Then we can write $\mathbf{w}$ as:

$$\mathbf{w} = [\mathbf{A}_1, \mathbf{A}_2, \hat{\mathbf{B}}_1, \hat{\mathbf{B}}_2] = (\alpha_1, \alpha_2, \hat{b}_1, \hat{b}_2).$$

Now given some data point x_i , we can rewrite f with variable weights $\mathbf{w}$ at fixed x_i as:

$$f(\mathbf{w}; x_i) = (\alpha_1, \alpha_2) \boldsymbol{\sigma} \left((\hat{b}_1, \hat{b}_2)^\top x_i \right) = \alpha_1 \boldsymbol{\sigma}(\hat{b}_1 x_i) + \alpha_2 \boldsymbol{\sigma}(\hat{b}_2 x_i) = \sum_{r=1}^{d_1} \alpha_r \boldsymbol{\sigma}(\hat{b}_r x_i).$$

Note this is stating f of $\mathbf{w}$ given *parameter* x , and the sum is over d_1 , not the number of training points m . The concept should appear natural as it is just a variation of how Bayesian people express their ideas. The purpose of writing $f(\mathbf{w}; x_i)$ is to take the gradient of f w.r.t $\mathbf{w}$, not x . In gradient descent, we minimize this loss function:

$$\text{loss}(\mathbf{w}) = \sum_{i=1}^m (y_i - f(\mathbf{w}; x_i))^2.$$

Where $f(\hat{\mathbf{w}}; x_i)$ is initialized at some $\hat{\mathbf{w}}$, so that after some delta we have $\mathbf{w} = \hat{\mathbf{w}} + \delta$. This $f(\mathbf{w}; x_i)$ can be computed using the n th order Taylor expansion:

$$f(\mathbf{w}; x_i) = f(\hat{\mathbf{w}}; x_i) + \underbrace{\left\langle \nabla f(\hat{\mathbf{w}}; x_i), (\mathbf{w} - \hat{\mathbf{w}}) \right\rangle}_{\text{gradient}} + \underbrace{\frac{1}{2} \left\langle (\mathbf{w} - \hat{\mathbf{w}}), \nabla^2 f(\hat{\mathbf{w}}; x_i) (\mathbf{w} - \hat{\mathbf{w}}) \right\rangle}_{\text{h.o.t}} + \dots$$

If we take the first order approximation with gradient only, and initialize training so that:

$$\hat{\mathbf{w}} = \mathbf{0} \quad f(\hat{\mathbf{w}}; x_i) = 0.$$

Then the first order approximation becomes:

$$f(\mathbf{w}; x_i) = f(\hat{\mathbf{w}}; x_i) + \left\langle \nabla f(\hat{\mathbf{w}}; x_i), (\mathbf{w} - \hat{\mathbf{w}}) \right\rangle = \langle \nabla f(\hat{\mathbf{w}}; x_i), \mathbf{w} \rangle.$$

The loss function over all the weights $\mathbf{w}$ is now rewritten:

$$\text{loss}(\mathbf{w}) = \sum_{i=1}^m (y_i - f(\mathbf{w}; x_i))^2 = \sum_{i=1}^m (y_i - \underbrace{\langle \nabla f(\hat{\mathbf{w}}; x_i), \mathbf{w} \rangle}_{\phi_{\mathbf{x}_i}})^2 \stackrel{(1)}{=} \sum_{i=1}^m (y_i - \langle \phi_{\mathbf{x}_i}, \mathbf{w} \rangle)^2.$$

Where going into (1) we note that $\nabla f(\hat{\mathbf{w}}; x_i)$ is in fact the corresponding feature map of x_i . So we can define the kernel using *just the gradient* with:

$$\kappa(x_i, x_j) = \phi_{\mathbf{x}_i}^\top \phi_{\mathbf{x}_j} = \underbrace{\left\langle \nabla f(\mathbf{w}; x_i), \nabla f(\mathbf{w}; x_j) \right\rangle}_{\text{neural tangent kernel}}.$$

Now we evaluate this kernel κ by considering $\nabla f(\mathbf{w}; x_i)$. Since $\hat{b}_1, \hat{b}_2$ are constant, we know $\frac{\partial f(\mathbf{w}; x_i)}{\partial \hat{b}_1} = 0$ and similarly for $\hat{b}_2$. So we simply take the gradient of f w.r.t $\mathbf{A}$ with:

$$\nabla_{\mathbf{A}} f(\mathbf{w}; x_i) = \frac{\partial f(\mathbf{w}; x_i)}{\partial \mathbf{A}} = \left(\frac{\partial f(\mathbf{w}; x_i)}{\partial \alpha_1}, \frac{\partial f(\mathbf{w}; x_i)}{\partial \alpha_2} \right) \quad (\text{E.30})$$

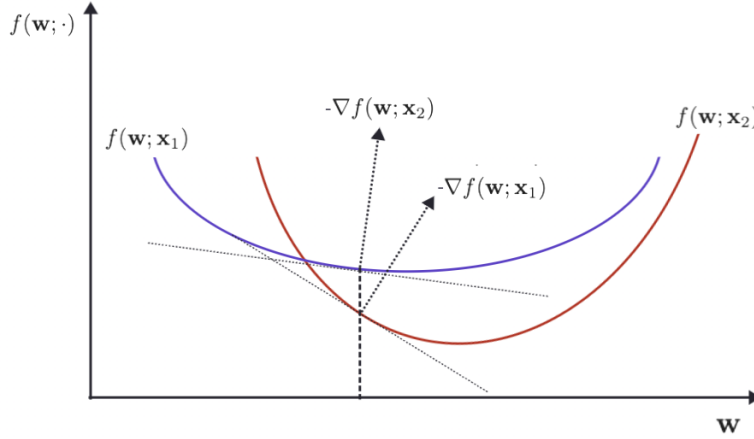


Figure E.12: Given two constants $\mathbf{x}_1$ and $\mathbf{x}_2$, two different functions of $f(\mathbf{w}; \mathbf{x}_1)$ and $f(\mathbf{w}; \mathbf{x}_2)$ can be drawn over the domain $\mathbf{w}$ of f . For every $\mathbf{w}$, the two functions have different gradients with $\nabla f(\mathbf{w}; \mathbf{x}_1)$ and $\nabla f(\mathbf{w}; \mathbf{x}_2)$. Their inner product is the neural tangent kernel, written: $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \langle \nabla f(\mathbf{w}; \mathbf{x}_1), \nabla f(\mathbf{w}; \mathbf{x}_2) \rangle$.

$$= \left(\sigma(\hat{b}_1 x_i), \sigma(\hat{b}_2 x_i) \right) = \sigma(\hat{\mathbf{B}} x_i), \quad (\text{E.31})$$

and similarly for $\nabla f(\mathbf{w}; x_j)$. Now note that in this case, all higher order terms (h.o.t) disappear as we have $\nabla_{\mathbf{A}}^2 f(\hat{\mathbf{w}}; x_j) = 0$, e.t.c. Thus the first order Taylor expansion of f is sufficient to define the kernel with:

$$\kappa(x_i, x_j) = \left\langle \nabla f(\mathbf{w}; x_i), \nabla f(\mathbf{w}; x_j) \right\rangle \sim \left\langle \sigma(\hat{\mathbf{B}} x_i), \sigma(\hat{\mathbf{B}} x_j) \right\rangle = \langle \phi_{\mathbf{x}_i}, \phi_{\mathbf{x}_j} \rangle,$$

which is what eqn (E.20) asserts. The point here is that the kernel of f can be derived by taking the gradient of f w.r.t $\mathbf{w}$, defined at two data points $\mathbf{x}_i$ and $\mathbf{x}_j$. This intuition will lead into the discussion for neural tangent kernel, see fig (E.12) for this remark in picture form.

Neural Tangent Kernel for Feed-Forward ANNs

The neural tangent kernel (NTK) is a natural extension of the NNGP of the previous section. However now we will consider two-layer feed-forward ANNs where all the parameters are learned. Recall when $\hat{\mathbf{B}}$ is fixed, the model $f(\mathbf{x}; \mathbf{w}) = \mathbf{A} \sigma(\hat{\mathbf{B}} \mathbf{x})$ reduces to a kernel regression problem so that:

1. the objective is convex;
2. by definition of eqn (E.10), we know the kernel is defined with $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi_{\mathbf{x}_i}, \phi_{\mathbf{x}_j} \rangle$, where $\phi_{\mathbf{x}_i} = \sigma(\hat{\mathbf{B}} \mathbf{x}_i)$.

In the case where both $(\mathbf{A}, \mathbf{B})$ are variable, 1) the objective is no longer convex, and 2) defining the feature map ϕ is more complicated, as during training the values of ϕ changes w.r.t regression parameters $\mathbf{A}$. The following proposition shows that in the limiting case where the width of $\mathbf{B}$ goes to infinity, there is a constant neural tangent kernel that can be derived even if all the ANN parameters are variable.

Proposition E.4.6. *The setting is similar to the previous section. Given:*

$$\begin{aligned}
 f : \mathbb{R}^{d_0} &\rightarrow \mathbb{R} \\
 f(\mathbf{x}; \mathbf{w}) &= \frac{1}{\sqrt{d_1}} \mathbf{A} \boldsymbol{\sigma}(\mathbf{B} \mathbf{x}) \quad \text{for } \mathbf{y} \in \mathbb{R} \quad \mathbf{x} \in \mathbb{R}^{d_0} \text{ with } \mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \\
 \text{where } \mathbf{A} &\in \mathbb{R}^{1 \times d_1} \quad \mathbf{B} \in \mathbb{R}^{d_1 \times d_0} \quad \mathbf{A}_{ij}, \mathbf{B}_{ij} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1) \\
 \boldsymbol{\sigma}(\cdot) : \mathbb{R} &\rightarrow \mathbb{R} \quad \ddot{\boldsymbol{\sigma}} : [-1, 1] \rightarrow \mathbb{R}.
 \end{aligned}$$

Then as $d_1 \rightarrow \infty$, the neural tangent kernel (NTK) is defined with:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \ddot{\boldsymbol{\sigma}}(\mathbf{x}_i^\top \mathbf{x}_j) + \ddot{\boldsymbol{\sigma}}'(\mathbf{x}_i^\top \mathbf{x}_j)(\mathbf{x}_i^\top \mathbf{x}_j), \quad (\text{E.32})$$

where $\ddot{\boldsymbol{\sigma}}'(\cdot)$ is the first derivative of $\ddot{\boldsymbol{\sigma}}$ w.r.t the terms inside.

Proof. This proof contains two parts: 1) using the gradient approximation of f to derive expression E.32; and 2) argue that the gradient is sufficient to express κ , since the Hessian goes to zero as the width of $\mathbf{B}$ increases. See fig (??) for intuition for the NTK.

1. Similar to remark E.4.5, we place all the parameters of $(\mathbf{A}, \mathbf{B})$ into a weights matrix of dimension $d_1 + d_1 \times d_0$:

$$\mathbf{w} = [\mathbf{A}_{11}, \dots, \mathbf{A}_{1d_1}, \mathbf{B}_{11}, \dots, \mathbf{B}_{d_0 d_1}] = [\alpha_1, \dots, \alpha_{d_1}, b_{11}, \dots, b_{d_0 d_1}], \quad (\text{E.33})$$

so that we have $f(\mathbf{w}; \mathbf{x}) : \mathbb{R}^{d_1 + d_0 d_1} \rightarrow \mathbb{R}$. Similar to eqn (E.21), we can write $f(\mathbf{w}; \mathbf{x})$ with dual activation $\ddot{\boldsymbol{\sigma}}$ as:

$$f(\mathbf{w}; \mathbf{x}) = \frac{1}{\sqrt{d_1}} \sum_{r=1}^{d_1} \alpha_r \ddot{\boldsymbol{\sigma}}(\langle \mathbf{b}_r, \mathbf{x} \rangle) = \frac{1}{\sqrt{d_1}} \left(\alpha_1 \ddot{\boldsymbol{\sigma}}(\mathbf{b}_1^\top \mathbf{x}) + \dots + \alpha_{d_1} \ddot{\boldsymbol{\sigma}}(\mathbf{b}_{d_1}^\top \mathbf{x}) \right).$$

Dropping the $1/\sqrt{d_1}$ for ease of notation, we take the gradient w.r.t $\{\alpha_r\}_{r \leq d_1}$:

$$\nabla_{\alpha_r} f(\mathbf{w}; \mathbf{x}) \sim \frac{\partial}{\partial \alpha_r} \left(\dots + \alpha_r \ddot{\boldsymbol{\sigma}}(\mathbf{b}_r^\top \mathbf{x}) + \dots \right) = \ddot{\boldsymbol{\sigma}}(\mathbf{b}_r^\top \mathbf{x}). \quad (\text{E.34})$$

Taking the gradient w.r.t $\{b_{rc}\}_{r \leq d_1, c \leq d_0}$:

$$\nabla_{b_{rc}} f(\mathbf{w}; \mathbf{x}) \sim \frac{\partial}{\partial b_{rc}} \left(\dots + \alpha_r \ddot{\boldsymbol{\sigma}}(\mathbf{b}_r^\top \mathbf{x}) + \dots \right) = \alpha_r \ddot{\boldsymbol{\sigma}}'(\mathbf{b}_r^\top \mathbf{x}) x_c.$$

Putting all the terms together, the gradient of f becomes:

$$\nabla f(\mathbf{w}; \mathbf{x}) = \frac{1}{\sqrt{d_1}} \left(\ddot{\boldsymbol{\sigma}}(\mathbf{b}_1^\top \mathbf{x}), \dots, \ddot{\boldsymbol{\sigma}}(\mathbf{b}_{d_1}^\top \mathbf{x}), \alpha_1 \ddot{\boldsymbol{\sigma}}'(\mathbf{b}_1^\top \mathbf{x}) x_1, \dots, \alpha_{d_1} \ddot{\boldsymbol{\sigma}}'(\mathbf{b}_{d_1}^\top \mathbf{x}) x_{d_0} \right).$$

Then the kernel for some fixed value of d_1 becomes:

$$\begin{aligned}
 \kappa(\mathbf{x}_i, \mathbf{x}_j) &= \left\langle \nabla f(\mathbf{w}; \mathbf{x}_i), \nabla f(\mathbf{w}; \mathbf{x}_j) \right\rangle \\
 &= \underbrace{\frac{1}{d_1} \sum_{r=1}^{d_1} \ddot{\boldsymbol{\sigma}}(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\boldsymbol{\sigma}}(\mathbf{b}_r^\top \mathbf{x}_j)}_{\text{NNGP}} + \underbrace{\frac{1}{d_1} \sum_{r=1}^{d_1} \sum_{c=1}^{d_0} \alpha_r^2 \cdot \ddot{\boldsymbol{\sigma}}'(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\boldsymbol{\sigma}}'(\mathbf{b}_r^\top \mathbf{x}_j) \cdot x_{ic} x_{jc}}_{\text{contribution from changing } \mathbf{B}}.
 \end{aligned}$$

Observe the first term under-bracketed is just the NNGP, which was derived in remark E.4.5. The second term with the double sums corresponds to contributions from the feature matrix $\mathbf{B}$ that is also updated via gradient descent.

Now we take the width of $\mathbf{B}$ to infinity with $d_1 \rightarrow \infty$, the NNGP part becomes:

$$\lim_{d_1 \rightarrow \infty} \left\langle \ddot{\sigma}(\mathbf{B}\mathbf{x}_i), \ddot{\sigma}(\mathbf{B}\mathbf{x}_j) \right\rangle = \ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j).$$

The equality follows as we are working with the dual activation with expectation and variance defined in eqn (E.26). Next we consider the contribution from $\mathbf{B}$:

$$\begin{aligned} & \lim_{d_1 \rightarrow \infty} \frac{1}{d_1} \sum_{r=1}^{d_1} \sum_{c=1}^{d_0} \alpha_r^2 \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_j) \cdot x_{ic} x_{jc} \\ &= \lim_{d_1 \rightarrow \infty} \frac{1}{d_1} \sum_{r=1}^{d_1} \alpha_r^2 \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_j) \sum_{c=1}^{d_0} x_{ic} x_{jc} \\ &= \lim_{d_1 \rightarrow \infty} \frac{1}{d_1} \sum_{r=1}^{d_1} \alpha_r^2 \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_j) \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ &= \mathbf{x}_i^\top \mathbf{x}_j \underbrace{\lim_{d_1 \rightarrow \infty} \frac{1}{d_1} \sum_{r=1}^{d_1} \alpha_r^2 \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_j)}_{\text{NNGP w/ } \ddot{\sigma}'(\cdot)} \end{aligned}$$

Now we examine NNGP w/ $\ddot{\sigma}'(\cdot)$ as $d_1 \rightarrow \infty$, consider:

$$\begin{aligned} & \lim_{d_1 \rightarrow \infty} \frac{1}{d_1} \sum_{r=1}^{d_1} \alpha_r^2 \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_j) \xrightarrow{p} \\ & \stackrel{(1)}{=} \mathbf{E}_{\alpha_r \sim \mathcal{N}(0,1), \mathbf{B} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\alpha_r^2 \left\langle \ddot{\sigma}'(\mathbf{B}\mathbf{x}_i), \ddot{\sigma}'(\mathbf{B}\mathbf{x}_j) \right\rangle \right] \\ & \stackrel{(2)}{=} \mathbf{E}_{\alpha_r \sim \mathcal{N}(0,1)} \left[\alpha_r^2 \right] \cdot \underbrace{\mathbf{E}_{\mathbf{B} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\langle \ddot{\sigma}'(\mathbf{B}\mathbf{x}_i), \ddot{\sigma}'(\mathbf{B}\mathbf{x}_j) \right\rangle \right]}_{\text{r.h.s}} \\ & \stackrel{(3)}{=} \ddot{\sigma}'(\mathbf{x}_i^\top \mathbf{x}_j). \end{aligned}$$

Where going into (1) is the consequence of the limit. Going into (2) we use the fact that α_r and $\mathbf{B}$ are independent variables. Going into (3) we note that $\mathbf{E}_{\alpha_r \sim \mathcal{N}(0,1)}[\alpha_r^2] = 1$, while the r.h.s is $\ddot{\sigma}'(\mathbf{x}_i^\top \mathbf{x}_j)$ by the same argument as E.26. Putting the above two expression together and we have the first order approximation of the kernel as: $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j) + \ddot{\sigma}'(\mathbf{x}_i^\top \mathbf{x}_j)(\mathbf{x}_i^\top \mathbf{x}_j)$ in the limit as expected.

Example E.4.7. (NTK for ReLU) We can complete the computation of NTK for a specific $\ddot{\sigma}$, namely the ReLU activation. Define appropriately normalized f with $f(\mathbf{x}; \mathbf{w}) = \frac{\sqrt{2}}{\sqrt{d_1}} \mathbf{A} \sigma(\mathbf{B}\mathbf{x})$.⁷ Its derivative is given by:

$$\ddot{\sigma}'(\mathbf{x}_i^\top \mathbf{x}_j) = \frac{1}{\pi} (\pi - \cos^{-1}(\mathbf{x}_i^\top \mathbf{x}_j)).$$

And so we have:

$$\begin{aligned} \kappa(\mathbf{x}_i, \mathbf{x}_j) &= \ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j) + \ddot{\sigma}'(\mathbf{x}_i^\top \mathbf{x}_j)(\mathbf{x}_i^\top \mathbf{x}_j) \\ &= \frac{1}{\pi} \left(\mathbf{x}_i^\top \mathbf{x}_j \left(\pi - \cos^{-1}(\mathbf{x}_i^\top \mathbf{x}_j) \right) + \sqrt{1 - (\mathbf{x}_i^\top \mathbf{x}_j)^2} \right) \\ &\quad + \frac{\mathbf{x}_i^\top \mathbf{x}_j}{\pi} (\pi - \cos^{-1}(\mathbf{x}_i^\top \mathbf{x}_j)), \end{aligned} \tag{E.35}$$

by using the prior expression for $\ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j)$ from E.28 [Rad+22d].

⁷The $\sqrt{2}$ is the normalization constant for $\ddot{\sigma}$.

2. In part two, we further the argument by stating that the first order approximation to the kernel in eqn (E.35) is sufficient. Recall in remark E.4.5, the Hessian is zero since $\nabla_{\mathbf{A}}^2 \sigma(\mathbf{B}\mathbf{x}) = 0$ as $\mathbf{B}$ is a constant by definition. In this case the $\mathbf{B}$ is variable, yet the claim is that:

- a) The Hessian $\nabla^2(\cdot)$ w.r.t the weights approaches zero as $d_1 \rightarrow \infty$, written:

$$\|\nabla^2(f(\mathbf{w}; \mathbf{x}))\|_2 = \mathcal{O}\left(\frac{1}{\sqrt{d_1}}\right). \quad (\text{E.36})$$

- b) So that given any particular training path via gradient descent, the neural tangent kernel is constant:

$$|\kappa(\mathbf{x}_i, \mathbf{x}_j)| = \mathcal{O}(1). \quad (\text{E.37})$$

- c) Thus the model f "transitions to linearity" as $d_1 \rightarrow \infty$ [LZB21], expressed by:

$$f(\mathbf{w}; \mathbf{x}) = f(\hat{\mathbf{w}}; \mathbf{x}) + \left\langle \nabla_{\mathbf{w}} f(\hat{\mathbf{w}}; \mathbf{x}), (\mathbf{w} - \hat{\mathbf{w}}) \right\rangle. \quad (\text{E.38})$$

Similar the gradient case, we have $f : \mathbb{R}^{d_1+d_0d_1} \rightarrow \mathbb{R}$, whose input ranges over the weights defined in E.33: $\mathbf{w} = [\alpha_1, \dots, \alpha_{d_1}, b_{11}, \dots, b_{d_0d_1}]$. The Hessian then is a square matrix of dimension $(d_1 + d_0d_1)^2$, and can be expressed in 2×2 block format:

$$\nabla^2(f(\mathbf{w}; \mathbf{x})) = \begin{bmatrix} \frac{\partial^2 f}{\partial \alpha_1 \partial \alpha_1} & \cdots & \frac{\partial^2 f}{\partial \alpha_1 \partial \alpha_{d_1}} & \left| & \frac{\partial^2 f}{\partial \alpha_1 \partial b_{11}} & \cdots & \frac{\partial^2 f}{\partial \alpha_1 \partial b_{d_0d_1}} \right. \\ \vdots & \ddots & \vdots & & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial \alpha_{d_1} \partial \alpha_1} & \cdots & \frac{\partial^2 f}{\partial \alpha_{d_1} \partial \alpha_{d_1}} & \left| & \frac{\partial^2 f}{\partial \alpha_{d_1} \partial b_{11}} & \cdots & \frac{\partial^2 f}{\partial \alpha_{d_1} \partial b_{d_0d_1}} \right. \\ \hline \frac{\partial^2 f}{\partial b_{11} \partial \alpha_1} & \cdots & \frac{\partial^2 f}{\partial b_{11} \partial \alpha_{d_1}} & \left| & \frac{\partial^2 f}{\partial b_{11} \partial b_{11}} & \cdots & \frac{\partial^2 f}{\partial b_{11} \partial b_{d_0d_1}} \right. \\ \vdots & \ddots & \vdots & & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial b_{d_0d_1} \partial \alpha_1} & \cdots & \frac{\partial^2 f}{\partial b_{d_0d_1} \partial \alpha_{d_1}} & \left| & \frac{\partial^2 f}{\partial b_{d_0d_1} \partial b_{11}} & \cdots & \frac{\partial^2 f}{\partial b_{d_0d_1} \partial b_{d_0d_1}} \right. \end{bmatrix}.$$

Recall the function is of form $f(\mathbf{w}; \mathbf{x}) \sim \sum_{r=1}^{d_1} \alpha_r \ddot{\sigma}(\langle \mathbf{b}_r, \mathbf{x} \rangle)$. Now observe:

- a) *Top left block*: all the terms are zero as it is taking the double derivative w.r.t. α_r :

$$\frac{\partial^2}{\partial \alpha_r \partial \alpha_s} (\dots + \alpha_r \ddot{\sigma}(\langle \mathbf{b}_r, \mathbf{x} \rangle) + \dots) = 0.$$

- b) *Top right block*: all non-diagonal terms are zero. Along the diagonal we have:

$$\frac{\partial^2}{\partial \alpha_r \partial b_{rr}} (\dots + \alpha_r \ddot{\sigma}(\langle \mathbf{b}_r, \mathbf{x} \rangle) + \dots) = \frac{\partial}{\partial b_{rr}} \ddot{\sigma}(\langle \mathbf{b}_r, \mathbf{x} \rangle) = \ddot{\sigma}'(\langle \mathbf{b}_r, \mathbf{x} \rangle) x_r.$$

- c) *Bottom left block*: this is the same as the top right block. So the non-diagonal terms are zero, and all the diagonal terms are the same as above.

- d) *Bottom right block*: the non-diagonal terms are zero again. Along the diagonal we have:

$$\frac{\partial^2}{\partial b_{rr} \partial b_{rr}} (\dots + \alpha_r \ddot{\sigma}(\langle \mathbf{b}_r, \mathbf{x} \rangle) + \dots) = \frac{\partial}{\partial b_{rr}} \alpha_r \ddot{\sigma}'(\langle \mathbf{b}_r, \mathbf{x} \rangle) x_r = \alpha_r \ddot{\sigma}''(\langle \mathbf{b}_r, \mathbf{x} \rangle) x_r^2.$$

All in all, the Hessian is sparse: where the top left block are all zeros, and in the other three blocks only the diagonal terms have non-zero values. Now the spectral norm of the Hessian can be written:

$$\begin{aligned} \|\nabla^2(f(\mathbf{w}; \mathbf{x}))\|_2 &= \frac{1}{\sqrt{d_1}} \max_{r \in d_1} \left| \frac{\alpha_r \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}) x_r^2 + \sqrt{\alpha_r^2 \ddot{\sigma}''(\mathbf{b}_r^\top \mathbf{x}) x_r^4 + 4 \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x})^2 x_r^2}}{2} \right| \\ &\stackrel{(1)}{=} \frac{C}{\sqrt{d_1}} = \mathcal{O}\left(\frac{1}{\sqrt{d_1}}\right), \end{aligned}$$

where going into (1), we use the fact that both $\mathbf{b}_r$ and $\mathbf{x}$ are bounded (see [LZB21, p. 6], and [Rad+22d, p. 8] for eigenvalue computation). Next consider claim E.37, the kernel can be written with ANN parameters:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{d_1} \left(\sum_{r=1}^{d_1} \ddot{\sigma}(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\sigma}(\mathbf{b}_r^\top \mathbf{x}_j) + \mathbf{x}_i^\top \mathbf{x}_j \sum_{r=1}^{d_1} \alpha_r^2 \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_i) \cdot \ddot{\sigma}'(\mathbf{b}_r^\top \mathbf{x}_j) \right).$$

Observe the kernel value is normalized by $1/d_1$, thus it remains constant as $d_1 \rightarrow \infty$. Therefore $|\kappa(\mathbf{x}_i, \mathbf{x}_j)| = \mathcal{O}(1)$, while Hessian of f approaches 0 as $d_1 \rightarrow \infty$. All in all, this implies the NTK becomes constant, and the the gradient is sufficient to characterize $f(\mathbf{w}; \mathbf{x})$ as E.38 states. Thus solving ANN with stochastic gradient descent is equivalent to solving kernel regression with the NTK [[LZB21] [Rad+22d]].

■

Remark E.4.8. (Limitations of proposition E.4.6) Note the function in E.4.6 is of form $f(\mathbf{x}; \mathbf{w}) = \mathbf{A}\sigma(\mathbf{B}\mathbf{x})$, that is it does not have nonlinear activation function applied to the output. If however f is defined such that $f(\mathbf{x}; \mathbf{w}) = \sigma(\mathbf{A}\sigma(\mathbf{B}\mathbf{x}))$, then expression E.38 would fail, as the Hessian elements would have additional terms from the nonlinear transformation on the output of the regression. That is to say the Hessian does not tend to zero as $d_1 \rightarrow \infty$, and the NTK does not stay constant throughout the training path. This observation rules out some ANNs from the analysis framework developed thus far, i.e. ones where the last output layer is logistic regression. Similarly, if a narrow "bottleneck" layer is added to the ANN, then even if the bottle neck layer is linear, the Hessian term would be large compared to the gradient term, and transition to linearity of the ANN would be lost. This is important as in many ANN architectures, "down-sampling" layers are interspersed throughout to "shake out" any spurious information that may accumulate by projecting the information onto higher dimensions [LZB21].

NTK for Deep Feed-Forward ANNs

This section derives the NNGP and NTK for fully connected, feed-forward ANNs.

Proposition E.4.9. *(NTK for Fully Connected Networks) Let f be an ANN with L hidden layers defined on $\mathbf{x}$'s and some $d_0 \geq 1$ s.t.:*

$$\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^{d_0 \times 1} \quad f(\mathbf{x}; \mathbf{w}) : \mathbb{R}^{d_0} \rightarrow \mathbb{R},$$

Now defined the $L + 1$ weights and element-wise activation function:

$$\mathbf{w} = [(\mathbf{W}^{L+1})_r, \dots, (\mathbf{W}^l)_{rc}, \dots, (\mathbf{W}^1)_{rc}] \quad \mathbf{W}^l \in \mathbb{R}^{d_l \times d_{l-1}} \quad \mathbf{W}^{L+1} \in \mathbb{R}^{1 \times d_L} \quad \sigma(\cdot) : \mathbb{R} \rightarrow \mathbb{R},$$

where $0 \leq l \leq L + 1$. And similar to E.4.6, redefine the function so that it is parameterized by some specific $\mathbf{x}$:

$$\begin{aligned} f : \mathbb{R}^{|\mathbf{w}|} &\rightarrow \mathbb{R} \\ f(\mathbf{w}; \mathbf{x}) &= \mathbf{W}^{L+1} \frac{1}{\sqrt{d_L}} \sigma \left(\mathbf{W}^L \frac{1}{\sqrt{d_{L-1}}} \sigma \left(\dots \mathbf{W}^2 \frac{1}{\sqrt{d_1}} \sigma(\mathbf{W}^1 \mathbf{x}) \dots \right) \right). \end{aligned}$$

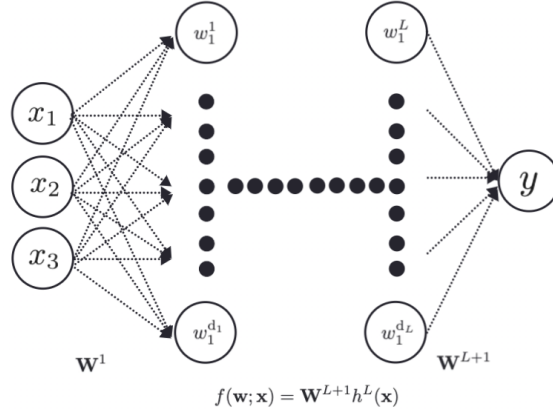


Figure E.13: Deep feed-forward ANN defined by function $f : \mathbb{R}^3 \rightarrow \mathbb{R}$, with $L + 1$ layers $\mathbf{w} = [\mathbf{W}^1, \dots, \mathbf{W}^L]$, and L hidden layers with functions $h^1(\mathbf{x}), \dots, h^L(\mathbf{x})$.

Alternatively we can write the function above in recurrence form with hidden layer functions $\{h^l\}_{0 \leq l \leq L}$, so that the 0th layer is the data $\mathbf{x} \in \mathbb{R}^{d_0 \times 1}$:

$$f(\mathbf{w}; \mathbf{x}) = \mathbf{W}^{L+1} h^L(\mathbf{w}; \mathbf{x}) \quad h^l(\mathbf{w}; \mathbf{x}) = \frac{1}{\sqrt{d_l}} \sigma(\mathbf{W}^l h^{l-1}(\mathbf{w}; \mathbf{x})) \quad h^0(\mathbf{w}; \mathbf{x}) = \mathbf{x}. \quad (\text{E.39})$$

Then as $d_1, \dots, d_L \rightarrow \infty$ in order, the NNGP and NTK are given by kernels τ and κ :

$$\begin{aligned} \tau^L(\mathbf{x}_i, \mathbf{x}_j) &= \ddot{\sigma}(\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j)) & \kappa^L(\mathbf{x}_i, \mathbf{x}_j) &= \tau^L(\mathbf{x}_i, \mathbf{x}_j) + \kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j) \ddot{\sigma}'(\kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j)) \\ \tau^0(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{x}_i^\top \mathbf{x}_j & \kappa^0(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{x}_i^\top \mathbf{x}_j. \end{aligned} \quad (\text{E.40})$$

Where the dual activation $\ddot{\sigma}'(\cdot)$ is defined so that $\ddot{\sigma}'(1) = 1$.

Proof. The proof is provided in [Rad+22c] and proceeds by induction.

1. *Base Case:* Let $L = 1$, this is the case in E.32. So the kernel is defined for the two layer feed-forward ANN, and we can directly write the kernels at the 0th (read: the data) and 1st layer with:

$$\tau^0(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j \quad \tau^1(\mathbf{x}_i, \mathbf{x}_j) = \ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j) \quad \kappa^0(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j.$$

And so similar to E.32 we have:

$$\begin{aligned} \kappa^1(\mathbf{x}_i, \mathbf{x}_j) &= \tau^1(\mathbf{x}_i, \mathbf{x}_j) + \kappa^0(\mathbf{x}_i, \mathbf{x}_j) \ddot{\sigma}'(\tau^0(\mathbf{x}_i, \mathbf{x}_j)) \\ &= \ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j) + (\mathbf{x}_i^\top \mathbf{x}_j) \ddot{\sigma}'(\mathbf{x}_i^\top \mathbf{x}_j), \end{aligned} \quad (\text{E.41})$$

per claim E.40.

2. *Inductive Case:* let $L > 1$, so that as $d_1, \dots, d_L \rightarrow \infty$, we can assume this recurrence exists:

$$\begin{aligned} \kappa^L(\mathbf{x}_i, \mathbf{x}_j) &\stackrel{(1)}{=} \tau^L(\mathbf{x}_i, \mathbf{x}_j) + \kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j) \ddot{\sigma}'(\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j)) \\ \tau^L(\mathbf{x}_i, \mathbf{x}_j) &\stackrel{(2)}{=} \ddot{\sigma}(\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j)) \\ \tau^0(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{x}_i^\top \mathbf{x}_j \quad \kappa^0(\mathbf{x}_i, \mathbf{x}_j) = \ddot{\sigma}'(\mathbf{x}_i^\top \mathbf{x}_j). \end{aligned} \quad (\text{E.42})$$

Next we will prove equivalence (1) and (2) by deconstructing f layer-by-layer: by taking the derivatives of each layer l w.r.t all the terms in that layer, whilst holding the $\{l-1, \dots, 1\}$ layers constant. Similar to E.33, the weight terms can be unrolled as:

$$\mathbf{w}^{L+1} = \underbrace{[w_{11}^{L+1}, \dots, w_{d_L+1}^{L+1}]}_{\mathbf{W}^{L+1}} \underbrace{[w_{11}^L, \dots, w_{d_L d_{L-1}}^L]}_{\mathbf{W}^L} \underbrace{[w_{11}^{L-1}, \dots, w_{11}^1, \dots, w_{d_1 d_0}^1]}_{\mathbf{w}^{L-1} = [\mathbf{W}^{L-1}, \dots, \mathbf{W}^1]}, \quad (\text{E.43})$$

$$\mathbf{w}^L = [\mathbf{W}^L, \dots, \mathbf{W}^1]$$

where $\mathbf{w} = \mathbf{w}^{L+1}$, and $\mathbf{w}^{L-1}$ is the set of weights without the terms from $\mathbf{W}^{L+1}$ and $\mathbf{W}^L$. Now similar to E.34, we can expand $f(\mathbf{w}^{L+1}; \mathbf{x})$ into its terms and apply it to some $\mathbf{x} = (x_1, \dots, x_{d_0})$. Dropping the normalization term, we can write then entire ANN as:

$$f(\mathbf{w}^{L+1}; \mathbf{x}) \sim w_{11}^{L+1} \left(\ddot{\sigma}(w_{11}^L \ddot{\sigma}(\dots \ddot{\sigma}(w_{11}^1 x_1 + \dots + w_{d_1 d_0}^1 x_{d_0}))) \right) + \dots + w_{d_L+1}^{L+1} \left(\ddot{\sigma}(w_{11}^L \ddot{\sigma}(\dots)) \right).$$

Now we can take the gradient w.r.t each term in E.43, and place them into a $1 \times |\mathbf{w}|$ vector $\nabla f(\mathbf{w}^{L+1}; \mathbf{x})$. The kernel at the $(L+1)$ th layer is:

$$\begin{aligned} \kappa^L(\mathbf{x}_i, \mathbf{x}_j) &= \left\langle \nabla f(\mathbf{w}^{L+1}; \mathbf{x}), \nabla f(\mathbf{w}^{L+1}; \mathbf{x}) \right\rangle \\ &= \underbrace{\frac{1}{d_L} \sum_{r=1}^{d_L+1} \nabla_{w_{1r}^{L+1}} f(\mathbf{w}^{L+1}; \mathbf{x}_i) \cdot \nabla_{w_{1r}^{L+1}} f(\mathbf{w}^{L+1}; \mathbf{x}_j)}_{\text{top layer}} + \underbrace{\left\langle \nabla_{\mathbf{w}^L} f(\mathbf{w}^L; \mathbf{x}_i), \nabla_{\mathbf{w}^L} f(\mathbf{w}^L; \mathbf{x}_j) \right\rangle}_{\text{bottom } L \text{ layers}}, \end{aligned}$$

where $f(\mathbf{w}^L; \mathbf{x}_i) = \mathbf{W}^{L+1} h^L(\mathbf{w}^L; \mathbf{x})$. Now we consider the two terms and take the width of the layers to the limit.

a) *The top layer* is the neural network Gaussian process. Consider:

$$\begin{aligned} \lim_{d_L \rightarrow \infty} \frac{1}{d_L} \sum_{r=1}^{d_L+1} \nabla_{w_{1r}^{L+1}} f(\mathbf{w}^{L+1}; \mathbf{x}_i) \cdot \nabla_{w_{1r}^{L+1}} f(\mathbf{w}^{L+1}; \mathbf{x}_j) \\ \stackrel{(1)}{=} \lim_{d_L \rightarrow \infty} \frac{1}{d_L} \sum_{r=1}^{d_L} \ddot{\sigma}(\langle \mathbf{w}_r^L, h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i) \rangle) \cdot \ddot{\sigma}(\langle \mathbf{w}_r^L, h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j) \rangle) \\ \stackrel{(2)}{=} \mathbf{E}_{\mathbf{W}^L \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\langle \ddot{\sigma}(\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i)), \ddot{\sigma}(\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j)) \right\rangle \right] \\ \stackrel{(3)}{=} \mathbf{E}_{(u,v) \sim \text{iid } \mathcal{N}(\mathbf{0}, \Lambda)} \left[\left\langle \ddot{\sigma}(u), \ddot{\sigma}(v) \right\rangle \right] \end{aligned}$$

$$\Lambda^{L-1} = \begin{bmatrix} \tau^{L-1}(\mathbf{x}_i, \mathbf{x}_i) & \tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j) \\ \tau^{L-1}(\mathbf{x}_j, \mathbf{x}_i) & \tau^{L-1}(\mathbf{x}_j, \mathbf{x}_j) \end{bmatrix} \stackrel{(4)}{=} \begin{bmatrix} 1 & \tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j) \\ \tau^{L-1}(\mathbf{x}_j, \mathbf{x}_i) & 1 \end{bmatrix}.$$

Where going into (1), we take the gradient of $f(\mathbf{w}; \mathbf{x}) = \mathbf{W}^{L+1} h^L(\mathbf{w}; \mathbf{x})$ w.r.t its r th entry, so that every $s \neq r$ entry becomes zero. Going into (2), we use the law of large numbers and definition of inner product. Note that by assumption, the terms of $\mathbf{W}^L$ are drawn i.i.d via $w_{r1}^L \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1)$, so $\mathbf{W}^L \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Now by inductive hypothesis, $h^{L-1}(\mathbf{w}; \mathbf{x}_i)$ is a Gaussian process, so that $\mathbf{W}^L \cdot h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i)$ is a product of two Gaussian distributed vectors, therefore it is also Gaussian distributed. Thus by a similar argument to E.23, we take step (3): by setting $u = \mathbf{W}^L \cdot h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i)$, and $v = \mathbf{W}^L \cdot h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j)$, we can construct a new Gaussian distribution with mean $\mathbf{0}$ and covariance Λ^{L-1} over vector u, v . Going

into (4), we first use the inductive hypothesis to recall in the base case this is true: $\tau^0(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$. Then note that in dual activation, $\ddot{\sigma}(\mathbf{x}_i^\top \mathbf{x}_j) = 1$ if $\mathbf{x}_i$ is drawn from a Gaussian distribution with support on the surface of a sphere. Thus we have $\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j) = 1$. Thus we have the recurrence over vectors (u, v) :

$$\tau^L(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{E}_{(u,v) \sim \text{iid } \mathcal{N}(\mathbf{0}, \Lambda)} \left[\left\langle \ddot{\sigma}(u), \ddot{\sigma}(v) \right\rangle \right] = \ddot{\sigma}(\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j)),$$

which is what equality two of E.42 states.

- b) *The "bottom L layers:"* consider the gradient over the hidden layers $\nabla_{\mathbf{w}^L} f(\mathbf{w}^L; \mathbf{x}_i)$. In this case, we generalize to any layer $1 \leq l \leq L-1$. Now fix two indices within some $w_{st}^l = (\mathbf{W}^l)_{st}$: $s \in \{1, \dots, d_{l+1}\}$, and $t \in \{1, \dots, d_l\}$. Now consider the gradient of the ANN w.r.t the values at coordinate st :

$$\begin{aligned} \nabla_{w_{st}^l}(\mathbf{W}^{L+1} h^L(\mathbf{w}; \mathbf{x})) &\stackrel{(1)}{=} \frac{1}{\sqrt{d_L}} \sum_{r=1}^{d_L} \mathbf{w}_{r1}^{L+1} \ddot{\sigma}'((\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}))_r) \\ &\quad \cdot (\nabla_{w_{st}^l}(h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}))_r). \end{aligned}$$

Where going into (1), we expand the product of $\mathbf{W}^{L+1}$ with $h^L(\mathbf{w}; \mathbf{x})$ by the rows of $\mathbf{W}^{L+1} = [\mathbf{w}_{11}^{L+1}, \dots, \mathbf{w}_{d_L 1}^{L+1}]$. And then apply the chain rule to differentiate the activation function, followed by pushing the differential to the terms inside the differentiated $\ddot{\sigma}'(\dots)$. Now consider the inner product of the gradients of the above term:

$$\begin{aligned} &\lim_{d_L \rightarrow \infty} \left\langle \nabla_{w_{st}^l}(\mathbf{W}^{L+1} h^L(\mathbf{w}; \mathbf{x}_i)), \nabla_{w_{st}^l}(\mathbf{W}^{L+1} h^L(\mathbf{w}; \mathbf{x}_j)) \right\rangle \\ &\stackrel{(1)}{=} \lim_{d_L \rightarrow \infty} \frac{1}{d_L} \sum_{r, r'=1}^{d_L} \mathbf{w}_{r1}^{L+1} \ddot{\sigma}'((\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i))_r) \\ &\quad \cdot \mathbf{w}_{r'1}^{L+1} \ddot{\sigma}'((\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j))_{r'}) \\ &\quad \cdot \left\langle \nabla_{w_{st}^l} h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i), \nabla_{w_{st}^l} h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j) \right\rangle \\ &\stackrel{(2)}{=} \lim_{d_L \rightarrow \infty} \frac{1}{d_L} \sum_{r=1}^{d_L} (\mathbf{w}_{r1}^{L+1})^2 \cdot \ddot{\sigma}'((\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i))_r) \\ &\quad \cdot \ddot{\sigma}'((\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j))_r) \cdot \kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j) \\ &\stackrel{(3)}{=} \lim_{d_L \rightarrow \infty} \frac{1}{d_L} \sum_{r=1}^{d_L} \ddot{\sigma}'((\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i))_r) \cdot \ddot{\sigma}'((\mathbf{W}^L h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j))_r) \cdot \kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j) \\ &\stackrel{(4)}{=} \ddot{\sigma}'(\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j)) \cdot \kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j). \end{aligned}$$

Here going into (1), we apply the expanded definition of $\nabla_{w_{st}^l}(\mathbf{W}^{L+1} h^L(\mathbf{w}; \mathbf{x}))$ from above. The indices of the sum range over r, r' where when $r = r'$, the inner product is $\kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j)$, and zero when $r \neq r'$. Going into (2), note that $\langle \nabla_{w_{st}^l} h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_i), \nabla_{w_{st}^l} h^{L-1}(\mathbf{w}^{L-1}; \mathbf{x}_j) \rangle$ is the definition of kernel at layer $L-1$: $\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j)$. Going into (3), we note that in expectation $(\mathbf{w}_{r1}^{L+1})^2 = 1$, as the terms of $\mathbf{W}^{L+1}$ are drawn i.i.d from $\mathcal{N}(0, 1)$. Going into (4), we use the definition of the kernel again.

Finally we have:

$$\kappa^L(\mathbf{x}_i, \mathbf{x}_j) = \tau^L(\mathbf{x}_i, \mathbf{x}_j) + \kappa^{L-1}(\mathbf{x}_i, \mathbf{x}_j) \ddot{\sigma}'(\tau^{L-1}(\mathbf{x}_i, \mathbf{x}_j)) \quad \kappa^0(\mathbf{x}_i, \mathbf{x}_j) = 0$$

as desired [[LB+18], [AD+19], [AD+20]].

■

APPENDIX F

Exponential Family Distributions

This section provides a bare minimum introduction to the exponential family, how it is constructed, and the basic characteristics of its partition function. The question here is how to price the difference in quality between two models of the exponential family, and how various representations of exponential models can be used in said pricing.

F.1 Derivation

In the motivating setting, suppose there is a random vector $\mathbf{x} = \{x_1, \dots, x_m\}$ taking value over sample space Ω^m , with distribution $\mathbf{p}$. Now we wish to express the distribution $\mathbf{p}$ using a finite set of parameters. The natural step is to compute the *moments* of the distribution,

$$\hat{\mu}_\alpha = \varphi_\alpha(\mathbf{x}) := \frac{1}{n} \sum_{s=1}^m x_s^\alpha \quad (1 \leq \alpha \leq d).$$

The value $\hat{\mu}_\alpha$ is read "the α th degree empirical mean of the dataset". The φ_α are the *sufficient statistics* of the distribution of degree α , each one maps realizations of some r.v. onto reals, $\varphi_\alpha : \Omega^m \rightarrow \mathbb{R}$. For example $\varphi_2(\mathbf{x}) = \frac{1}{n} \sum_{s=1}^m x_s^2$, and note the sufficient statistics in the first and second degree are the sample mean and variance of the dataset. Now we wish to select the probability distribution $\mathbf{p}$ that generates the sample means *in expectation*, namely this condition must be satisfied:

$$\mathbf{E}_{\mathbf{p}}[\varphi_\alpha(\mathbf{x})] := \int_{\Omega^m} \varphi_\alpha(\mathbf{x}) \mathbf{p}(\varphi_\alpha(\mathbf{x})) \nu(d\mathbf{x}) = \hat{\mu}_\alpha \quad (1 \leq \alpha \leq d).$$

That is to say the expected value of the sufficient statistics under every possible sample of $\mathbf{x}$ is the same as the sample mean we observe from the current selection. This set of constraints is called the *moment matching conditions* of selecting the distribution $\mathbf{p}$. It turns out, there are many distributions $\mathbf{p}$ that lies within the boundaries defined by the moment-matching constraints, and the principle of minimum entropy is used to select the best $\mathbf{p}$. Recall entropy is $H(\mathbf{p}) = - \int \mathbf{p}(\mathbf{x}) \log(\mathbf{p}(\mathbf{x})) \nu(d\mathbf{x})$.

So the complete optimization problem solves for:

$$\mathbf{p}^* = \arg \max_{\mathbf{p} \in \mathfrak{P}} H(\mathbf{p}) \quad \text{subject to } \mathbf{E}_{\mathbf{p}}[\varphi_\alpha(\mathbf{x})] = \hat{\mu}_\alpha \quad (1 \leq \alpha \leq d).$$

Solving this program involves finding the minimum point $\mathbf{p}^*$ on the level sets of $H(\mathbf{p})$ that is incident to the curves traced out by the moment matching constraints. At the solution point, the gradient of $H(\mathbf{p})$ is equal to the gradient of the constraints up to some constant factor - that is we have $\nabla H(\mathbf{p}) = \theta_\alpha \nabla(\mathbf{E}_{\mathbf{p}}[\varphi_\alpha(\mathbf{x})] = \hat{\mu}_\alpha)$ for every α . The θ_α 's are Lagrange multipliers (although typically the symbol λ is used) that parameterize the distribution $\mathbf{p}$ alongside the sufficient statistics [BV04]. In the context of exponential family, the θ 's are also called *canonical parameters*. Solving for $\mathbf{p}^*$, we find the distribution of maximum entropy that agrees with the data is of form $\exp(\langle \theta, \varphi(\mathbf{x}) \rangle)$, with the proper normalization factor omitted here for clarity.

F.2 Canonical Representation

Formally, let the random vector $\mathbf{x} = (x_1, \dots, x_s, \dots, x_m)$ take value in the Cartesian product space $\Omega^m = \Omega_1 \times \dots \times \Omega_m$. Ω may be discrete, ie $\{0, 1\}$ or it may be continuous. Ω^m is endowed with measure ν defined as $d\nu = h(\mathbf{x})d\mathbf{x}$, where $h: \Omega^m \rightarrow \mathbb{R}$. In the discrete case, $\nu(d\mathbf{x})$ is simply the counting measure, in the continuous case it may be the Lebesgue measure. Now define the sufficient statistic $\varphi_\alpha: \Omega^m \rightarrow \mathbb{R}$ as $\varphi_\alpha(\mathbf{x}) = \frac{1}{n} \sum_{\mathbf{s}} \mathbf{x}_{\mathbf{s}}^\alpha$. And define $\boldsymbol{\varphi}: \Omega^m \rightarrow \mathbb{R}^d$ as $\boldsymbol{\varphi} = (\varphi_1, \dots, \varphi_d)$, a vector of functions that computes the sufficient statistics of values $\mathbf{x} \in \Omega^m$ up to the d th degree. A statistic $\boldsymbol{\varphi}(\mathbf{x})$ is sufficient w.r.t parameter $\boldsymbol{\theta}$, if $\mathbf{p}(\mathbf{x}|\boldsymbol{\varphi}(\mathbf{x}); \boldsymbol{\theta})$ does not depend on any value of this statistic $\boldsymbol{\varphi}(\mathbf{x})$. Next associate $\boldsymbol{\varphi}$ with the canonical parameters $\boldsymbol{\theta} = (\theta_1, \dots, \theta_d)$, then the *exponential family* is a collection of density functions:

$$\mathbf{p}(\mathbf{x}; \boldsymbol{\theta}) = \exp\left(\langle \boldsymbol{\varphi}(\mathbf{x}), \boldsymbol{\theta} \rangle - A(\boldsymbol{\theta})\right).$$

Where the normalization factor, or *log partition function* $A(\boldsymbol{\theta})$ is:

$$A(\boldsymbol{\theta}) = \log \int_{\Omega^m} \exp\left(\langle \boldsymbol{\varphi}(\mathbf{x}), \boldsymbol{\theta} \rangle\right) \nu(d\mathbf{x}).$$

This quantity is of particular interest in our case as it is the cost function. The function A has some important properties w.r.t its first and second derivatives that will be useful in the model market, namely the derivatives of A are the moments of the density, and A is convex.

Theorem F.2.1. *The log partition function has derivatives of all orders on its domain Θ , and the α th derivative of the partition function is the α th centered-moment of the sufficient statistic. In particular, the first two derivatives yield the cumulants of the random vector $\boldsymbol{\varphi}(\mathbf{x})$ with:*

$$\frac{\partial A}{\partial \theta_\alpha}(\boldsymbol{\theta}) = \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta})}[\varphi_\alpha(\mathbf{x})] = \int_{\Omega^m} \varphi_\alpha(\mathbf{x}) \mathbf{p}(\mathbf{x}; \boldsymbol{\theta}) \nu(d\mathbf{x}). \quad (\text{F.1})$$

$$\frac{\partial^2 A}{\partial \theta_\alpha \partial \theta_\beta}(\boldsymbol{\theta}) = \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta})}[\varphi_\alpha(\mathbf{x}) \varphi_\beta(\mathbf{x})] - \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta})}[\varphi_\alpha(\mathbf{x})] \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta})}[\varphi_\beta(\mathbf{x})]. \quad (\text{F.2})$$

Note the expectations are taken with respect to probability distribution parameterized by $\boldsymbol{\theta}$. We then have the Jacobian:

$$\nabla A(\boldsymbol{\theta}) = \left(\frac{\partial A}{\partial \theta_\alpha}(\boldsymbol{\theta}) : 1 \leq \alpha \leq d \right), \quad (\text{F.3})$$

and the Hessian matrix:

$$\nabla^2 A(\boldsymbol{\theta}) = \left[\frac{\partial^2 A}{\partial \theta_\alpha \partial \theta_\beta}(\boldsymbol{\theta}) : (1, 1) \leq (\alpha, \beta) \leq (d, d) \right]. \quad (\text{F.4})$$

Note the Hessian specifies the variance of the distribution. And because the variance is always > 0 , A must be convex in every $\boldsymbol{\theta}$. Now define the set Θ :

$$\Theta = \{\boldsymbol{\theta} \in \mathbb{R}^d : A(\boldsymbol{\theta}) < \infty\},$$

note Θ must be a convex set as A is a convex function. Θ is strictly convex if the representation of A is minimal, that is to say the density is not over-parameterized with more sufficient statistics φ_α than necessary [[WJ04], [WJ08]].

Example F.2.2 (Ising Model). The Ising model is a kind of Markov Random Field (MRF), whereby the graph $G = (V, E)$ has Bernoulli random variables x_s associated with every vertex $s \in V$. The edges on the graph is associated with weight θ_{st} for $s, t \in V$, and every vertex is associated with potential θ_s . The density for this model is then:

$$p(\mathbf{x}; \boldsymbol{\theta}) = \exp\left(\sum_{s \in V} \theta_s x_s + \sum_{(s,t) \in E} \theta_{st} x_s x_t - A(\boldsymbol{\theta})\right).$$

Note the canonical parameter characterize the graph with $\boldsymbol{\theta} = (\theta_1, \dots, \theta_{|V \cup E|})$ [WJ04].

F.3 Mean Representation

It is useful to define an alternate formulation of the exponential family using the *mean parameters* $\boldsymbol{\mu}$, with $A(\boldsymbol{\mu})$ as the partition function. Translating between the two representations will be of interest because although $A(\boldsymbol{\theta})$ is used to price the market updates, it is very difficult to evaluate in practice.

Since $A(\boldsymbol{\mu})$ is defined over the mean parameters, it is useful to name the set:

$$\mathcal{M} = \left\{ \boldsymbol{\mu} \in \mathbb{R}^d : \int_{\Omega^m} \boldsymbol{\varphi}(\mathbf{x}) p(\mathbf{x}) \nu(d\mathbf{x}) = \boldsymbol{\mu} \text{ for some } p \in \Omega \right\},$$

and notice the set Ω may include distributions that are not in the exponential family. However the probability distribution that maximizes entropy is from the exponential family. In the case where the r.v. X is discrete, we have:

$$\mathcal{M} = \left\{ \boldsymbol{\mu} \in \mathbb{R}^d : \sum_{\mathbf{x} \in \Omega^m} \boldsymbol{\varphi}(\mathbf{x}) p(\mathbf{x}) = \boldsymbol{\mu} \text{ for some } p(\mathbf{x}) > 0, \sum_{\mathbf{x} \in \Omega^m} p(\mathbf{x}) = 1 \right\}.$$

That is the mean parameter set is just the convex hull of the sufficient statistics, this follows from the fact that the expression $\sum_{\mathbf{x} \in \Omega^m} \boldsymbol{\varphi}(\mathbf{x}) p(\mathbf{x})$ is the convex combination of $\boldsymbol{\varphi}(\mathbf{x})$'s.

Example F.3.1 (Gaussian Markov Random Field). Returning to the MRF, the mean parameters of MRF are the mean vector $\boldsymbol{\mu} = \mathbf{E}[X]$ and the covariance $\Sigma = \mathbf{E}[XX^\top]$. Thus its mean parameter set is:

$$\mathcal{M} = \left\{ (\boldsymbol{\mu}, \Sigma) : \Sigma - \boldsymbol{\mu}^\top \boldsymbol{\mu} \succeq 0 \right\}.$$

And the density is:

$$p(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = h(\mathbf{x}) \exp\left(-\frac{1}{2} \text{tr}(\Sigma^{-1} \mathbf{x} \mathbf{x}^\top) + \boldsymbol{\mu}^\top \Sigma^{-1} \mathbf{x} - \frac{1}{2} A(\boldsymbol{\mu}, \Sigma)\right),$$

where $A(\boldsymbol{\mu}, \Sigma)$ and $h(\mathbf{x})$ are normalization factors. This is the more familiar representation of the Gaussian [WJ04].

F.4 Kullback-Leibler Divergence

KL divergence measures the distance between two distributions, this quantity will be used to price the contribution of successive updates on the underlying statistical model. There are many ways to introduce KL divergence, in this context it is illuminating to express it w.r.t. $A(\boldsymbol{\theta})$, and the hyperplanes that supports the epigraph of A at various $\boldsymbol{\theta}$'s along the curve. Referencing the figure F.1, there are $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$, which parametrize $p(\cdot; \boldsymbol{\theta}_1)$ and $p(\cdot; \boldsymbol{\theta}_2)$ respectively. Now suppose we want to find the "distance" between the two $\boldsymbol{\theta}$'s. The simplest way to do so is to integrate along the curve $A(\boldsymbol{\theta})$, but this would be a prohibitive

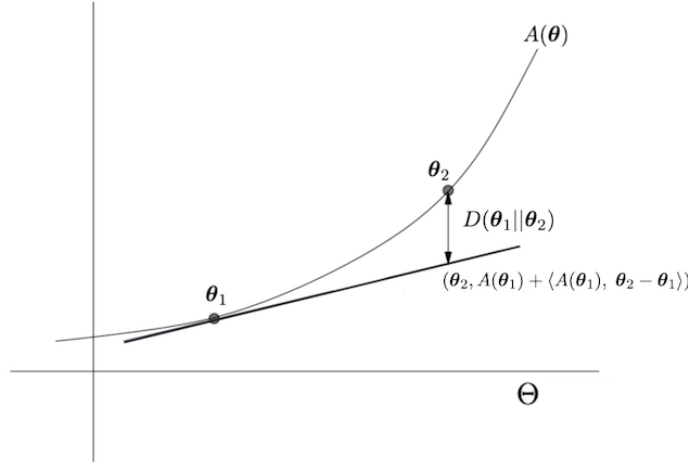


Figure F.1: The KL divergence of two distributions parameterized by θ is the gap between the hyperplane supporting θ_1 evaluated at the point θ_2 , and the curve $A(\theta_2)$. Observe if we fix θ_1 and move θ_2 so that it is close to right side of where $A(\theta)$ is defined, then the KL divergence grows very large [WJ04].

task depending on the form of A . Instead, we do a first order approximation by drawing a hyperplane from θ_1 with slope $\nabla A(\theta_1)$. The plane intersects θ_2 at the value:

$$A(\theta_1) + \left\langle \nabla A(\theta_1), \theta_2 - \theta_1 \right\rangle.$$

And we see there is a gap between the actual value of $A(\theta_2)$ and the value of the expression above (see fig (F.1)). This gap is the KL divergence, written:

$$\begin{aligned} D(\theta_1 || \theta_2) &= A(\theta_2) - \left(A(\theta_1) + \left\langle \nabla A(\theta_1), \theta_2 - \theta_1 \right\rangle \right) \\ &= A(\theta_2) - A(\theta_1) - \left\langle \nabla A(\theta_1), \theta_2 - \theta_1 \right\rangle \\ &= A(\theta_2) - A(\theta_1) - \left\langle \mu_1, \theta_2 - \theta_1 \right\rangle. \end{aligned} \quad (F.5)$$

Where the last line follows from the fact that by the moment matching condition, we have $\nabla A(\theta_1) = \mu_1$. This form is called the *primal form* of the KL divergence, or the *Bregman Divergence*. Alternatively we can arrive at the expression above starting from the more familiar definition of KL divergence:

$$D(q || p) = \int_{\Omega^m} q(\mathbf{x}) \log \frac{q(\mathbf{x})}{p(\mathbf{x})} \nu(d\mathbf{x}) = \mathbf{E}_q \left[\log \frac{q(\mathbf{x})}{p(\mathbf{x})} \right]. \quad (F.6)$$

Applying the definition of p , we have:

$$\begin{aligned} D(p(\mathbf{x}; \theta_1) || p(\mathbf{x}; \theta_2)) &= \mathbf{E}_{p(\cdot; \theta_1)} \left[\log \frac{p(\mathbf{x}; \theta_1)}{p(\mathbf{x}; \theta_2)} \right] \\ &= \mathbf{E}_{p(\cdot; \theta_1)} \left[\log(p(\mathbf{x}; \theta_1)) - \log(p(\mathbf{x}; \theta_2)) \right] \\ &= \mathbf{E}_{p(\cdot; \theta_1)} \left[\left(\langle \varphi(\mathbf{x}), \theta_1 \rangle - A(\theta_1) \right) - \left(\langle \varphi(\mathbf{x}), \theta_2 \rangle - A(\theta_2) \right) \right] \end{aligned}$$

$$\begin{aligned}
&= \mathbf{E}_{\mathbf{p}(\cdot; \boldsymbol{\theta}_1)} \left[\left(\langle \boldsymbol{\varphi}(\mathbf{x}), \boldsymbol{\theta}_1 \rangle - \langle \boldsymbol{\varphi}(\mathbf{x}), \boldsymbol{\theta}_2 \rangle \right) - \left(\mathbf{A}(\boldsymbol{\theta}_1) - \mathbf{A}(\boldsymbol{\theta}_2) \right) \right] \\
&= \int_{\Omega^m} \mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_1) \left(\langle \boldsymbol{\varphi}(\mathbf{x}), \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2 \rangle - (\mathbf{A}(\boldsymbol{\theta}_1) - \mathbf{A}(\boldsymbol{\theta}_2)) \right) \nu(d\mathbf{x}) \\
&= \mathbf{A}(\boldsymbol{\theta}_2) - \mathbf{A}(\boldsymbol{\theta}_1) + (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2) \int_{\Omega^m} \boldsymbol{\varphi}(\mathbf{x}) \mathbf{p}(\mathbf{x}; \boldsymbol{\theta}_1) \nu(d\mathbf{x}) \\
&= \mathbf{A}(\boldsymbol{\theta}_2) - \mathbf{A}(\boldsymbol{\theta}_1) - \langle \boldsymbol{\mu}_1, \boldsymbol{\theta}_2 - \boldsymbol{\theta}_1 \rangle.
\end{aligned}$$

Where going to last line, we use the fact that the expected value of the sufficient statistics under density $\mathbf{p}(\cdot; \boldsymbol{\theta}_1)$ is $\boldsymbol{\mu}_1$ by the moment matching constraints. And thus the two expressions of KL divergence are in correspondence. Alternatively we can express the divergence in terms of its dual form:

$$\mathbf{D}(\boldsymbol{\mu}_1 || \boldsymbol{\mu}_2) = \mathbf{A}^*(\boldsymbol{\mu}_1) - \mathbf{A}^*(\boldsymbol{\mu}_2) - \langle \boldsymbol{\theta}_1, \boldsymbol{\mu}_1 - \boldsymbol{\mu}_2 \rangle,$$

and note the signs are flipped because $\mathbf{A}(\boldsymbol{\theta})$ is dual with $-\mathbf{A}^*(\boldsymbol{\mu})$.

We can also start from $\boldsymbol{\theta}_2$ and find its distance to $\boldsymbol{\theta}_1$:

$$\begin{aligned}
&\mathbf{A}(\boldsymbol{\theta}_2) + \langle \nabla \mathbf{A}(\boldsymbol{\theta}_2), \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2 \rangle && \text{(hyperplane from } \boldsymbol{\theta}_2 \text{)} \\
&\mathbf{D}(\boldsymbol{\theta}_2 || \boldsymbol{\theta}_1) = \mathbf{A}(\boldsymbol{\theta}_1) - (\mathbf{A}(\boldsymbol{\theta}_2) + \langle \nabla \mathbf{A}(\boldsymbol{\theta}_2), \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2 \rangle) && \text{(KL divergence from } \boldsymbol{\theta}_2 \text{ to } \boldsymbol{\theta}_1 \text{)}.
\end{aligned}$$

Notice $\mathbf{D}(\boldsymbol{\theta}_1 || \boldsymbol{\theta}_2) \neq \mathbf{D}(\boldsymbol{\theta}_2 || \boldsymbol{\theta}_1)$, so KL divergence is not a symmetric distance function, this is clear from the picture: the curve has different gradients at different points. Similarly, we can define the KL divergence for the conditional distribution. In this case, a set of random variables is partitioned into observed variables $\mathbf{y}$ and unobserved variables $\mathbf{x}$, and the probability model is a joint exponential family distribution for $(\mathbf{x}, \mathbf{y})$ [WJ04]:

$$\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}) = \frac{\exp(\langle \boldsymbol{\theta}, \boldsymbol{\varphi}(\mathbf{x}, \mathbf{y}) \rangle)}{\int_{\Omega^m} \exp(\langle \boldsymbol{\theta}, \boldsymbol{\varphi}(\mathbf{x}, \mathbf{y}) \rangle) \nu(d\mathbf{x})} = \exp(\langle \boldsymbol{\theta}, \boldsymbol{\varphi}(\mathbf{x}, \mathbf{y}) \rangle + \mathbf{A}_y(\boldsymbol{\theta})).$$

The Bregman divergence of conditional distribution is then comparable to eqn (F.5):

$$\begin{aligned}
\mathbf{D}(\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) || \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_2)) &= \mathbf{E}_{\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1)} \left[\log(\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1)) - \log(\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_2)) \right] \\
&= \int_{\text{dom}(Y)} \mathbf{p}(\mathbf{y}) \int_{\Omega^m} \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) \left(\langle \boldsymbol{\varphi}(\mathbf{x}, \mathbf{y}), \boldsymbol{\theta}_1 \rangle - \mathbf{A}_y(\boldsymbol{\theta}_1) - \langle \boldsymbol{\varphi}(\mathbf{x}, \mathbf{y}), \boldsymbol{\theta}_2 \rangle + \mathbf{A}_y(\boldsymbol{\theta}_2) \right) \nu(d\mathbf{x}) \nu(d\mathbf{y}) \\
&= \mathbf{A}_y(\boldsymbol{\theta}_2) - \mathbf{A}_y(\boldsymbol{\theta}_1) + (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2) \int_{\mathbf{y} \in \text{dom}(Y)} \mathbf{p}(\mathbf{y}) \int_{\Omega^m} \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) \boldsymbol{\varphi}(\mathbf{x}, \mathbf{y}) \nu(d\mathbf{x}) \nu(d\mathbf{y}) \\
&= \mathbf{A}_y(\boldsymbol{\theta}_2) - \mathbf{A}_y(\boldsymbol{\theta}_1) - \langle \boldsymbol{\mu}_1, \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2 \rangle. \tag{F.7}
\end{aligned}$$

Where going into the last line, we used the moment matching condition to set

$$\int_{\mathbf{y} \in \text{dom}(Y)} \mathbf{p}(\mathbf{y}) \int_{\Omega^m} \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) \boldsymbol{\varphi}(\mathbf{x}, \mathbf{y}) \nu(d\mathbf{x}) \nu(d\mathbf{y}) = \mathbf{E}_{\mathbf{p}(\cdot | \cdot; \boldsymbol{\theta}_1)}[\boldsymbol{\varphi}(\mathbf{x}, \mathbf{y})] = \nabla \mathbf{A}_y(\boldsymbol{\theta}_1) = \boldsymbol{\mu}_1.$$

In more familiar form, we have:

$$\begin{aligned}
\mathbf{D}(\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) || \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_2)) &= \mathbf{E}_{\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1)} \left[\mathbf{D}(\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) || \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_2)) \right] \\
&= \int_{\text{dom}(Y)} \mathbf{p}(\mathbf{y}) \left(\int_{\Omega^m} \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) \cdot \log \frac{\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1)}{\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_2)} \nu(d\mathbf{x}) \right) \nu(d\mathbf{y}). \tag{F.8}
\end{aligned}$$

Its discrete analogue is:

$$\mathbf{D}(\mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_1) || \mathbf{p}(\mathbf{x} | \mathbf{y}; \boldsymbol{\theta}_2)) = \sum_{\mathbf{y}_s \in \text{dom}(Y)} \mathbf{p}(\mathbf{y}_s) \cdot \sum_{(\mathbf{x}_s, \mathbf{y}_s)} \mathbf{p}(\mathbf{x}_s | \mathbf{y}_s; \boldsymbol{\theta}_1) \log \frac{\mathbf{p}(\mathbf{x}_s | \mathbf{y}_s; \boldsymbol{\theta}_1)}{\mathbf{p}(\mathbf{x}_s | \mathbf{y}_s; \boldsymbol{\theta}_2)}.$$

This form can be computed via a great number of sampling methods.

Bibliography

- [ACV12] Abernethy, J., Chen, Y. and Vaughan, J. W. ‘Efficient Market Making via Convex Optimization, and a Connection to Online Learning’. In: *ACM Transactions on Economics and Computation* (2012).
- [AD+19] Arora, S., Du, S. et al. ‘On Exact Computation with an Infinitely Wide Neural Net’. In: *Neural Information Processing Systems* (2019).
- [AD+20] Arora, S., Du, S. et al. ‘Harnessing the Power of Infinitely Wide Deep Nets on Small-data Tasks’. In: *ICLR* (2020).
- [Agr+09] Agrawal, S. et al. ‘A Unified Framework for Dynamic Pari-Mutuel Information Market Design’. In: *Proceedings of the 10th ACM conference on Electronic commerce* (2009).
- [AK+14] Abernethy, J., Kuttly, S. et al. ‘Information Aggregation in Exponential Family Markets’. In: *Proceedings of the fifteenth ACM conference on Economics and computation* (2014).
- [AM+23] A., K., M., G. et al. ‘Will the real stablecoin please stand up?’ In: *BIS Papers* (2023).
- [AS22] A., K.-M. and S., S. ‘Designing Autonomous Markets for Stablecoin Monetary Policy’. In: *arxiv* (2022).
- [AW+21] Alemohammad, S., Wang, Z. et al. ‘The Recurrent Neural Tangent Kernel’. In: *ICRL* (2021).
- [Axl20] Axler, S. *Measure, Integration and Real Analysis*. Springer, 2020.
- [Bar08] Bartlett, P. ‘Representer theorem and kernel examples’. 2008.
- [Bea73] Beals, R. *Advanced Mathematical Analysis: Periodic Functions and Distributions, Complex Analysis, Laplace Transform and Applications*. Springer, 1973.
- [Bel21] Belkin, M. ‘Fit without fear: remarkable mathematical phenomena of deep learning through the prism of interpolation’. In: *Cambridge University Press* (2021).
- [BMR21] Bartlett, P., Montanari, A. and Rakhlin, A. ‘Deep learning: a statistical viewpoint’. In: *Cambridge University Press* (2021).
- [BP98] Brin, S. and Page, L. ‘The anatomy of a large-scale hypertextual Web search engine’. In: *Computer Networks* (1998).
- [BV04] Boyd, S. and Vandenberghe, L. *Convex Optimization*. Cambridge University Press, 2004.
- [CA10] Chen, Y. and Abernethy, J. ‘An Optimization-Based Framework for Automated Market-Making’. In: *12th ACM Conference on Electronic Commerce* (2010).
- [CL06] Cesa-Bianchi, N. and Lugosi, G. *Prediction, Learning, and Games*. Cambridge University Press, 2006.

- [CM99] Cho, G. and Meyer, C. ‘Markov chain sensitivity measured by mean first passage times’. In: *Linear Algebra and its Applications* (1999).
- [CP07] Chen, Y. and Pennock, D. ‘A Utility Framework for Bounded-Loss Market Makers’. In: *Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence* (2007).
- [CV10] Chen, Y. and Vaughan, J. W. ‘A New Understanding of Prediction Markets via No-Regret Learning’. In: *11th ACM Conference on Electronic Commerce* (2010).
- [D25] D., C. ‘Petrodollar to Digital Yuan’. In: *asiasociety* (2025).
- [Dom20] Domingos, P. ‘Every Model Learned by Gradient Descent Is Approximately a Kernel Machine’. In: *arxiv* (2020).
- [DR14] Dwork, C. and Roth, A. *The Algorithmic Foundations of Differential Privacy*. Foundations and Trends in Theoretical Computer Science, 2014.
- [FBS20] Farina, G., Bianchi, T. and Sandholm1, T. ‘Coarse Correlation in Extensive-Form Games’. In: *AAAI* (2020).
- [For+07] Fortnow, C. et al. ‘Betting on Permutations’. In: *EC* (2007).
- [FPW23] Frongillo, R., Papireddygar, M. and Waggoner, B. ‘An Axiomatic Characterization of CFMMs and Equivalence to Prediction Markets’. In: *arXiv* (2023).
- [GB16] Goodfellow, I. and Bengio, Y. *Deep Learning*. MIT Press, 2016.
- [GH+24] Globus-Harris, I., Harrison, D. et al. ‘Diversified Ensembling: An Experiment in Crowdsourced Machine Learning’. In: *FACCT* (2024).
- [GKD19] Görtler, J., Kehlbeck, R. and Deussen, O. ‘A Visual Exploration of Gaussian Processes’. 2019.
- [GP+14] Goodfellow, I., Pouget-Abadie, J. et al. ‘Generative Adversarial Nets’. In: *Advances in neural information processing systems (pp. 2672–2680)* (2014).
- [GR07] Gneiting, T. and Raftery, A. ‘Strictly Proper Scoring Rules, Prediction, and Estimation’. In: *Journal of American Statistical Association* (2007).
- [Han02] Hanson, R. ‘Logarithmic Market Scoring Rules for Modular Combinatorial Information Aggregation’. In: *The Journal of Prediction Markets* (2002).
- [Han03] Hanson, R. ‘Combinatorial Information Market Design’. In: *Information Systems Frontiers* (2003).
- [Har+20] Harz, A. K.-M. et al. ‘Stablecoins 2.0: Economic Foundations and Risk-based Models’. In: *ACM* (2020).
- [HB+20] Hron, J., Bahri, Y. et al. ‘Infinite attention: NNGP and NTK for deep attention networks’. In: *Proceedings of Machine Learning Research* (2020).
- [Hil24] Hill, E. ‘Artificial Bubbles:The Rise and Fall of "AI"’. In: *The Observer Vol 48*. (2024).
- [hkm22] hkma. ‘mBridge: Building a multi CBDC platform for international payments’. In: *hkma* (2022).
- [HSW89] Hornik, K., Stinchcombe, M. and White, H. ‘Multilayer feedforward networks are universal approximators’. In: *Neural Networks* (1989).
- [Hug24] HuggingFace. ‘huggingface.co/models’. In: (2024).
- [HW19] Harris, J. and Waggoner, B. ‘Decentralized and Collaborative AI on Blockchain’. In: *IEEE* (2019).
- [IW06] Ipsen, C. and Wills, R. ‘Mathematical Properties and Analysis of Google’s PageRank’. In: *Bol. Soc. Esp. Mat.* (2006).

- [JG+18] Jacot, A., Gabriel, F. et al. ‘Neural Tangent Kernel: Convergence and Generalization in Neural Networks’. In: *NIPS’18: Proceedings of the 32nd International Conference on Neural Information Processing Systems* (2018).
- [JM24] Jurafsky, D. and Martin, J. *Speech and Language Processing (3rd ed. draft)*. 2024.
- [KP+23] Kwon, Y., Pongmala, K. et al. ‘What Drives the (In)stability of a Stablecoin?’ In: *arxiv* (2023).
- [Kut15] Kutty, S. ‘all men count with you, but none too much: Information Aggregation and Learning in Prediction Markets’. In: (2015).
- [Lal18] Lalley, S. *Random Walks on Infinite Discrete Groups*. Northwestern, 2018.
- [LB+18] Lee, J., Bahri, Y. et al. ‘Deep Neural Networks as Gaussian Processes’. In: *ICRL* (2018).
- [LB18] Lee, J. and Bahri, Y. ‘Deep Neural Networks as Gaussian Processes’. In: *neurips* (2018).
- [LM98] Langville, A. and Meyer, C. ‘Deeper Inside PageRank’. In: *Internet Mathematics Vol. 1, No. 3: 335-380* (1998).
- [LPW17] Levin, D., Peres, Y. and Wilmer, E. *Markov Chains and Mixing Times, second edition*. AMS, 2017.
- [LX+19] Lee, J., Xiao, L. et al. ‘Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent’. In: *neurips* (2019).
- [LZB21] Liu, C., Zhu, L. and Belkin, M. ‘On the linearity of large non-linear models: when and why the tangent kernel is constant’. In: *neurips* (2021).
- [Mat20] Matheau-Raven, O. *Random Walks deon the Symmetric Group: Cutoff for One-sided Transposition Shuffles*. arXiv, 2020.
- [MC+13] Mikolov, T., Chen, K. et al. ‘Efficient Estimation of Word Representations in Vector Space’. In: *arxiv* (2013).
- [Mit13] Mitzenmacher, M. ‘Tossing a Biased Coin’. In: (2013).
- [MP+24] Metaxa, ., Park, J. et al. *Auditing Algorithms: Understanding Algorithmic Systems from the Outside In*. Human-Computer Interaction: Vol. 14, No. 4, 2024.
- [MR95] Motwani, R. and Raghavan, P. *Randomized Algorithms*. Cambridge University Press, 1995.
- [Nea96] Neal, R. *Bayesian Learning for Neural Networks*. Springer-Verlag New York, Inc, 1996.
- [Nic19] Nichani, E. ‘Random Walks on Groups’. In: (2019).
- [ODo21] O’Donnell, R. *Analysis of Boolean Functions*. arXiv, 2021.
- [Pen04] Pennock, D. ‘A Dynamic Pari-Mutuel Market for Hedging, Wagering, and Information Aggregation’. In: *Proceedings of the 5th ACM conference on Electronic commerce* (2004).
- [Pow23] Powell, W. *China, Trust and Digital Supply Chains*. Routledge, 2023.
- [Rad+22a] Radhakrishnan, A. et al. ‘Kernel Regression’. 2022.
- [Rad+22b] Radhakrishnan, A. et al. ‘NNGP, Dual Activations, and Over-Parameterization’. 2022.
- [Rad+22c] Radhakrishnan, A. et al. ‘NTK for Deep Networks and Convolutional NTK (CNTK)’. 2022.
- [Rad+22d] Radhakrishnan, A. et al. ‘NTK Origin and Derivation’. 2022.

- [Rak09] Rakhlin, A. ‘Lecture Notes on Online Learning’. 2009.
- [Rou14] Roughgarden, T. *Lectures Notes on Algorithmic Game Theory*. Tim Roughgarden, 2014.
- [RW06] Rasmussen, C. and Williams, I. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- [SC04] Shawe-Taylor, J. and Cristianini, N. *Kernel Methods for Pattern Analysis*. The Edinburgh Building, Cambridge CB2 2RU, UK: cambridge university press, 2004.
- [Sch00] Scholkopf, B. ‘The Kernel Trick for Distances’. In: *Neural Information Processing Systems* (2000).
- [SCM21] Salehi, M., Clark, J. and Mannan, M. ‘Red-Black Coins: Dai Without Liquidations’. In: *Financial Cryptography and Data Security* (2021).
- [Sha48] Shannon. ‘A Mathematical Theory of Communication’. In: *The Bell System Technical Journal* (1948).
- [Shr10] Shreve, S. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2010.
- [SM+23] S., W., M., B. et al. ‘Central Bank Digital Currency Global Interoperability Principles’. In: *The World Economic Forum* (2023).
- [SS+17] Silver, D., Schrittwieser, J. et al. ‘Mastering the game of Go without human knowledge’. In: *Nature* (2017).
- [SS16] Saitoh, S. and Sawano, Y. *Theory of Reproducing Kernels and its Applications*. Springer, 2016.
- [SW22] Seabrook, E. and Wiskott, L. ‘A Tutorial on the Spectral Theory of Markov Chains’. In: *arxiv* (2022).
- [Tad13] Tadelis, S. *Game Theory: An Introduction*. Princeton University Press, 2013.
- [Tan23] Tanaka, Y. ‘On the Average Hitting Times of Weighted Caley Graphs’. In: *arxiv* (2023).
- [TJ04] Thibaux, R. and Jordan, M. ‘CS281B/Stat241B: Advanced Topics in Learning and Decision Making’. 2004.
- [UBS21] UBS. ‘Project mBridge’. In: *UBS* (2021).
- [Van23] Vanschoren, J. ‘Lecture 3: Kernelization’. 2023.
- [VS+17] Vaswani, A., Shazeer, N. et al. ‘Attention Is All You Need’. In: *neurips* (2017).
- [Was19] Wasserman, L. ‘Function Spaces’. 2019.
- [Wel19] Welling, M. ‘Kernel ridge Regression’. 2019.
- [WFA15] Waggoner, B., Frongillo, R. and Abernethy, J. ‘A Market Framework for Eliciting Private Data’. In: *NIPS* (2015).
- [WJ04] Wainwright, M. and Jordan, M. *Graphical Models, Exponential Families, and Variational Inference*. Berkeley 94720, CA: Foundations and Trends in Machine Learning, 2004.
- [WJ08] Wainwright, M. and Jordan, M. *Graphical Models, Exponential Families, and Variational Inference*. Berkeley 94720, CA: Foundations and Trends in Machine Learning, 2008.
- [Yan20] Yang, G. ‘Tensor Programs II: Neural Tangent Kernel for Any Architecture’. In: *arxiv* (2020).
- [YBS22] Yuan, J., Barmpoutis, P. and Stathaki, T. ‘Effectiveness of Vision Transformer for Fast and Accurate Single-Stage Pedestrian Detection’. In: *neurips* (2022).

- [YL20] Yang, G. and Littwin, E. ‘Tensor Programs IIb: Architectural Universality of Neural Tangent Kernel Training Dynamics’. In: *Proceedings of Machine Learning Research* (2020).
- [ZJ+12] Zhang, Jin et al. ‘Efficient Online Learning for Large-Scale Sparse Kernel Logistic Regression’. In: *AAAI* (2012).